Texas A&M University

Department of Engineering Technology and Industrial Distribution

STAR Autonomous Mapping Robot

Formal Technical Proposal

Locked Inc.

Sponsor: Dr. Logan Porter

Technical Advisor: Dr. Logan Porter

December 8, 2025

The attached document is the formal technical proposal for the Simultaneous Tracking and Robotics (STAR) Autonomous Mapping Robot project by Locked Inc.

Locked Inc. consists of six team members. All six members have a wide spectrum of experience and can contribute to all aspects of the project. The project team includes Cesar Magana, the project manager; Jared Bartz, the software lead; Brighton Sikarskie, the electrical lead; Michael Norton, the mechanical lead; Jeremie Hews, the embedded systems lead; and Shawn Ciaciura, testing and quality lead.

The STAR Robot provides a remote-controlled and autonomous platform for real-time 2D floor plan generation. To address the high cost of entry in the Simultaneous Localization and Mapping (SLAM) market, the system is engineered with a sub-$1,000 target budget for materials. By developing a system capable of integrating both industrial-grade and budget-friendly Light Detection and Ranging (LiDAR) sensors, STAR allows users to scale hardware costs according to their needs. This method makes SLAM technology accessible by using standard components and open-source tools like ROS 2.

The project began development on August 26, 2025, with continued development through May 15, 2026, spanning the ESET-419 and ESET-420 senior design courses. At the conclusion, the final demonstration of the fully functional prototype will be conducted.

Locked Inc. is completely committed to finishing this project on time and within budget. Locked Inc. will retain all intellectual property for the STAR Robot. See Section 1.2 for details.

Sincerely,

Cesar Magana

Cesar Magana
Project Manager
Locked Inc.

Contents

List of Figures

List of Tables

List of Code Snippets

1.0 Introduction

Section Author(s): CM, BS, JB, MN, JH, SC

1.1 Project Overview

This Formal Technical Proposal presents the STAR (Simultaneous Tracking and Autonomous Roaming) Robot, an autonomous indoor mapping platform developed by Locked Inc. for the ESET Senior Design course at Texas A&M University. The STAR Robot addresses the high cost barrier of commercial SLAM (Simultaneous Localization and Mapping) systems by providing an affordable, platform using common hardware and open-source software.

The project team, Locked Inc., consists of six members with complementary expertise spanning embedded systems, mechanical design, electrical engineering, and software development. The team is committed to delivering a fully functional prototype that demonstrates professional project management practices while meeting all technical requirements.

The team has secured many critical components from personal collections and industry donations, including the SK-TDA4VM (Texas Instruments Edge AI) board, PYNQ-Z2 (Xilinx) board, Raspberry Pi 5, test equipment, backup RPLiDAR sensor, ultrasonic sensors, stereo camera, motors, batteries, and other hardware. This resource availability, combined with donated industrial-grade LiDAR equipment from SICK Inc. valued at approximately $3,500, enables the project to achieve professional-level capabilities while maintaining an accessible materials budget.

Table 1: Team Member Roles and Responsibilities

Name	Role	Responsibilities
Cesar Magana (CM)	Project Manager	Project coordination, documentation, stakeholder communication
Jared Bartz (JB)	Software Lead	ROS 2 development, SLAM implementation, navigation stack
Brighton Sikarskie (BS)	Electrical Lead	PCB design, power systems, sensor interfaces
Michael Norton (MN)	Mechanical Lead	Chassis design, 3D printing, assembly, drivetrain integration, CAD modeling
Jeremie Hews (JH)	Embedded Systems Lead	Component integration, communication protocols, ESP32 Firmware
Shawn Ciaciura (SC)	Testing & QA	Test planning, validation, risk management

Team member GitHub profiles and LinkedIn pages are available in Section 17.6.1.

1.2 Intellectual Property Statement

All intellectual property developed during this project remains the property of Locked Inc. team members unless otherwise agreed in writing with the project sponsor. The team has committed to releasing all software as open-source to support the educational robotics community. See Section 17.1 for acronyms and definitions referenced throughout this document.

- While copyright is retained by the team, all source code and hardware assets (including Computer-Aided Design (CAD) files and Printed Circuit Boards (PCB)) are released under the MIT License (Open Source Initiative) on GitHub (GitHub, Inc.) (https://github.com/Locked-Inc/STAR). The MIT License is a permissive open-source license that allows free use, modification, and distribution with minimal restrictions. GitHub is a web-based platform for version control and collaborative software development where the project source code is publicly accessible and managed.
- The team retains intellectual property rights to all design documents and schematics.
- The sponsor may use the final deliverables for research, educational, and demonstration purposes within Texas A&M University.
- Publication rights remain with the team with proper attribution.

2.0 Problem & Solution Statement

Section Author(s): MN

2.1 Problem Statement

Indoor mapping and navigation remain a challenge across multiple industries. Typical methods for developing floor plans and navigating complex indoor environments require substantial manual effort, specialized equipment, and technical expertise. This creates barriers for applications from facility management to emergency response.

The challenge is particularly relevant in environments that change frequently or are difficult for human access. One such situation is that of emergency responders who need real-time spatial awareness in unfamiliar buildings. Current solutions either rely on expensive commercial systems costing tens of thousands of dollars, or require significant technical expertise to assemble and configure from open-source components.

2.2 Industry Statistics and Impact

The autonomous mobile robot market has demonstrated substantial growth and demand for mapping solutions:

- The global market for autonomous mobile robots was valued at $4.07 billion in 2024 and is projected to grow to $9.56 billion by 2030, representing a compound annual growth rate of 15.1% (Grand View Research).
- The robotics industry faces a significant price barrier: commercial SLAM-based robots range from $25,000 for simple line-following AGVs (Automated Guided Vehicles) to over $150,000 for advanced warehouse automation systems, limiting adoption by smaller organizations and educational institutions (Standard Bots).
- Mobile mapping systems significantly reduce data collection time compared to traditional surveying methods. A single operator with autonomous mapping technology can capture survey-quality data in minutes, whereas traditional methods require teams working for hours or days (UtilitiesOne).
- According to the Bureau of Labor Statistics, 5,283 workers suffered fatal work injuries in 2023, equivalent to one worker death every 99 minutes. Many incidents occurred in environments where better spatial awareness and automated inspection could have reduced human exposure to hazards (U.S. Bureau of Labor Statistics).

2.3 Solution Statement

The STAR Robot addresses the challenges of costly commercial mapping systems, time-intensive manual surveying, and hazardous human-operated inspections by providing an autonomous indoor mapping platform at a fraction of commercial system costs. The system combines a SLAM algorithm with readily available hardware components to create a practical, modular, and rapidly deployable solution.

The STAR Robot delivers the following core capabilities:

1. **Real-Time Mapping**: Utilizing 2D LiDAR sensors (SICK TiM561 primary with 270° FoV (Field of View), RPLiDAR C1 backup with 360° FoV), the system generates real-time 2D occupancy grid maps suitable for floor plan generation and navigation.
2. **Autonomous Navigation**: The robot navigates indoor environments without human intervention, able to avoid obstacles and explore unmapped areas systematically.
3. **Modular Architecture**: The system design supports additional expansion and modification, enabling iterative development and future capability enhancements.

2.4 Solution Alignment

Implementation of the STAR Robot provides the following benefits for consumers:

- **Safety Enhancement:** Remote operation capabilities keep humans out of potentially hazardous environments during mapping.
- **Cost Reduction:** With a materials budget under $1,000, the system costs less than 4% of entry-level commercial SLAM robots ($25,000+) and less than 1% of advanced warehouse automation systems ($150,000+) (Standard Bots).
- **Accuracy:** LiDAR-based mapping is able to achieve centimeter-level accuracy suitable for floor plan generation and navigation.
- **Time:** Autonomous operation eliminates the need for constant human oversight during mapping operations
- **Extensibility:** A modular system supports the integration of additional sensors, algorithms, and capabilities.

3.0 Conceptual Block Diagram

Section Author(s): MN

3.1 System Overview

A Conceptual Block Diagram (CBD) provides a high-level visual representation of a system's major functional components and their relationships. Unlike detailed schematics, the CBD abstracts implementation specifics to communicate the overall system architecture and data flow to stakeholders. This top-down approach enables early validation of system concepts before committing to detailed design decisions.

The conceptual block diagram in Figure 1 illustrates the main sequence of operations performed by the system. To begin the process, the small mobile platform robot is placed within the environment to be mapped. The robot then systematically navigates throughout the space using an autonomous control algorithm. As it moves, it generates a map with its onboard LiDAR sensor and transfers the resulting data over the intranet to a computer for user access.

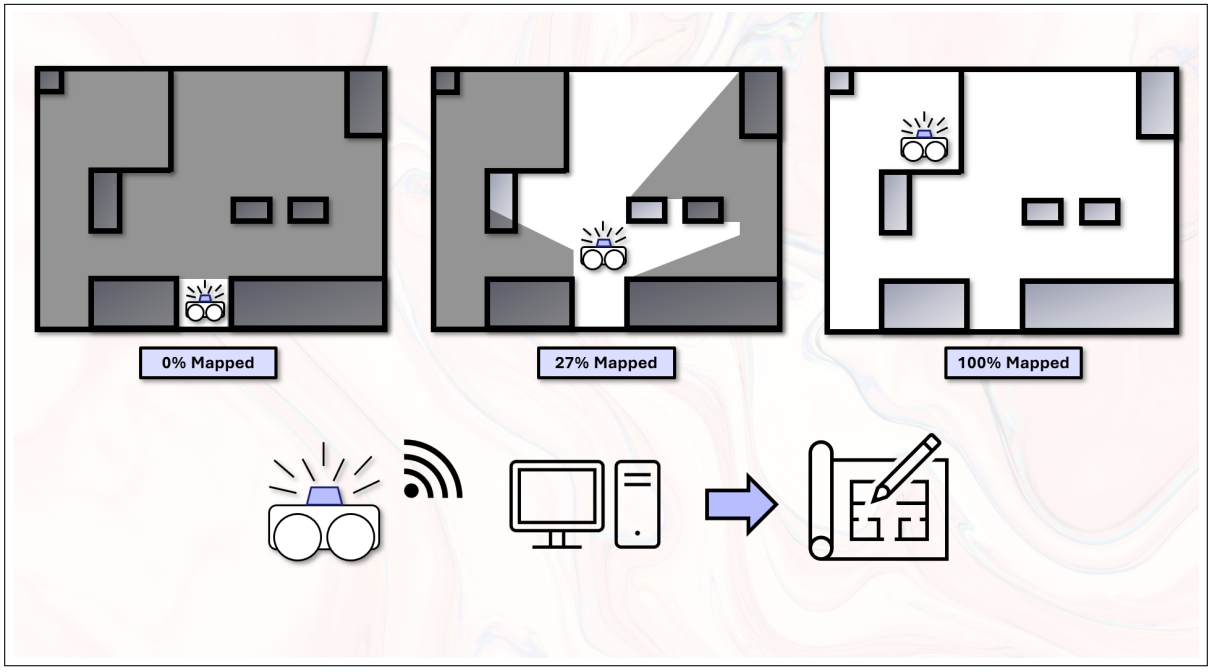


Figure 1: System Conceptual Block Diagram

4.0 Project Scope

Section Author(s): CM, BS

4.1 Scope Overview

This section defines the boundaries of the STAR Robot project, clearly establishing what is included and excluded from the project deliverables. The STAR Robot is a LiDAR SLAM robot with remote control capabilities designed for autonomous indoor mapping.

4.2 Scope Definition

4.2.1 What IS Included

The following capabilities and deliverables are within the project scope:

- **Autonomous Navigation System**
  - Self-directed exploration of indoor environments
  - Real-time obstacle detection and avoidance
  - Path planning and trajectory execution
- **SLAM-Based Mapping**
  - 2D occupancy grid map generation
  - Real-time localization within generated maps
  - Map saving and loading capabilities
  - Integration with ROS 2 navigation stack
- **Hardware Platform**
  - Custom PETG/aluminum chassis with high-compliance wheel drive
  - SICK TiM561 2D LiDAR: 270° FoV, 10m range, 15Hz scan rate, $\pm$60mm accuracy (donated, \$3,500 value including cables)
  - RPLiDAR C1 2D LiDAR (backup/development): 360° FoV, 12m range, 10Hz scan rate, $\pm$30mm accuracy, \$55
  - Ultrasonic/sonar sensors for obstacle detection redundancy and DWA (Dynamic Window Approach) local planner interruption, a real-time collision avoidance algorithm that evaluates safe velocity trajectories
  - DS18B20+ temperature sensor for HC-SR04 ultrasonic calibration, enabling speed-of-sound compensation ($v = 331.3 + 0.606 \cdot T$) to improve distance measurement accuracy from $\pm$3% to $\pm$0.5%

  - NTC thermistors integrated into power management ICs for motor driver and battery thermal monitoring with automatic safety shutdown
  - Raspberry Pi 5 compute platform (high-level processing)
  - ESP32-S3 microcontroller operating as SPI peripheral to Raspberry Pi 5, responsible for DS18B20+ temperature sensing and motor control via DRV8243-Q1 drivers using hardware MCPWM (Motor Control Pulse Width Modulator)
  - BNO055 IMU connected to Raspberry Pi 5 via I^2C for orientation sensing with onboard sensor fusion
  - LiPo battery power system with fuel gauge monitoring (low-level control via SMBus)
  - DRV8243-Q1 motor drivers (4x automotive-grade H-bridge) configured by ESP32 via SPI at boot, then controlled via EN (direction) and PH (speed) PWM signals; encoder feedback at 100Hz for closed-loop velocity control
  - HC-SR04 ultrasonic sensors (7x) connected to Raspberry Pi 5 GPIO for close-range obstacle detection (2–400 cm)
  - A7670G 4G LTE cellular module for remote connectivity and telemetry
  - Power management system: USB-C DRP(Dual-Role Power) PD(Power Delivery) port with buck-boost charger for LiPo battery charging, fuel gauge (BQ78350) for state-of-charge monitoring, and regulated 6V/5V/3.3V rails for system power
- **Software System**
  - ROS 2 Humble framework implementation
  - Nav2 navigation stack integration
  - SLAM Toolbox for mapping
  - Custom Buildroot configuration generating a minimal Linux image for Raspberry Pi 5, optimized for ROS 2 and real-time sensor processing
  - RViz2 (ROS Visualization 2) for real-time 3D visualization of sensor data, robot state, and generated maps
- **Documentation**
  - Complete assembly instructions
  - Software installation guide
  - User operation manual
  - Technical design documentation

4.2.2 What is NOT Included

The following items are explicitly excluded from the project scope:

- **Outdoor Operation**: The system is designed for indoor, structured environments only. Out-

door terrain, weather resistance, and GPS integration are excluded.

- **Manipulation Capabilities**: No robotic arm, gripper, or object manipulation functionality is included.
- **Multi-Robot Coordination**: The project focuses on a single-robot system. Swarm behavior and multi-robot communication are excluded.
- **Paid Cloud-Based Processing**: All computation occurs on-board. Cloud connectivity and remote processing are excluded; a local web-based interface for map visualization is included.
- **3D Reconstruction**: Our 2D LiDAR sensors (SICK TiM561, RPLiDAR C1) scan in a single horizontal plane and cannot generate 3D volumetric data. The system produces 2D occupancy grid maps only.
- **Machine Learning Training**: Pre-trained models may be used, but custom ML model development is excluded. Locked Inc. will finetune models, but not create any from scratch.
- **Return-to-home functionality**: Autonomous return-to-origin requires accurate long-term odometry which is subject to cumulative drift errors. The system relies on manual teleoperation via remote control for return navigation, which provides more reliable control in the prototype stage.

4.3 Requirements

4.3.1 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Priority
FR-001	Robot shall navigate autonomously through indoor environments, completing a 50 m path without human intervention	High
FR-002	Robot shall detect and avoid obstacles greater than 10 cm at speeds up to 0.3 m/s with 85% success rate	High
FR-003	System shall generate 2D occupancy grid maps with accuracy within 5 cm of actual room dimensions	High
FR-004	LiDAR shall provide continuous scanning (270° SICK TiM561 or 360° RPLiDAR C1) at minimum 10 Hz update rate	High

ID	Requirement	Priority
FR-005	Robot shall localize within generated maps with position error less than 10 cm after loop closure	High
FR-006	System shall export maps in standard ROS 2 format (.pgm and .yaml files)	Medium
FR-007	Motor controllers shall provide encoder feedback at minimum 100Hz for odometry calculations	Medium
FR-008	Robot shall support manual teleoperation mode via remote control interface	Medium
FR-009	System shall provide real-time visualization of sensor data and maps through RViz2	Low

4.3.2 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Requirement	Priority
NFR-001	System shall operate continuously for minimum 60 minutes on a single battery charge under typical navigation load	High
NFR-002	Maximum robot velocity shall not exceed 0.5m/s for safety in indoor environments	High
NFR-003	Robot physical dimensions shall allow passage through standard doorways (width $<$76cm, height $<$100cm)	High
NFR-004	ROS 2 software shall maintain CPU utilization below 80% on Raspberry Pi 5 during SLAM operations	High
NFR-005	Total material cost for reproducible build shall remain under $1,000	Medium
NFR-006	System shall be maintainable by a single technician with standard tools	Medium
NFR-007	Software shall be fully documented with installation guides and API references	Medium
NFR-008	Robot weight shall not exceed 10 kg to ensure safe handling and adequate battery runtime	Low

4.4 Key Deliverables

The project will produce the following deliverables with associated completion dates. These deliverables align with project milestones shown on (Table 25).

Table 4: Project Deliverables and Schedule

ID	Deliverable	Description	Due Date
D1	Design Package	Complete mechanical, electrical, and software design documentation	Feb 28, 2026
D2	Hardware Platform	Assembled robot chassis with all sensors and electronics integrated	Mar 31, 2026
D3	Software System	Functional ROS 2 software with navigation and mapping capabilities	Apr 15, 2026
D4	System Integration	Fully integrated robot demonstrating autonomous mapping	May 1, 2026
D5	Final Prototype	Tested and validated system with complete documentation	May 8, 2026

4.5 Test Methods

Each requirement has an associated test method to verify compliance:

Detailed test procedures, acceptance criteria, and validation results are documented in the Implementation section (see Section 16.0) and the Test Plan section (Table 89).

1. **Navigation Test**: Robot navigates predefined path in lab environment; success measured by completion without human intervention.
2. **Obstacle Avoidance Test**: Course with obstacles of various sizes; robot must detect and avoid all obstacles above 10cm threshold.
3. **Mapping Accuracy Test**: Map generated room with known dimensions; compare generated map to laser-measured ground truth.
4. **Sensor Validation**: Verify LiDAR reports valid data across full 360-degree range with no blind spots.
5. **Endurance Test**: Continuous operation under typical navigation workload until battery depletion; measure total runtime.
6. **Performance Monitoring**: Log CPU, memory, and sensor rates during operation using

`ros2 topic hz` for message rates (Open Robotics, ROS 2 Documentation: Jazzy Jalisco), `diagnostic_common_diagnostics` for system health (diagnostic_common_diagnostics — ROS 2 Package), and `htop` for process-level resource usage.

5.0 Assumptions

Section Author(s): CM, MN

5.1 Assumptions Overview

This section documents the assumptions that the team has taken into consideration for the project. These assumptions define the boundaries within which the project will operate and help the team focus their efforts on defined deliverables.

5.2 Technical Assumptions

5.2.1 Hardware Platform

1. **Compute Platform**: The Raspberry Pi 5 with 8GB RAM provides sufficient computational resources for running ROS 2, SLAM Toolbox, and Nav2 navigation stack simultaneously at acceptable frame rates. The ESP32 microcontroller handles motor control and encoder feedback via SPI communication, enabling sensor fusion between the main processor and motor subsystem.
2. **LiDAR Performance**: The SICK TiM561 2D LiDAR serves as the primary sensor, providing accurate range measurements within its 10-meter range across its 270-degree field of view at 15Hz scan rate with $\pm$60mm accuracy. This industrial-grade sensor was donated by SICK Inc. (valued at $3,500 including cables). The RPLiDAR C1 2D LiDAR (360° FoV, 12m range, 10Hz, $\pm$30mm accuracy, $55), purchased separately by the team, serves as a backup sensor for redundancy and development testing.
3. **Depth Camera Compatibility**: The IMX219-83 Stereo Camera connects to the Raspberry Pi 5 via the CSI-2 (Camera Serial Interface) ribbon cable connector, using V4L2 camera drivers and OpenCV for stereo depth computation.
4. **Motor Control**: The DC gear motors with encoders provide sufficient torque to move the robot with all components at the target velocity of 0.5 m/s on smooth, level surfaces.
5. **Battery Capacity**: A custom 3S LiPo battery pack (3S denotes three lithium polymer cells in series, yielding 11.1V nominal / 12.6V fully charged; parallel cell count to be determined during testing, approximately 2500mAh per cell) provides at least 60 minutes of operation under typical navigation workloads.

5.2.2 Software Environment

1. **Operating System**: A custom Linux image built with Buildroot runs stably on the Raspberry Pi 5, providing a minimal footprint optimized for ROS 2 and SLAM operations with all required drivers and packages.
2. **ROS 2 Humble**: ROS 2 Humble Hawksbill is the target distribution and is assumed to be stable and well-supported throughout the project duration.
3. **SLAM Toolbox**: SLAM Toolbox is a ROS 2-based SLAM algorithm that creates 2D occupancy grid maps using LiDAR data, requiring wheel odometry.
4. **Nav2 Stack**: Nav2 (Navigation 2) is the ROS 2 navigation framework that provides autonomous navigation capabilities including global and local path planning, costmap generation, obstacle avoidance, and behavior trees for complex navigation behaviors.
5. **Python 3**: Python 3.10+ is available on the target platform for custom node development and scripting.

5.2.3 Operating Environment

1. **Indoor Environment**: The robot will operate exclusively in indoor, structured environments with:
    - Flat, hard flooring (tile, concrete, wood)
    - Standard doorways (minimum 76cm width)
    - Adequate lighting for depth camera operation
    - Primarily static obstacles, dynamic obstacle avoidance using DWA (Dynamic Window Approach) is a stretch goal
2. **WiFi Availability**: WiFi connectivity is available in operating environments for remote monitoring and visualization. As a stretch goal, cellular module integration may provide connectivity in environments without WiFi infrastructure.
3. **Temperature Range**: Operating temperature remains within 15-30°C (typical indoor climate control).
4. **Obstacle Characteristics**: Obstacles are solid, opaque objects detectable by both LiDAR (range) and depth camera (visual).

5.3 Resource Assumptions

5.3.1 Team Availability

1. **Team Commitment**: All six team members remain enrolled and committed to the project for the full duration (August 2025 - May 2026).

2. **Work Hours**: Team members dedicate an average of 8-10 hours per week to project work during academic semesters.
3. **Skill Availability**: Team members possess or can acquire necessary skills in robotics, ROS 2, CAD, and electronics within the project timeline.
4. **Communication**: Team members are responsive to communications within 24 hours on weekdays.

5.3.2 Facilities and Equipment

1. **Lab Access**: The team has access to university fabrication labs and testing spaces throughout the project.
2. **Tools**: Standard hand tools, soldering equipment, and 3D printers are available through university facilities.
3. **Test Environment**: Suitable indoor test spaces are available, including university facilities (minimum 100 sq ft) and TEEX's Disaster City complex for realistic post-disaster scenario testing.
4. **Personal Computers**: Team members have personal computers capable of running ROS 2 development tools and simulation.

5.3.3 Budget and Procurement

1. **Funding**: The department provides less than $1,000 for materials. Team members have self-funded additional components to accelerate development and reduce dependency on the limited department allocation.
2. **Component Availability**: All specified components are available for purchase from domestic and international suppliers within stated lead times.

5.4 Project Management Assumptions

1. **Sponsor Engagement**: The project sponsor and technical advisor (Dr. Logan Porter) is available for bi-weekly status meetings and provides timely feedback on deliverables.
2. **Requirements Stability**: Core project requirements remain stable after Research phase completion. Any changes must follow formal change control and be approved by all members and the sponsor in a meeting.
3. **Course Requirements**: ESET-419 course requirements and deadlines do not change significantly from current expectations.

5.5 Assumption Validation

Assumptions are validated through the following methods:

Table 5: Assumption Validation Methods

Assumption Category	Validation Approach
Compute Platform Performance	Benchmark testing during Research phase
Sensor Compatibility	Prototype integration during Design phase
Battery Runtime	Endurance testing during Build phase
Operating Environment	Testing in representative environments
Team Availability	Weekly check-ins and commitment tracking using the Gantt Chart
Facility Access	Advance scheduling and confirmation
Component Availability	Early procurement of long-lead items

5.6 Assumption Tracking

Assumptions are reviewed bi-monthly and updated as:

- **Confirmed**: Assumption validated as true
- **Adjusted**: Assumption modified based on new information or finding
- **Invalidated**: Assumption found to be false, mitigation required

Invalidated assumptions are escalated as risks and addressed through the risk management process. For more information on risk management and mitigation strategies for invalidated assumptions, see Section 12.0 (Risk Assessment).

6.0 Work Breakdown Structure

Section Author(s): CM, MN

6.1 WBS Overview

The Work Breakdown Structure (WBS) breaks the project scope into manageable work packages (WorkBreakdownStructure.com). It organizes all work under six main categories: Research, Design, Build, Test, Close, and Project Management Overhead. This structure adds detail at every level to clearly define specific tasks and deliverables.

6.1.1 Team Member Assignments

Each leaf-level activity is assigned a primary owner who is accountable for its completion.

6.1.2 Ownership Structure

Activity ownership follows a RACI-based(Responsible Accountable Consulted Informed) assignment model (Project Management Institute, RACI Chart: Roles and Responsibilities):

- **Owner (A/R)**: Primary team member accountable for and responsible for completing the activity. Shown in the "Owner" column of WBS tables.
- **Sub-Owners**: Supporting team members who contribute as Responsible (R), Consulted (C), or Informed (I) parties.

Ownership ensures clear accountability, balanced workload distribution, and skills alignment. **For detailed resource allocation including secondary assignments and person-day calculations, see the WBS Activity Assignment List in Section 11.0, starting in Table 36.**

6.2 Control Accounts

The WBS is organized under six primary control accounts (Defense Acquisition University):

Table 6: WBS Control Account Summary

ID	Control Account	Tasks	Description
1.1	Research	13	Requirements gathering, technology evaluation
1.2	Design	24	Architecture, mechanical, electrical, software design
1.3	Build	20	Procurement, fabrication, assembly, integration
1.4	Test	15	Validation, verification, system testing
1.5	Close	7	Documentation, handoff, final report
1.6	PM Overhead	5	Meetings, status reports, risk management
	Total	**84**	

6.3 WBS Hierarchy

The hierarchical structure follows this pattern:

- **Level 1**: Project Root (1.0 Project Name)
- **Level 2**: Control Accounts (Research, Design, Build, Test, Close, PM Overhead)
- **Level 3**: Categories (Requirements Analysis, System Architecture)
- **Level 4**: Work Packages (Functional Requirements, Algorithm Design)
- **Level 5**: Tasks (Document user stories, Evaluate options)

6.4 WBS Diagram

The complete WBS overview diagram is included in Appendix 17.2 (Figure 17.2). The following subsections show the detailed WBS structure in table format, generated from the project's activities database.

6.5 Research Phase Work Breakdown

Total Elements: 20
Work Packages: 13

Note: This phase diagram is split across three figures to improve readability and ensure the work breakdown structure elements remain visible at a legible size.

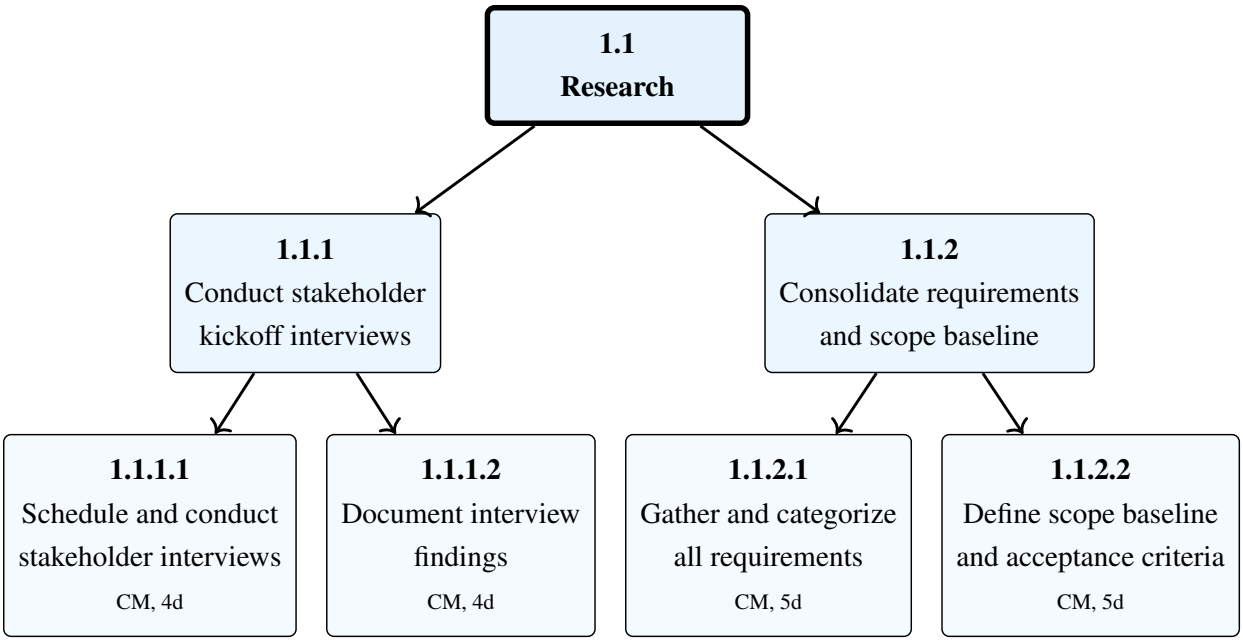


Figure 2: Research Phase Work Breakdown Structure (Part 1: Stakeholder & Requirements)

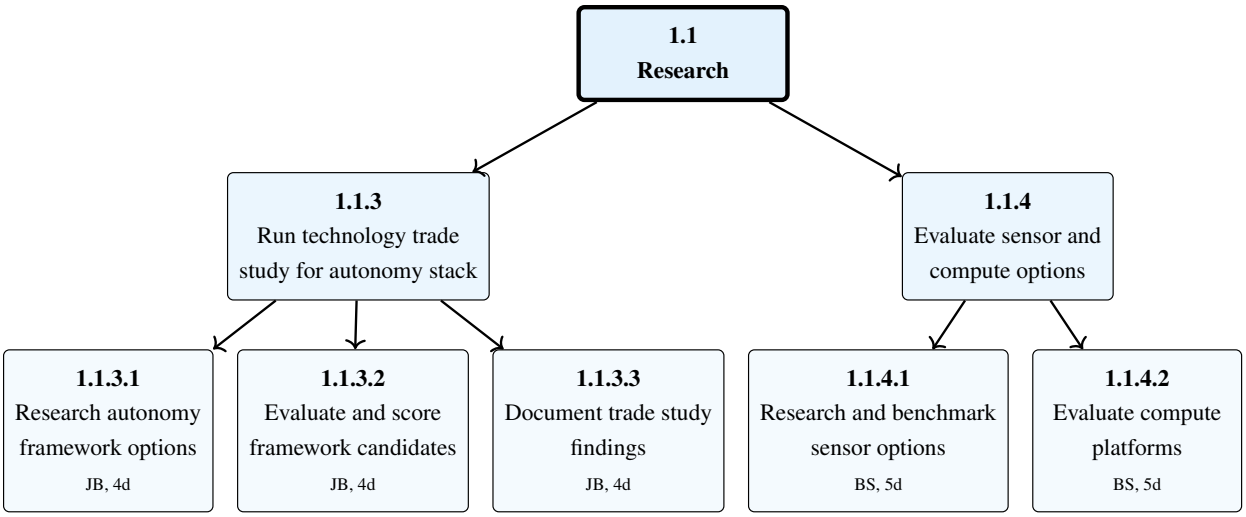


Figure 3: Research Phase Work Breakdown Structure (Part 2: Technology Trade Studies)

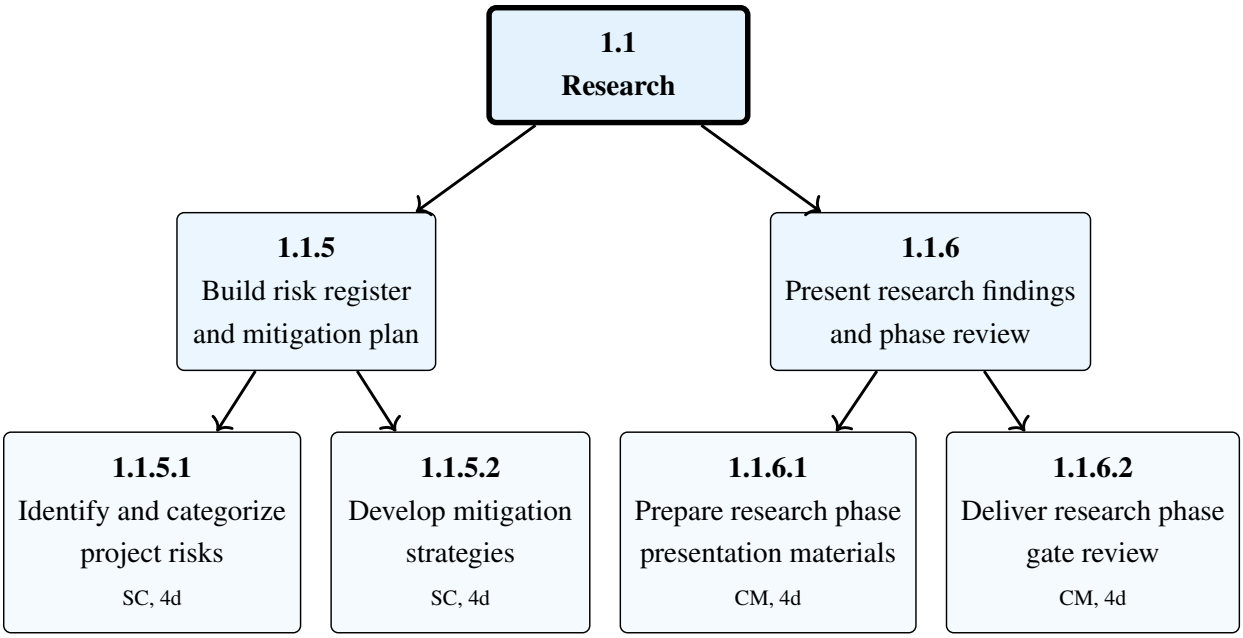


Figure 4: Research Phase Work Breakdown Structure (Part 3: Risk & Review)

WBS ID	Task Name	Days	Owner
1.1.1	**Conduct stakeholder kickoff interviews**	–	–
1.1.1.1	Schedule and conduct stakeholder interviews	4	CM
1.1.1.2	Document interview findings and requirements	4	CM
1.1.2	**Consolidate requirements and scope baseline**	–	–
1.1.2.1	Gather and categorize all requirements	5	CM
1.1.2.2	Define scope baseline and acceptance criteria	5	CM
1.1.3	**Run technology trade study for autonomy stack**	–	–
1.1.3.1	Research autonomy framework options	4	JB
1.1.3.2	Evaluate and score framework candidates	4	JB
1.1.3.3	Document trade study findings and recommendation	4	JB
1.1.4	**Evaluate sensor and compute options with team**	–	–
1.1.4.1	Research and benchmark sensor options	5	BS
1.1.4.2	Evaluate compute platforms and make selection	5	BS
1.1.5	**Build risk register and mitigation plan**	–	–
1.1.5.1	Identify and categorize project risks	4	SC
1.1.5.2	Develop mitigation strategies and contingency plans	4	SC
1.1.6	**Present research findings and phase review**	–	–
1.1.6.1	Prepare research phase presentation materials	4	CM
1.1.6.2	Deliver research phase gate review	4	CM

Table 7: Research Phase Work Breakdown

See Appendix 17.2.1 for the complete Research phase WBS diagram.
See Table 36 for table including co-owners of tasks.

6.6 Design Phase Work Breakdown

Total Elements: 32

Work Packages: 24

Note: This phase diagram is split across four figures to improve readability and ensure the work breakdown structure elements remain visible at a legible size.

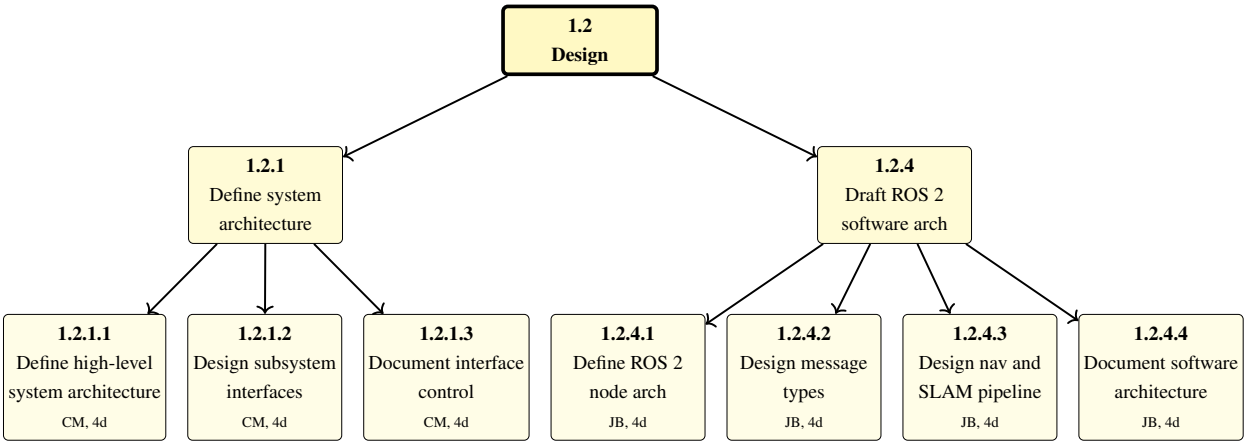


Figure 5: Design Phase Work Breakdown Structure (Part 1: System Architecture)

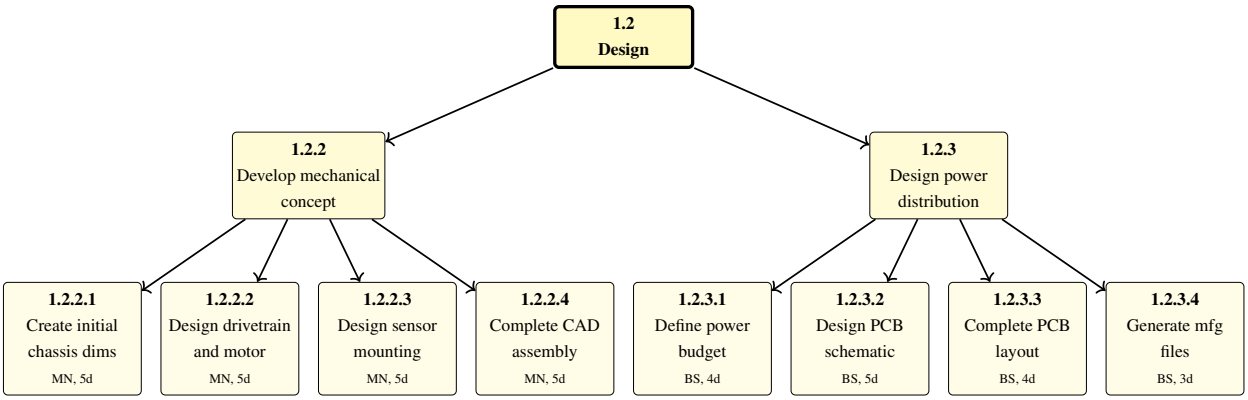


Figure 6: Design Phase Work Breakdown Structure (Part 2: Mechanical & Electrical)

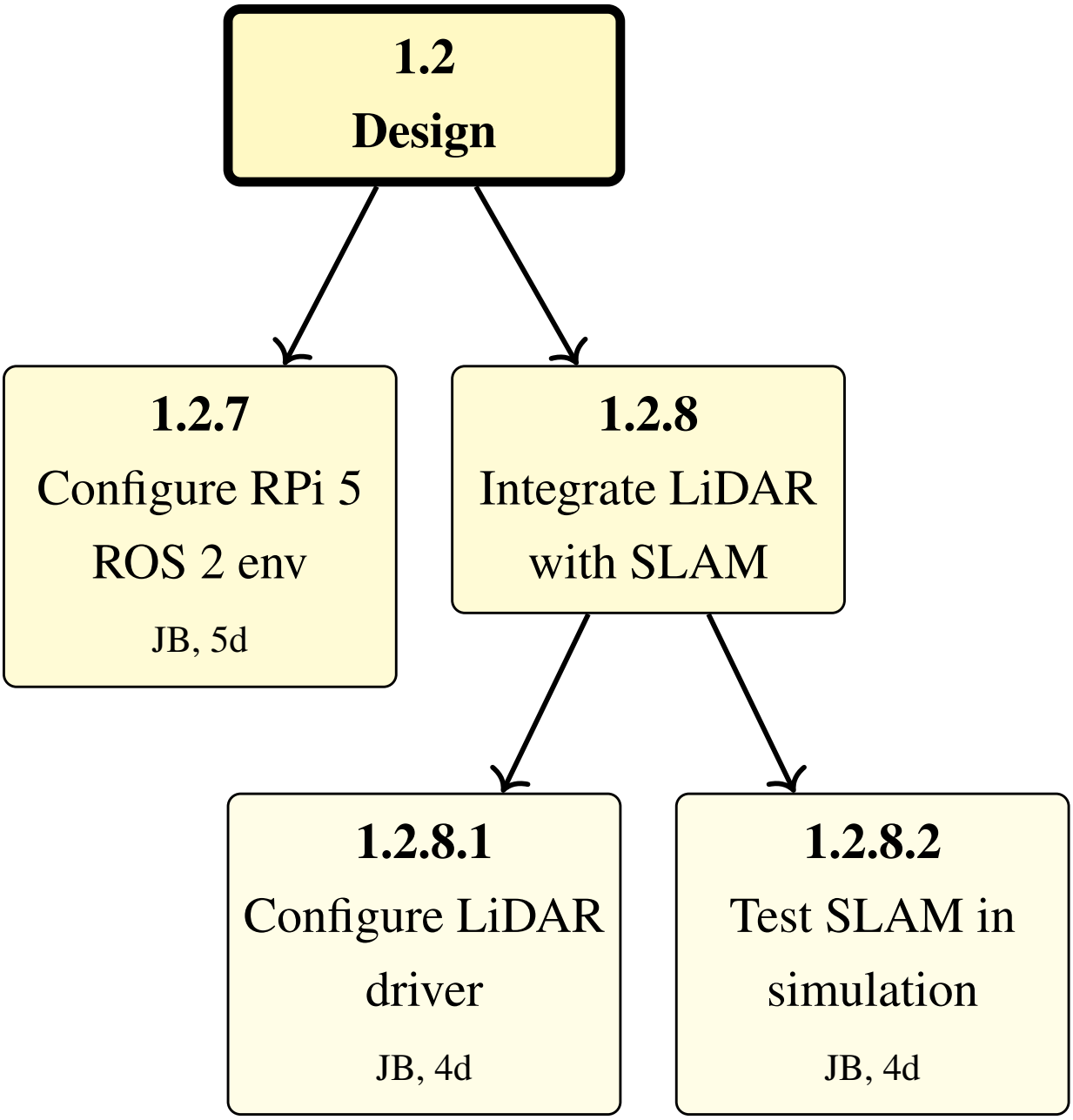


Figure 7: Design Phase Work Breakdown Structure (Part 3: Software Configuration)

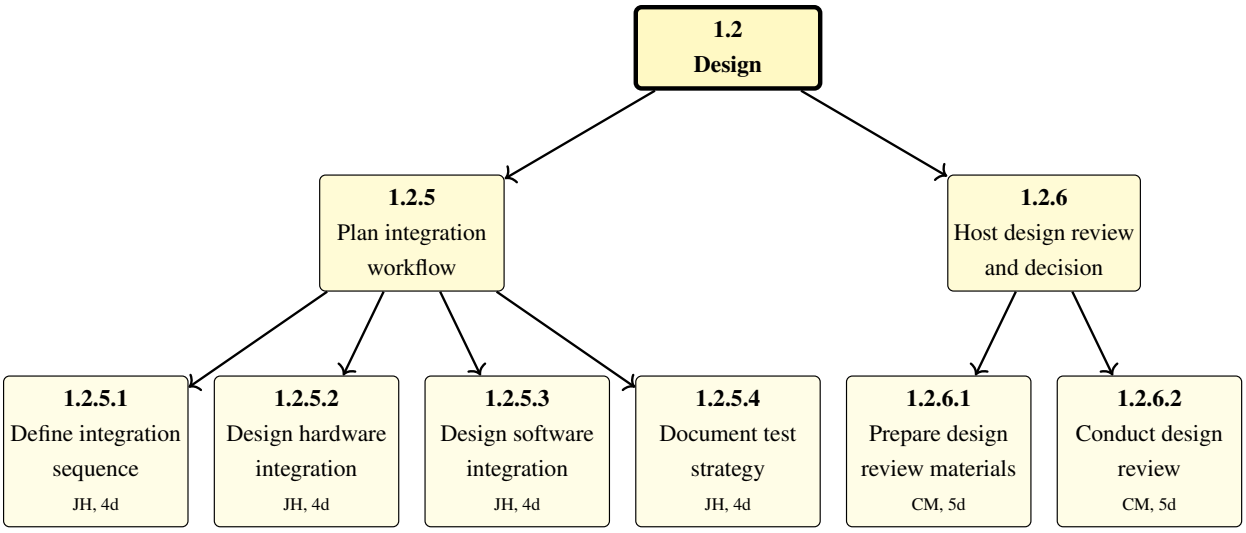


Figure 8: Design Phase Work Breakdown Structure (Part 4: Integration & Review)

Table 8: Design Phase Work Breakdown

WBS ID	Task Name	Days	Owner
1.2.1	**Define system architecture and interface map**	–	–
1.2.1.1	Define high-level system architecture	4	CM
1.2.1.2	Design subsystem interfaces and protocols	4	CM
1.2.1.3	Document interface control and data flow	4	CM
1.2.2	**Develop mechanical concept and CAD package**	–	–
1.2.2.1	Create initial chassis dimensions and constraints	5	MN
1.2.2.2	Design drivetrain and motor mounting	5	MN
1.2.2.3	Design sensor mounting and enclosures	5	MN
1.2.2.4	Complete CAD assembly and generate drawings	5	MN
1.2.3	**Design power distribution and PCB**	–	–
1.2.3.1	Define power budget and distribution architecture	4	BS
1.2.3.2	Design PCB schematic and component selection	5	BS
1.2.3.3	Complete PCB layout and routing	4	BS
1.2.3.4	Generate manufacturing files and submit order	3	BS
1.2.4	**Draft ROS 2 software architecture**	–	–
1.2.4.1	Define ROS 2 node architecture	4	JB
1.2.4.2	Design message types and topic structure	4	JB
1.2.4.3	Design navigation and SLAM pipeline	4	JB
1.2.4.4	Document software architecture and APIs	4	JB
1.2.5	**Plan integration workflow and test strategy**	–	–
1.2.5.1	Define integration sequence and dependencies	4	JH

Continued on next page

Table 8 continued from previous page

WBS ID	Task Name	Days	Owner
1.2.5.2	Design hardware integration test procedures	4	JH
1.2.5.3	Design software integration test procedures	4	JH
1.2.5.4	Document test strategy and success criteria	4	JH
1.2.6	**Host design review and decision gate**	–	–
1.2.6.1	Prepare design review materials and agenda	5	CM
1.2.6.2	Conduct design review and document decisions	5	CM
1.2.7	Configure Raspberry Pi 5 ROS 2 environment	5	JB
1.2.8	**Integrate LiDAR sensor with SLAM algorithm**	–	–
1.2.8.1	Configure LiDAR driver and ROS interface	4	JB
1.2.8.2	Test SLAM algorithm in simulation	4	JB

See Appendix 17.2.2 for the complete Design phase WBS diagram.

6.7 Build Phase Work Breakdown

Total Elements: 27
Work Packages: 20

Note: This phase diagram is split across three figures to improve readability and ensure the work breakdown structure elements remain visible at a legible size.

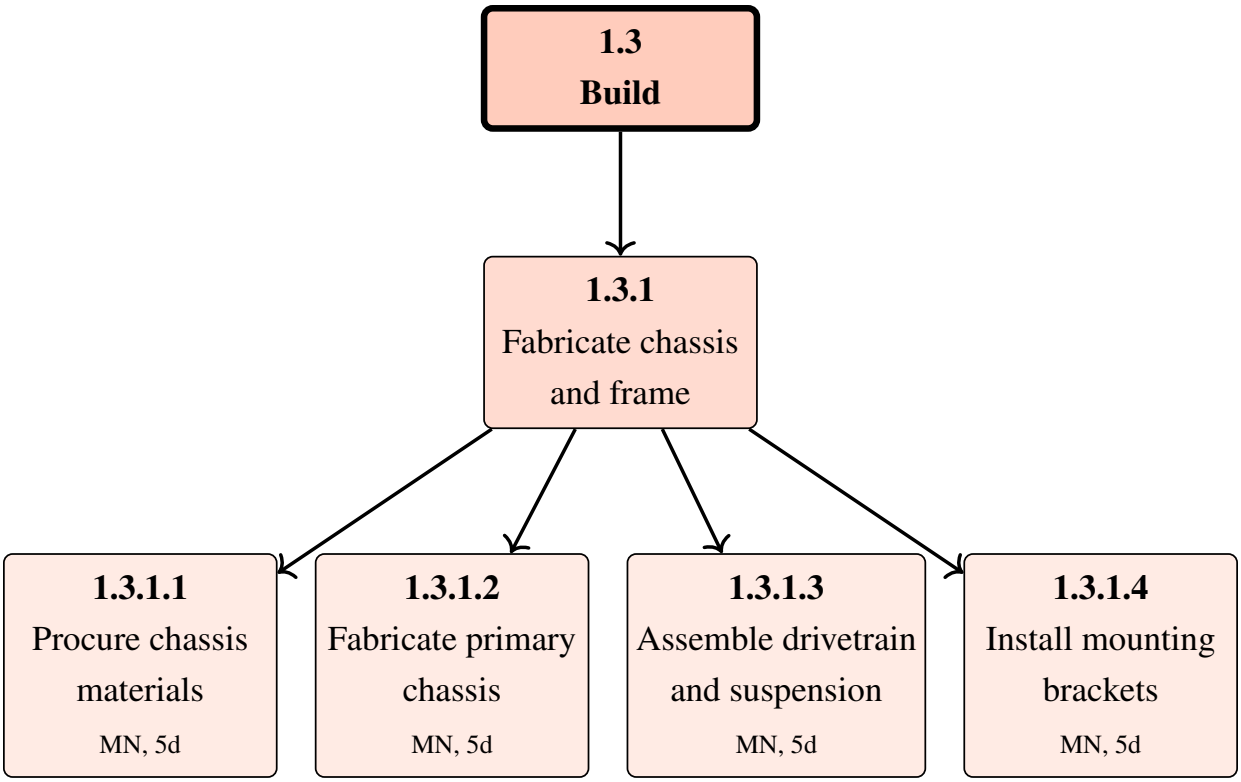


Figure 9: Build Phase Work Breakdown Structure (Part 1: Mechanical Fabrication)

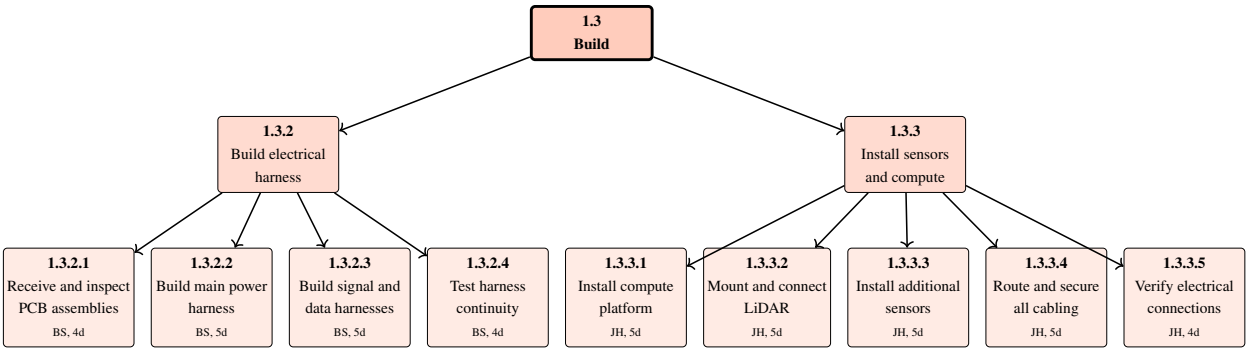


Figure 10: Build Phase Work Breakdown Structure (Part 2: Electrical Assembly)

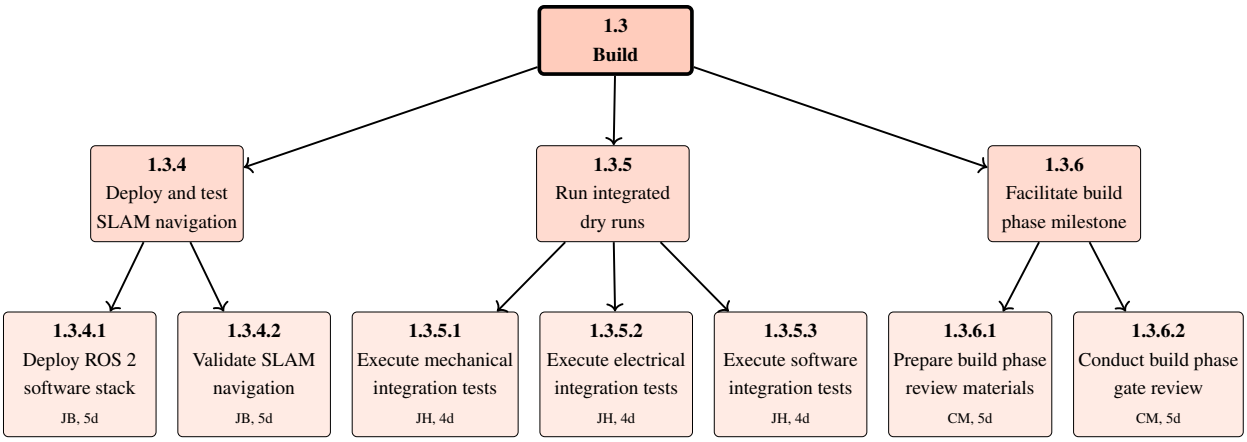


Figure 11: Build Phase Work Breakdown Structure (Part 3: Software Deployment & Review)

WBS ID	Task Name	Days	Owner
1.3.1	**Fabricate chassis and assemble mechanical frame**	–	–
1.3.1.1	Procure chassis materials and components	5	MN
1.3.1.2	Fabricate primary chassis structure	5	MN
1.3.1.3	Assemble drivetrain and suspension	5	MN
1.3.1.4	Install mounting brackets and verify alignment	5	MN
1.3.2	**Build electrical harness and power bus**	–	–
1.3.2.1	Receive and inspect PCB assemblies	4	BS
1.3.2.2	Build main power distribution harness	5	BS
1.3.2.3	Build signal and data harnesses	5	BS
1.3.2.4	Test harness continuity and isolation	4	BS
1.3.3	**Install sensors, compute, and cabling**	–	–
1.3.3.1	Install compute platform and configure OS	5	JH
1.3.3.2	Mount and connect LiDAR sensor	5	JH
1.3.3.3	Install additional sensors and peripherals	5	JH
1.3.3.4	Route and secure all cabling	5	JH
1.3.3.5	Verify all electrical connections and power-on test	4	JH
1.3.4	**Deploy and test SLAM navigation on physical robot**	–	–
1.3.4.1	Deploy ROS 2 software stack to robot	5	JB
1.3.4.2	Validate SLAM navigation on physical robot	5	JB
1.3.5	**Run integrated dry runs and readiness checks**	–	–
1.3.5.1	Execute mechanical integration tests	4	JH
1.3.5.2	Execute electrical integration tests	4	JH
1.3.5.3	Execute software integration tests	4	JH
1.3.6	**Facilitate build phase milestone review**	–	–
1.3.6.1	Prepare build phase review materials	5	CM
1.3.6.2	Conduct build phase gate review	5	CM

Table 9: Build Phase Work Breakdown

See Appendix 17.2.3 for the complete Build phase WBS diagram.
See Table 36 for table including co-owners of tasks.

6.8 Test Phase Work Breakdown

Total Elements: 21
Work Packages: 15

Note: This phase diagram is split across three figures to improve readability and ensure the work breakdown structure elements remain visible at a legible size.

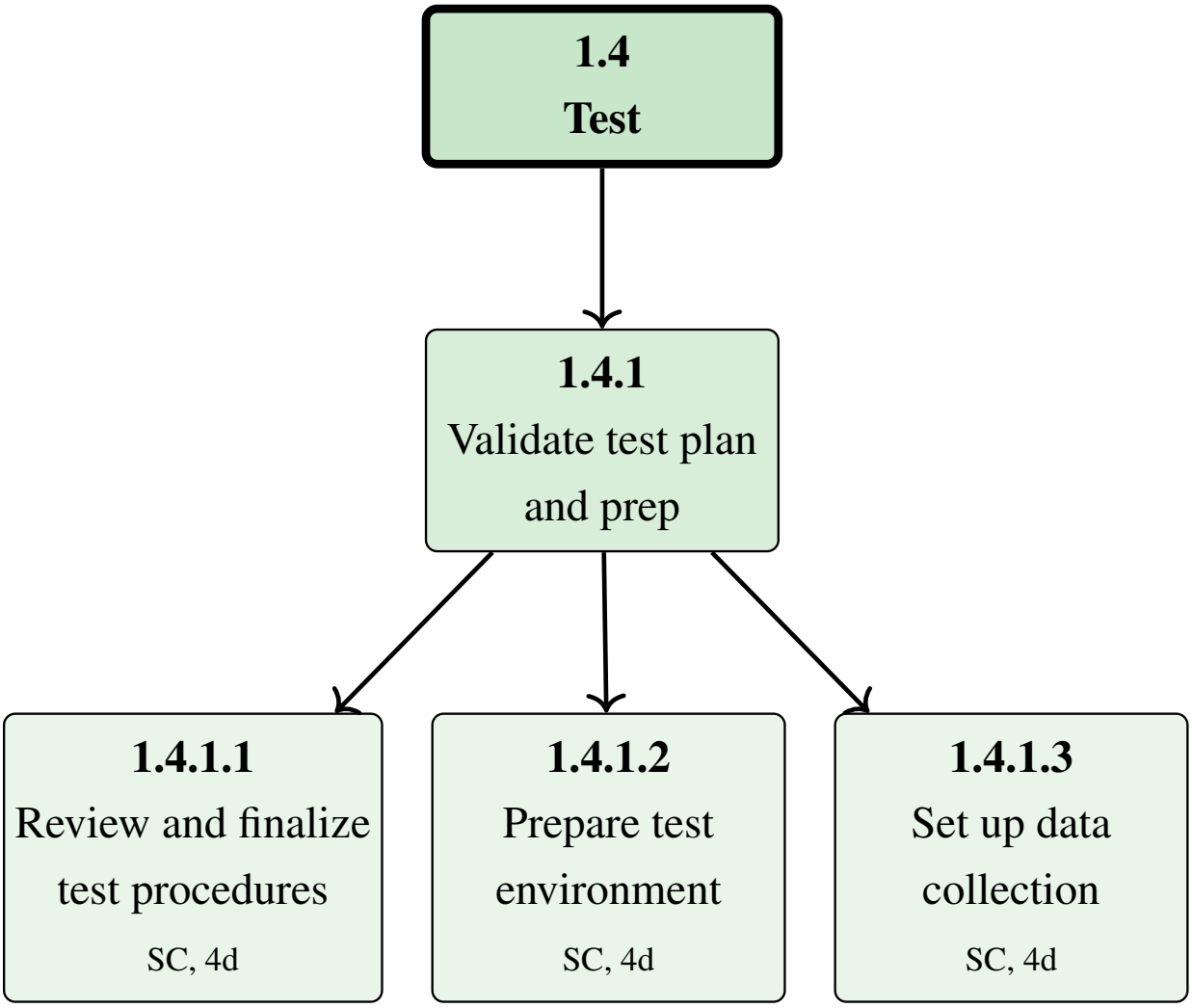


Figure 12: Test Phase Work Breakdown Structure (Part 1: Test Planning)

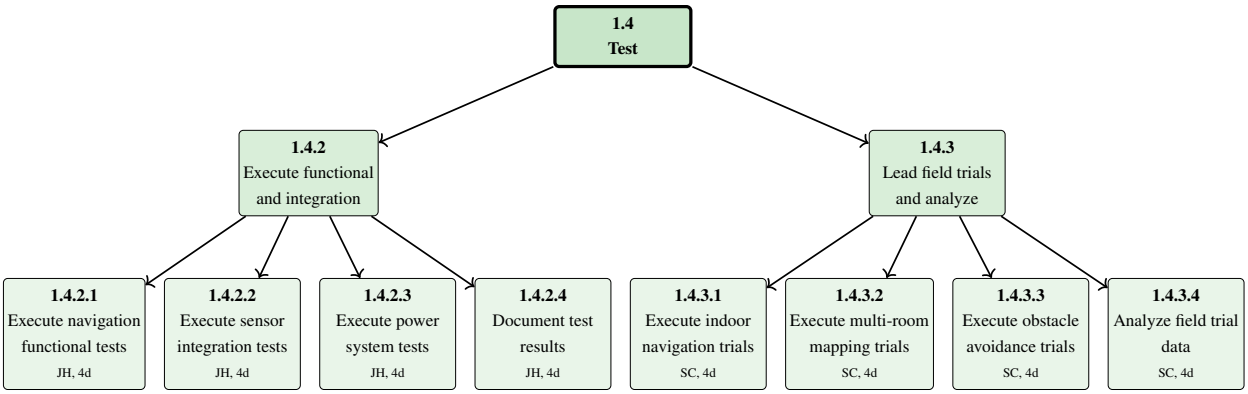


Figure 13: Test Phase Work Breakdown Structure (Part 2: Test Execution)

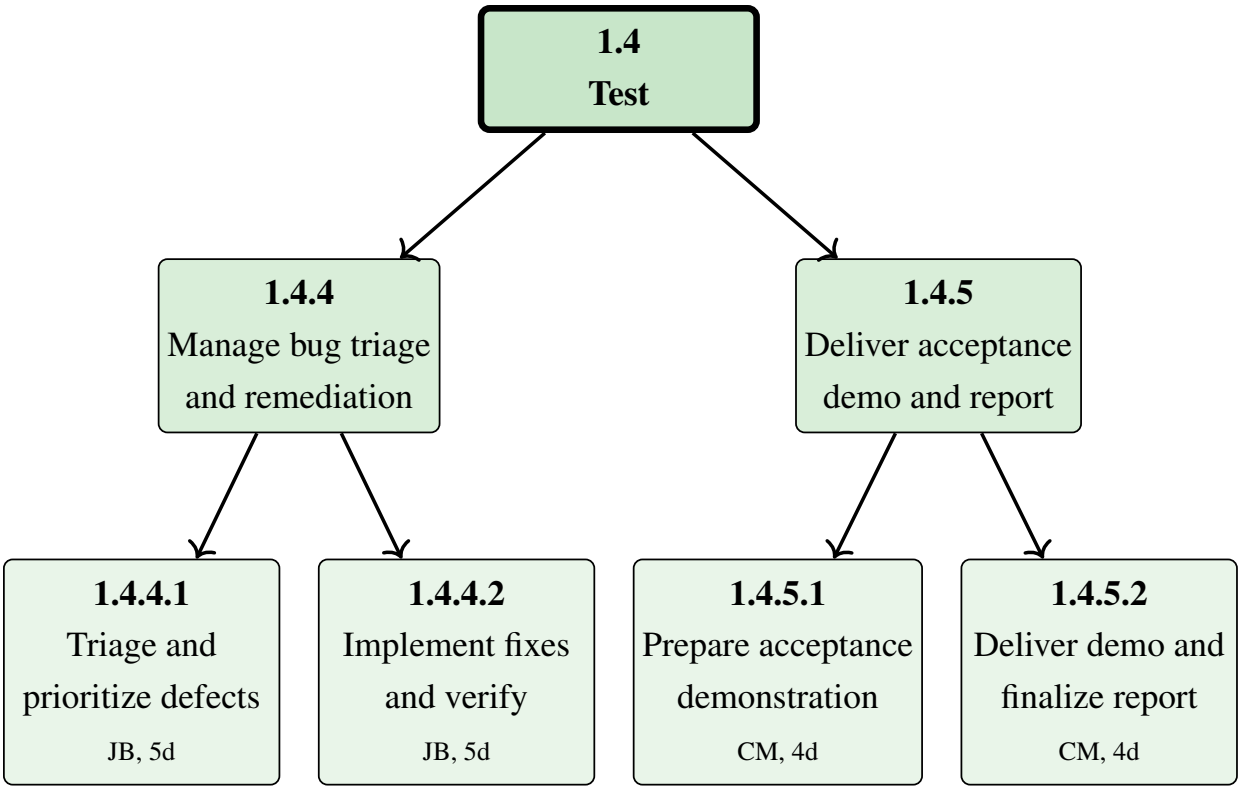


Figure 14: Test Phase Work Breakdown Structure (Part 3: Remediation & Acceptance)

WBS ID	Task Name	Days	Owner
1.4.1	**Validate test plan and prep proving ground**	–	–
1.4.1.1	Review and finalize test procedures	4	SC
1.4.1.2	Prepare test environment and equipment	4	SC
1.4.1.3	Set up data collection and monitoring systems	4	SC
1.4.2	**Execute functional and integration test cycles**	–	–
1.4.2.1	Execute navigation functional tests	4	JH
1.4.2.2	Execute sensor integration tests	4	JH
1.4.2.3	Execute power system tests	4	JH
1.4.2.4	Document test results and defects	4	JH
1.4.3	**Lead field trials and analyze performance data**	–	–
1.4.3.1	Execute indoor navigation trials	4	SC
1.4.3.2	Execute multi-room mapping trials	4	SC
1.4.3.3	Execute obstacle avoidance trials	4	SC
1.4.3.4	Analyze field trial data and document findings	4	SC
1.4.4	**Manage bug triage and remediation sprints**	–	–
1.4.4.1	Triage and prioritize defects	5	JB
1.4.4.2	Implement fixes and verify corrections	5	JB
1.4.5	**Deliver acceptance demo and test report**	–	–
1.4.5.1	Prepare acceptance demonstration	4	CM
1.4.5.2	Deliver demo and finalize test report	4	CM

Table 10: Test Phase Work Breakdown

See Appendix 17.2.4 for the complete Test phase WBS diagram.
See Table 36 for table including co-owners of tasks.

6.9 Close Phase Work Breakdown

Total Elements: 11
Work Packages: 7

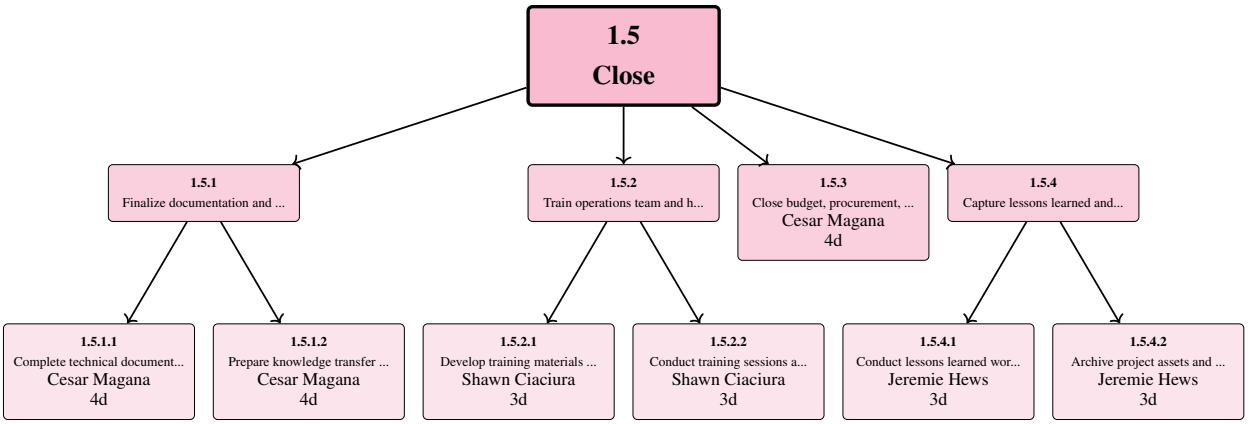


Figure 15: Close Phase Work Breakdown Structure

WBS ID	Task Name	Days	Owner
1.5.1	**Finalize documentation and knowledge transfer**	–	–
1.5.1.1	Complete technical documentation	4	CM
1.5.1.2	Prepare knowledge transfer materials	4	CM
1.5.2	**Train operations team and hand off system**	–	–
1.5.2.1	Develop training materials and procedures	3	SC
1.5.2.2	Conduct training sessions and hand off	3	SC
1.5.3	Close budget, procurement, and inventory actions	4	CM
1.5.4	**Capture lessons learned and archive project assets**	–	–
1.5.4.1	Conduct lessons learned workshop	3	JH
1.5.4.2	Archive project assets and documentation	3	JH

Table 11: Close Phase Work Breakdown

See Appendix 17.2.5 for the complete Close phase WBS diagram.
See Table 36 for table including co-owners of tasks.
See Table 36 for table including co-owners of tasks.

6.10 Activity Naming Convention

All WBS activities follow the noun-verb format as required:

- **Correct**: “Research algorithms”, “Design chassis”, “Test system”
- **Incorrect**: “Algorithm research”, “Chassis design”, “Testing”

6.11 Duration Guidelines

Per FTP requirements, all leaf-level activities are limited to a maximum of 5 days. Activities originally estimated at longer durations have been decomposed into smaller tasks.

7.0 Network Logic Diagram

Section Author(s): CM, JH

7.1 NLD Overview

The Network Logic Diagram illustrates the chronological sequence and logical dependencies of all project activities. It links every task from the WBS to its necessary predecessors and successors to visualize the workflow. This structure allows the team to identify the Critical Path which is the longest chain of dependent tasks that determines the minimum project duration. Isolating these critical tasks helps the team manage schedule risks and monitor float within non-critical activities. The complete diagram includes 84 activities and 89 dependencies organized by phase.

7.2 NLD Structure

The diagrams use the Activity-on-Node (AON) format. Each activity node displays information as follows:

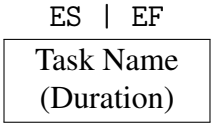


Figure 16: Activity-on-Node (AON) Structure

Node Components:

- **ES (Early Start)**: Earliest day the task can begin, based on predecessor completion
- **EF (Early Finish)**: Earliest day the task can complete (ES + Duration)
- **Task Name**: Descriptive activity name
- **Duration**: Number of working days for the task

Arrow Styles:

- **Red arrows**: Critical path dependencies; any delay in these connections will extend the project duration
- **Gray arrows**: Non-critical dependencies with available float/slack time

Phase Color Coding: Nodes are colored by project phase for visual clarity: Research (blue), Design (yellow), Build (orange), Test (green), and Close (pink).

All arrows indicate Finish-to-Start (FS) dependencies, where the successor cannot start until the predecessor completes.

7.3 NLD Diagram

The network diagrams below show task dependencies organized by phase, with critical path indicated by red arrows.

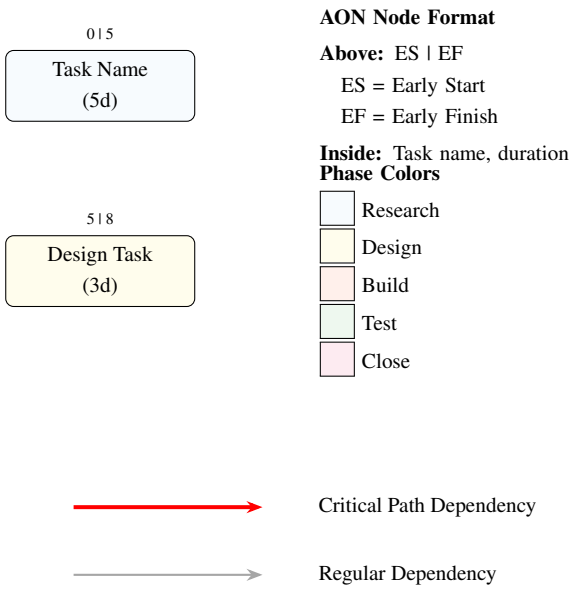


Figure 17: NLD Node Format Legend (Activity-on-Node)

Reading the Diagrams:

- Each node represents a project activity/task
- **ES** (Early Start): Earliest day the task can begin
- **EF** (Early Finish): Earliest day the task can complete
- **Red arrows**: Critical path, any delay extends project duration
- **Gray arrows**: Non-critical dependencies with float/slack time

7.4 Network Logic Diagram Overview

The NLD shows task dependencies and the critical path through the project. Tasks are organized by phase, with arrows indicating predecessor-successor relationships.

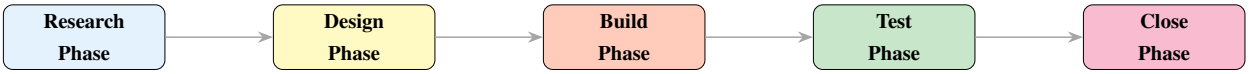


Figure 18: Project Phase Flow

Table 12: Phase Statistics

Phase	Tasks	Duration	Critical
Research	13	56d	9
Design	24	103d	9
Build	20	94d	16
Test	15	62d	11
Close	7	24d	3
Total	**79**	**339d**	**48**

7.5 Critical Path Analysis

Critical Path Length: 48 tasks

Project Duration: 214 working days

The critical path determines the minimum project duration. Any delay in these tasks will delay the project completion. The diagram below shows critical tasks in a grid layout with ES|EF (Early Start | Early Finish) values displayed above each of the nodes.

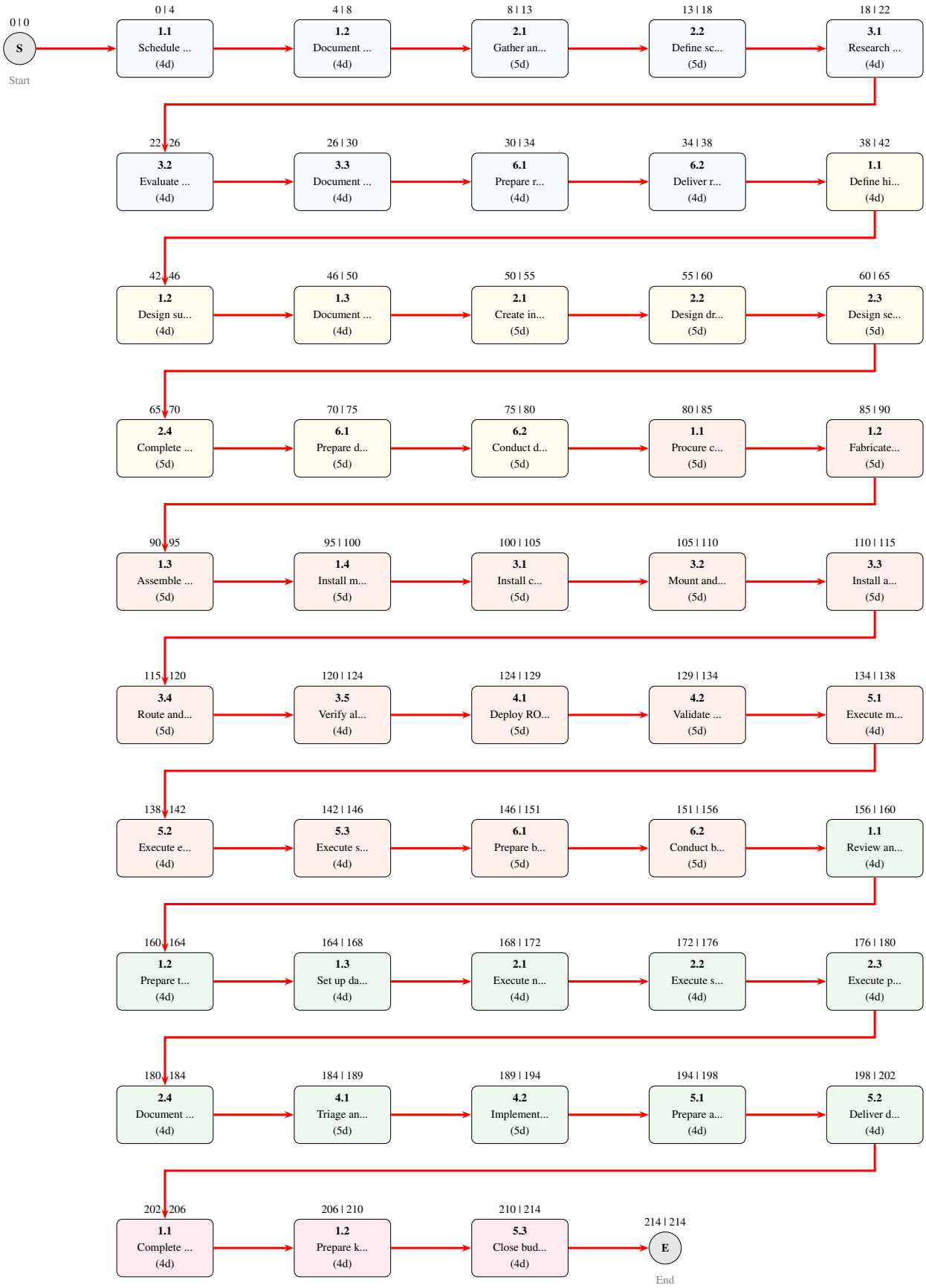


Figure 19: STAR Robot Project Critical Path (Grid Layout)

Note: S = Start milestone, E = End milestone. ES\|EF values shown above each node. Node colors indicate phase (blue=Research, yellow=Design, orange=Build, green=Test, pink=Close). See Appendix C.1 for complete CPM calculations.

For a tabular representation of the critical path with complete timing calculations (ES, EF, LS, LF, and Float values), see the CPM Calculation Table in Appendix C.1.

Table 13: Critical Path Summary by Phase

Phase	Tasks	Days	Cumulative
Research	9	38	38d
Design	9	42	80d
Build	16	76	156d
Test	11	46	202d
Close	3	12	214d
Total	**48**	**214**	**214d**

7.6 Phase-Specific Network Diagrams

The following diagrams show detailed task dependencies within each project phase. Red arrows indicate critical path dependencies; gray arrows indicate non-critical dependencies with available float time. ES\|EF values are displayed above each node.

7.7 Research Phase Network Diagram

The Research phase contains **13 tasks** totaling **56 working days**. Of these, 9 tasks (38 days) are on the critical path, and 4 tasks (18 days) are supporting activities with available float time.

7.7.1 Critical Path Tasks

The following 9 tasks are on the critical path. Any delay in these tasks will directly extend the project completion date. These tasks span 38 working days.

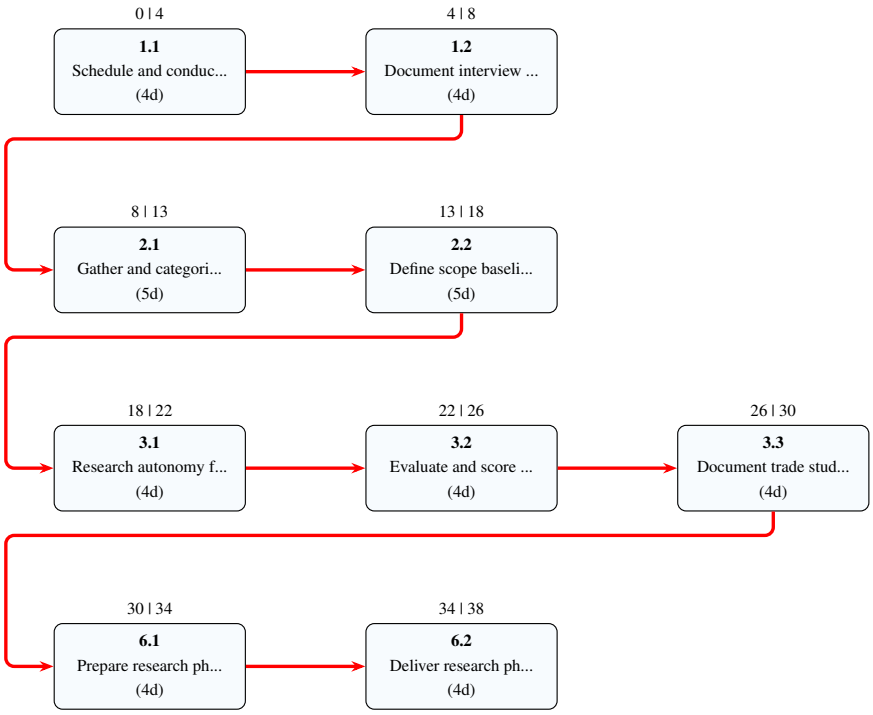


Figure 20: Research Phase Critical Path Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.1.1.1	Schedule and conduct stakeholder interviews	4	0	4	CM
1.1.1.2	Document interview findings and requirements	4	4	8	CM
1.1.2.1	Gather and categorize all requirements	5	8	13	CM
1.1.2.2	Define scope baseline and acceptance criteria	5	13	18	CM
1.1.3.1	Research autonomy framework options	4	18	22	JB
1.1.3.2	Evaluate and score framework candidates	4	22	26	JB
1.1.3.3	Document trade study findings and recommendation	4	26	30	JB
1.1.6.1	Prepare research phase presentation materials	4	30	34	CM
1.1.6.2	Deliver research phase gate review	4	34	38	CM

Table 14: Research Phase Critical Path Tasks

7.7.2 Supporting Tasks

The following 4 supporting tasks have available float time. These tasks can be delayed without impacting the project end date, as long as they complete within their late finish dates. These tasks span 18 working days.

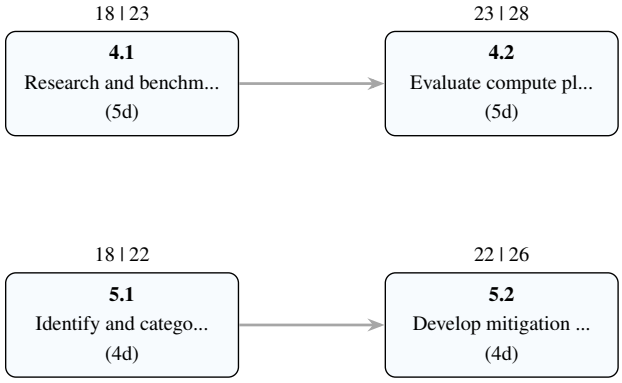


Figure 21: Research Phase Supporting Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.1.4.1	Research and benchmark sensor options	5	18	23	BS
1.1.4.2	Evaluate compute platforms and make selection	5	23	28	BS
1.1.5.1	Identify and categorize project risks	4	18	22	SC
1.1.5.2	Develop mitigation strategies and contingency plans	4	22	26	SC

Table 15: Research Phase Supporting Tasks

See Appendix C for the complete Research phase network logic diagram.

7.8 Design Phase Network Diagram

The Design phase contains **24 tasks** totaling **103 working days**. Of these, 9 tasks (42 days) are on the critical path, and 15 tasks (61 days) are supporting activities with available float time.

7.8.1 Critical Path Tasks

The following 9 tasks are on the critical path. Any delay in these tasks will directly extend the project completion date. These tasks span 42 working days.

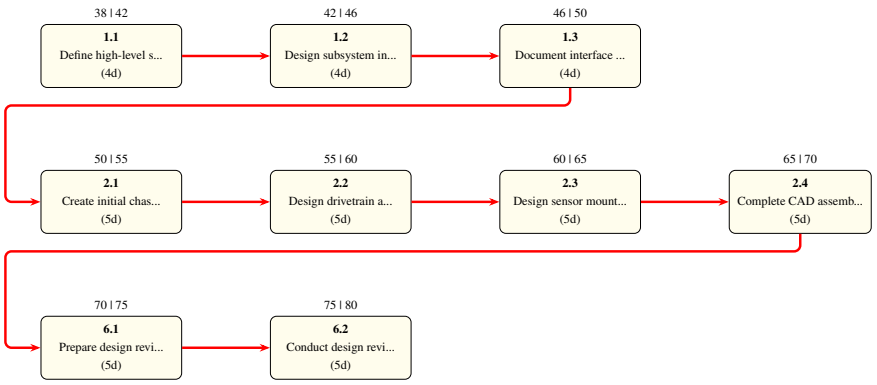


Figure 22: Design Phase Critical Path Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.2.1.1	Define high-level system architecture	4	38	42	CM
1.2.1.2	Design subsystem interfaces and protocols	4	42	46	CM
1.2.1.3	Document interface control and data flow	4	46	50	CM
1.2.2.1	Create initial chassis dimensions and constraints	5	50	55	MN
1.2.2.2	Design drivetrain and motor mounting	5	55	60	MN
1.2.2.3	Design sensor mounting and enclosures	5	60	65	MN
1.2.2.4	Complete CAD assembly and generate drawings	5	65	70	MN
1.2.6.1	Prepare design review materials and agenda	5	70	75	CM
1.2.6.2	Conduct design review and document decisions	5	75	80	CM

Table 16: Design Phase Critical Path Tasks

7.8.2 Supporting Tasks

The following 15 supporting tasks have available float time. These tasks can be delayed without impacting the project end date, as long as they complete within their late finish dates. These tasks span 61 working days.

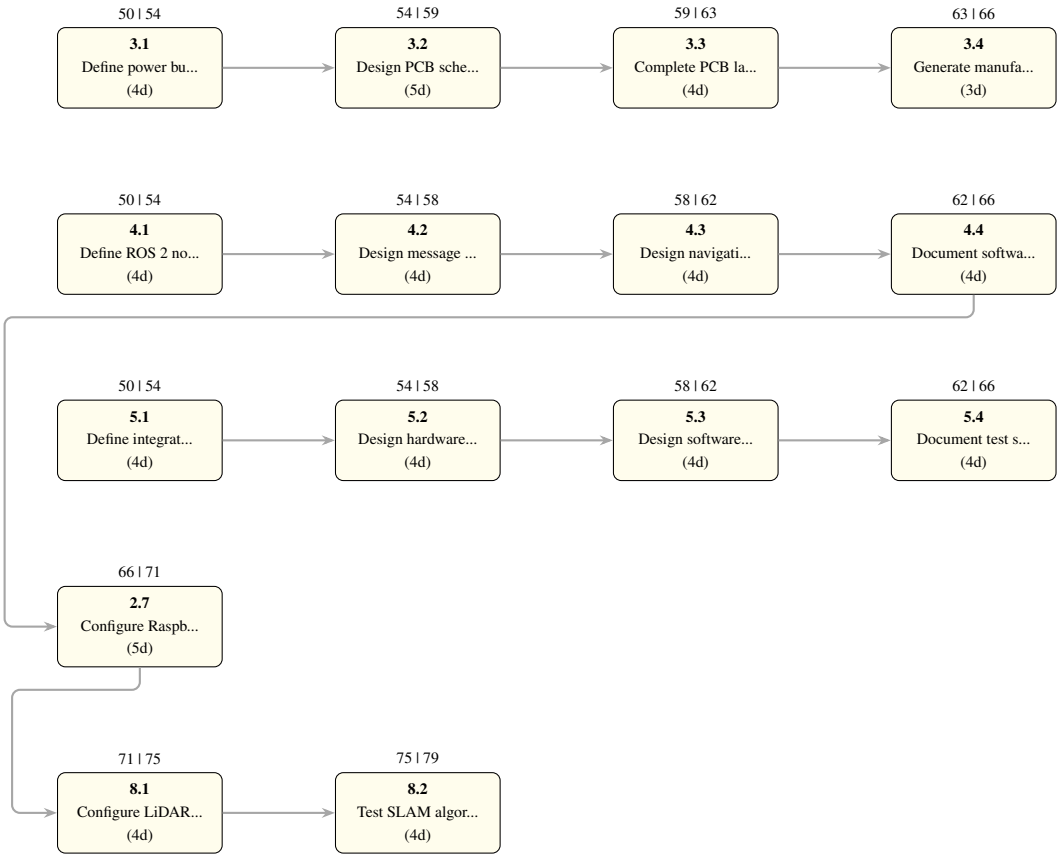


Figure 23: Design Phase Supporting Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.2.3.1	Define power budget and distribution architecture	4	50	54	BS
1.2.3.2	Design PCB schematic and component selection	5	54	59	BS
1.2.3.3	Complete PCB layout and routing	4	59	63	BS
1.2.3.4	Generate manufacturing files and submit order	3	63	66	BS
1.2.4.1	Define ROS 2 node architecture	4	50	54	JB
1.2.4.2	Design message types and topic structure	4	54	58	JB
1.2.4.3	Design navigation and SLAM pipeline	4	58	62	JB
1.2.4.4	Document software architecture and APIs	4	62	66	JB
1.2.5.1	Define integration sequence and dependencies	4	50	54	JH
1.2.5.2	Design hardware integration test procedures	4	54	58	JH
1.2.5.3	Design software integration test procedures	4	58	62	JH
1.2.5.4	Document test strategy and success criteria	4	62	66	JH
1.2.7	Configure Raspberry Pi 5 ROS 2 environment	5	66	71	JB
1.2.8.1	Configure LiDAR driver and ROS interface	4	71	75	JB
1.2.8.2	Test SLAM algorithm in simulation	4	75	79	JB

Table 17: Design Phase Supporting Tasks

See Appendix C for the complete Design phase network logic diagram.

7.9 Build Phase Network Diagram

The Build phase contains **20 tasks** totaling **94 working days**. Of these, 16 tasks (76 days) are on the critical path, and 4 tasks (18 days) are supporting activities with available float time.

7.9.1 Critical Path Tasks

The following 16 tasks are on the critical path. Any delay in these tasks will directly extend the project completion date. These tasks span 76 working days.

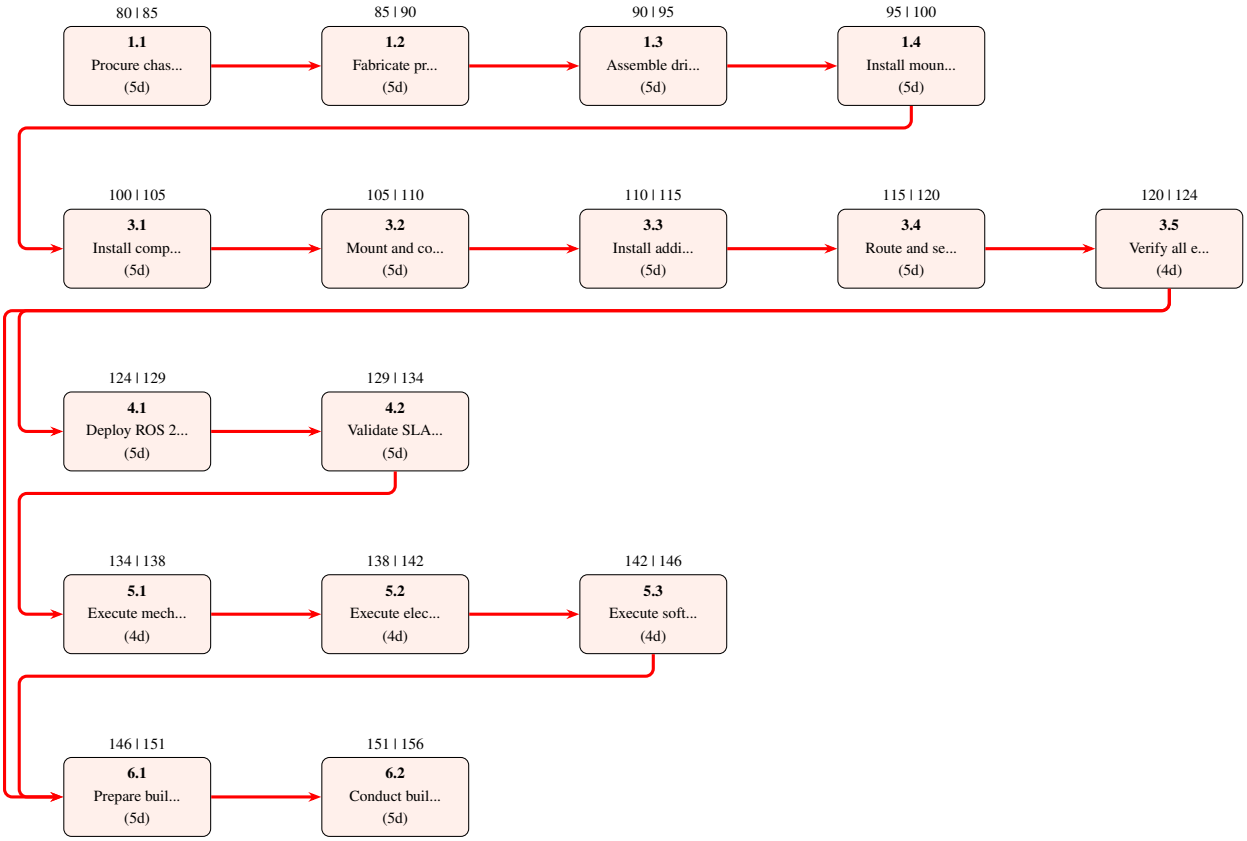


Figure 24: Build Phase Critical Path Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.3.1.1	Procure chassis materials and components	5	80	85	MN
1.3.1.2	Fabricate primary chassis structure	5	85	90	MN
1.3.1.3	Assemble drivetrain and suspension	5	90	95	MN
1.3.1.4	Install mounting brackets and verify alignment	5	95	100	MN
1.3.3.1	Install compute platform and configure OS	5	100	105	JH
1.3.3.2	Mount and connect LiDAR sensor	5	105	110	JH
1.3.3.3	Install additional sensors and peripherals	5	110	115	JH
1.3.3.4	Route and secure all cabling	5	115	120	JH
1.3.3.5	Verify all electrical connections and power-on test	4	120	124	JH
1.3.4.1	Deploy ROS 2 software stack to robot	5	124	129	JB
1.3.4.2	Validate SLAM navigation on physical robot	5	129	134	JB
1.3.5.1	Execute mechanical integration tests	4	134	138	JH
1.3.5.2	Execute electrical integration tests	4	138	142	JH
1.3.5.3	Execute software integration tests	4	142	146	JH
1.3.6.1	Prepare build phase review materials	5	146	151	CM
1.3.6.2	Conduct build phase gate review	5	151	156	CM

Table 18: Build Phase Critical Path Tasks

7.9.2 Supporting Tasks

The following 4 supporting tasks have available float time. These tasks can be delayed without impacting the project end date, as long as they complete within their late finish dates. These tasks

span 18 working days.

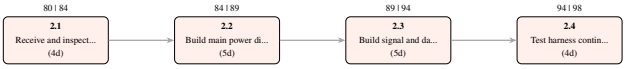


Figure 25: Build Phase Supporting Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.3.2.1	Receive and inspect PCB assemblies	4	80	84	BS
1.3.2.2	Build main power distribution harness	5	84	89	BS
1.3.2.3	Build signal and data harnesses	5	89	94	BS
1.3.2.4	Test harness continuity and isolation	4	94	98	BS

Table 19: Build Phase Supporting Tasks

See Appendix C for the complete Build phase network logic diagram.

7.10 Test Phase Network Diagram

The Test phase contains **15 tasks** totaling **62 working days**. Of these, 11 tasks (46 days) are on the critical path, and 4 tasks (16 days) are supporting activities with available float time.

7.10.1 Critical Path Tasks

The following 11 tasks are on the critical path. Any delay in these tasks will directly extend the project completion date. These tasks span 46 working days.

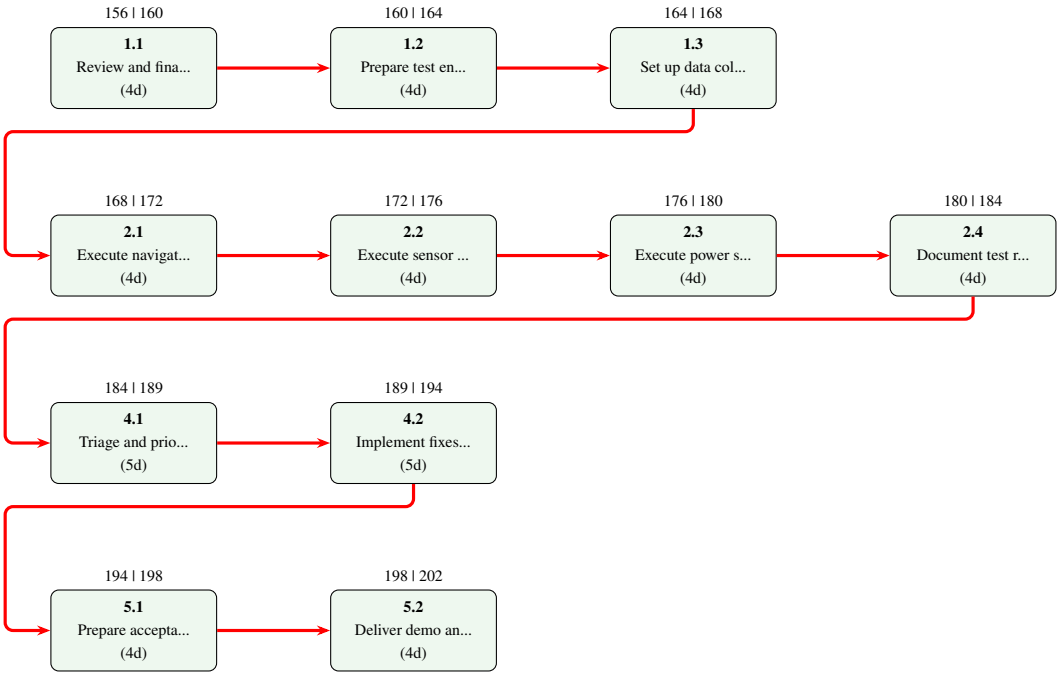


Figure 26: Test Phase Critical Path Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.4.1.1	Review and finalize test procedures	4	156	160	SC
1.4.1.2	Prepare test environment and equipment	4	160	164	SC
1.4.1.3	Set up data collection and monitoring systems	4	164	168	SC
1.4.2.1	Execute navigation functional tests	4	168	172	JH
1.4.2.2	Execute sensor integration tests	4	172	176	JH
1.4.2.3	Execute power system tests	4	176	180	JH
1.4.2.4	Document test results and defects	4	180	184	JH
1.4.4.1	Triage and prioritize defects	5	184	189	JB
1.4.4.2	Implement fixes and verify corrections	5	189	194	JB
1.4.5.1	Prepare acceptance demonstration	4	194	198	CM
1.4.5.2	Deliver demo and finalize test report	4	198	202	CM

Table 20: Test Phase Critical Path Tasks

7.10.2 Supporting Tasks

The following 4 supporting tasks have available float time. These tasks can be delayed without impacting the project end date, as long as they complete within their late finish dates. These tasks span 16 working days.

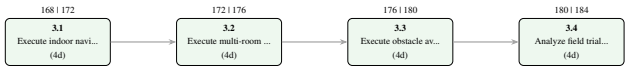


Figure 27: Test Phase Supporting Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.4.3.1	Execute indoor navigation trials	4	168	172	SC
1.4.3.2	Execute multi-room mapping trials	4	172	176	SC
1.4.3.3	Execute obstacle avoidance trials	4	176	180	SC
1.4.3.4	Analyze field trial data and document findings	4	180	184	SC

Table 21: Test Phase Supporting Tasks

See Appendix C for the complete Test phase network logic diagram.

7.11 Close Phase Network Diagram

The Close phase contains **7 tasks** totaling **24 working days**. Of these, 3 tasks (12 days) are on the critical path, and 4 tasks (12 days) are supporting activities with available float time.

7.11.1 Critical Path Tasks

The following 3 tasks are on the critical path. Any delay in these tasks will directly extend the project completion date. These tasks span 12 working days.

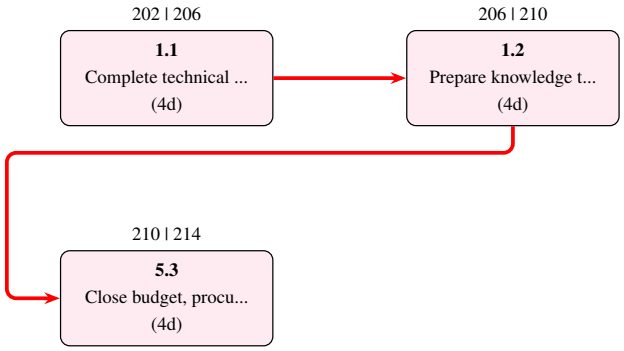


Figure 28: Close Phase Critical Path Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.5.1.1	Complete technical documentation	4	202	206	CM
1.5.1.2	Prepare knowledge transfer materials	4	206	210	CM
1.5.3	Close budget, procurement, and inventory actions	4	210	214	CM

Table 22: Close Phase Critical Path Tasks

7.11.2 Supporting Tasks

The following 4 supporting tasks have available float time. These tasks can be delayed without impacting the project end date, as long as they complete within their late finish dates. These tasks span 12 working days.

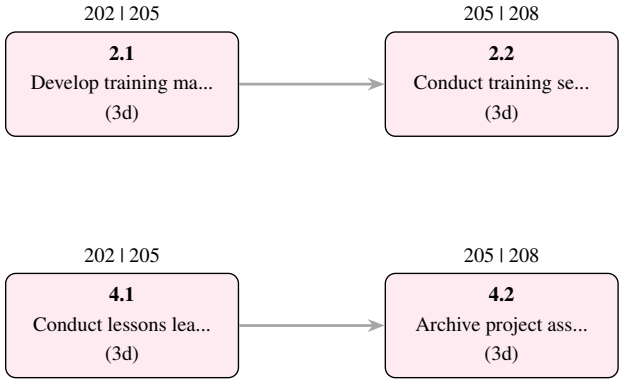


Figure 29: Close Phase Supporting Tasks

WBS ID	Task Name	Days	ES	EF	Owner
1.5.2.1	Develop training materials and procedures	3	202	205	SC
1.5.2.2	Conduct training sessions and hand off	3	205	208	SC
1.5.4.1	Conduct lessons learned workshop	3	202	205	JH
1.5.4.2	Archive project assets and documentation	3	205	208	JH

Table 23: Close Phase Supporting Tasks

See Appendix C for the complete Close phase network logic diagram.

7.12 Critical Path Method (CPM)

The critical path was calculated using the Critical Path Method, which involves a forward pass to calculate Early Start (ES) and Early Finish (EF) times, followed by a backward pass to calculate Late Start (LS) and Late Finish (LF) times. Total Float for each task is calculated as LS - ES.

Tasks with zero float form the critical path and any delay in these tasks will delay the project completion date. The complete CPM calculation table showing ES, EF, LS, LF, and Float for all 84 activities is provided in Appendix C.1.

7.13 NLD Validation

The NLD has been validated against the WBS to ensure:

- All 84 WBS leaf activities appear in the NLD
- Every activity (except START/END) has at least one input and output
- No circular dependencies exist
- The critical path is clearly identifiable
- Activity durations match WBS estimates

7.14 Dependency Chains

The following dependency chains represent critical project flows:

1. **Hardware Development Chain**: Requirements → Component Selection → Mechanical Design → Chassis Fabrication → System Integration
2. **Software Development Chain**: Requirements → Software Architecture → SLAM Development → Navigation Development → System Integration
3. **Validation Chain**: System Integration → Unit Testing → Integration Testing → System Validation → Acceptance Testing

These chains inform resource allocation and risk management priorities.

8.0 Gantt Chart

Section Author(s): CM, JB

8.1 Schedule Overview

The Gantt chart provides a time-based visualization of the project schedule, showing all 84 activities (including PM overhead) from the WBS with their durations, dependencies, and resource assignments. The project spans from August 26, 2025 through May 15, 2026, a total of 263 calendar days (approximately 9 months).

8.2 Project Timeline

Table 24: Project Schedule Summary

Phase	Duration	Dates
Research	67 days	Aug 26 – Oct 31, 2025
Design	120 days	Nov 1, 2025 – Feb 28, 2026
Build	106 days	Jan 15 – Apr 30, 2026
Test	57 days	Mar 15 – May 10, 2026
Close	15 days	May 1 – May 15, 2026
Total Project	**263 days**	**Aug 26, 2025 – May 15, 2026**

Note: Phases overlap due to parallel activities and dependencies between phases.

8.3 Calendar Modifications

The project calendar has been modified to reflect the team's actual work schedule:

8.3.1 Academic Calendar Considerations

- **Fall 2025 Semester**: August 26 – December 12, 2025
- **Winter Break**: December 13, 2025 – January 12, 2026 (reduced availability)
- **Spring 2026 Semester**: January 13 – May 15, 2026
- **Spring Break**: March 8-15, 2026 (no project work)

8.3.2 Weekly Work Schedule

The team operates on the following weekly schedule:

- **Work Days**: Monday through Friday
- **Weekend**: Saturday and Sunday (non-working)
- **Team Meetings**: Tuesday and Thursday, 5:00 PM – 7:00 PM
- **Individual Work**: Distributed throughout week
- **Average Hours**: 8-10 hours per person per week during semester

8.3.3 Impact on Schedule

The modified calendar affects the schedule in the following ways:

- Winter break (December 13, 2025 – January 12, 2026) reduces team availability as members travel and formal meetings are suspended. Light asynchronous work such as documentation and planning continues during this period, with regular progress resuming January 13.
- Spring break represents 1 week of reduced project activity
- Final exams periods (December, May) have reduced team availability
- The Build phase is scheduled during Spring semester when lab access is maximized

8.4 Dependencies in Gantt

All 89 dependencies identified in the NLD are represented in the Gantt chart:

- Dependencies shown as connector arrows between task bars
- Finish-to-Start (FS) relationships are used exclusively
- Critical path activities are highlighted in red
- Slack time is visible as the difference between early and late dates

8.5 Phase Gantt Charts

The following subsections present the detailed Gantt charts for each project phase. Tasks on the critical path are highlighted in red. Dependencies within each phase are shown as connector arrows.

8.6 Research Phase Schedule

Phase Duration: 67 days (Aug 26, 2025 – Oct 31, 2025)
Total Tasks: 13

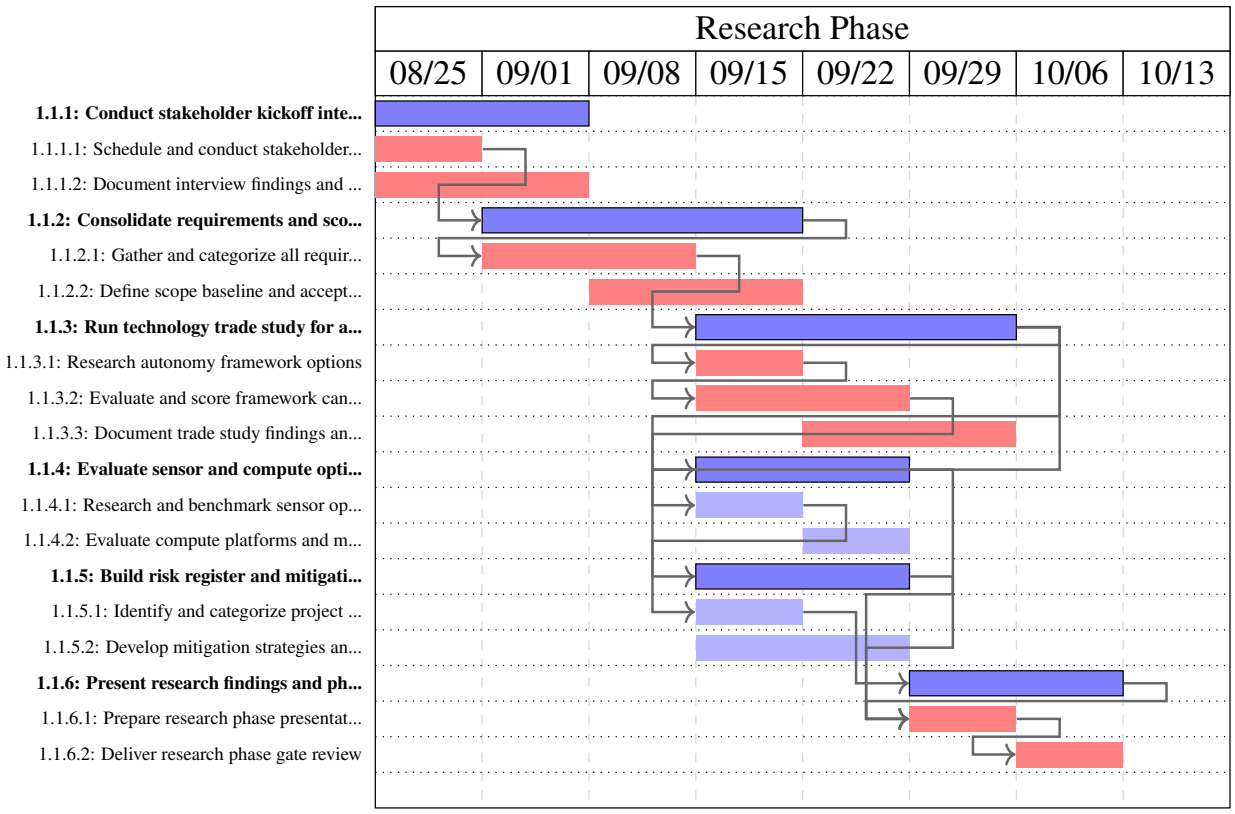


Figure 30: Research Phase Gantt Chart

Note: Tasks highlighted in red are on the critical path (9 tasks in this phase).

8.7 Design Phase Schedule

Phase Duration: 120 days (Nov 1, 2025 – Feb 28, 2026)

Total Tasks: 24

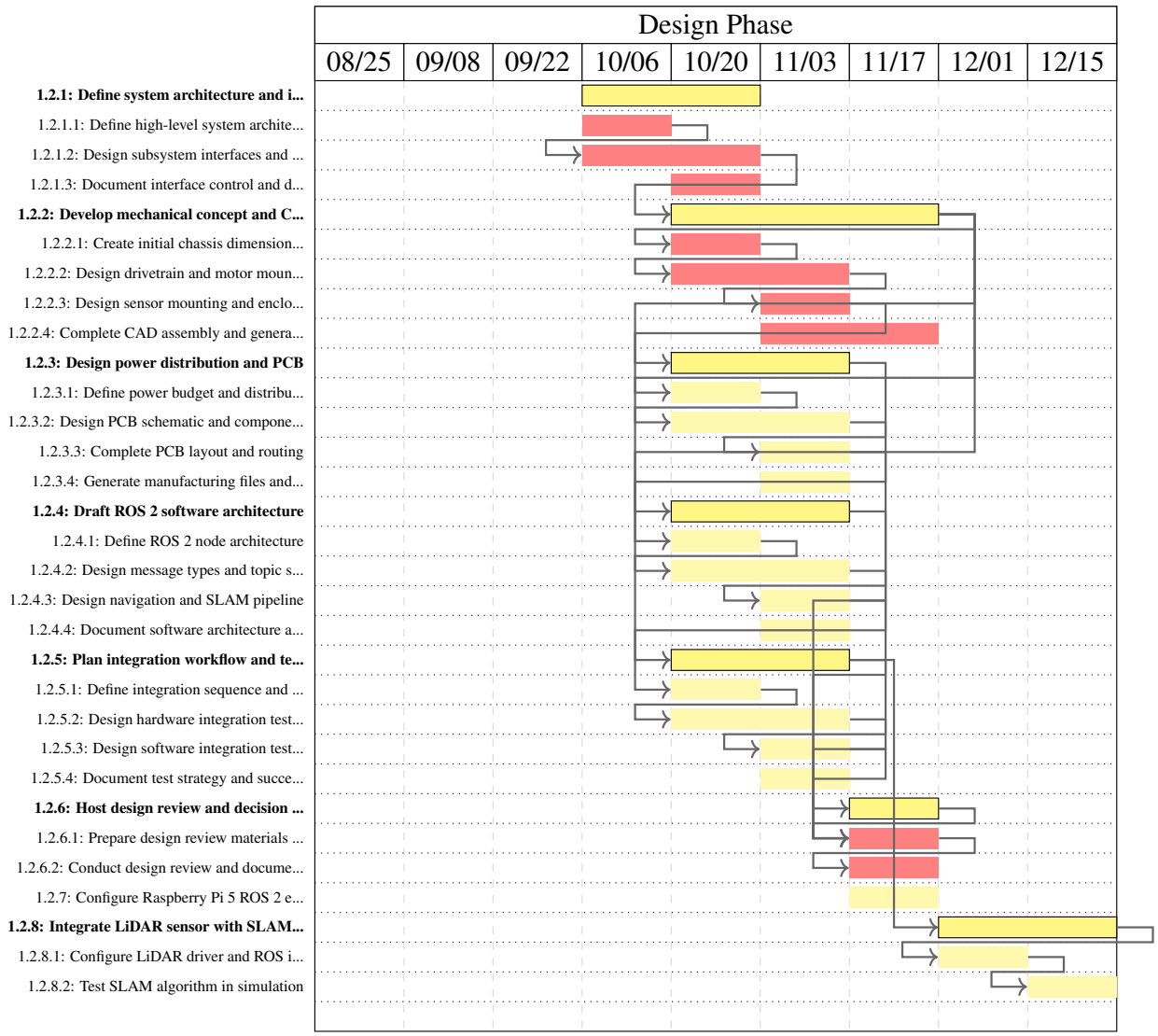


Figure 31: Design Phase Gantt Chart

Note: Tasks highlighted in red are on the critical path (9 tasks in this phase).

8.8 Build Phase Schedule

Phase Duration: 106 days (Jan 15, 2026 – Apr 30, 2026)

Total Tasks: 20

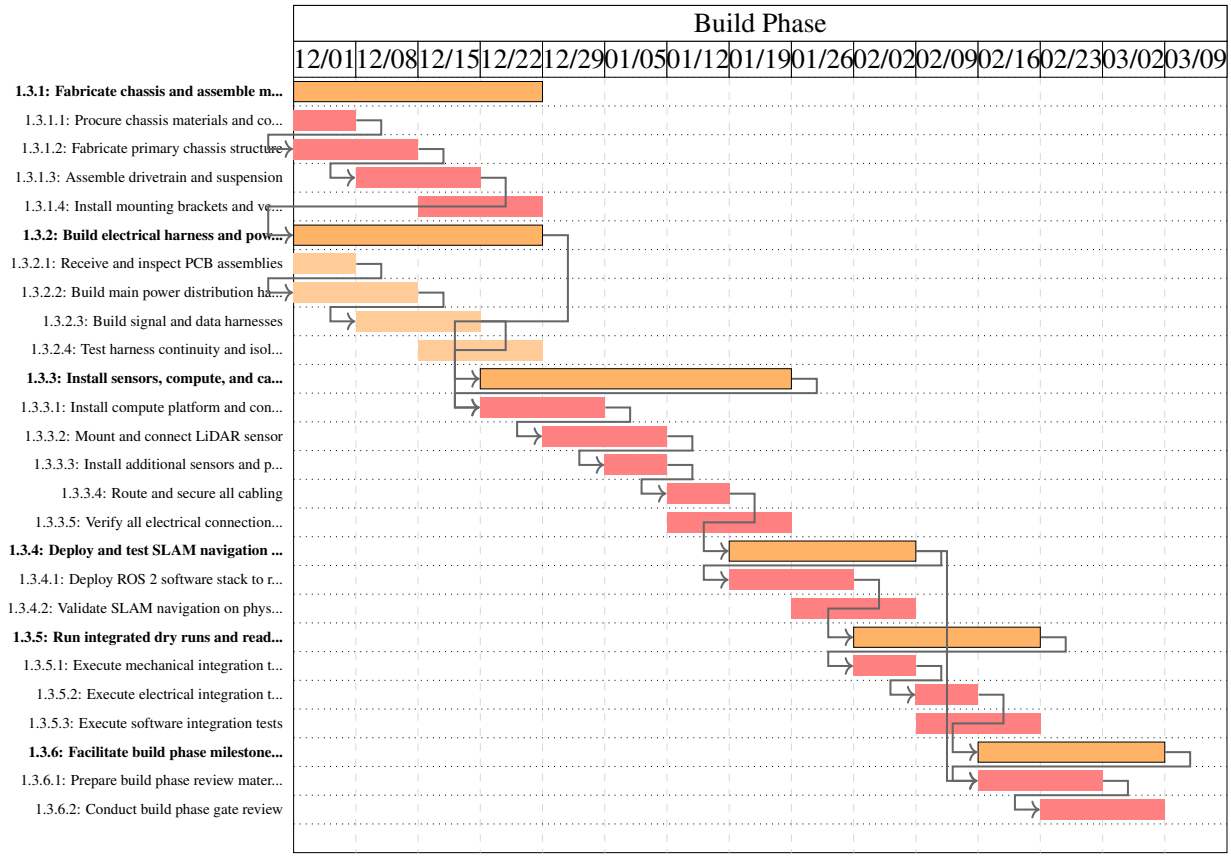


Figure 32: Build Phase Gantt Chart

Note: Tasks highlighted in red are on the critical path (16 tasks in this phase).

8.9 Test Phase Schedule

Phase Duration: 57 days (Mar 15, 2026 – May 10, 2026)

Total Tasks: 15

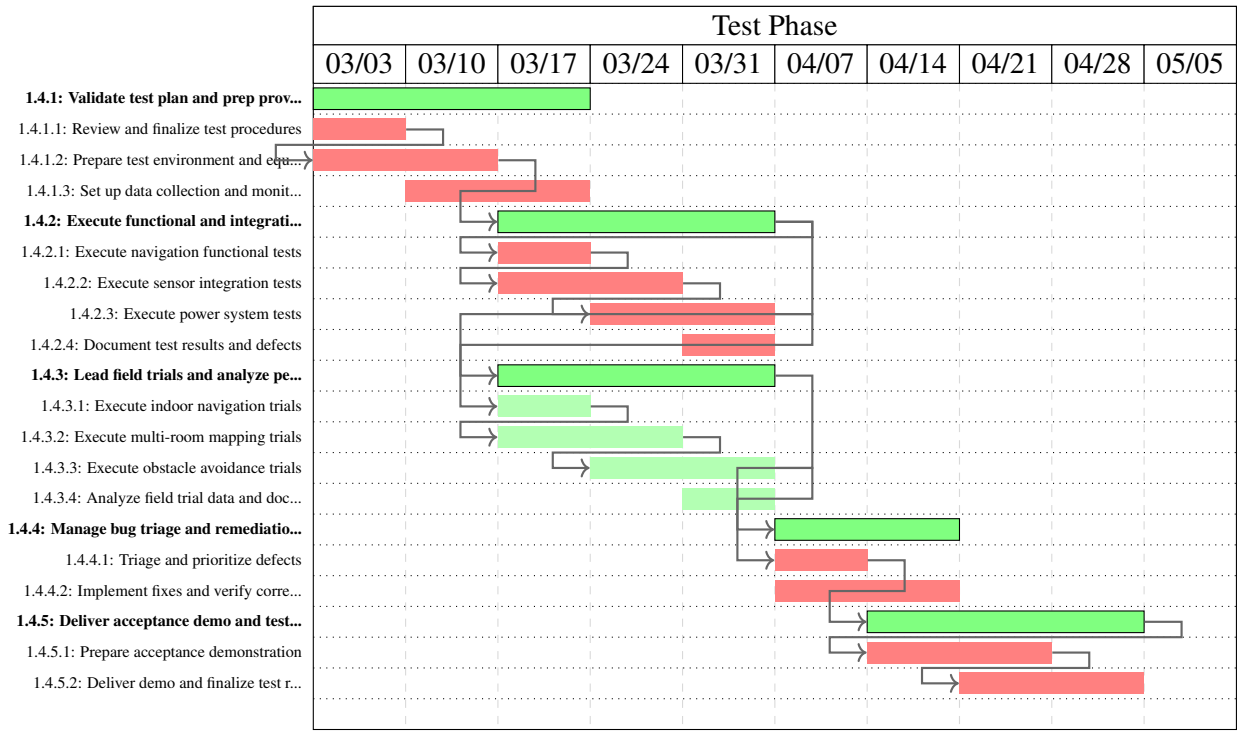


Figure 33: Test Phase Gantt Chart

Note: Tasks highlighted in red are on the critical path (11 tasks in this phase).

8.10 Close Phase Schedule

Phase Duration: 10 days (Apr 21, 2026 – Apr 30, 2026)

Total Tasks: 7

Note: Task 1.5.3 "Close budget, procurement, and inventory" runs throughout the project lifecycle for continuous tracking and is not shown in this focused view.

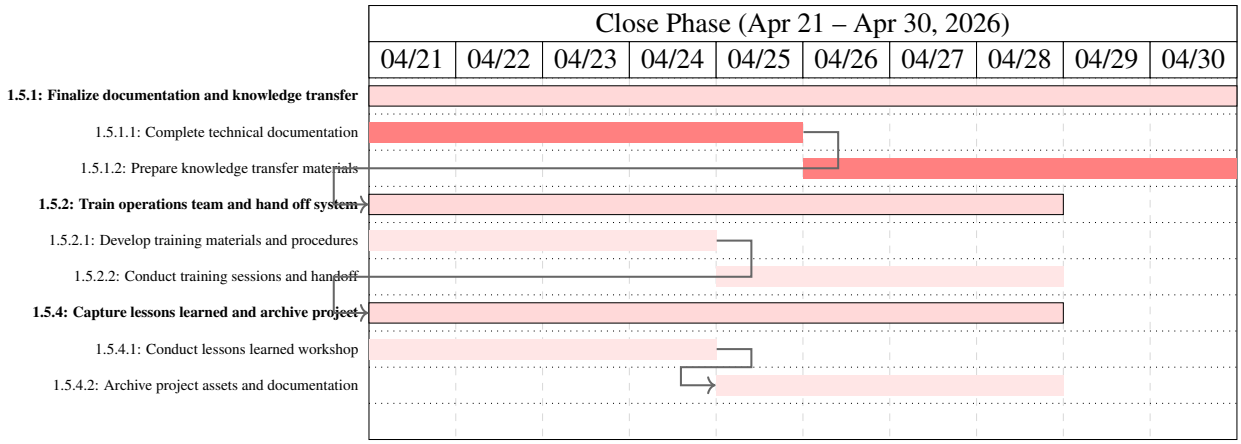


Figure 34: Close Phase Gantt Chart

Note: Tasks highlighted in red are on the critical path (2 tasks in this phase).

8.11 Resource Loading

The Gantt chart includes resource assignments showing which team members are responsible for each task. This enables identification of:

- Resource conflicts (multiple assignments on same dates)
- Underutilization periods
- Critical resource constraints

8.12 Schedule Monitoring

The Gantt chart serves as the primary schedule monitoring tool. Weekly progress reviews compare:

- Planned dates vs. actual completion dates
- Percent complete for in-progress activities
- Impact of delays on downstream activities
- Critical path status and total float consumption

Schedule updates are recorded in the project management tool (GanttProject) and shared with the sponsor at bi-weekly status meetings.

9.0 Milestones

Section Author(s): CM, MN

9.1 Milestone Overview

The milestones outline the project's key development checkpoints and provides a structured method for tracking team progress throughout the schedule.

9.2 Project Milestones

Eight key milestones mark critical checkpoints throughout the project lifecycle. These milestones align with the Gantt chart (Table 25) and represent decision gates for sponsor review and team progress verification.

Table 25: Project Milestones

ID	Date	Milestone	Deliverable
M1	Oct 29, 2025	PDR	PDR presentation and documentation
M2	Oct 31, 2025	Research Phase Complete	Requirements document
M3	Dec 03, 2025	Software Demo	Working software demonstration
M4	Feb 28, 2026	Design Review	Design specifications
M5	Mar 31, 2026	Hardware Integration Complete	Assembled prototype
M6	Apr 15, 2026	Software Integration Complete	Working software
M7	May 1, 2026	System Testing Complete	Test reports
M8	May 8, 2026	Final Demonstration	Working system

9.3 Milestone Acceptance Criteria

Each milestone has specific acceptance criteria that must be met. These criteria follow SMART principles: Specific, Measurable, Achievable, Relevant, and Time-Bound.

9.3.1 M1: PDR (October 29, 2025)

- PDR presentation delivered
- All functional and non-functional requirements documented
- Project scope and timeline approved
- Sponsor feedback incorporated into project plan

9.3.2 M2: Research Phase Complete (October 31, 2025)

- All requirements documented and approved by sponsor
- Technology selection justified with trade-off analysis
- Risk assessment completed with mitigation strategies
- Competitive analysis finalized

9.3.3 M3: Software Demo (December 3, 2025)

- Core software functionality demonstrated to sponsor
- Basic SLAM algorithm validated with test data
- Software architecture documented

9.3.4 M4: Design Review (February 28, 2026)

- All design documents reviewed and approved
- No critical design issues outstanding
- Bill of materials finalized and procurement initiated
- Hardware-software interface specifications complete

9.3.5 M5: Hardware Integration Complete (March 31, 2026)

- All components procured and received
- Physical robot assembly completed
- Initial power-on successful with basic diagnostics
- Sensor integration verified with test readings

9.3.6 M6: Software Integration Complete (April 15, 2026)

- All software modules integrated on physical hardware
- Basic navigation functionality verified
- Communication between all subsystems operational
- Ready for system-level testing

9.3.7 M7: System Testing Complete (May 1, 2026)

- All test cases executed per test plan
- Critical defects resolved with verification
- Performance requirements verified against specifications
- Test reports documented and reviewed

9.3.8 M8: Final Demonstration (May 8, 2026)

- System demonstrates all required functionality
- Autonomous mapping of 10 m × 10 m area completed
- Sponsor accepts deliverable and signs off
- All documentation complete and delivered

9.4 Milestone Timeline

Figure 35 shows the Fall 2025 milestones and Figure 36 shows the Spring 2026 milestones.

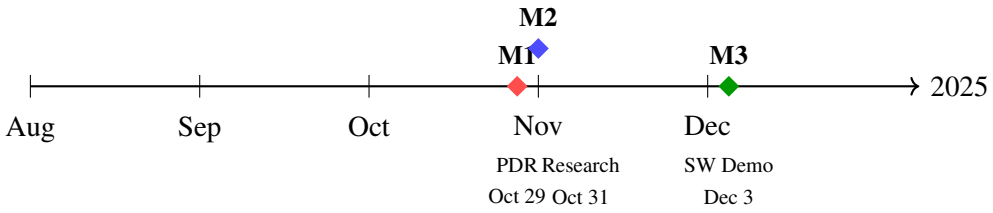


Figure 35: Fall 2025 Milestones

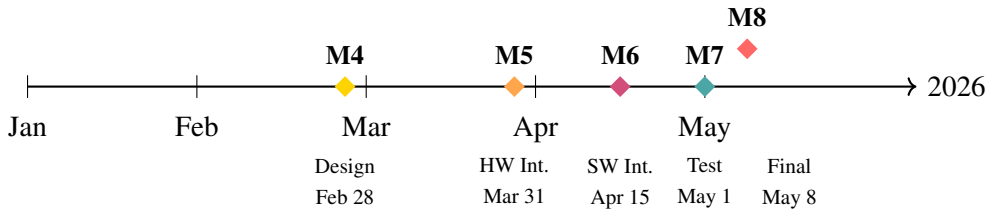


Figure 36: Spring 2026 Milestones

10.0 Critical Path

Section Author(s): CM, JB

10.1 Critical Path Definition

The critical path is the longest sequence of dependent activities that determines the minimum project completion time. Activities on the critical path have zero slack (float), meaning any delay in a critical path activity directly delays the project end date.

For the STAR Robot project, the critical path spans 214 working days and includes 47 activities across all project phases.

10.2 Critical Path Summary

Table 26: Critical Path Statistics

Metric	Value
Total Activities on Critical Path	47
Critical Path Duration	214 days
Total Project Activities	84
Percentage on Critical Path	56%
Total Float (Non-Critical Activities)	Varies (1-45 days)

10.3 Critical Path Visualization

The critical path is highlighted in red on the NLD (see Figure 19 in Section 7.0) and the Gantt chart. Full diagrams are available in the Appendix.

10.4 Critical Path Activities

The following activities comprise the critical path, organized by phase. Row colors match the phase color coding used in the NLD and WBS diagrams.

10.4.1 Research Phase Critical Activities

Table 27: Research Phase Critical Activities

WBS ID	Activity	Duration
1.1.1.1	Gather stakeholder requirements	5 days
1.1.1.2	Document functional requirements	5 days
1.1.2.1	Research SLAM algorithms	4 days
1.1.2.2	Evaluate sensor technologies	4 days
1.1.3.1	Select compute platform	5 days
1.1.3.2	Select sensors	4 days
1.1.6.1	Compile research findings	5 days
1.1.6.2	Create research report	5 days

10.4.2 Design Phase Critical Activities

Table 28: Design Phase Critical Activities

WBS ID	Activity	Duration
1.2.1.1	Define system architecture	5 days
1.2.1.2	Create interface specifications	5 days
1.2.1.3	Document design decisions	5 days
1.2.2.1	Design chassis frame	5 days
1.2.2.2	Design drive system	5 days
1.2.2.3	Design sensor mounts	5 days
1.2.2.4	Create CAD assembly	5 days
1.2.6.1	Prepare design review	5 days
1.2.6.2	Conduct design review	5 days

10.4.3 Build Phase Critical Activities

Table 29: Build Phase Critical Activities

WBS ID	Activity	Duration
1.3.1.1	Fabricate chassis components	5 days
1.3.1.2	Assemble chassis frame	5 days
1.3.1.3	Install drive system	5 days
1.3.1.4	Verify mechanical assembly	5 days
1.3.3.1	Install LiDAR sensor	5 days
1.3.3.2	Install depth camera	5 days
1.3.3.3	Install compute platform	5 days
1.3.3.4	Configure sensor interfaces	5 days
1.3.3.5	Verify sensor operation	5 days
1.3.4.1	Develop motor control firmware	5 days
1.3.4.2	Develop ROS 2 interface	5 days
1.3.5.1	Integrate hardware subsystems	5 days
1.3.5.2	Integrate software components	5 days
1.3.5.3	System bring-up testing	5 days
1.3.6.1	Verify build completeness	5 days
1.3.6.2	Document build status	5 days

10.4.4 Test Phase Critical Activities

Table 30: Test Phase Critical Activities

WBS ID	Activity	Duration
1.4.1.1	Test sensor subsystems	5 days
1.4.1.2	Test motor control	5 days
1.4.1.3	Test software modules	5 days
1.4.2.1	Test SLAM performance	5 days
1.4.2.2	Test navigation functions	5 days
1.4.2.3	Test obstacle avoidance	5 days
1.4.2.4	Document integration results	5 days
1.4.4.1	Measure system accuracy	5 days
1.4.4.2	Measure system latency	5 days
1.4.5.1	Conduct acceptance tests	5 days
1.4.5.2	Document test results	5 days

10.4.5 Close Phase Critical Activities

Table 31: Close Phase Critical Activities

WBS ID	Activity	Duration
1.5.1.1	Complete user documentation	5 days
1.5.1.2	Complete technical documentation	5 days
1.5.3	Deliver final report	5 days

10.5 Critical Path Analysis

10.5.1 Why These Activities Are Critical

The critical path activities share these characteristics:

- **Sequential Dependencies**: Each activity depends on completion of the previous activity
- **No Parallel Alternatives**: Work cannot be redistributed to other paths
- **Resource Constraints**: Key team members are required for specific expertise
- **Technical Dependencies**: Outputs are required inputs for subsequent work

10.5.2 Critical Path Risk Factors

The following factors pose the greatest risk to critical path activities:

1. **Component Procurement**: Delays in receiving LiDAR or depth camera affect sensor integration
2. **SLAM Performance**: Algorithm tuning may require iteration if accuracy targets not met
3. **Integration Complexity**: Hardware-software integration often reveals unexpected issues
4. **Testing Failures**: Failed tests require debugging and re-testing cycles

10.6 Critical Path Management

10.6.1 Monitoring

Critical path status is monitored through:

- Weekly progress review against baseline schedule
- Early warning indicators from in-progress activities
- Dependency tracking for predecessor completion

10.6.2 Mitigation Strategies

To protect the critical path:

- **Buffer Time**: Non-critical activities provide schedule buffer if resources shift
- **Parallel Development**: Software development proceeds parallel to hardware
- **Early Procurement**: Long-lead items ordered during Research phase
- **Resource Flexibility**: Cross-trained team members can support critical activities
- **Risk Reserves**: Schedule contingency available for high-risk activities

10.6.3 Recovery Options

If critical path delays occur:

1. **Crashing**: Add resources to critical activities (limited effectiveness)
2. **Fast-Tracking**: Overlap sequential activities where possible
3. **Scope Reduction**: Defer non-essential features to recover schedule
4. **Schedule Compression**: Reduce activity durations through focused effort

10.7 Float Analysis

Activities not on the critical path have float (slack) time:

Table 32: Non-Critical Activities with Highest Float

Activity	Phase	Float (Days)
Risk Assessment	Research	15
Software Architecture	Design	12
Electrical Design	Design	10
PM Documentation	PM Overhead	45
Knowledge Transfer	Close	8

Float provides schedule flexibility and opportunities to redirect resources to critical path activities if needed.

11.0 Resource Planning

Section Author(s): CM, BS

11.1 Resource Overview

This section documents team resource allocation across all WBS activities. The six-member team operates approximately 8-10 hours per member weekly during the academic semester, with adjusted availability during exam periods. Resource planning ensures balanced workload distribution while aligning individual expertise with task requirements.

11.2 Resource Allocation Matrix (RAM)

The RAM defines team member responsibilities using the RACI model (Project Management Institute, RACI Chart: Roles and Responsibilities):

- **R** - Responsible: Does the work
- **A** - Accountable: Ultimately answerable (one per activity)
- **C** - Consulted: Provides input
- **I** - Informed: Kept updated

Table 33: Resource Allocation Matrix by Phase

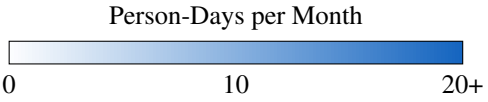
Phase	CM	JB	BS	MN	JH	SC
Research	A/R	R	R	C	C	R
Design - Architecture	A/R	R	C	C	C	I
Design - Mechanical	C	I	C	A/R	I	C
Design - Electrical	C	C	A/R	C	R	I
Design - Software	C	A/R	C	I	R	C
Build - Hardware	I	C	R	A/R	R	C
Build - Software	I	A/R	C	I	R	C
Test	C	R	R	R	R	A/R
Close	A/R	R	R	R	R	R

11.3 Monthly Workload Heat Map

Table 34 presents a heat map visualization of monthly person-day allocation by team member. Darker cells indicate higher workload intensity during that month. The complete detailed breakdown with primary and secondary assignments is provided in Appendix 17.5.

Table 34: Monthly Resource Allocation Heat Map (Person-Days)

Member	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Total
CM	2	10	14	12	6	8	10	10	12	12	**96**
JH	–	2	3	6	4	8	12	16	17	6	**74**
JB	–	6	6	12	8	9	8	5	5	2	**61**
SC	–	4	4	6	4	4	4	8	8	4	**46**
BS	–	5	5	8	6	6	6	4	4	–	**44**
MN	–	–	–	10	5	5	10	6	4	–	**40**
Monthly	**2**	**27**	**32**	**54**	**33**	**40**	**50**	**49**	**50**	**24**	**361**



The heat map reveals key workload patterns:

- **CM** maintains consistent involvement across all months due to project management coordination
- **JH** peaks during Build phase (Feb–Apr) for hardware/software integration
- **JB** concentrates effort in Design phase (Nov–Jan) for ROS 2 architecture
- **SC** increases intensity during Test phase (Mar–May)
- **MN** focuses on mechanical design and fabrication (Nov–Mar)
- **BS** spans electrical design through Build phase (Nov–Mar)

11.4 Effort Distribution

Table 35: Effort Distribution by Team Member

Team Member	Person-Days	Hours	Percentage	Role
Cesar Magana (CM)	96	768	26.6%	Project Manager
Jeremie Hews (JH)	74	592	20.5%	Embedded Systems
Jared Bartz (JB)	61	488	16.9%	Software Lead
Shawn Ciaciura (SC)	46	368	12.7%	Testing & QA
Brighton Sikarskie (BS)	44	352	12.2%	Electrical Lead
Michael Norton (MN)	40	320	11.1%	Mechanical Lead
Total	**361**	**2,888**	**100%**	

Note: Hours calculated at 8 hours per person-day. Project Manager (CM) has higher allocation due to coordination responsibilities across all phases.

11.5 WBS Activity Assignment List

Table 36 documents primary and secondary team member assignments for all WBS leaf activities, including duration and person-day calculations.

Table 36: WBS Activity Assignment List

ID	Activity	Primary	Days	Secondary	P-Days
1.1	**Research Phase**				
1.1.1.1	Schedule and conduct stakeholder interviews	CM	4	BS, MN, JB	4.0
1.1.1.2	Document interview findings and requirements	CM	4	JH, SC	4.0
1.1.2.1	Gather and categorize all requirements	CM	5	BS, MN, JB	5.0
1.1.2.2	Define scope baseline and acceptance criteria	CM	5	JH, SC	5.0
1.1.3.1	Research autonomy framework options	JB	4	CM, BS	4.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.1.3.2	Evaluate and score framework candidates	JB	4	CM, JH	4.0
1.1.3.3	Document trade study findings and recommendation	JB	4	BS	4.0
1.1.4.1	Research and benchmark sensor options	BS	5	JB, CM	5.0
1.1.4.2	Evaluate compute platforms and make selection	JH	5	BS, JB	5.0
1.1.5.1	Identify and categorize project risks	SC	4	CM, BS, MN	4.0
1.1.5.2	Develop mitigation strategies and contingency plans	SC	4	CM, JB, JH	4.0
1.1.6.1	Prepare research phase presentation materials	CM	4	JB, BS, SC	4.0
1.1.6.2	Deliver research phase gate review	CM	4	JB, BS, MN	4.0
	Research Phase Total				*56.0*
1.2	**Design Phase**				
1.2.1.1	Define high-level system architecture	CM	4	JB, BS, MN	4.0
1.2.1.2	Design subsystem interfaces and protocols	CM	4	JB, JH, BS	4.0
1.2.1.3	Document interface control and data flow	CM	4	JB, SC	4.0
1.2.2.1	Create initial chassis dimensions and constraints	MN	5	BS, CM	5.0
1.2.2.2	Design drivetrain and motor mounting	MN	5	JH, BS	5.0
1.2.2.3	Design sensor mounting and enclosures	MN	5	BS, JB	5.0
1.2.2.4	Complete CAD assembly and generate drawings	MN	5	CM	5.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.2.3.1	Define power budget and distribution architecture	BS	4	JB, CM	4.0
1.2.3.2	Design PCB schematic and component selection	BS	5	JH, CM	5.0
1.2.3.3	Complete PCB layout and routing	BS	4	JH	4.0
1.2.3.4	Generate manufacturing files and submit order	BS	3	CM	3.0
1.2.4.1	Define ROS 2 node architecture	JB	4	BS, CM	4.0
1.2.4.2	Design message types and topic structure	JB	4	JH, CM	4.0
1.2.4.3	Design navigation and SLAM pipeline	JB	4	CM, BS	4.0
1.2.4.4	Document software architecture and APIs	JB	4	CM, JH	4.0
1.2.5.1	Define integration sequence and dependencies	JH	4	SC, JB	4.0
1.2.5.2	Design hardware integration test procedures	SC	4	JH, BS, MN	4.0
1.2.5.3	Design software integration test procedures	SC	4	JH, JB	4.0
1.2.5.4	Document test strategy and success criteria	SC	4	JH, CM	4.0
1.2.6.1	Prepare design review materials and agenda	CM	5	MN, BS, JB	5.0
1.2.6.2	Conduct design review and document decisions	CM	5	JB, BS, MN	5.0
1.2.7	Configure Raspberry Pi 5 ROS 2 environment	JB	5	–	5.0
1.2.8.1	Configure LIDAR driver and ROS interface	JB	4	JH	4.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.2.8.2	Test SLAM algorithm in simulation	JB	4	CM, JH	4.0
	Design Phase Total				*103.0*
1.3	**Build Phase**				
1.3.1.1	Procure chassis materials and components	MN	5	CM	5.0
1.3.1.2	Fabricate primary chassis structure	MN	5	JH, BS	5.0
1.3.1.3	Assemble drivetrain and suspension	MN	5	JH, BS	5.0
1.3.1.4	Install mounting brackets and verify alignment	MN	5	JH	5.0
1.3.2.1	Receive and inspect PCB assemblies	BS	4	JH	4.0
1.3.2.2	Build main power distribution harness	BS	5	JH, MN	5.0
1.3.2.3	Build signal and data harnesses	BS	5	JH, JB	5.0
1.3.2.4	Test harness continuity and isolation	SC	4	BS, JH	4.0
1.3.3.1	Install compute platform and configure OS	JB	5	JH, BS	5.0
1.3.3.2	Mount and connect LIDAR sensor	MN	5	JH, BS, JB	5.0
1.3.3.3	Install additional sensors and peripherals	JH	5	BS, MN	5.0
1.3.3.4	Route and secure all cabling	BS	5	JH, MN	5.0
1.3.3.5	Verify all electrical connections and power-on test	SC	4	JH, BS, JB	4.0
1.3.4.1	Deploy ROS 2 software stack to robot	JB	5	JH, BS	5.0
1.3.4.2	Validate SLAM navigation on physical robot	JB	5	JH, SC	5.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.3.5.1	Execute mechanical integration tests	SC	4	JH, MN	4.0
1.3.5.2	Execute electrical integration tests	SC	4	JH, BS, JB	4.0
1.3.5.3	Execute software integration tests	SC	4	JH, JB	4.0
1.3.6.1	Prepare build phase review materials	CM	5	JH, JB, BS	5.0
1.3.6.2	Conduct build phase gate review	CM	5	JH, JB, BS	5.0
	Build Phase Total				*94.0*
1.4	**Test Phase**				
1.4.1.1	Review and finalize test procedures	SC	4	JH, CM	4.0
1.4.1.2	Prepare test environment and equipment	SC	4	JH, JB	4.0
1.4.1.3	Set up data collection and monitoring systems	SC	4	JB, BS	4.0
1.4.2.1	Execute navigation functional tests	SC	4	JH, JB	4.0
1.4.2.2	Execute sensor integration tests	SC	4	JH, BS	4.0
1.4.2.3	Execute power system tests	SC	4	JH, BS, CM	4.0
1.4.2.4	Document test results and defects	SC	4	JH	4.0
1.4.3.1	Execute indoor navigation trials	SC	4	JH, JB	4.0
1.4.3.2	Execute multi-room mapping trials	SC	4	JH, JB	4.0
1.4.3.3	Execute obstacle avoidance trials	SC	4	JB, JH	4.0
1.4.3.4	Analyze field trial data and document findings	SC	4	JB	4.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.4.4.1	Triage and prioritize defects	JB	5	BS, JH	5.0
1.4.4.2	Implement fixes and verify corrections	JB	5	BS, MN, JH	5.0
1.4.5.1	Prepare acceptance demonstration	CM	4	JH, SC, JB	4.0
1.4.5.2	Deliver demo and finalize test report	CM	4	JH, SC, BS	4.0
	Test Phase Total				*62.0*
1.5	**Close Phase**				
1.5.1.1	Complete technical documentation	CM	4	JB, BS, JH	4.0
1.5.1.2	Prepare knowledge transfer materials	CM	4	JB, MN, SC	4.0
1.5.2.1	Develop training materials and procedures	SC	3	JH, JB	3.0
1.5.2.2	Conduct training sessions and hand off	SC	3	JH, CM	3.0
1.5.3	Close budget, procurement, and inventory actions	CM	4	–	4.0
1.5.4.1	Conduct lessons learned workshop	JH	3	CM, JB, BS	3.0
1.5.4.2	Archive project assets and documentation	JH	3	CM, MN, JB	3.0
	Close Phase Total				*24.0*
1.6	**PM Overhead Phase**				
1.6.1	Conduct weekly team meetings	CM	5	BS, MN, JB	5.0
1.6.2	Submit bi-weekly status reports	CM	5	BS, MN, JB	5.0
1.6.3	Conduct risk management reviews	SC	4	CM, BS, JB	4.0
1.6.4	Maintain project documentation	CM	5	JH, JB, BS	5.0

Continued on next page

ID	Activity	Primary	Days	Secondary	P-Days
1.6.5	Manage scope and change requests	CM	3	SC, BS	3.0
	PM Overhead Phase Total				*22.0*
Project Total					**361.0**

Note: Person-days (P-Days) represent primary owner effort. Secondary assignments contribute additional support effort estimated at 15-25% of primary.

11.6 Resource Constraints

- Team members available 8-10 hours per week during academic semester
- Reduced availability during exam periods (midterms, finals)
- Reduced productivity during university breaks (Thanksgiving, Winter, Spring Break)
- Critical skills requiring coverage: ROS 2 (JB, JH), PCB design (BS), mechanical CAD (MN)
- Hardware lab access available 24/7 during semester

11.7 Resource Leveling

Resource conflicts are resolved by:

1. Adjusting activity timing within available float
2. Reassigning activities to available team members with appropriate skills
3. Splitting activities across multiple team members where feasible
4. Prioritizing critical path activities over non-critical work

12.0 Risk Assessment

Section Author(s): MN, JH

12.1 Risk Management Overview

This section identifies the project's technical risks by highest priority, evaluates their probability and impact, and defines mitigation strategies. Risk management follows a systematic approach: each risk is assigned a **probability** rating (likelihood of occurrence) and an **impact** rating (severity of consequences if it occurs). These ratings combine to determine overall risk priority, guiding resource allocation for mitigation efforts.

The primary technical risks center on the drivetrain, sensing, computation, and navigation subsystems. The drivetrain utilizes 3D-printed bevel gears, which require precise design due to potential chafing and wear under continuous load. SLAM performance may degrade in environments with limited geometric features, reducing map accuracy. SLAM failure occurs when the algorithm cannot reliably determine the robot's position within its map. This manifests as pose estimate drift, loop closure failures, or complete localization loss. Contributing factors include featureless environments (long corridors, blank walls), repetitive geometry (identical rooms), dynamic obstacles, and sensor noise. LiDAR reliability can be affected by different surface types, particularly reflective (glass, mirrors) or absorptive (dark fabrics) materials, leading to noisy range data. Processing demands may exceed the Raspberry Pi 5's resources since the SLAM algorithm, LiDAR processing, and Nav2 framework run concurrently, potentially causing timing delays.

The most relevant risks are presented in detail below; the complete risk assessment is available in Appendix H.

12.2 Risk Matrix

Table 37: Risk Assessment Matrix (Probability × Impact)

Probability	**Impact**				
	1	2	3	4	5
5	5	10	15	20	25
4	4	8	12	16	20
3	3	6	9	12	15
2	2	4	6	8	10
1	1	2	3	4	5

Risk Level Legend:

- Critical: Immediate action required; may halt project
- High: Priority mitigation needed; close monitoring
- Medium: Mitigation plan required; regular monitoring
- Low: Accept with contingency plan
- Minimal: Accept; no action required

12.3 Identified Risks

Table 38: Risk Register

ID	Risk	Prob	Impact	Risk Level	Mitigation
R-001	SLAM failure in feature-poor environments	3	4	12	Perform tests in low-feature areas; tune scan matching parameters; integrate IMU fusion for drift reduction
R-002	Compute Throughput Saturation on Raspberry Pi 5	3	3	9	Profile system load; optimize code & ROS 2 node configurations to reduce CPU / memory usage
R-003	3D-printed gears wearing under continuous load	4	5	20	Apply post-processing and validate durability through torque and runtime testing

12.4 Risk Response Strategies

For each risk category, the following response strategies are used:

- **Avoid**: Eliminate the threat by changing project plan
- **Mitigate**: Reduce probability or impact
- **Accept**: Acknowledge and prepare contingency

12.5 Top Risks Detail

12.5.1 R-001: SLAM failure in feature-poor environments

- **Description**: SLAM failure is a significant risk because unstable scan matching can produce incomplete or inaccurate maps, especially in environments without many geometric features or which have repetitive geometry. Under these conditions, the algorithm may accumulate drift or lose its pose estimate, reducing the system's ability to deliver a reliable map. Performance issues can also emerge if the compute pipeline cannot maintain consistent timing, further degrading data alignment and SLAM quality.

- **Probability**: Medium. SLAM algorithms are mature and the SICK TiM561 provides high-quality scan data, but feature-poor environments (hallways, empty rooms) are common in target use cases.
- **Impact**: High. SLAM failure directly prevents the robot from completing its primary mission of generating accurate maps; the system cannot recover without operator intervention.
- **Mitigation**: Mitigation focuses on improving the stability of the sensor data and the reliability of the SLAM algorithm. Tuning the LiDAR's scan-matching and filtering parameters can ensure that the LiDAR is able to return consistent values even in environments with poor geometric features. Integrating IMU data would provide an additional constraint on the pose estimation, limiting drift. SLAM performance is to be validated through the use of several controlled tests to measure map completeness and percentage of error over a full mission. Finally, additional behaviors could be programmed to allow the robot to automatically restore its pose estimate when confidence drops.
- **Owner**: Jared Bartz

12.5.2 R-002: Raspberry Pi 5 compute load saturation

- **Description**: Since the LiDAR processing, Nav2 planning, SLAM, and communication are all running concurrently on the Raspberry Pi 5, they may exceed the board's available CPU and memory resources. Whenever the processing pipeline becomes overloaded, node update rates and real-time tasks may be delayed in their response, leading to less accuracy, slowed down navigation, or intermittent communication with the motor controllers.
- **Probability**: Medium. The Raspberry Pi 5 has significant compute capability (4-core Cortex-A76, 8GB RAM), but running SLAM, Nav2, and LiDAR processing concurrently is demanding.
- **Impact**: Medium. Degraded performance causes slower operation rather than complete failure; timing-critical motor control tasks are offloaded to the ESP32 running FreeRTOS.
- **Mitigation**: System resource usage would need to be refined in programming in order to maintain real-time performance. This can start by benchmarking the Raspberry Pi 5 under full operational load in order to identify the specific nodes and processes that are responsible for any CPU or memory spikes. Update frequencies can then be adjusted and parameters tuned so that the CPU utilization can remain within a safer threshold. The Raspberry Pi 5 is a single-board computer without real-time guarantees, so all timing-sensitive tasks (motor control, encoder feedback) run on the ESP32 which executes FreeRTOS for deterministic timing. If ESP32 performance issues arise, Segger Ozone can be used for real-time debugging and profiling. If Pi 5 issues persist, hardware accelerators or a higher-performance compute module are to be evaluated. Other mitigation can include quantization of the model.

- **Owner**: Jeremie Hews

12.5.3 R-003: 3D-printed gears wearing under continuous load

- **Description**: Due to the extensive price of gears currently and a motor shaft that is an uncommon shape, 3D-printed bevel gears will be used to transfer torque from the motor shafts to the wheel axes. Inherently, any kind of plastic is a mechanical reliability risk at such a critical position in the system. The gears will be made from PETG, which is tougher than other filaments like PLA, but still have much less wear resistance and stiffness compared to more typical engineering-grade polymers. Gradual tooth wear may result under sustained loads, so their design is critical. Over time, these factors may produce increased noise in the system, reduced torque transfer, or even full gear failure, impacting the robot's mobility.
- **Probability**: High. 3D-printed PETG gears inherently have lower wear resistance than machined metal gears, and the drivetrain experiences continuous mechanical stress during operation.
- **Impact**: High. Gear failure immobilizes the robot completely, requiring physical repair and gear replacement before operation can resume.
- **Mitigation**: Mitigation focuses on optimizing the fabrication quality and validating the mechanical performance of the gears through numerous tests. The gears should be printed with a high infill and wall thickness. When printing, the layer lines should run in the same plane as the gears teeth to maximize the gears shear strength. Post-processing, such as smoothing the tooth profiles or applying a lubricant, can reduce the friction the gears face and improve wear-resistance. Still, the gear dimensions need to be near perfect for the most reliable results, so the gears will be evaluated through controlled torque and endurance testing to confirm the PETG gears are able to maintain acceptable performance over the lifespan of the robot.
- **Owner**: Michael Norton

12.6 Risk Monitoring

Risks are monitored through:

1. During weekly meetings evaluate the current risk status, discuss any emerging issues, and determine any corrective actions.
2. Maintaining and updating a risk register to ensure that all risks are being tracked with current ratings and mitigation progress.
3. Establishing trigger conditions that prompt changes, or the introduction of new risks into the register.

4. Reviewing test results and field data at regular intervals to identify any trends that may indicate increasing or newly developing risks.

13.0 Budget

Section Author(s): CM, BS, MN

13.1 Budget Overview

The project budget encompasses both material costs and labor costs, providing a comprehensive view of project investment. Material costs include all electronic components (with a 1.2× spare parts buffer) and mechanical hardware. Labor costs total 2,888 person-hours across all six team members at a standard rate of $20 per hour. A 10% contingency reserve is included to account for unforeseen expenses and scope adjustments.

Table 39: Budget Summary

Category	**Amount**
Electronics (w/ 1.2× Spare Buffer)	$294.81
Mechanical Components	$377.25
Total Material Value	**$672.06**
Labor Costs (2,888 hours @ $20/hr)	$57,760.00
Subtotal	$58,432.06
Reserve (10%)	$5,843.21
Total Project Value	**$64,275.27**

13.2 Material Costs - Bill of Materials

The Bill of Materials (BOM) details all physical components required for the project. Pricing uses the lowest available price between DigiKey and Mouser for each component. A 1.2× spare parts buffer (20% additional) is applied to electronics costs to account for assembly losses and testing. For complete component details including part numbers and supplier links, see Appendix 17.9.

Table 40: Bill of Materials: PCB Summary (1.2× Buffer Applied)

Board	**Parts**	**Raw Cost**	**Buffered**	**Ref**
Power Management				

Continued on next page

Table 40 (Continued from previous page)

Board	Parts	Raw Cost	Buffered	Ref
3.3V Buck Regulator	15	$7.00	$8.40	123
5V Buck Regulator	23	$16.29	$19.55	124
6V Buck Regulator	34	$37.78	$45.34	125
Battery Management				
Analog Front-End (AFE)	81	$37.91	$45.49	118
Buck-Boost Charger	43	$17.51	$21.01	120
Fuel Gauge	46	$15.18	$18.22	121
Gate Driver	12	$5.03	$6.04	122
Control/Interface				
LiDAR Breakout	5	$11.15	$13.38	126
MCU Board	42	$53.83	$64.60	119
Motor Driver	42	$28.39	$34.07	116
USB-C PD Controller	51	$15.59	$18.71	117
Electronics Subtotal	**394**	**$245.66**	**$294.81**	
Mechanical Components				
Chassis, Motors, Hardware	–	–	$377.25	128
Total Material Value			**$672.06**	

13.3 Material Costs by Category

Table 41: Material Value Distribution by Category

Category	Raw Cost	With Buffer	Percentage
Power Management	$61.07	$73.29	10.9%
Battery Management	$75.63	$90.76	13.5%
Control/Interface	$108.96	$130.76	19.5%
Mechanical	$377.25	$377.25	56.1%
Total	**$622.91**	**$672.06**	**100%**

13.4 Labor Costs

Labor costs are calculated based on team member effort allocation at a standard rate of $20 per hour. Hours are derived from the Work Breakdown Structure activities, with each person-day representing 8 hours of effort:

Table 42: Labor Cost by Team Member

Team Member	Hours	Rate	Cost
Cesar Magana (CM)	768	$20/hr	$15,360.00
Jeremie Hews (JH)	592	$20/hr	$11,840.00
Jared Bartz (JB)	488	$20/hr	$9,760.00
Shawn Ciaciura (SC)	368	$20/hr	$7,360.00
Brighton Sikarskie (BS)	352	$20/hr	$7,040.00
Michael Norton (MN)	320	$20/hr	$6,400.00
Total Labor	**2,888**		**$57,760.00**

13.5 Monthly Budget Breakdown

The following table shows planned expenditures by month, aligned with the five project phases (Research, Design, Build, Test, Close). Material purchases are concentrated in the Build phase when components are ordered, while labor costs follow the activity schedule:

Table 43: Monthly Budget Distribution

Month	Phase	Materials	Labor	Total
Aug 2025	Research	$0.00	$320.00	$320.00
Sep 2025	Research	$0.00	$4,320.00	$4,320.00
Oct 2025	Research	$0.00	$5,120.00	$5,120.00
Nov 2025	Design	$25.00	$8,640.00	$8,665.00
Dec 2025	Design	$25.00	$5,280.00	$5,305.00
Jan 2026	Design/Build	$200.00	$6,400.00	$6,600.00
Feb 2026	Build	$250.00	$8,000.00	$8,250.00
Mar 2026	Build/Test	$122.06	$7,840.00	$7,962.06
Apr 2026	Test	$50.00	$8,000.00	$8,050.00
May 2026	Test/Close	$0.00	$3,840.00	$3,840.00
Total		**$672.06**	**$57,760.00**	**$58,432.06**

13.6 Budget Visualization

Figure 37 illustrates the budget distribution across material categories. The pie chart shows the relative allocation by subsystem, while the bar chart displays per-board costs with the 1.2× buffer applied.

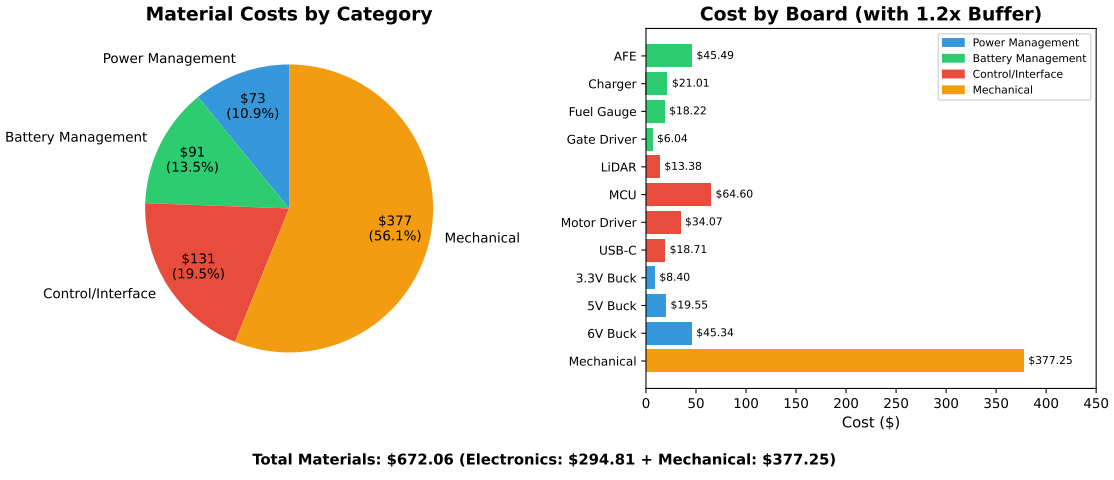


Figure 37: Budget Distribution by Category and Board

13.7 Contingency Reserve

A 10% contingency reserve ($5,843.21) is allocated to address:

- Component price fluctuations or supply chain delays
- Design iterations requiring additional materials
- Scope adjustments approved through change control
- Unforeseen testing or integration requirements

The reserve is managed under strict change control and requires Project Manager approval for any withdrawals.

14.0 Lessons Learned

Section Author(s): CM, BS, JH, MN, SC, JB

This section documents key insights gained during the project lifecycle, demonstrating continuous improvement and knowledge capture. These lessons have been applied iteratively throughout the project phases.

14.1 Research Phase Lessons

14.1.1 LL-1: Iterative Compute Platform Selection

Situation: During compute platform selection, the team evaluated multiple options for the main processing unit. The initial candidate was the Texas Instruments SK-TDA4VM, selected for its integrated vision accelerators and automotive-grade reliability.

Discovery #1: Power analysis revealed the SK-TDA4VM consumed 15–20W under typical SLAM workloads and 100W at max load, exceeding our 10W power budget for the compute subsystem. This would have required a larger battery and increased robot weight.

Pivot #1: Research into low-power FPGA alternatives led to the Pynq-Z2 board, inspired by work on unified SLAM accelerators for low-end FPGAs (Sugiura and Matsutani). The Pynq-Z2 offered hardware acceleration within our power envelope.

Discovery #2: Compatibility testing revealed that ROS 2 Humble had limited support for the Pynq-Z2's ARM architecture and required significant manual configuration. The FPGA development toolchain also added complexity.

Pivot #2: The team selected Raspberry Pi 5, which provides native Ubuntu 22.04 and ROS 2 Humble support, extensive community documentation, proven reliability in robotics applications, and acceptable power consumption (5–8W under load).

Result: Zero schedule impact. Both pivots occurred during Research before any hardware procurement. Software development proceeded without compatibility issues.

Lesson: *Evaluate multiple candidates against all constraints (power, software compatibility, development complexity) before procurement. Early pivots cost hours; late pivots cost weeks.*

14.1.2 LL-2: Sponsor Relationship Investment

Situation: The project budget constrained sensor options, with professional-grade LiDARs costing $2,000–$5,000.

Action: Proactive engagement with SICK Inc. through multiple meetings with Dr. Porter clarified project scope and demonstrated educational value. The team presented the project's alignment

with SICK's educational outreach goals.

Result: SICK Inc. donated a TiM561 LiDAR valued at $3,500+, providing professional-grade sensors at educational budget levels.

Lesson: *Sponsor relationships provide resources beyond funding: expertise, equipment, and scope guidance. Invest time in demonstrating mutual value.*

14.2 Design Phase Lessons

14.2.1 LL-3: Multi-Board PCB Strategy

Situation: The original electrical design called for a single large PCB containing all motor drivers, power distribution, and sensor interfaces.

Risk Analysis: Risk assessment revealed that a single board failure would be catastrophic; the entire system would be non-functional, and replacement would require complete board redesign and fabrication

Action: Redesigned to six smaller PCBs. Each board uses 2-layer or 4-layer and can be replaced independently.

Result: If one board fails, it's cheaper to replace one board than the entire system.

Lesson: *Design for failure isolation. Modular hardware architecture reduces both cost and catastrophic failure risk.*

14.2.2 LL-4: Interim Hardware for Parallel Development

Situation: The donated SICK TiM561 LiDAR requires a specific power adapter board. Initial testing revealed we did not possess the correct adapter, and sourcing would take 2–3 weeks.

Problem: Without a functional LiDAR, software development (ROS 2 drivers, SLAM integration, visualization) would stall.

Action: Ordered an RPLiDAR C1 ($55) as an interim development sensor. Both sensors use similar ROS 2 driver interfaces, allowing software development to proceed.

Result: Software team continued SLAM development and ROS 2 integration while the SICK adapter was sourced. When the correct adapter arrived, the software was ready for immediate integration.

Lesson: *Identify blockers early and find workarounds to maintain parallel progress. A $55 interim sensor prevented weeks of idle time.*

14.3 Implementation Insights

14.3.1 LL-5: Backup Hardware Strategy Validated

Incident: During initial testing in November 2025, jumper wires connected to the SICK TiM561 LiDAR melted due to high inrush current at startup. The LiDAR draws significant current during power-on, which exceeded the current capacity of the temporary jumper wire connections.

Impact: Initial testing was temporarily halted while the team investigated the root cause. Testing with a current-limited bench power supply confirmed the LiDAR itself was undamaged and operated normally under controlled conditions.

Mitigation: Because the RPLiDAR C1 had been ordered as a backup/development sensor (per LL-4), software development continued uninterrupted using the backup sensor. The team was able to proceed with SLAM development and ROS 2 integration while addressing the power delivery issue.

Resolution: The SICK TiM561 remains the primary sensor with proper power management. The RPLiDAR C1 continues to serve as the backup/development sensor, providing redundancy and enabling parallel testing.

Lesson: *Always have backup hardware for critical components, and test power delivery carefully before connecting sensitive equipment. Inrush current management is critical for industrial sensors.*

14.4 Applied Improvements Summary

These lessons were applied iteratively throughout the project:

Table 44: Lessons Applied During Project

Lesson	Phase Applied	Action Taken	Outcome
LL-1	Research	Pynq-Z2 $\rightarrow$ RPi 5 pivot	Zero schedule impact
LL-2	Research	SICK sponsorship engagement	$3,500+ equipment donated
LL-3	Design	Single PCB $\rightarrow$ 6 boards	Cost reduction
LL-4	Design	Ordered RPLiDAR C1 interim	Parallel development
LL-5	Build (early)	Used backup during power issue	No schedule slip

14.5 Recommendations for Future Projects

Based on these lessons, we recommend:

1. **Validate Early**: Test software/hardware compatibility during Research, not Build
2. **Design for Failure**: Modular architectures enable graceful degradation
3. **Budget for Backups**: Include backup components for critical-path items
4. **Engage Sponsors**: Industry partnerships provide more than funding

15.0 Preliminary Implementation

Section Author(s): BS, CM, JH, MN

15.1 Implementation Overview

The development follows three parallel workstreams (electrical, embedded, software) that will converge during Capstone 420 integration. A modular design philosophy enables subsystems to be developed and tested both simultaneously and independently of each other.

Table 45: Implementation Status by Phase

Phase	Status	Deliverables
Capstone 419 (Complete)		
Research	Done	Component selection (power, BMS, sensors, actuation)
Design	Done	6 PCB schematics + layouts (DRC passed)
Software	Done	ROS 2 architecture, communication protocols
Firmware	Done	Motor control and sensor interface framework
Capstone 420 (Planned)		
Build	Pending	PCB fabrication, assembly, firmware deployment
Test	Pending	Unit, integration, SLAM/navigation validation

15.2 Electrical Preliminary Implementation

15.2.1 Component Selection and Research

Table 46: Hardware Subsystem Overview

Subsystem	Function	Key Components
Power Input	USB-C charging, power path control	TPS25751D, BQ25798
Power Distribution	Voltage conversion (6V, 5V, 3.3V)	TPS40305, TPS566238, TLV62569
Battery Management	Cell monitoring, protection, fuel gauge	BQ76920, BQ78350, BQ76200
Compute & Control	SLAM/navigation + real-time control	Raspberry Pi 5, ESP32-S3
Sensing & Actuation	Environmental perception + motion	LiDAR, IMU, motors, drivers

Power Input and Charging Power enters the system through USB-C, managed by the TPS25751D Dual Role Power (DRP) controller. This enables the robot to either sink power for battery charging or source power to the Raspberry Pi 5.

Table 47: USB-C Power Modes

Mode	Power	Use Case
Sink	Up to 100W (20V/5A)	Battery charging from adapter
Source	5V/3A (15W)	Powering Raspberry Pi 5

The BQ25798 buck-boost charger manages 3S Li-ion charging while providing the SYS output for downstream regulators. Buck-boost topology handles USB PD voltages above/below pack voltage seamlessly.

Table 48: BQ25798 Charger Specifications

Parameter	Value
Input voltage	3.6–24V (USB PD 3.0 compatible)
Charge current	Up to 5A (10 mA resolution)
Battery config	3S Li-ion (12.6V charge voltage)
Peak efficiency	96.5%
Interface	I^2C

Power Distribution The SYS output feeds three switching regulators that provide isolated rails for different load types.

Table 49: Power Rail Summary

Rail	Regulator	Current	Source	Loads
6V	TPS40305DRCT	25A max	PACK+ (AFE)	4× DC motors
5V	TPS566238PRQFR	6A	SYS	Encoders, USB PD
3.3V	TLV62569ADRLR	2A	SYS	ESP32, IMU, sensors

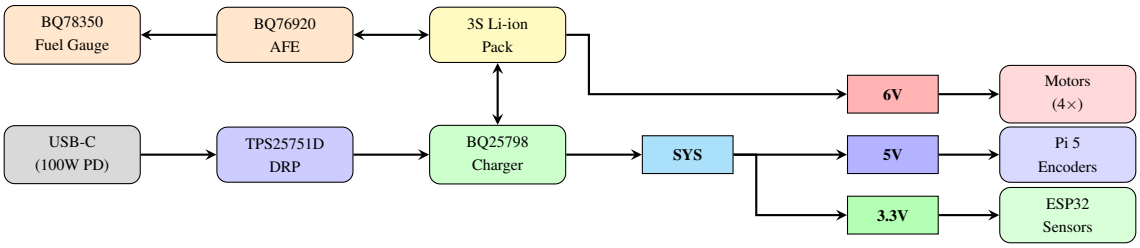


Figure 38: Power Distribution Architecture

Table 50: Power Regulator Details

Rail	Key Feature	Notes
6V (TPS40305)	1.2 MHz, FSS	Voltage mode with feed-forward; worst-case 12.8A (4×3.2A stall)
5V (TPS566238)	D-CAP3	No external compensation; 20.8/10.6 mΩ HS/LS MOSFETs
3.3V (TLV62569)	35 μA I_q	95% efficiency; $<$2 μA shutdown

Design Evolution (3.3V): Originally used LDO from 5V rail, but ESP32-S3 WiFi peaks (500 mA) caused 0.85W dissipation. Switched to dedicated regulator from SYS output.

Battery System The BMS employs three ICs: an Analog Front End (AFE) for cell monitoring/protection, a fuel gauge for state-of-charge estimation, and a gate driver for power path control. The 3S Li-ion pack provides 11.1V nominal (12.6V fully charged).

Table 51: BMS IC Summary

IC	Part Number	Function
AFE	BQ76920PW	Cell voltage monitoring (14-bit ADC), hardware protection (OV/UV/OCD/SCD), cell balancing
Fuel Gauge	BQ78350DBT-R1A	CEDV algorithm for SOC/SOH estimation, SMBus interface to host
Gate Driver	BQ76200PW	High-side N-FET driver with charge pump, controls CHG/DSG/PCHG paths

AFE Protection Features (BQ76920) Hardware-level protection operates independently of software:

Table 52: BQ76920 Protection Thresholds

Protection	Threshold	Method
Overvoltage (OV)	4.25V/cell	Digital comparator
Undervoltage (UV)	2.8V/cell	Digital comparator
Overcurrent (OCD)	Configurable	2 mΩ sense resistor
Short Circuit (SCD)	Configurable	Fast analog comparator
Cell Balancing	50–100 mA	Passive resistive

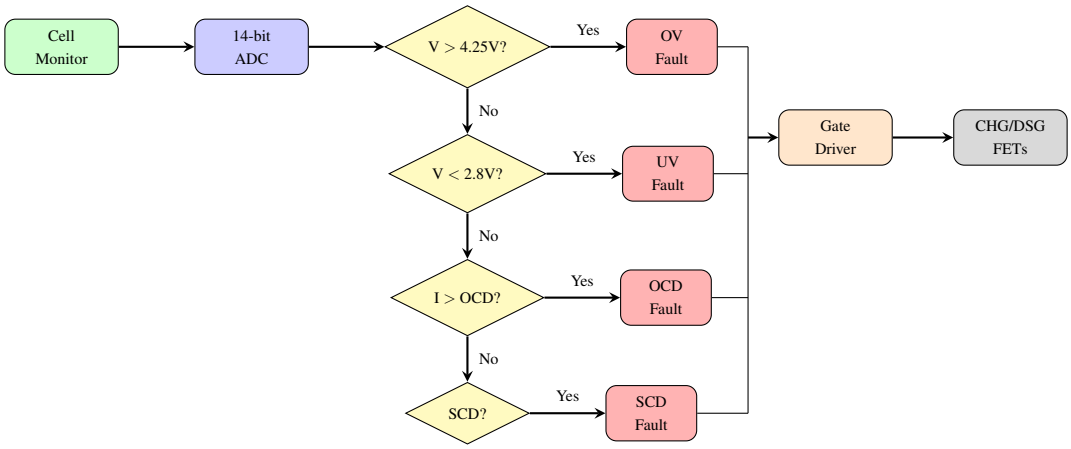


Figure 39: BMS Protection Logic Flow

Table 53: Power Stage MOSFETs

Function	Part	Key Spec
Discharge (DSG)	CSD19531Q5AT	1.6 mΩ $R_{DS(on)}$, 100A
Charge (CHG)	N-channel	Series with DSG path
Precharge (PCHG)	FDD4141	P-channel, current-limited soft-start

Power FET Configuration (BQ76200) N-channel high-side FETs chosen for lower $R_{DS(on)}$ and maintained ground continuity. Integrated charge pump generates 9–14V above PACK+ for gate drive.

Compute and Control Architecture The STAR robot employs a distributed compute architecture that separates high-level processing from real-time control. This section describes the platform selection process, the rationale for choosing specific compute modules, and how they integrate to form a cohesive system.

Platform Selection and Evolution The compute platform underwent evolution during Research phase evaluation against project requirements.

Table 54: Compute Platform Evaluation

Platform	Pros	Cons	Decision
PYNQ-Z2 (Zynq)	FPGA acceleration	32-bit ARM (ROS 2 needs 64-bit)	Rejected
SK-TDA4VM (TI)	AI accelerators	100W TDP (battery impractical)	Rejected
Raspberry Pi 5	64-bit, 5–10W, $95	–	**Selected**

Raspberry Pi 5: Single Board Computer The Raspberry Pi 5 runs a custom Buildroot Linux image and handles complex software stacks, while the ESP32 microcontroller executes real-time motor control firmware.

Table 55: Raspberry Pi 5 Specifications

Spec	Value	Spec	Value
Processor	BCM2712 (4× A76 @ 2.4 GHz)	Memory	8 GB LPDDR4X
Storage	MicroSD (Buildroot)	WiFi	802.11ac dual-band
USB	2× 3.0, 2× 2.0	GPIO	40-pin (SPI, I^2C)
Power	5V/5A (USB-C from DRP)	Ethernet	Gigabit

Responsibilities: SLAM processing, Nav2 navigation, sensor fusion (LiDAR/IMU/encoders), web UI hosting, velocity commands to ESP32 via SPI.

ESP32-S3-WROOM-1-N16: Real-Time Motor Controller Handles time-critical operations (motor PWM, sensor polling) that cannot tolerate Linux latency variability.

Table 56: ESP32-S3 Specifications

Spec	Value	Spec	Value
Processor	2× LX7 @ 240 MHz	SRAM	512 KB
Flash	16 MB	FPU	Hardware
USB	Native OTG	MCPWM	Center-aligned PWM

Why N16 (No PSRAM): PSRAM (Pseudo Static RAM) variants dedicate GPIO 35–37 to the memory interface. The N16 frees these pins for the second SPI bus to motor drivers. 512 KB internal SRAM is sufficient.

Communication: Pi 5 (controller) ↔ ESP32 (peripheral) over SPI for velocity commands and sensor data. ESP32 autonomously maintains PWM generation between updates.

Table 57: Sensor Specifications

Sensor	Type	Range	FOV	Rate	Interface
SICK TiM561	2D LiDAR	10 m	270°	15 Hz	Ethernet
RPLiDAR C1	2D LiDAR	12 m	360°	10 Hz	UART
BNO055	9-axis IMU	–	–	100 Hz	I^2C
HC-SR04 (7×)	Ultrasonic	2–400 cm	15°	–	GPIO

Sensor Array **LiDAR Selection:** The SICK TiM561 (primary) was donated by SICK Inc., saving $3,500. The RPLiDAR C1 ($65) provides 360° backup coverage. The TiM561's higher angular resolution (0.33° vs 0.72°) and IP67 rating make it preferred for precision mapping.

IMU (BNO055): On-chip sensor fusion outputs quaternions and Euler angles directly, eliminating host-side Kalman filter complexity. Provides dead reckoning in featureless environments.

Ultrasonics (HC-SR04): Seven sensors around perimeter detect close-range obstacles (2–30 cm) and LiDAR-invisible objects (glass). Temperature-compensated via DS18B20 ($v = 331.3 + 0.606T$ m/s).

Actuation and Drive Four-wheel differential drive with independently controlled DC gear motors and integrated H-bridge drivers.

Table 58: Motor and Driver Specifications

Motor (DFRobot FIT0520)		**Driver (DRV8243SQDGQRQ1)**	
Voltage	6V DC	Max current	12A/channel
Speed	210 RPM (no-load)	$R_{DS(on)}$	98 mΩ
Stall torque	10 kg·cm	PWM freq	Up to 25 kHz
Stall current	3.2A	Interface	SPI + PWM
Gear ratio	34:1	Protection	OCP, OVP, TSD, UVLO
Encoder	341.2 counts/rev	Current sense	IPROPI output

Table 59: Motor Control Data Flow

Step	**Component**	**Function**
1	Pi 5 (ROS 2)	Calculate wheel velocities from Nav2
2	SPI bus	Transmit velocity commands
3	ESP32 (1 kHz)	PID loop: encoder feedback vs. command
4	MCPWM	Generate center-aligned PWM
5	Encoders	Close velocity loop (341.2 counts/rev)

Control Loop Architecture **Design Rationale:** 144 mm wheels yield 1.1 m/s max speed. AEC-Q100 drivers provide fault protection. Integrated current sensing enables stall detection. Architecture separates timing-critical control (ESP32/FreeRTOS) from navigation (Pi 5/Linux).

15.2.2 Schematic Design

All schematics were designed in KiCad 9.0.

AFE (Analog Front End) The BQ76920 AFE schematic provides cell voltage monitoring and hardware protection.

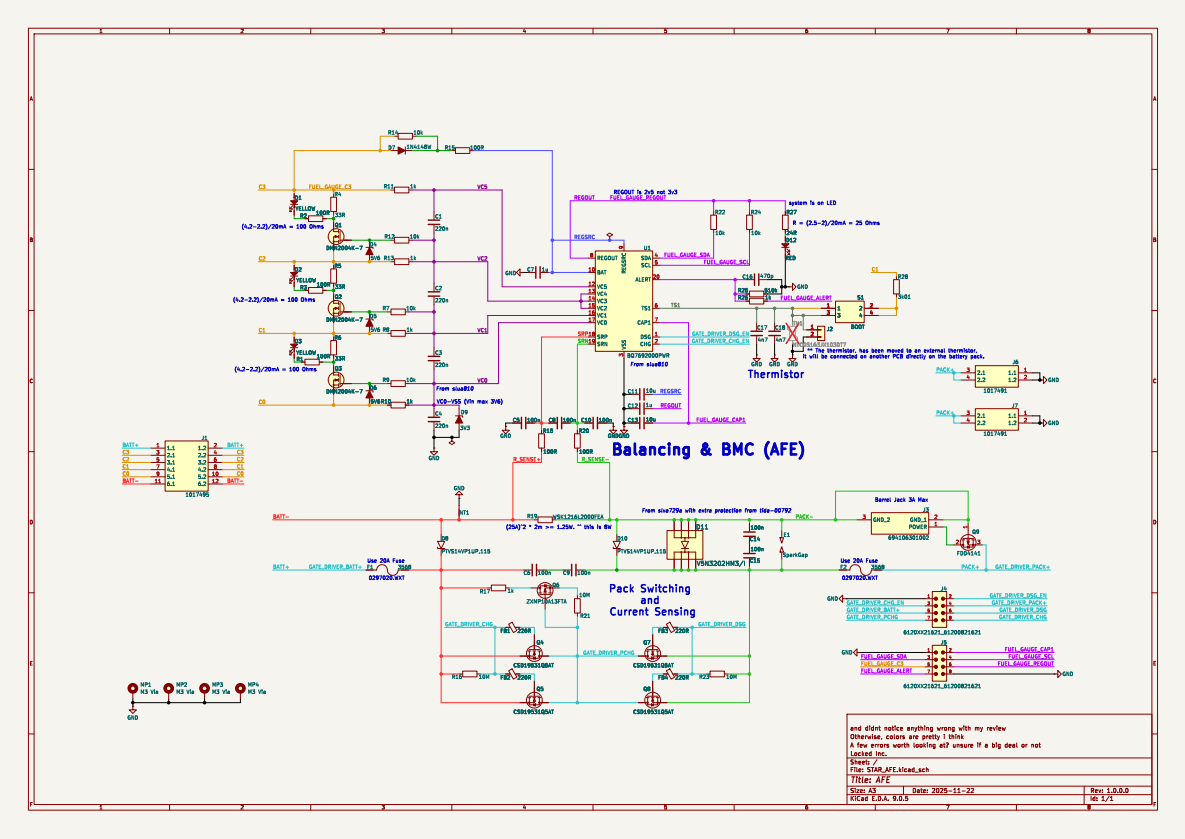


Figure 40: AFE Schematic (BQ76920)

Buck-Boost Charger The BQ25798 charger schematic handles USB-PD input and battery charging.

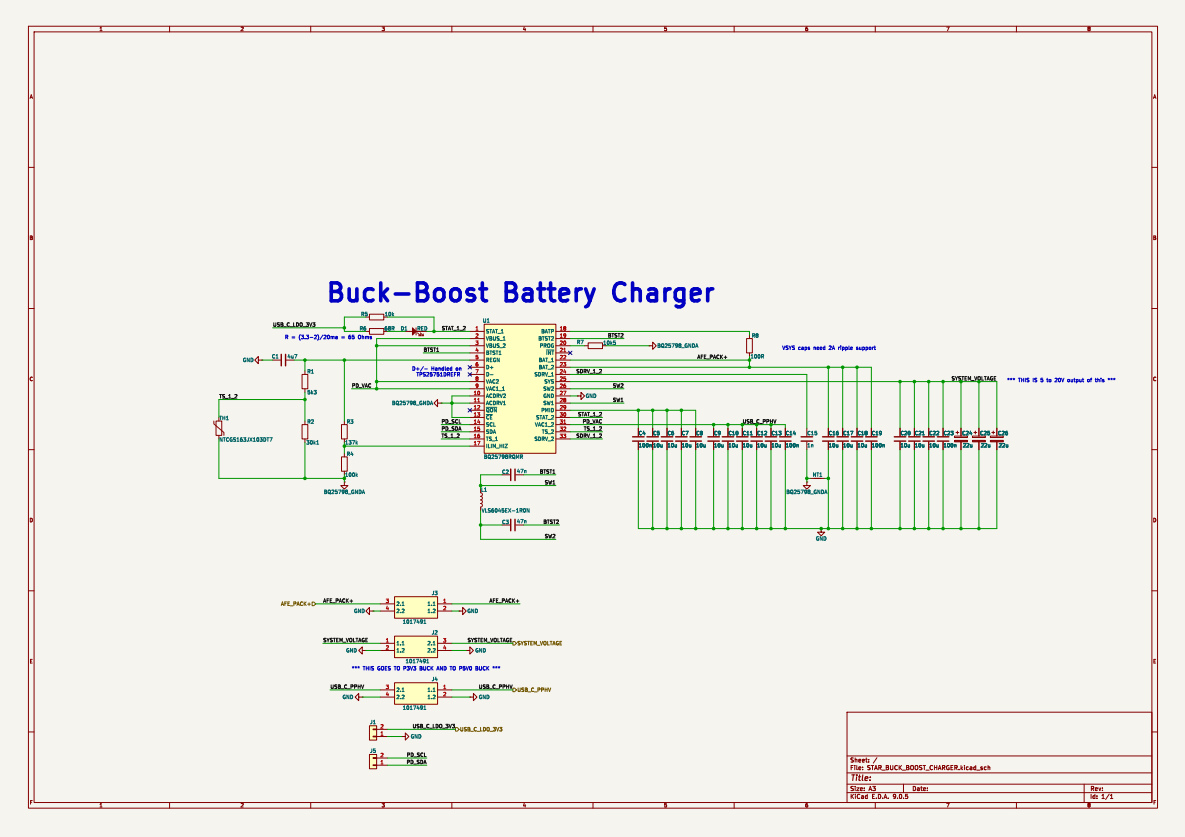


Figure 41: Buck-Boost Charger Schematic (BQ25798)

MCU The ESP32-S3 microcontroller schematic for real-time motor control.

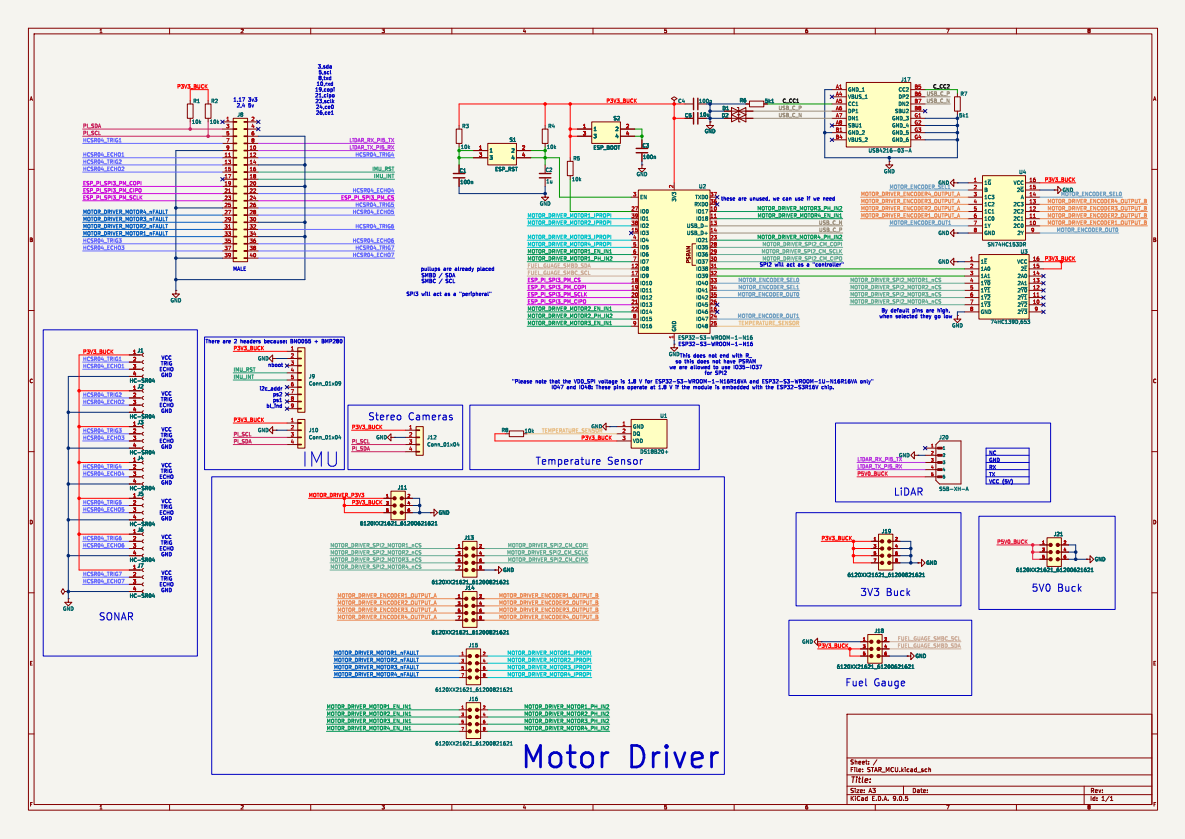


Figure 42: MCU Schematic (ESP32-S3)

Motor Driver The DRV8243 H-bridge driver schematic for DC motor control.

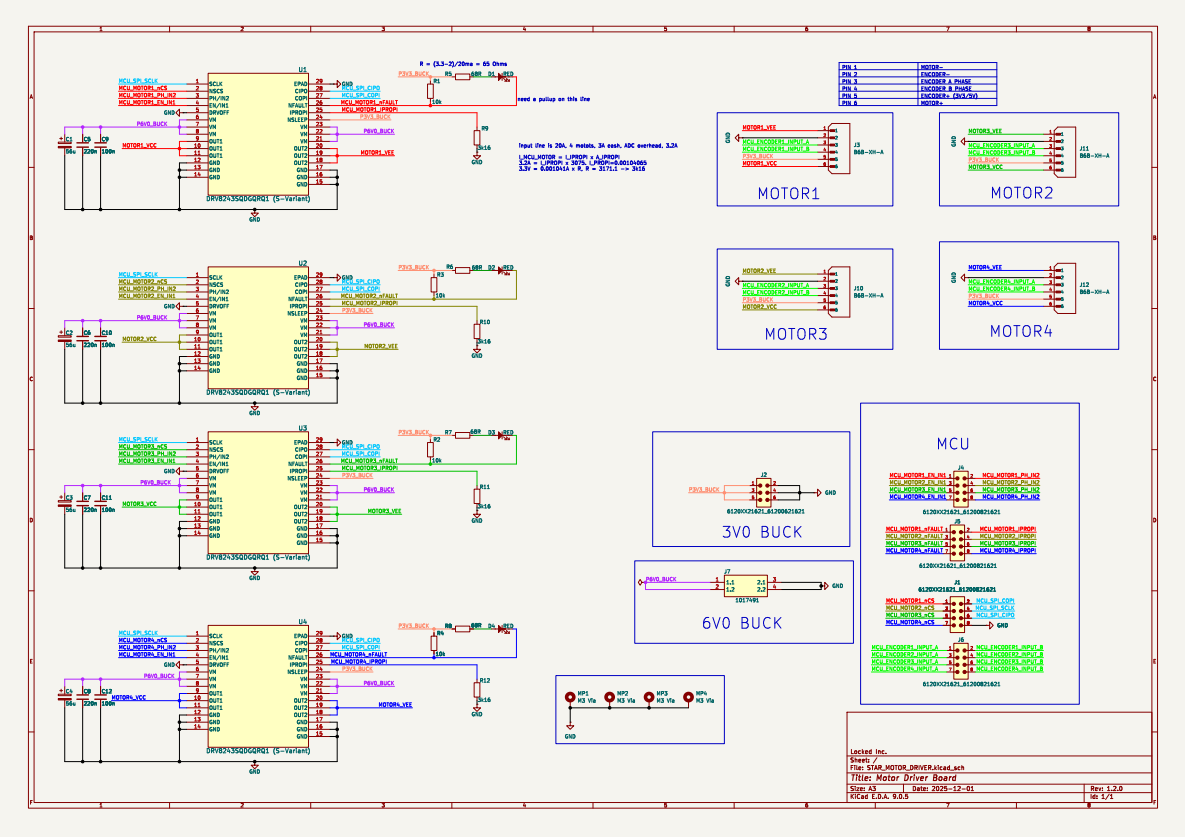


Figure 43: Motor Driver Schematic (DRV8243)

USB-C The TPS25751D USB-C PD controller schematic.

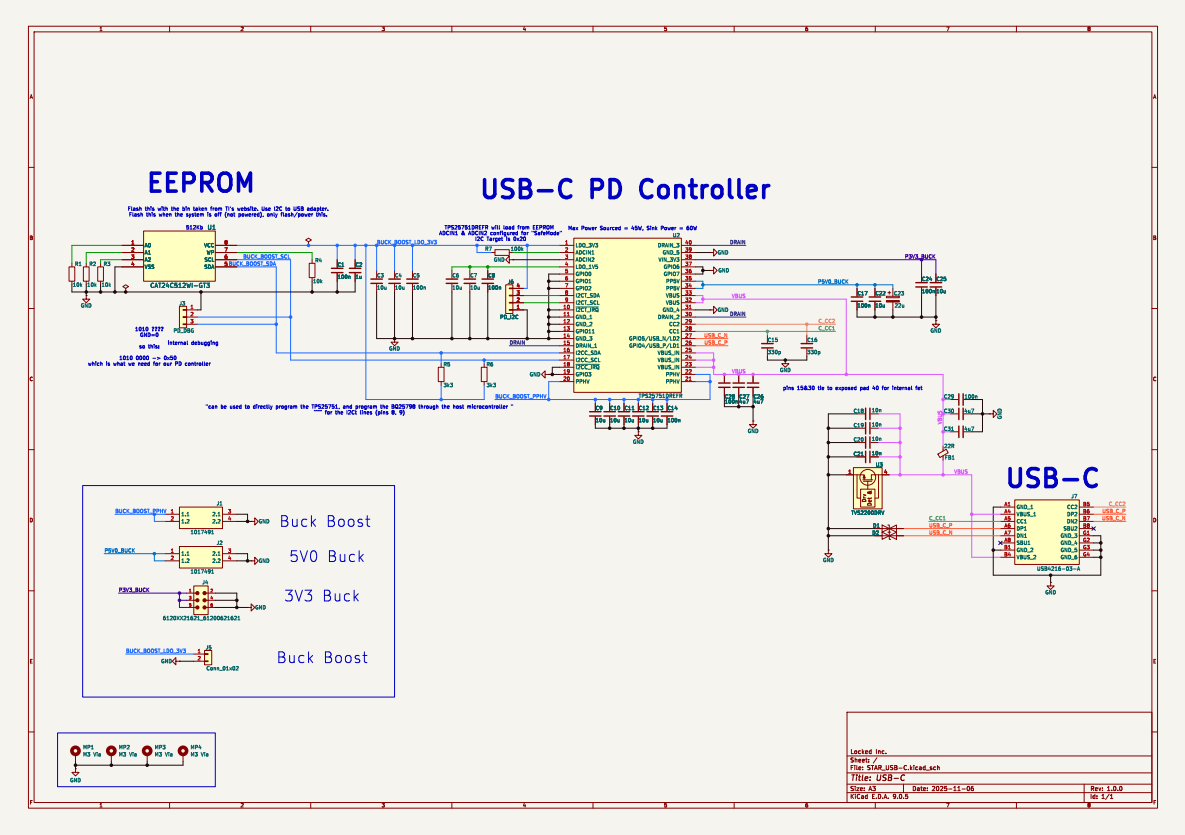


Figure 44: USB-C PD Controller Schematic (TPS25751D)

Fuel Gauge The BQ78350 fuel gauge schematic for battery state estimation.

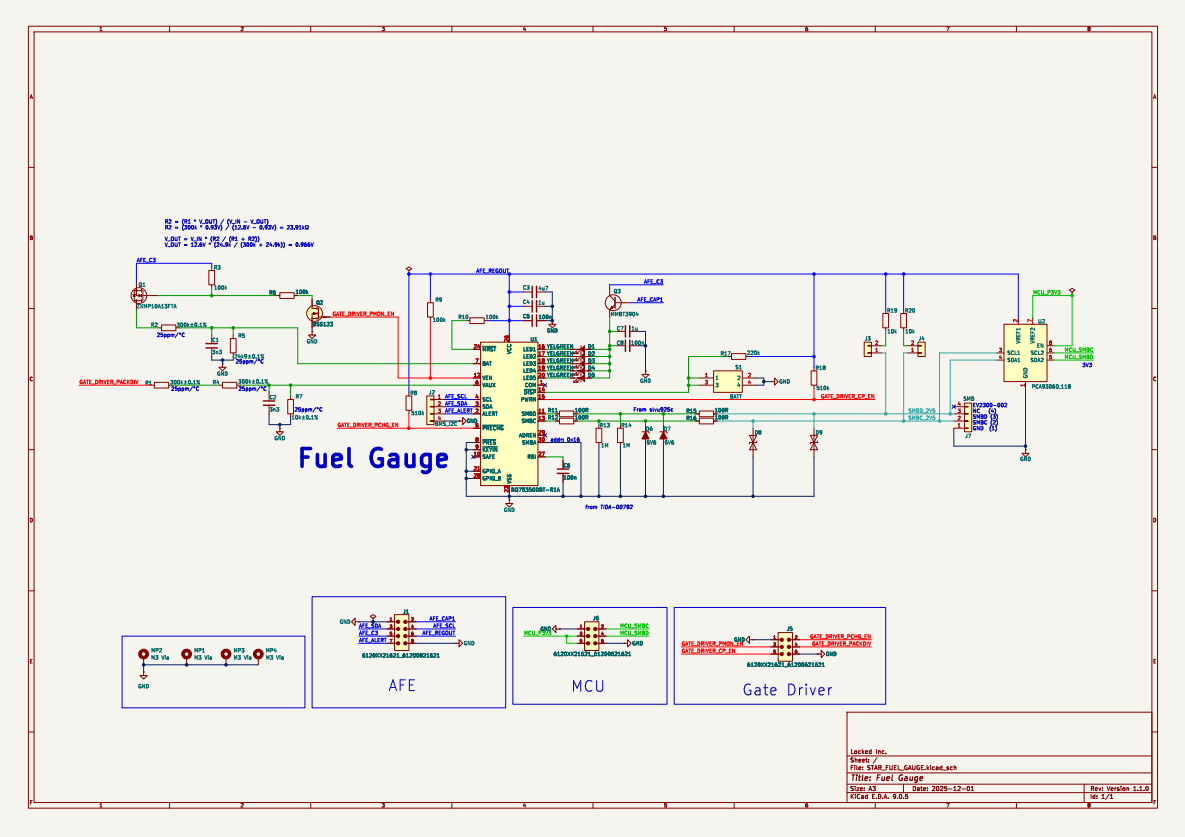


Figure 45: Fuel Gauge Schematic (BQ78350)

Gate Driver The BQ76200 gate driver schematic for power FET control.

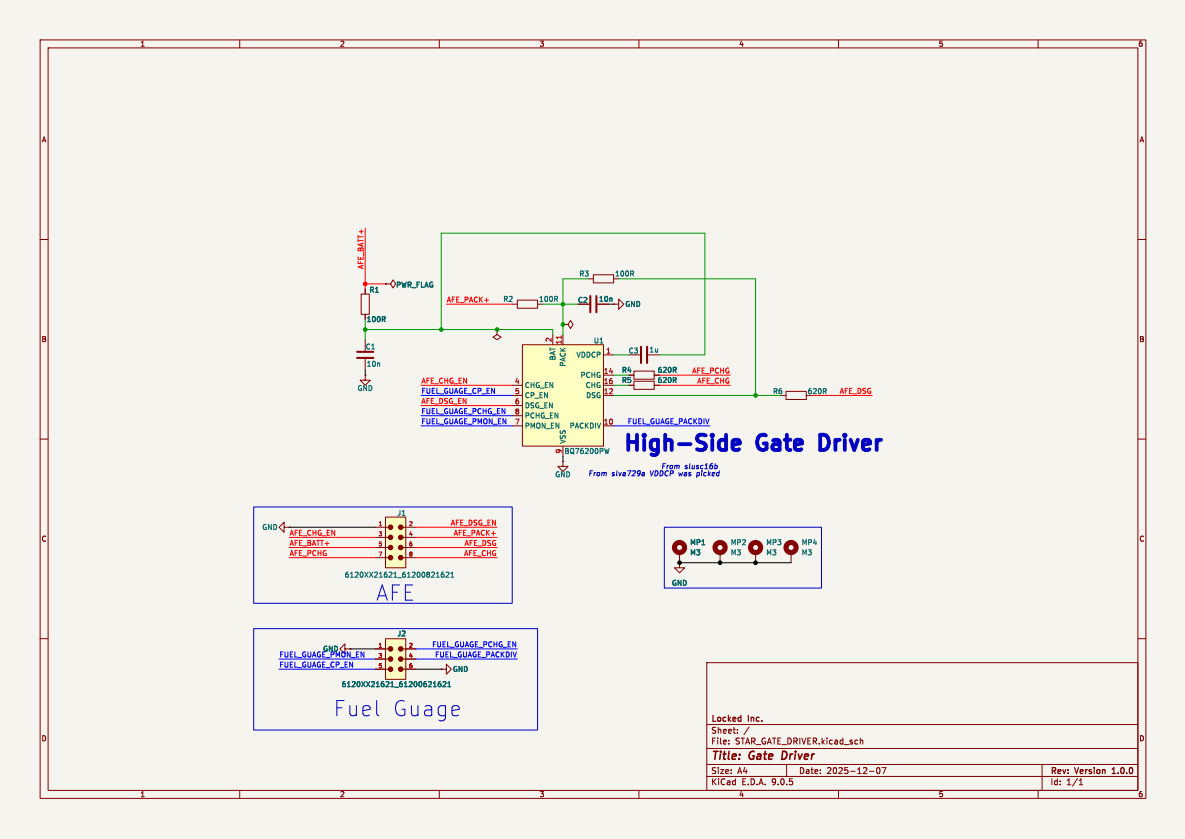


Figure 46: Gate Driver Schematic (BQ76200)

3.3V Buck Regulator The TLV62569 buck regulator schematic for 3.3V rail.

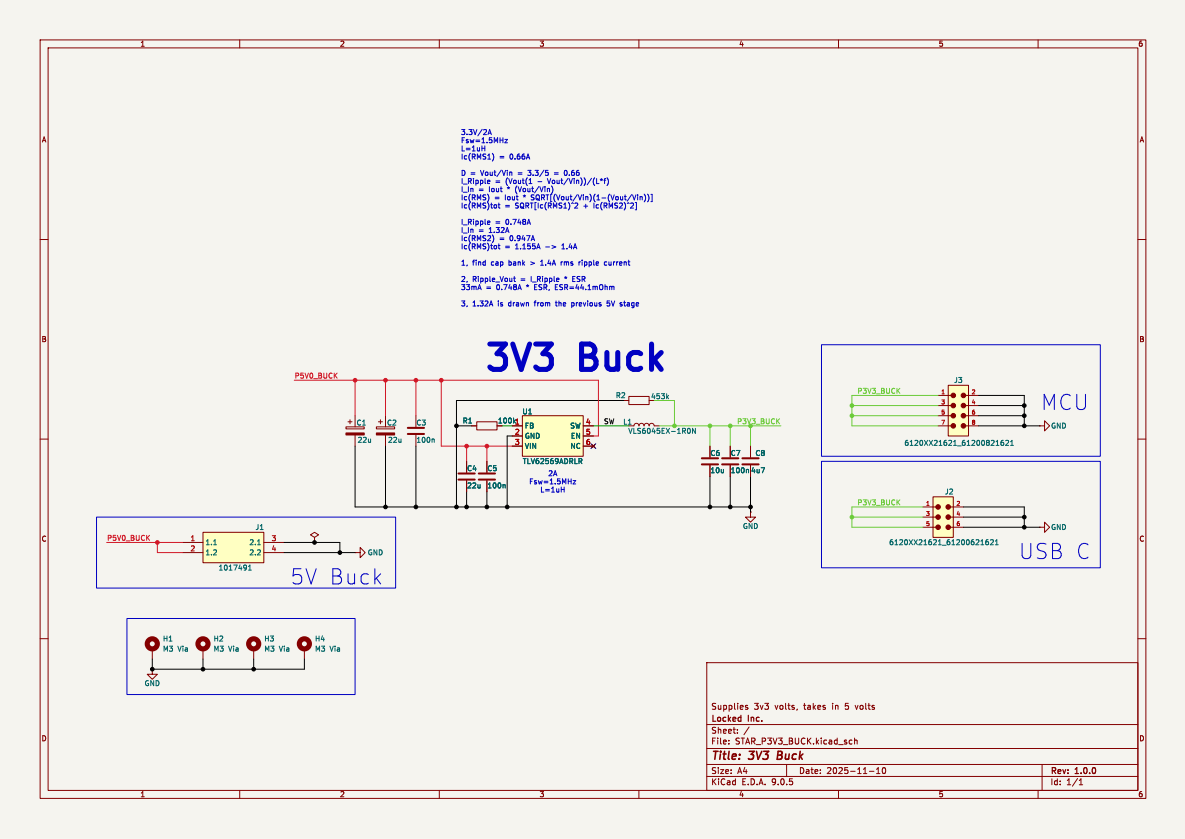


Figure 47: 3.3V Buck Regulator Schematic (TLV62569)

5V Buck Regulator The TPS566238 buck regulator schematic for 5V rail.

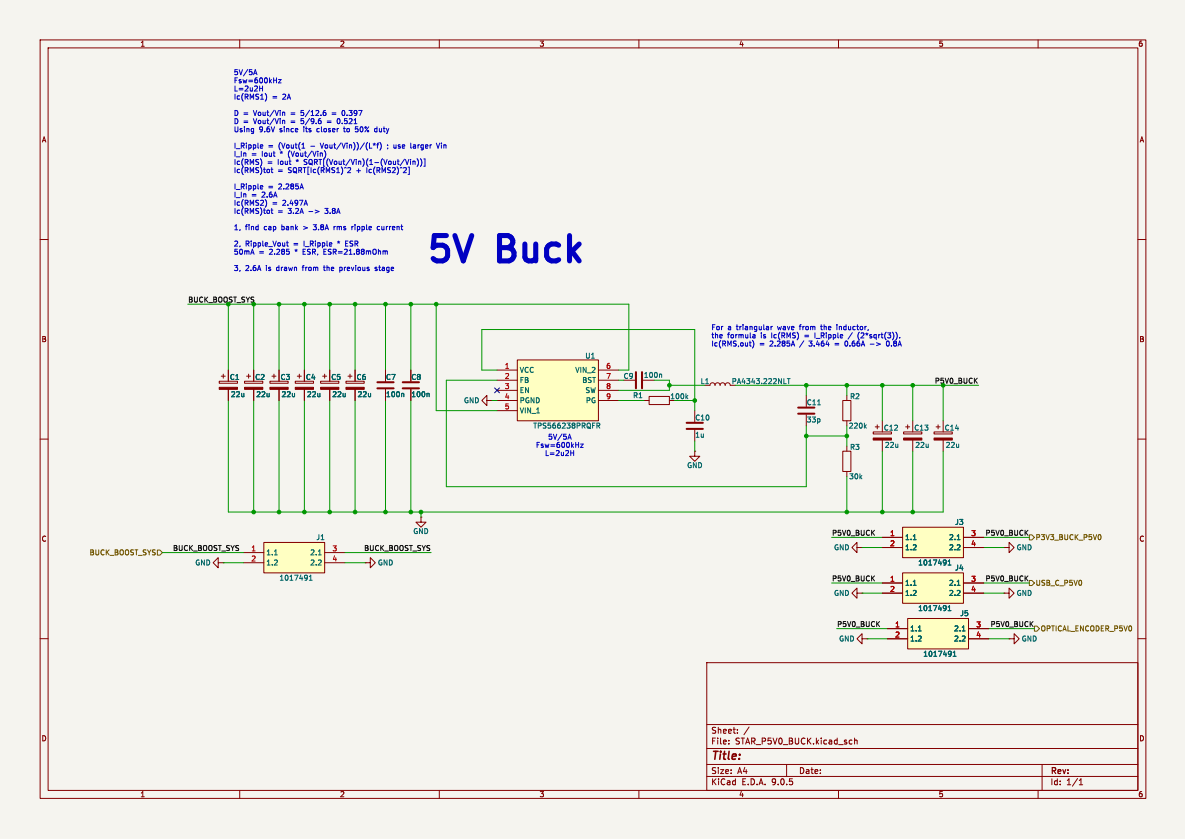


Figure 48: 5V Buck Regulator Schematic (TPS566238)

6V Buck Regulator The TPS40305 buck regulator schematic for 6V motor rail.

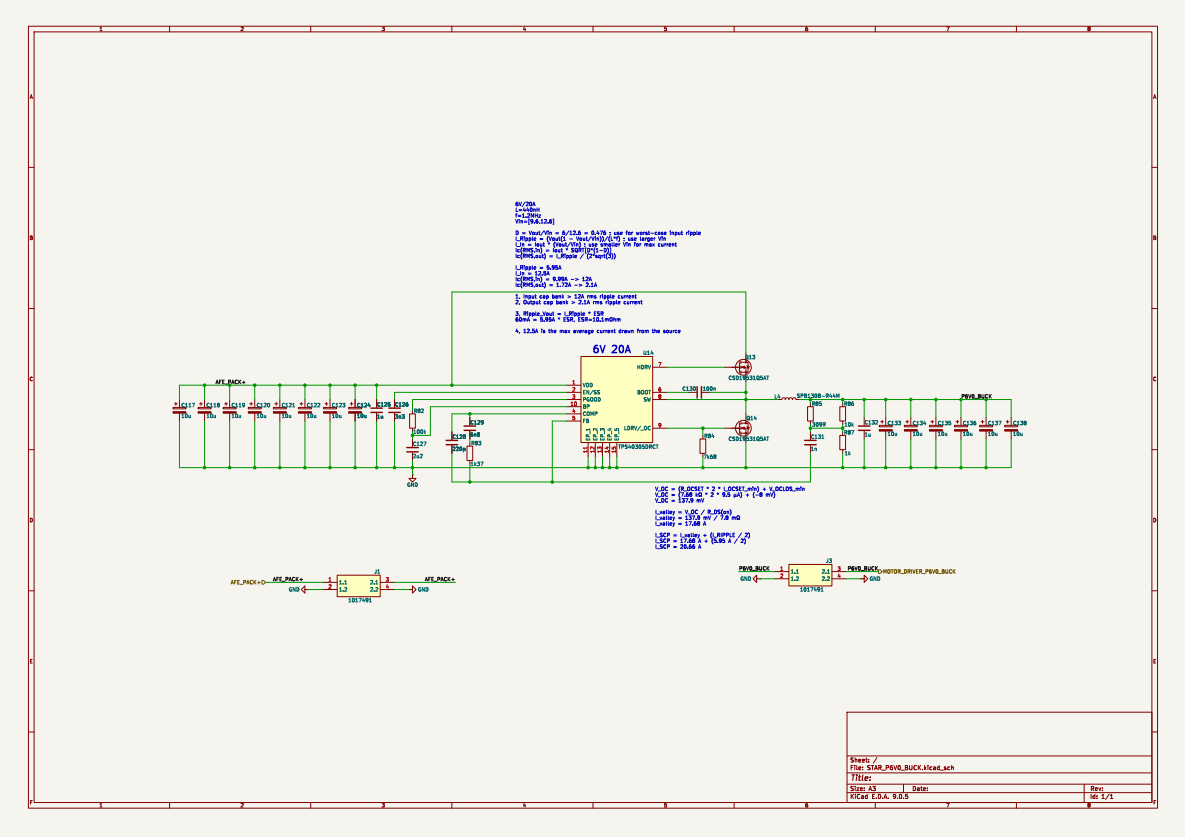


Figure 49: 6V Buck Regulator Schematic (TPS40305)

LiDAR Breakout The LiDAR breakout board schematic for sensor interfacing.

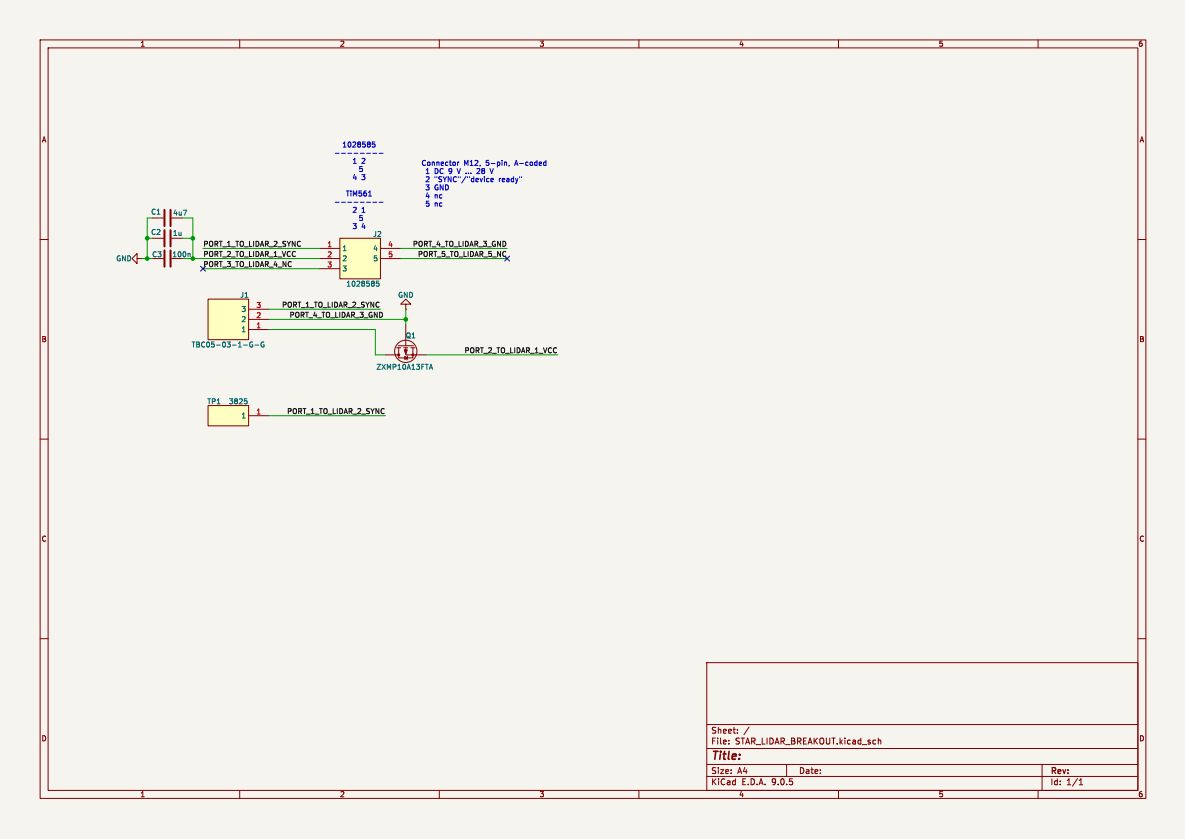


Figure 50: LiDAR Breakout Board Schematic

15.2.3 Printed Circuit Board (PCB) Design

All PCBs were designed in KiCad 9.0. Most boards use 2-layer stackups; the AFE and Motor Driver use 4-layer stackups for improved signal integrity, current handling, and thermal management. DRC passed on all boards.

AFE PCB (4-Layer) The AFE PCB implements the BQ76920 Analog Front End, which provides critical cell voltage monitoring using a 14-bit ADC and hardware protection features such as Overvoltage (OV), Undervoltage (UV), Overcurrent (OCD), and Short Circuit (SCD) detection. This hardware-level protection operates independently of software, ensuring robust battery safety. The 4-layer stackup provides improved signal integrity for the mixed-signal design and better ground plane coverage for noise-sensitive analog measurements.

Figure 51: AFE PCB: Layer 1 (Top Cu) and Layer 2 (In1.Cu)

Figure 52: AFE PCB: Layer 3 (In2.Cu) and Layer 4 (Bottom Cu)

Buck-Boost Charger PCB The Buck-Boost Charger PCB integrates the BQ25798 buck-boost charger, crucial for managing 3S Li-ion battery charging and providing the SYS output for downstream regulators. Its buck-boost topology efficiently handles USB PD voltages, adapting seamlessly whether the input voltage is above or below the battery pack voltage. The PCB is designed to optimize power flow and thermal performance for reliable charging.

Figure 53: Buck-Boost Charger PCB Layout (Top and Bottom)

MCU PCB The MCU PCB is built around the ESP32-S3 microcontroller, specifically chosen for its role in real-time motor control. This board handles time-critical operations such as motor PWM generation and sensor polling, tasks that require precise timing and cannot tolerate the latency variability of a Linux-based system. Its design prioritizes efficient signal routing and robust power delivery for the microcontroller.

Motor Driver PCB (4-Layer) The Motor Driver PCB features the DRV8243 H-bridge drivers, enabling independent control of the four DC gear motors for the robot's differential drive system. This board utilizes a 4-layer stackup, a critical design choice for improved current handling capabilities and enhanced thermal management, ensuring stable and reliable motor operation under varying load conditions. The motor driver uses a 4-layer stackup for improved current handling and thermal management.

Figure 54: Motor Driver PCB: Layer 1 (Top Cu) and Layer 2 (In1.Cu)

Figure 55: Motor Driver PCB: Layer 3 (In2.Cu) and Layer 4 (Bottom Cu)

USB-C PCB The USB-C PCB incorporates the TPS25751D Dual Role Power (DRP) controller, serving as the primary interface for power input to the system. This controller enables versatile power management, allowing the robot to either sink power for efficient battery charging or source power to the Raspberry Pi 5. The board is designed to handle up to 100W (20V/5A) for charging and provide 5V/3A (15W) when sourcing power.

Figure 56: USB-C PCB Layout (Top and Bottom)

Fuel Gauge PCB The Fuel Gauge PCB is dedicated to accurate battery state-of-charge (SOC) and state-of-health (SOH) estimation, utilizing the BQ78350 fuel gauge IC with its CEDV algorithm. This component provides vital battery telemetry via an SMBus interface to the host controller, offering real-time insights into battery performance and remaining capacity. The PCB design focuses on precise measurement and reliable communication.

Figure 57: Fuel Gauge PCB Layout (Top and Bottom)

Gate Driver PCB The Gate Driver PCB integrates the BQ76200, a specialized high-side N-FET driver with an integrated charge pump. This component is essential for robust power path control, managing the Charge (CHG), Discharge (DSG), and Precharge (PCHG) FETs. The PCB is engineered to provide efficient and reliable switching of these power FETs, crucial for battery protection and power management.

Figure 58: Gate Driver PCB Layout (Top and Bottom)

Power Regulator PCBs **Note:** The 3.3V, 5V, 6V buck regulators and buck-boost charger are currently being redesigned into a single unified power board to reduce complexity and improve integration.

3.3V Buck PCB The 3.3V Buck Regulator PCB utilizes the TLV62569ADRLR buck converter to efficiently generate a stable 3.3V power rail from the SYS output. This rail supplies critical low-power components such as the ESP32 microcontroller, IMU, and various sensors. The PCB is designed for high efficiency and low noise, providing clean power for sensitive digital and analog circuitry.

Figure 59: 3.3V Buck Regulator PCB Layout (Top and Bottom)

5V Buck PCB The 5V Buck Regulator PCB integrates the TPS566238PRQFR buck converter, responsible for producing a stable 5V power rail from the SYS output. This 5V rail is vital for powering components like the Raspberry Pi 5, encoders, and other USB PD related circuitry. The design prioritizes high current capability and stable voltage regulation for these essential system elements.

Figure 60: 5V Buck Regulator PCB Layout (Top and Bottom)

6V Buck PCB The 6V Buck Regulator PCB employs the TPS40305DRCT buck converter to supply a robust 6V power rail. This rail is primarily dedicated to powering the four DC motors, drawing up to 25A maximum from the PACK+ (AFE) output. The PCB design emphasizes high current delivery, thermal dissipation, and efficient power conversion to meet the demanding requirements of motor actuation.

LiDAR Breakout PCB The LiDAR Breakout PCB serves as an essential interface board for integrating the 2D LiDAR sensors into the system, including both the SICK TiM561 (primary) and

RPLiDAR C1 (backup). This PCB facilitates the necessary electrical connections and signal conditioning for these sensors, which provide critical data for SLAM and environmental perception. Its design ensures reliable communication and power delivery to the LiDAR units.

Figure 61: LiDAR Breakout PCB Layout (Top and Bottom)

15.2.4 3D PCB Renders

High-quality 3D renders were generated from KiCad to visualize component placement and board assembly. Boards with incomplete PCB layouts (MCU, 6V Buck) are excluded pending design updates.

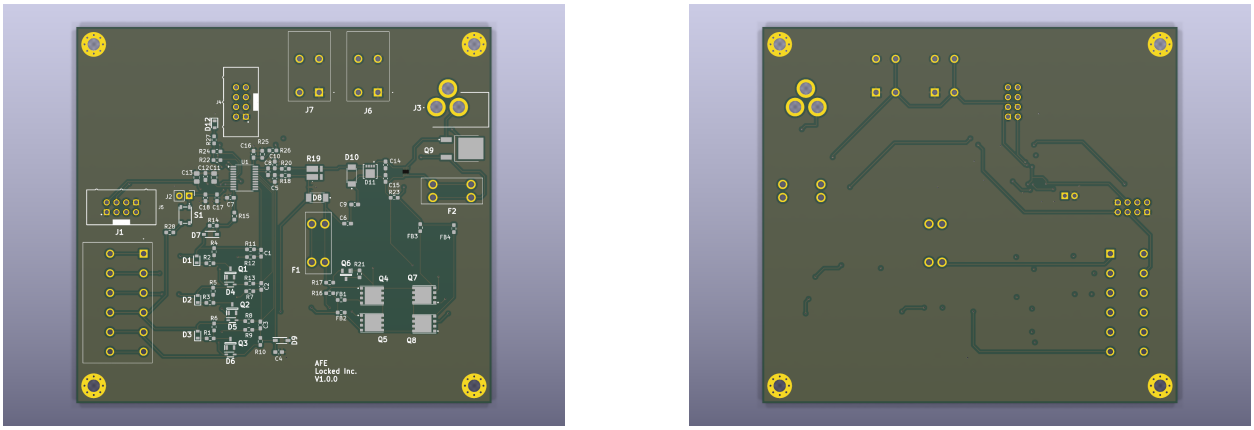


Figure 62: AFE Board 3D Render (Top and Bottom)

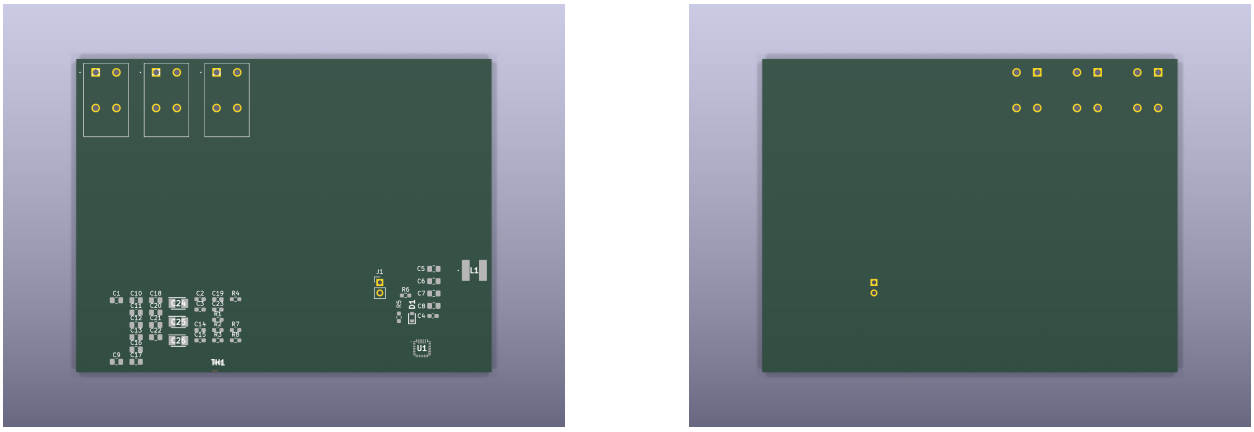


Figure 63: Buck-Boost Charger Board 3D Render (Top and Bottom)

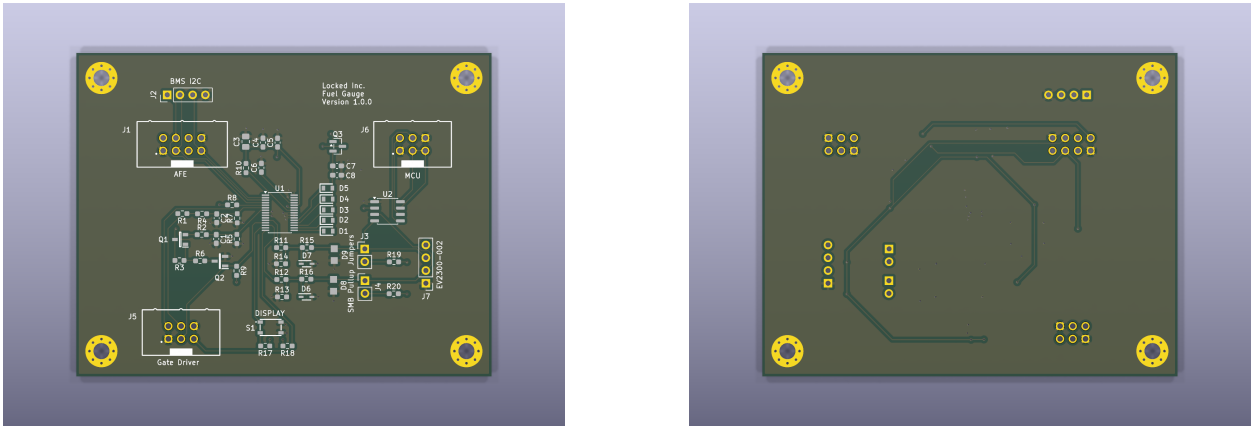


Figure 64: Fuel Gauge Board 3D Render (Top and Bottom)

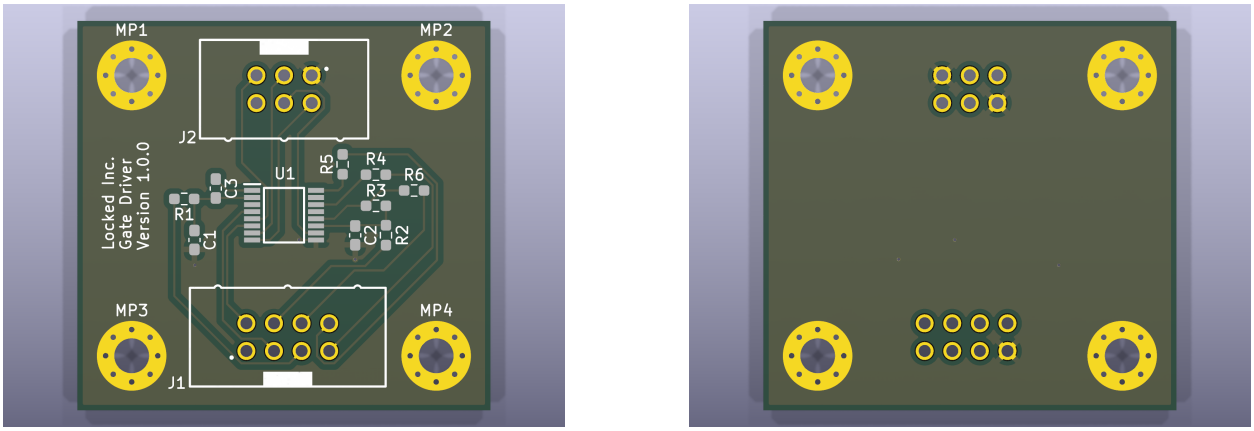


Figure 65: Gate Driver Board 3D Render (Top and Bottom)

Power Management Boards

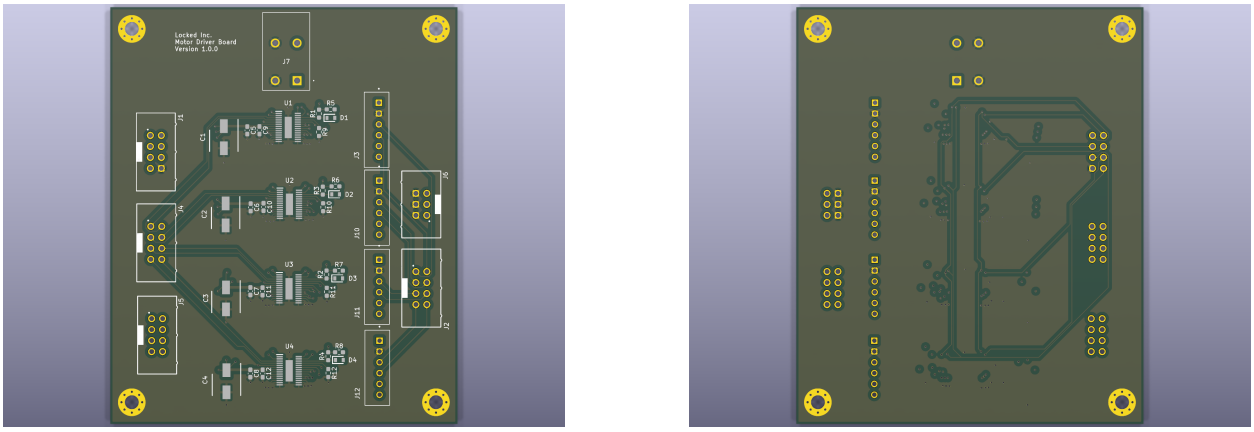


Figure 66: Motor Driver Board 3D Render (Top and Bottom)

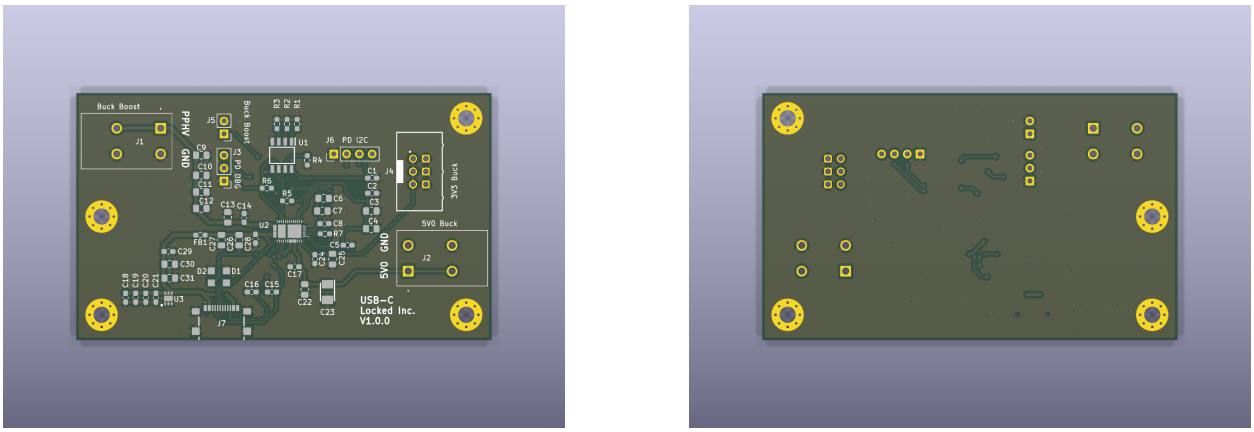


Figure 67: USB-C Board 3D Render (Top and Bottom)

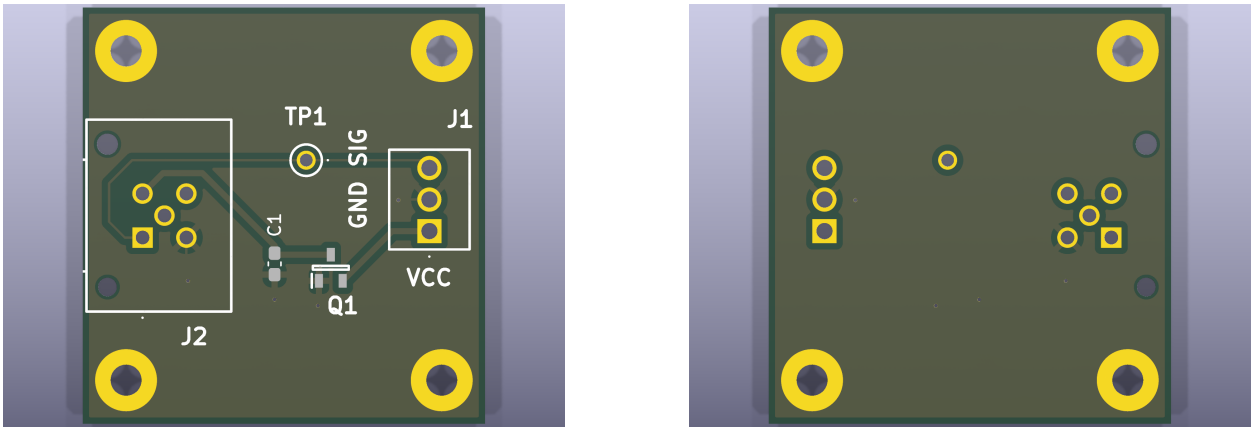


Figure 68: LiDAR Breakout Board 3D Render (Top and Bottom)

Control and Interface Boards

Power Regulator Boards **Note:** These boards are being consolidated into a unified power board.

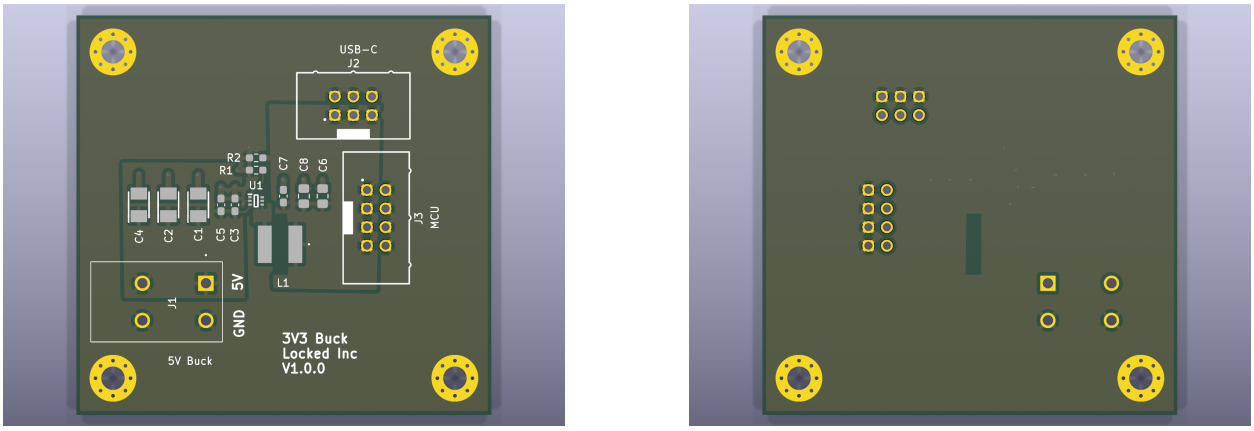


Figure 69: 3.3V Buck Regulator Board 3D Render (Top and Bottom)

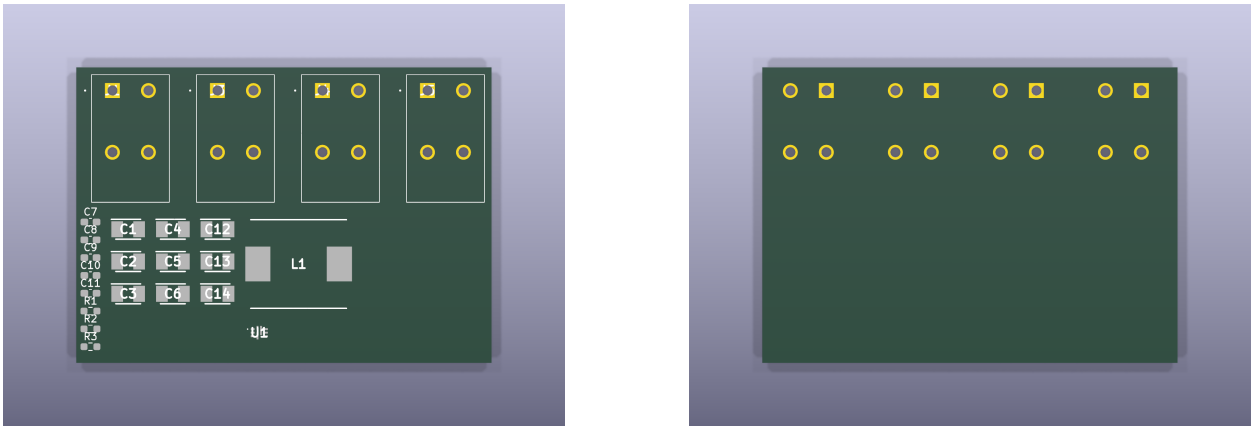


Figure 70: 5V Buck Regulator Board 3D Render (Top and Bottom)

15.2.5 Bill of Materials

Complete BOMs were exported from KiCad 9.0 for each PCB with the following fields:

- Reference designator, Value, Footprint, Quantity
- Manufacturer name and part number (MPN)
- Digi-Key part number and unit price
- Mouser part number and unit price
- Component description

Full BOMs are provided in CSV format in the project repository (`figures/bom_full/`).

Table 60: PCB Layer Configuration

Board	Layers	Stackup	Rationale
AFE	4	F.Cu / In1.Cu / In2.Cu / B.Cu	Mixed-signal integrity, ground plane coverage
Motor Driver	4	F.Cu / In1.Cu / In2.Cu / B.Cu	High current (12A), thermal management
Buck-Boost Charger	2	F.Cu / B.Cu	Power stage, adequate copper area
Fuel Gauge	2	F.Cu / B.Cu	Digital + analog, low power
Gate Driver	2	F.Cu / B.Cu	Simple gate drive circuit
MCU	2	F.Cu / B.Cu	Digital signals, connectors
USB-C	2	F.Cu / B.Cu	USB PD controller, standard routing
3.3V Buck	2	F.Cu / B.Cu	Low-power regulator
5V Buck	2	F.Cu / B.Cu	Medium-power regulator
6V Buck	2	F.Cu / B.Cu	Motor power regulator
LiDAR Breakout	2	F.Cu / B.Cu	Simple interface board

PCB Layer Stackup Summary

Breadboard Testing Limitations Breadboards cannot be used for prototyping or testing the power electronics in this project. The 6V motor rail must deliver up to 25A peak current (four motors at 3.2A stall each), while standard breadboard contact clips are rated for approximately 1A. Exceeding this limit causes resistive heating at the contacts, leading to unreliable connections and potential fire hazards. This current limitation necessitated moving directly to custom PCB fabrication for the power distribution, motor driver, and battery management subsystems. Phase 1 hardware verification in Section 16.0 (Table 93) therefore focuses on PCB-level tests (TC-PCB-001 through TC-PCB-006) to verify solder quality and trace continuity under load conditions that breadboards cannot safely support.

Table 61: BOM Summary by Board

Board	Layers	Components	Primary IC	Package
AFE	4	81	BQ76920PW	TSSOP-20
Buck-Boost Charger	2	41	BQ25798RQMR	QFN-32
Fuel Gauge	2	46	BQ78350DBT-R1A	TSSOP-30
Gate Driver	2	12	BQ76200PW	TSSOP-14
MCU	2	42	ESP32-S3-WROOM-1-N16	Module
Motor Driver	4	42	DRV8243SQDGQRQ1 (4×)	WQFN-29
USB-C	2	49	TPS25751DREFR	QFN-64
3.3V Buck	2	15	TLV62569ADRLR	SOT-5X3
5V Buck	2	23	TPS566238PRQFR	QFN-18
6V Buck	2	34	TPS40305DRCT	SON-10
LiDAR Breakout	2	5	–	–
Total	–	**390**	–	–

Component Count Summary

Table 62: Motor Driver Board BOM with Pricing

Ref	Value	Qty	MPN	DigiKey	Mouser
U1–U4	DRV8243S	4	DRV8243SQDGQRQ1	$4.72	$6.37
C1–C4	56μF	4	865080343007	$0.26	$0.27
C5–C8	220nF	4	CL10B224KB8NNNC	$0.11	$0.11
C9–C12	100nF	4	CC603JRX7R9BB104	$0.10	$0.10
R1–R4	10kΩ	4	RT0603BRE0710KL	$0.14	$0.10
R5–R8	68Ω	4	RT0603FRE0768RL	$0.10	$0.10
R9–R12	3.16kΩ	4	ERA-3AEB3161V	$0.10	$0.10
D1–D4	Red LED	4	150060RS75000	$0.15	$0.16
J3,J10–12	B6B-XH-A	4	B6B-XH-A (JST)	–	–
J7	Phoenix 2P	1	1017491	–	–

Motor Driver BOM (42 components)

Table 63: USB-C Board BOM with Pricing

Ref	Value	Qty	MPN	DigiKey	Mouser
U2	TPS25751D	1	TPS25751DREFR	$2.90	$2.91
U1	EEPROM	1	CAT24C512WI-GT3	$0.59	$0.59
U3	TVS	1	TVS2200DRVR	$0.53	$0.83
J7	USB-C	1	USB4216-03-A	$0.62	$0.72
C3–C13	10μF	11	GRM21BR61H106KE3L	$0.28	$0.28
C23	22μF	1	T527T226M025ATE100	N/A	$1.13
C26–C31	4.7μF	4	GRT21BR61H475KE3K	$0.17	$0.17
D1,D2	TVS	2	82306050029 (Würth)	$0.73	$0.73
R1–R4	10kΩ	4	RT0603BRE0710KL	$0.14	$0.10

USB-C PD Controller BOM (49 components)

Table 64: MCU Board BOM with Pricing

Ref	Value	Qty	MPN	DigiKey	Mouser
U2	ESP32-S3	1	ESP32-S3-WROOM-1-N16	$3.50	$3.50
U1	Temp Sensor	1	DS18B20+	$4.25	$4.25
U3	Decoder	1	74HC139D,653	$0.45	$0.45
U4	Mux	1	SN74HC153DR	$0.35	$0.35
J1–J7	HC-SR04	7	4-pin header	$0.10	$0.10
J8	Pi GPIO	1	2x20 header	$1.50	$1.50
D1,D2	ESD	2	82306050029	$0.73	$0.73
R1–R8	10kΩ	8	RT0603BRE0710KL	$0.14	$0.10

MCU Board BOM (42 components)

Table 65: Power Board BOM Summary with Key Component Pricing

Board	Primary IC	Qty	MPN	DigiKey	Mouser
Buck-Boost Charger	BQ25798	41	BQ25798RQMR	$5.50	$5.50
Fuel Gauge	BQ78350	46	BQ78350DBT-R1A	$8.25	$8.25
Gate Driver	BQ76200	12	BQ76200PW	$2.15	$2.15
AFE	BQ76920	81	BQ7692000PWR	$3.75	$3.75
3.3V Buck	TLV62569	15	TLV62569ADRLR	$1.25	$1.25
5V Buck	TPS566238	23	TPS566238PRQFR	$1.85	$1.85
6V Buck	TPS40305	34	TPS40305DRCT	$2.45	$2.45

Power Management BOMs Complete BOMs with all component fields for each PCB are provided in Appendix 17.9.

15.3 Embedded Preliminary Implementation

Section Author(s): BS, JH

This section covers embedded systems architecture, communication protocols, and motor control implementation. See Section 15.2 for hardware specifications and schematics.

15.3.1 System Architecture Overview

The distributed architecture separates high-level processing from real-time control:

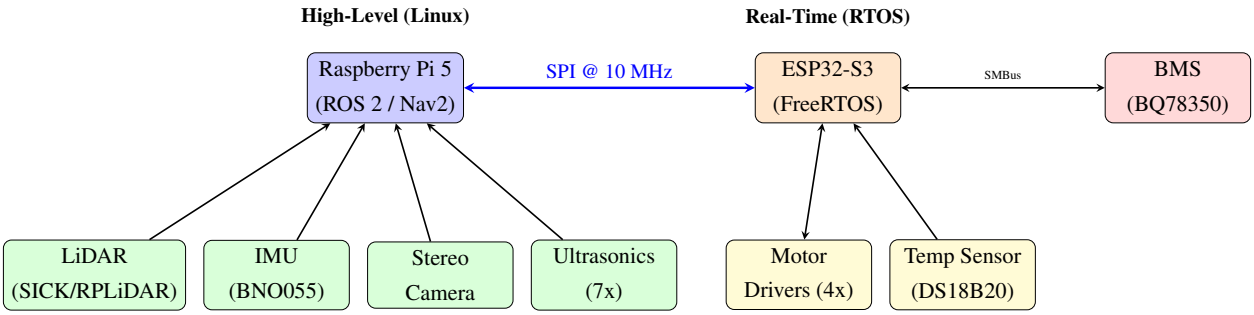


Figure 71: Distributed Compute Architecture

15.3.2 Communication Protocols

Three communication buses connect system components:

Table 66: Communication Bus Summary

Bus	Speed	Controller	Peripheral	Purpose
SPI-1	10 MHz	Pi 5	ESP32	Velocity commands, sensor data
SPI-2	–	ESP32	DRV8243	Motor config, fault status
SMBus	100 kHz	ESP32	BQ78350	Battery SOC, alerts
1-Wire	–	ESP32	DS18B20	Temperature compensation

Table 67: SPI-1 Pin Mapping (Pi 5 $\leftrightarrow$ ESP32)

Signal	Pi 5 GPIO	ESP32 GPIO
COPI (Controller Out)	GPIO11	GPIO13
CIPO (Controller In)	GPIO9	GPIO12
SCLK	GPIO11	GPIO14
CS	GPIO8	GPIO15

SPI Bus Configuration The N16 variant (no PSRAM) frees GPIO 35–37 for SPI-2 to connect to motor drivers. Firmware uses a unified bus abstraction layer for driver interchangeability.

SMBus for Battery Management SMBus (vs. I2C) provides BMS-specific features (System Management Interface Forum; Analog Devices, Guide to Comparing I2C Bus to the SMBus):

Table 68: SMBus Features for BMS

Feature	Benefit
Mandatory Timeout	25–35 ms timeout prevents bus lockups
Packet Error Check	CRC-8 checksum for noisy environments
SMBALERT#	Interrupt-driven fault notification (OV, OC, thermal)
SBS Commands	Standardized SOC, voltage, current, temperature queries

Temperature Compensation The DS18B20 provides $\pm 0.5^{\circ}$C accuracy for ultrasonic speed-of-sound compensation (DS18B20 Programmable Resolution 1-Wire Digital Thermometer):

$$v_{sound} = 331.3 + 0.606 \cdot T_{celsius} \text{ m/s} \tag{1}$$

This improves ultrasonic distance accuracy from $\pm 3\%$ to $\pm 0.5\%$.

15.3.3 Motor Control Implementation

The ESP32's MCPWM peripheral drives four DRV8243 H-bridges (see Section 15.0 for motor/-driver specs).

Center-Aligned PWM MCPWM uses center-aligned (up-down count) rather than edge-aligned PWM (Espressif Systems, Motor Control Pulse Width Modulator (MCPWM) - ESP-IDF Programming Guide; Texas Instruments, So, Which PWM Technique is Best?).

PWM Fundamentals PWM controls motor speed by rapidly switching power on and off. The *duty cycle* (percentage of time the output is HIGH) determines average voltage delivered to the motor. Hardware PWM uses a counter that continuously increments and a compare register holding the duty cycle threshold. When counter $<$ threshold, output is HIGH; when counter $\geq$ threshold, output is LOW.

Edge-Aligned vs. Center-Aligned The key difference is how the counter behaves. Each motor has a different duty cycle at any given moment (one motor might be at 25%, another at 75%, depending on how the robot is turning). The comparison rule is: output = HIGH when counter $<$ threshold.

Edge-aligned (sawtooth): Counter counts from 0 to MAX, then instantly resets to 0. When the counter resets, it goes below *all* thresholds simultaneously, causing all outputs to go HIGH at the same instant. Both motors turn ON at reset, creating a current spike.

Center-aligned (triangle): Counter counts from 0 to MAX, then back down to 0. There is no instant reset. The output toggles twice per cycle (once on the way up, once on the way down), and since each motor has a different threshold, they cross at different times.

Figure 72 illustrates this with two motors at 25% and 75% duty cycles. Edge-aligned bunches two transitions at reset (current spike), while center-aligned spreads four transitions across the entire cycle.

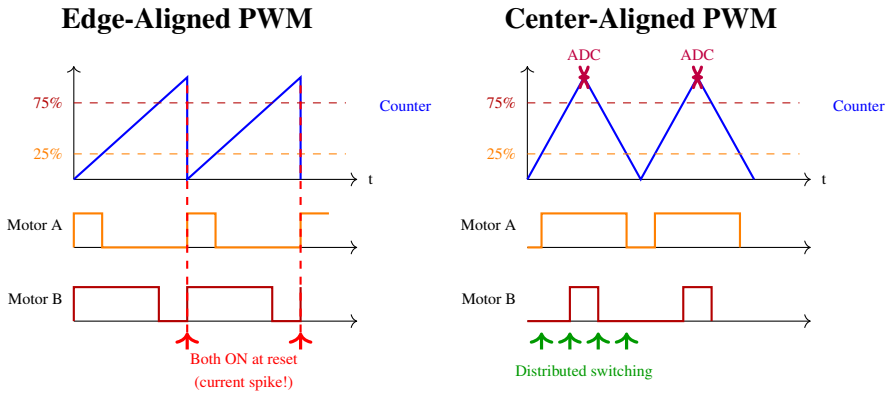


Figure 72: Edge-Aligned vs. Center-Aligned PWM with Two Motors (25% and 75% duty)

ADC Sampling Window At the triangle wave peak, the counter has reached MAX and is about to count down. At this instant, all PWM outputs are in a stable state (no transitions occurring)

because the counter is above all duty cycle thresholds. This "quiet zone" is ideal for sampling motor current via ADC, as switching noise is minimized.

Why This Matters With four motors drawing up to 3.2A each at stall, simultaneous switching in edge-aligned mode would create 12.8A current spikes. Center-aligned PWM staggers these transitions, reducing peak current draw, lowering EMI (Electromagnetic Interference), and enabling accurate current sensing for overcurrent protection.

Table 69: Center-Aligned PWM Benefits

Benefit	Mechanism
Reduced current spikes	Switching events staggered across time, lowering peak di/dt
Lower EMI	Distributed transitions reduce high-frequency harmonic content
Accurate current sensing	Quiet period at counter peak allows noise-free ADC sampling
Smoother torque	Symmetric pulse placement reduces odd harmonics in motor current

MCPWM also provides dead-time insertion (prevents H-bridge shoot-through), hardware fault inputs for OCP, and multi-phase synchronization (Espressif Systems, Motor Control Pulse Width Modulator (MCPWM) - ESP-IDF Programming Guide).

Encoder Feedback Quadrature encoders (341.2 counts/rev) connect to Pi 5 GPIO for closed-loop velocity control and odometry.

15.3.4 Sensor Integration

See Section 15.0 for sensor specifications. Key integration notes:

Table 70: Sensor Integration Summary

Sensor	Interface	Implementation Notes
HC-SR04 (7×)	Pi 5 GPIO	Temperature-compensated; staggered triggering avoids acoustic interference
BNO055 + BMP280	I^2C	Onboard fusion outputs quaternions; BMP280 adds barometric data
SICK TiM561	Pi 5 Ethernet	Primary LiDAR for SLAM
RPLiDAR C1	Pi 5 UART	Backup 360° coverage
IMX219-83 Stereo	Pi 5 USB	Dual 8 MP cameras for depth/visual odometry
A7670G 4G	Pi 5 UART/USB	LTE CAT1 (5/10 Mbps up/down) for remote connectivity

15.3.5 STAR Firmware Framework

The ESP32 firmware implements a production-ready embedded framework called **STAR** (Sensor and Actuator Abstraction Runtime). This modular architecture follows SOLID principles to ensure loose coupling, testability, and maintainability across all subsystems.

SOLID Principles Overview SOLID is an acronym for five object-oriented design principles that promote maintainable, flexible software:

- **S**ingle Responsibility: Each module has one reason to change
- **O**pen/Closed: Open for extension, closed for modification
- **L**iskov Substitution: Subtypes must be substitutable for their base types
- **I**nterface Segregation: Many specific interfaces over one general-purpose interface
- **D**ependency Inversion: Depend on abstractions, not concretions

While SOLID originated in object-oriented programming, these principles apply equally to embedded C through careful use of function pointers and opaque types. The STAR framework particularly emphasizes the **Dependency Inversion Principle (DIP)**.

The Dependency Inversion Principle DIP addresses a fundamental problem in embedded systems: *tight coupling between application logic and hardware access*. Traditional embedded C code often looks like this:

This approach makes unit testing impossible (requires real hardware), prevents hardware swapping (changing I2C to SPI requires rewriting the driver), and creates circular dependencies.

Code Snippet 1: Traditional Coupled Approach (Anti-Pattern)

```c
/* sensor_driver.c - Tightly coupled to specific I2C implementation */
float read_temperature(void) {
    uint8_t data[2];
    i2c_master_read(I2C_NUM_0, 0x48, data, 2);  /* Hardcoded */
    return convert_to_celsius(data);
}
```

DIP states: *"High-level modules should not depend on low-level modules. Both should depend on abstractions."* In our implementation, sensor drivers depend on abstract bus interfaces, not concrete I^2C/SPI implementations:

Code Snippet 2: DIP Approach with Abstract Interface

```c
/* sensor_driver.c - Depends on abstraction, not implementation */
float read_temperature(star_bus_manager_t* bus_mgr, const char* bus_name) {
    uint8_t data[2];
    star_bus_read(bus_mgr, bus_name, 0x48, data, 2);  /* Abstracted */
    return convert_to_celsius(data);
}
```

The bus manager internally dispatches to the correct protocol (I^2C, SPI, GPIO, etc.) based on bus configuration. This enables:

- **Testability**: Pass a mock bus manager that records calls without hardware
- **Swappability**: Change I^2C sensor to SPI by reconfiguring the bus, not rewriting drivers
- **Optional Dependencies**: Components accept NULL interfaces for graceful degradation
- **No Circular Dependencies**: Clean dependency graph from interfaces to implementations

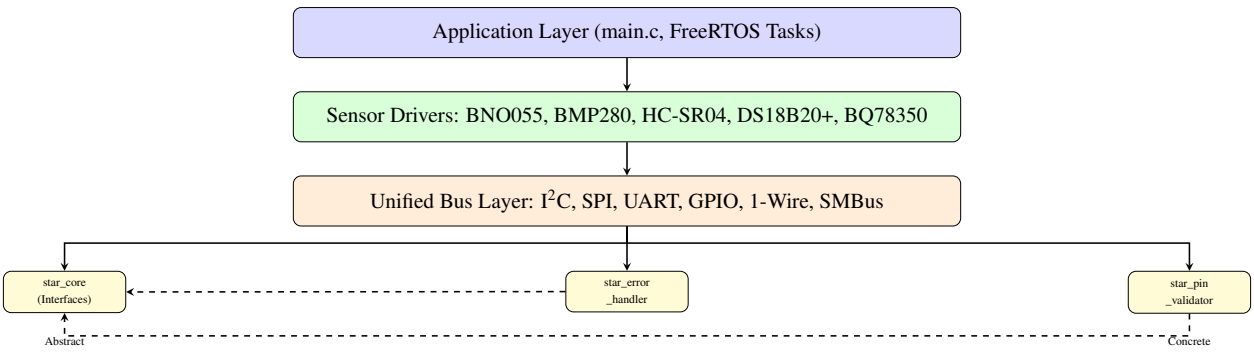


Figure 73: STAR Firmware Layered Architecture

Custom Library Ecosystem The team developed 11 custom libraries totaling over 15,000 lines of C code:

Table 71: STAR Firmware Library Summary

Library	Type	Purpose
`star_core`	Infrastructure	Abstract interfaces (`star_error_interface_t`, `star_pin_interface_t`)
`star_error_handler`	Infrastructure	Error handling with retry logic and exponential backoff
`star_pin_validator`	Infrastructure	GPIO conflict detection at initialization
`star_bus`	Abstraction	Unified bus manager for I^2C, SPI, UART, GPIO, 1-Wire, SMBus
`star_sensor_mpu6050`	Driver	6-axis IMU with DIP interface
`star_sensor_bno055_bmp280`	Driver	10-DOF IMU (9-axis + barometer)
`star_sensor_hcsr04`	Driver	Ultrasonic distance with temperature compensation
`star_sensor_pca9685`	Driver	16-channel PWM controller for LED/servo
`star_bms_bq7850`	Driver	Battery management system SMBus interface
`star_servo`	Utility	Stateless servo angle calculations

Repository File Structure The firmware repository follows a modular structure with clear separation of concerns:

Shared Data Module The `shared_data` module provides thread-safe inter-task communication using FreeRTOS primitives:

Table 72: Shared Data Module Components

Component	Type	Purpose
`sensor_data_queue`	QueueHandle_t	Sensor readings from sensor task to LED task
`temperature_data_queue`	QueueHandle_t	Temperature data from DHT22 task
`health_mutex`	SemaphoreHandle_t	Protects health statistics
`state_mutex`	SemaphoreHandle_t	Protects system state transitions
`system_health_t`	struct	Failure counters, uptime, state
`latest_sensors[]`	sensor_data_t[]	Most recent readings per sensor

Dependency Injection Pattern All components use dependency injection through interface structs with function pointers:

Code Snippet 3: ESP32 Firmware Repository Structure

```
ESP32-Firmware/
|-- src/                          # Application source code
|   |-- main.c                    # Entry point, initialization sequence
|   |-- system_config.c           # Hardware configuration definitions
|   |-- include/
|   |   |-- system_config.h       # Type-safe configuration structures
|   |-- modules/
|   |   |-- shared_data.c         # Thread-safe inter-task communication
|   |   |-- led_controller.c      # Distance-to-color mapping logic
|   |-- tasks/                    # FreeRTOS task implementations
|       |-- dht22_task.c          # Temperature/humidity monitoring
|       |-- sensor_task.c         # HC-SR04 distance measurement
|       |-- led_task.c            # RGB LED visual feedback
|       |-- watchdog_task.c       # System health monitoring
|
|-- lib/                          # Custom libraries (DIP architecture)
|   |-- star_core/                # Abstract interfaces
|   |   |-- star_error_interface.h
|   |   |-- star_pin_interface.h
|   |-- star_bus/                 # Unified bus abstraction
|   |   |-- star_bus_manager.c    # Bus lifecycle management
|   |   |-- star_bus_i2c.c        # I2C protocol implementation
|   |   |-- star_bus_spi.c        # SPI protocol implementation
|   |   |-- star_bus_gpio.c       # GPIO bus implementation
|   |   |-- star_bus_dht22.c      # DHT22 proprietary protocol
|   |-- star_error_handler/       # Error handling with retry logic
|   |-- star_pin_validator/       # GPIO conflict detection
|   |-- star_sensor_hcsr04/       # Ultrasonic sensor driver
|   |-- star_sensor_pca9685/      # PWM controller driver
|   |-- star_sensor_mpu6050/      # 6-axis IMU driver
|   |-- star_sensor_bno055_bmp280/ # 9-axis IMU + barometer
|   |-- star_bms_bq7850/          # Battery management SMBus
|   |-- star_servo/               # Servo angle calculations
|
|-- docs/                         # Generated documentation
|   |-- doxygen/html/             # Doxygen API reference
|   |-- callgraph/                # Call graph analysis
|       |-- visual_levels/        # Hierarchical call graphs
|           |-- callgraph_application.pdf
|           |-- callgraph_framework.pdf
|           |-- callgraph_system.pdf
|           |-- callgraph_complete.pdf
|
|-- platformio.ini                # Build configuration
|-- sdkconfig.esp32_wroom         # ESP32-WROOM config
|-- sdkconfig.esp32s3             # ESP32-S3 config
```

Call Graph Analysis Comprehensive call graph tooling was implemented using GNU cflow, Egypt, Doxygen, and Graphviz to analyze execution flow and validate the architecture. Figure 74 shows the application-level call graph from `app_main()`.

Code Snippet 4: Dependency Injection Pattern in STAR Firmware

```
// 1. Create error handler implementation
error_handler_t* error_handler = error_handler_create_custom(
    2,    // max retries
    10,   // base delay ms
    50,   // max delay ms
    NULL, NULL);

// 2. Get abstract interface from implementation
star_error_interface_t error_iface;
error_handler_get_interface(&error_iface, error_handler);

// 3. Get pin validator interface
star_pin_interface_t pin_iface;
pin_validator_get_interface(&pin_iface);

// 4. Initialize bus manager with injected interfaces
star_bus_manager_t bus_manager;
star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);

// 5. Create and add I2C bus for sensors
star_bus_config_t* i2c_bus = star_bus_config_create_i2c(
    "sensor_bus", I2C_NUM_0, 0x68, GPIO_NUM_21, GPIO_NUM_22, 400000);
star_bus_manager_add_bus(&bus_manager, i2c_bus);

// 6. Initialize sensor driver with bus manager reference
pca9685_handle_t pca_handle;
star_sensor_pca9685_init(&pca_handle, &bus_manager, "sensor_bus",
                         &error_iface, GPIO_NUM_4, false, &config);
```

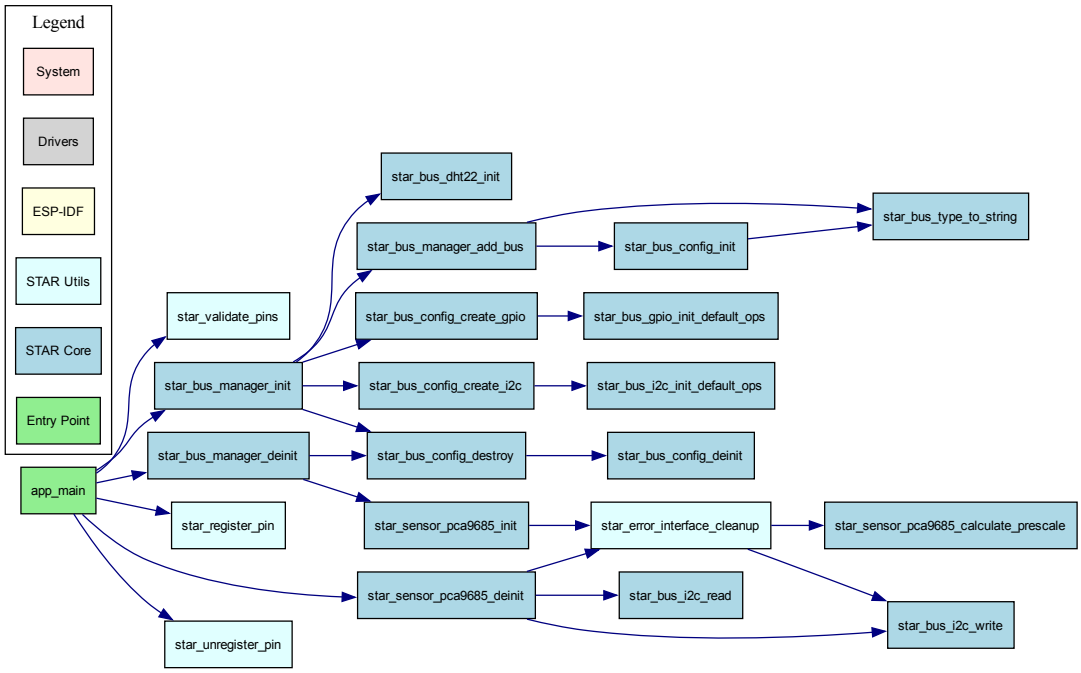


Figure 74: Application Layer Call Graph (from `app_main`)

The call graphs show the key architectural patterns:

1. **Dependency Injection Flow**: `app_main()` → interface creation → component initialization
2. **Bus Abstraction**: Sensor drivers → `star_bus_manager` → protocol implementations

3. **Error Handling**: Centralized error interface with configurable retry logic
4. **Resource Management**: Pin validation and bus lifecycle management

FreeRTOS Task Architecture The firmware runs as concurrent FreeRTOS tasks with dedicated responsibilities:

Table 73: FreeRTOS Task Configuration

Task	Priority	Interval	Function
DHT22 Task	Normal	2000 ms	Temperature/humidity monitoring
Sensor Task	Normal	100 ms	HC-SR04 distance measurements
LED Task	Normal	50 ms	RGB LED visual feedback via PCA9685
Watchdog Task	High	1000 ms	System health monitoring, fault recovery

Thread Safety All sensor drivers implement mutex protection with configurable timeouts (default 1000 ms):

Code Snippet 5: Thread-Safe State Modification

```
esp_err_t star_sensor_pca9685_set_duty_cycle(pca9685_handle_t* handle,
                                             uint8_t channel,
                                             float duty_percent) {
    if (handle == NULL || !handle->initialized) {
        return ESP_ERR_INVALID_STATE;
    }

    // Acquire mutex with timeout
    if (xSemaphoreTake(handle->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
        ESP_LOGE(TAG, "Failed to acquire mutex");
        return ESP_ERR_TIMEOUT;
    }

    // Perform operation while holding mutex
    esp_err_t ret = internal_set_duty(handle, channel, duty_percent);

    xSemaphoreGive(handle->mutex);
    return ret;
}
```

Error Handling with Retry Logic The error handler provides automatic retry with exponential backoff for transient bus failures:

Table 74: Error Handler Configuration

Parameter	Default	Fast Mode
Max Retries	3	2
Base Delay	100 ms	10 ms
Max Delay	5000 ms	50 ms

This enables graceful handling of I^2C bus lockups, sensor timeouts, and communication errors without system crashes.

Documentation and Code Quality The firmware includes comprehensive documentation generated via Doxygen with:

- Function-level call/caller graphs
- Cross-referenced source code
- API documentation for all public functions
- Architecture diagrams

For the complete firmware source code, see Appendix 17.10. The full repository is available at the project GitHub.

See Software Implementation section for ROS 2 integration and SLAM algorithms.

15.4 Software Preliminary Implementation

Section Author(s): BS, JB, CM

This section covers the software architecture: operating system, SLAM, navigation, control, ML inference, and user interface. See Embedded Implementation for ESP32 firmware details.

15.4.1 System Architecture Overview

Table 75: Software Stack Layers

Layer	Platform	Responsibilities
Embedded	ESP32-S3	Motor PWM, sensor polling, hardware abstraction
Edge	Raspberry Pi 5	ROS 2, SLAM, Nav2, sensor fusion, local ML
Remote (Optional)	Server	ML offloading, data storage, web UI

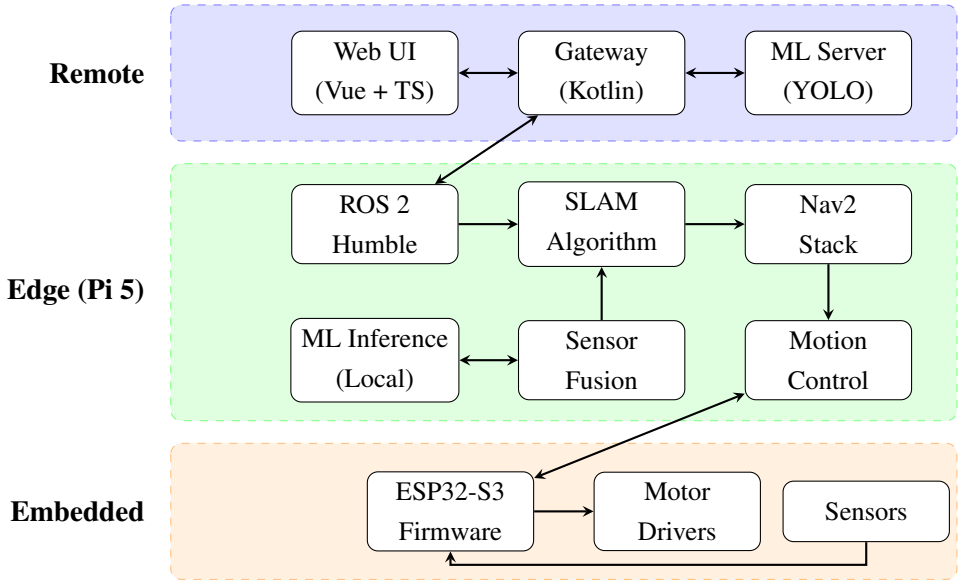


Figure 75: STAR Robot Software Architecture Overview

15.4.2 Software Process Overview

The software system follows a modular 6-layer architecture. Each layer operates independently while communicating through defined interfaces. The complete integrated process map is available

in Appendix 17.11 for full reference.

Layer 1: User Interface The UI layer provides real-time visualization and operator control through map display (800×600), camera feeds (640×480), and teleoperation inputs via keyboard-/joystick at 50 Hz.

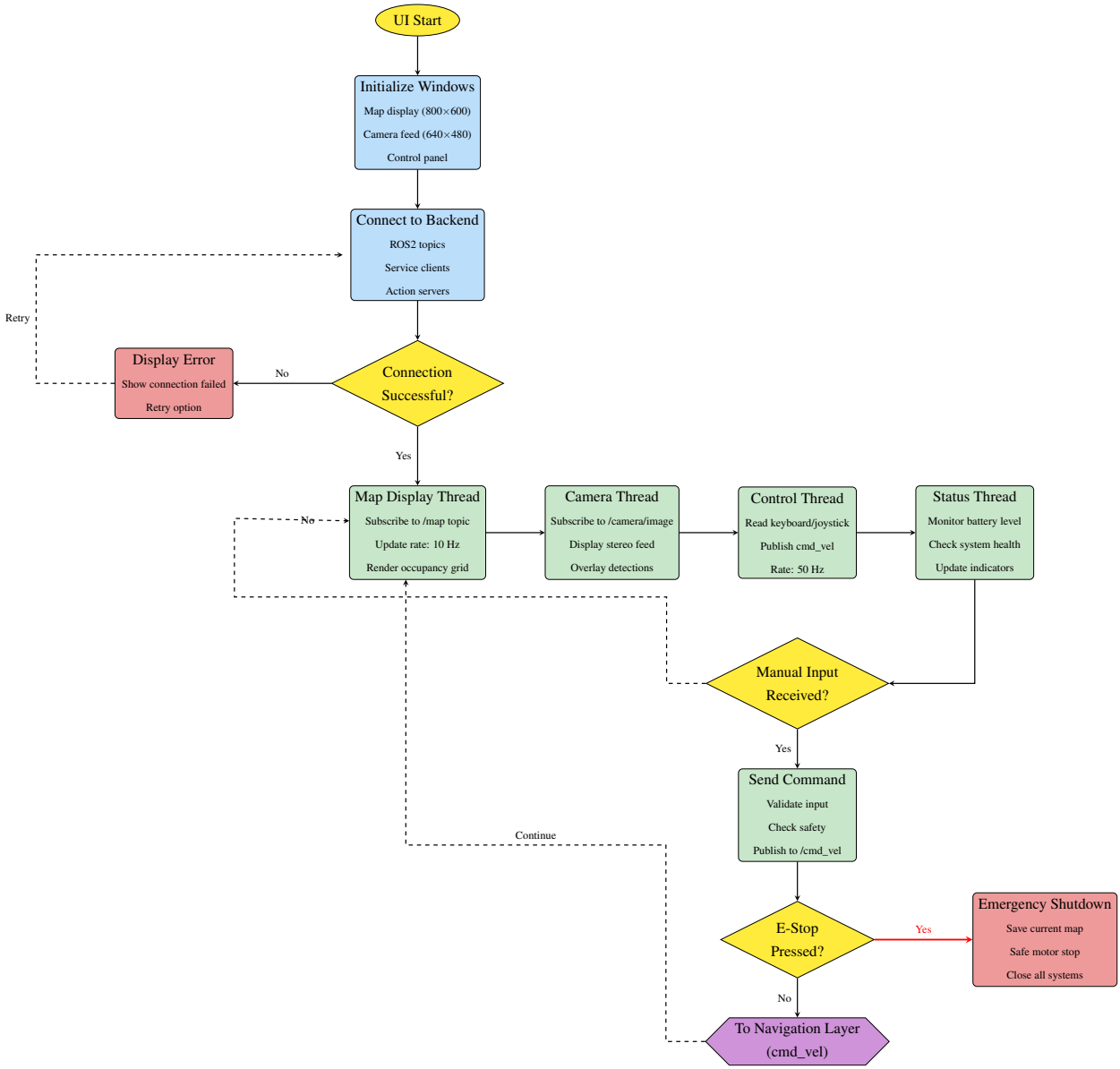


Figure 76: User Interface Layer Process Flow

Layer 2: Navigation & Control The navigation layer executes path planning via A* algorithm on the global costmap and local trajectory control using DWA for real-time obstacle avoidance.

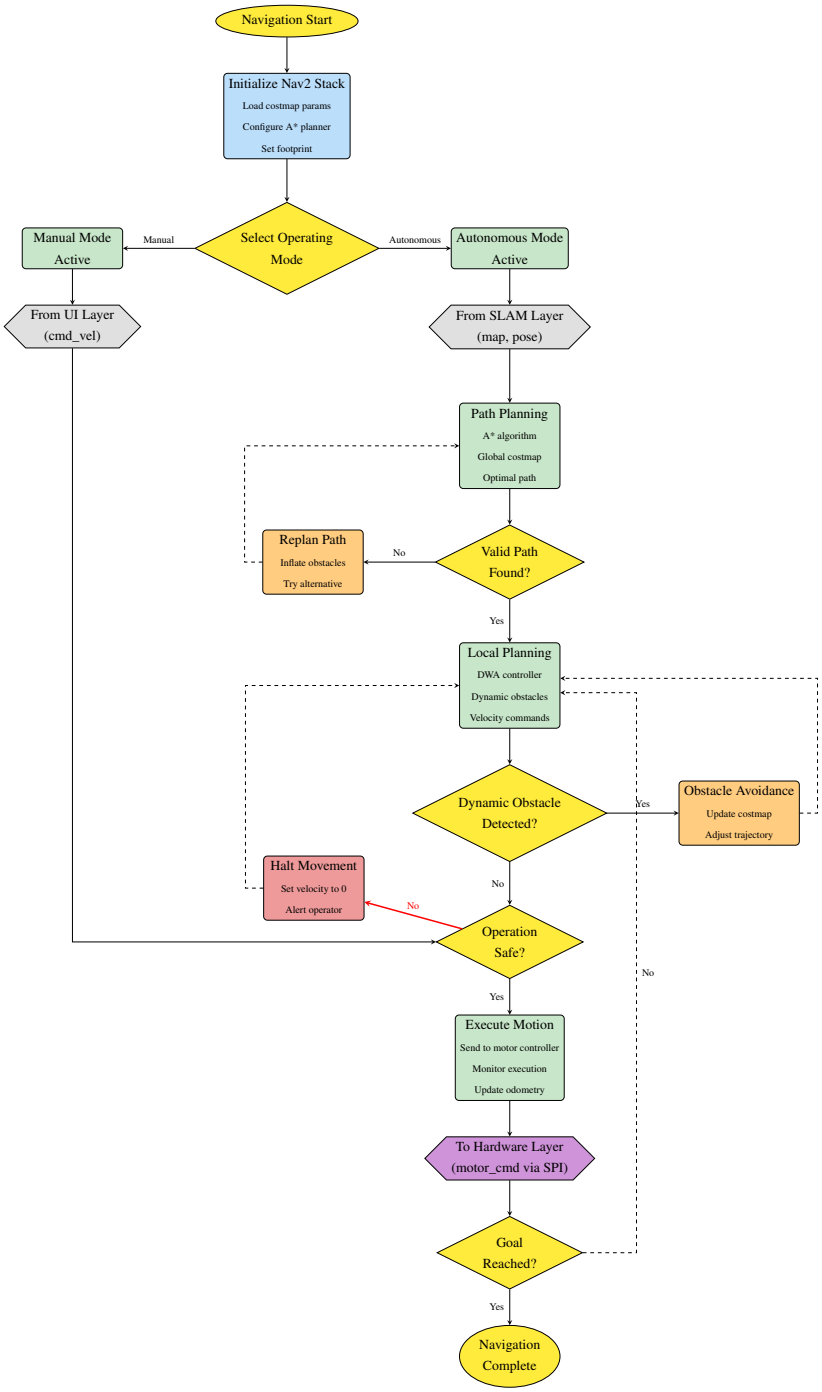


Figure 77: Navigation & Control Layer Process Flow

Layer 3: SLAM & Mapping (Hector-SLAM) Hector-SLAM processes LiDAR scans (SICK TiM561 at 15 Hz) to build occupancy grid maps while estimating robot pose through scan matching and loop closure detection.

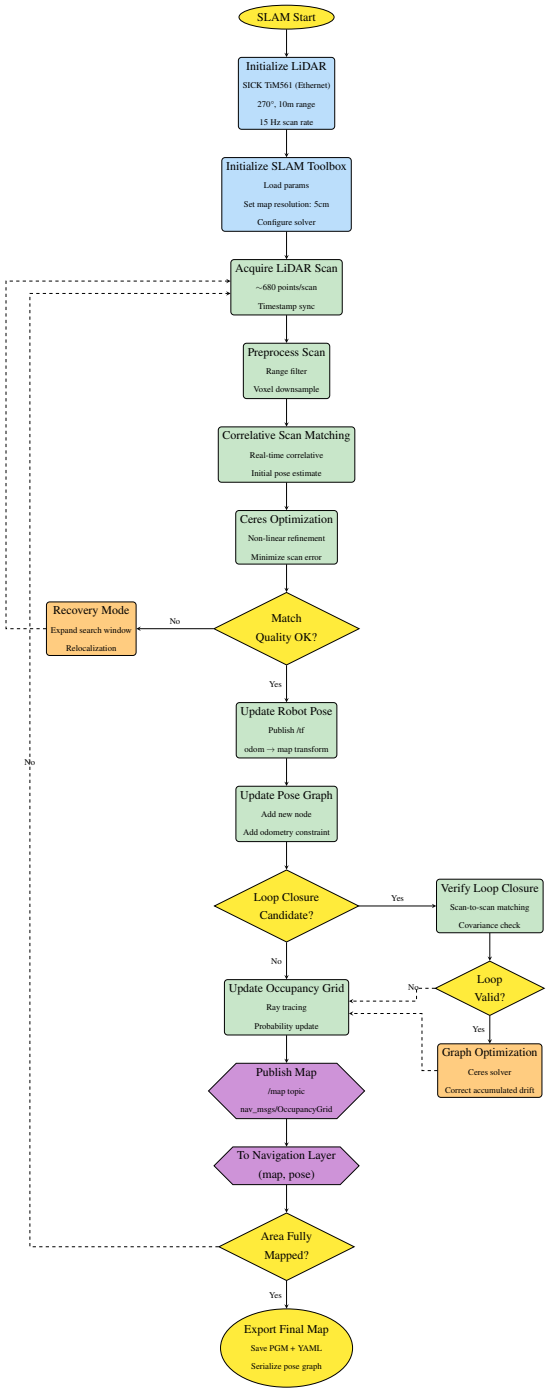


Figure 78: SLAM & Mapping Layer Process Flow (SLAM Toolbox)

Layer 4: Computer Vision Safety The CV layer performs stereo depth estimation (640×480 @ 30 FPS) and YOLO-based person detection to enable safety-critical obstacle avoidance with emergency stop at distances below 0.5 m.

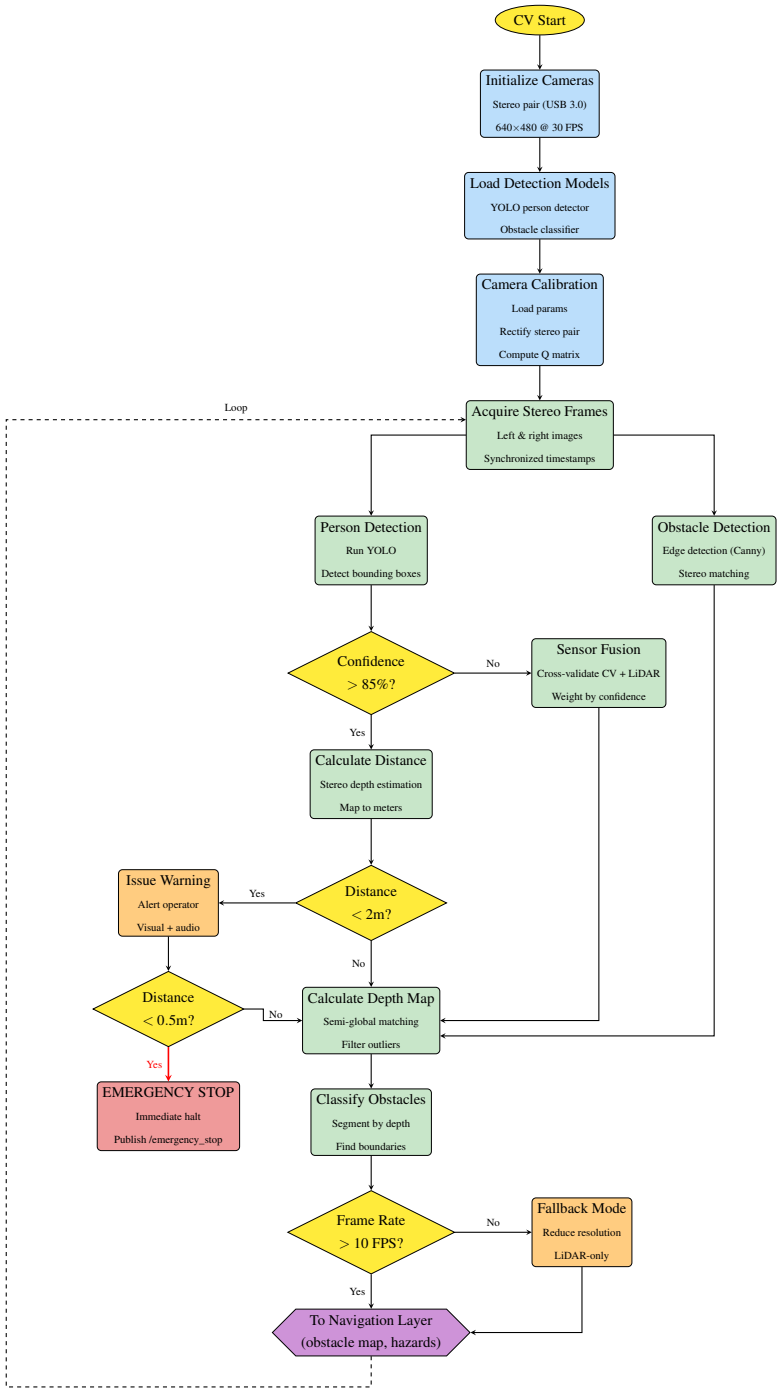


Figure 79: Computer Vision Safety Layer Process Flow

Layer 5: Hardware & Sensors This layer manages motor control via SPI, sensor polling (IMU via I2C, encoders, temperature via ADC), power system monitoring via SMBus, and publishes odometry to /odom and /tf.

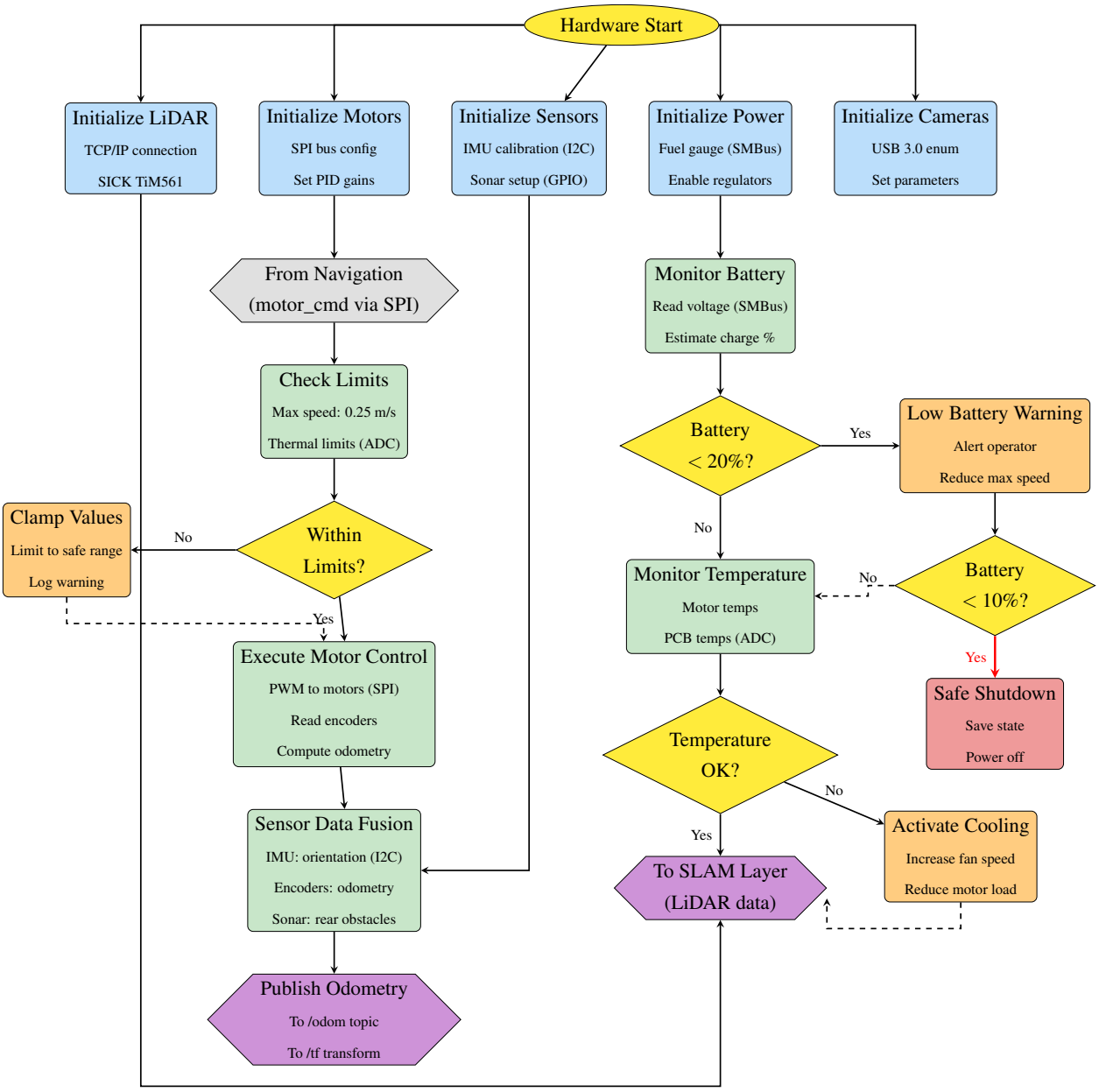


Figure 80: Hardware & Sensor Layer Process Flow

Layer 6: ESP32-Firmware Demo (STAR Framework) The demo layer showcases STAR Framework capabilities with FreeRTOS tasks, DHT22 temperature/humidity sensors, HC-SR04 ultrasonic sensors, and PCA9685 PWM output for LED patterns.

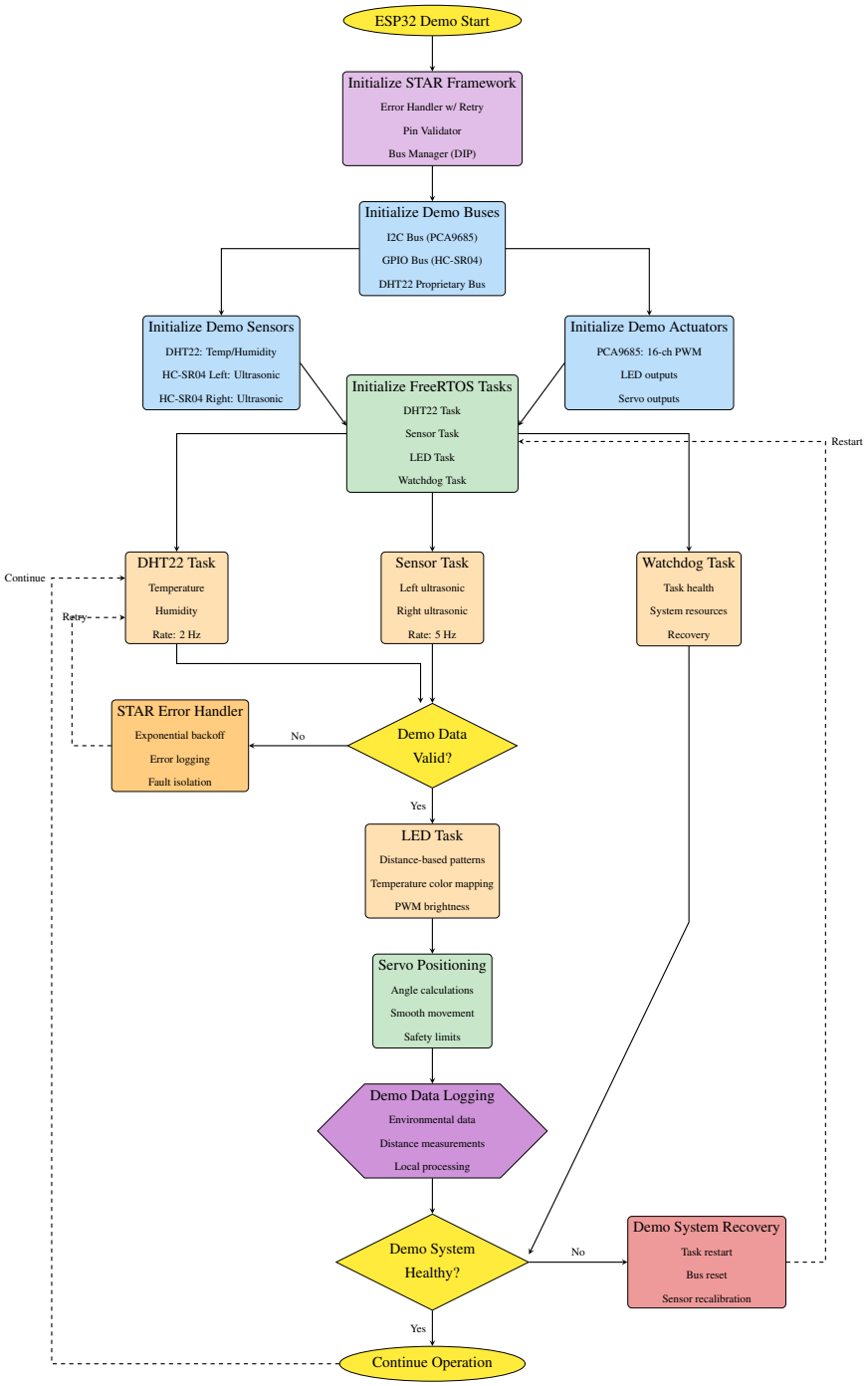


Figure 81: ESP32-Firmware Demo Layer Process Flow (STAR Framework)

15.4.3 Operating System

Custom **Buildroot 2025.02 LTS** Linux for Raspberry Pi 5 (Buildroot Developers, Buildroot: Making Embedded Linux Easy):

Table 76: Buildroot Image Configuration

Component	Details
Base image	2 GB (vs. 4+ GB Raspberry Pi OS)
Boot time	15 seconds
ROS 2	Jazzy LTS with Nav2, SLAM Toolbox
Vision	OpenCV 4 with DNN module
ML Stack	Python 3.11, NumPy, SciPy, TFLite
Networking	mDNS (star-robot.local), WiFi + Ethernet

15.4.4 SLAM Algorithm Selection

Simultaneous Localization and Mapping (SLAM) is the core algorithmic capability enabling autonomous navigation (Thrun et al.). We evaluated three prominent 2D LiDAR SLAM algorithms available in the ROS 2 ecosystem.

Candidate Algorithms Three 2D LiDAR SLAM algorithms were evaluated:

- **Hector SLAM** (Kohlbrecher et al.): Scan-matching without odometry; low compute but no loop closure
- **SLAM Toolbox** (Macenski and Jambrecic): ROS 2 Nav2 default; pose graph optimization with loop closure
- **Google Cartographer** (Hess et al.): Graph-based with submaps; high accuracy but complex configuration

Table 77: SLAM Algorithm Decision Matrix

Criterion	Weight	Hector	SLAM Toolbox	Cartographer
ROS 2 Nav2 Integration	25%	2	**5**	4
Computational Efficiency	20%	**5**	4	2
Loop Closure Support	15%	1	4	**5**
Odometry Requirement	15%	**5**	3	3
Documentation Quality	10%	3	**5**	4
Configuration Complexity	10%	**5**	4	2
Active Development	5%	2	**5**	3
Weighted Score	100%	3.35	**4.20**	3.30

Scoring: 1 = Poor, 2 = Below Average, 3 = Average, 4 = Good, 5 = Excellent

Decision Matrix

Selection: SLAM Toolbox **SLAM Toolbox** selected for: native ROS 2 Nav2 integration, loop closure support, effective odometry fusion with our quadrature encoders, and balanced computational load on Pi 5. Hector SLAM remains as fallback if odometry proves unreliable.

SLAM Process Flow Figure 82 illustrates the SLAM algorithm data flow, from sensor input through map generation.

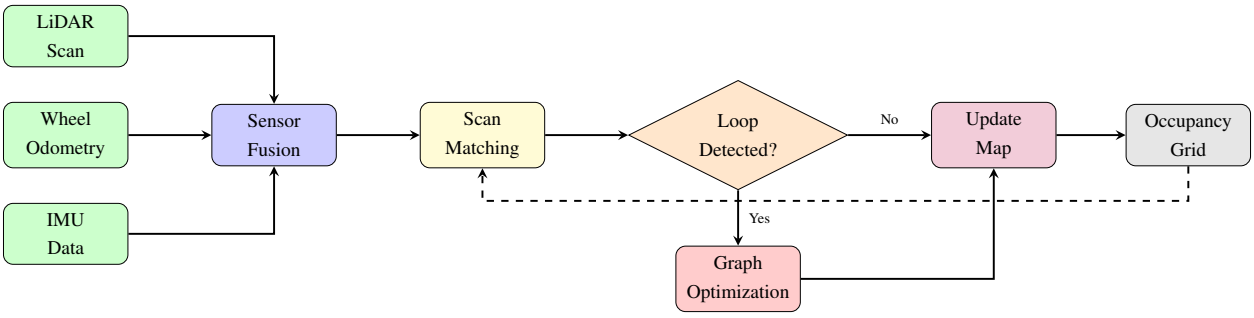


Figure 82: SLAM Toolbox Process Flow

15.4.5 LiDAR Sensor Selection

The STAR robot supports two LiDAR sensors, providing flexibility for different deployment scenarios. Both sensors have been validated with the ROS 2 software stack.

Table 78: LiDAR Sensor Comparison

Specification	SICK TiM561	RPLiDAR C1
Field of View	270°	**360°**
Range	10 m	**12 m** (white), 6 m (black)
Scan Rate	**15 Hz**	10 Hz
Angular Resolution	**0.33°** (810 points)	0.45°(800 points)
Interface	Ethernet TCP/IP	UART (USB adapter)
Accuracy	±**60 mm** (1-10 m)	±30 mm (0.15-0.5 m)
IP Rating	IP67	IP54
Operating Temp	-25°C to +50°C	0°C to 40°C
Power Consumption	4 W	**2.5 W**
Cost	$3,000+ (donated)	**$60**
ROS 2 Driver	sick_scan_xd	rplidar_ros

Sensor Comparison

Selection Rationale The STAR Robot uses a dual-sensor strategy. The **SICK TiM561** serves as the primary production sensor due to:

- Higher accuracy (0.33° angular resolution, ±60mm distance accuracy)
- Industrial reliability (IP67 rating, automotive-grade)
- 15Hz scan rate for faster SLAM updates
- Donated by SICK Inc. ($3,500+ value)

The **RPLiDAR C1** serves as a backup/development sensor, providing 360° FOV for software development and testing scenarios where the primary sensor is unavailable.

15.4.6 ROS 2 Architecture

ROS 2 (Robot Operating System 2) provides the middleware framework for inter-process communication, sensor data distribution, and navigation algorithm coordination (Open Robotics, ROS 2 Documentation: Jazzy Jalisco). The STAR robot uses ROS 2 Jazzy, the latest LTS release.

Node Architecture The ROS 2 node graph defines how software components communicate through topics (publish-subscribe), services (request-response), and actions (long-running tasks with feedback).

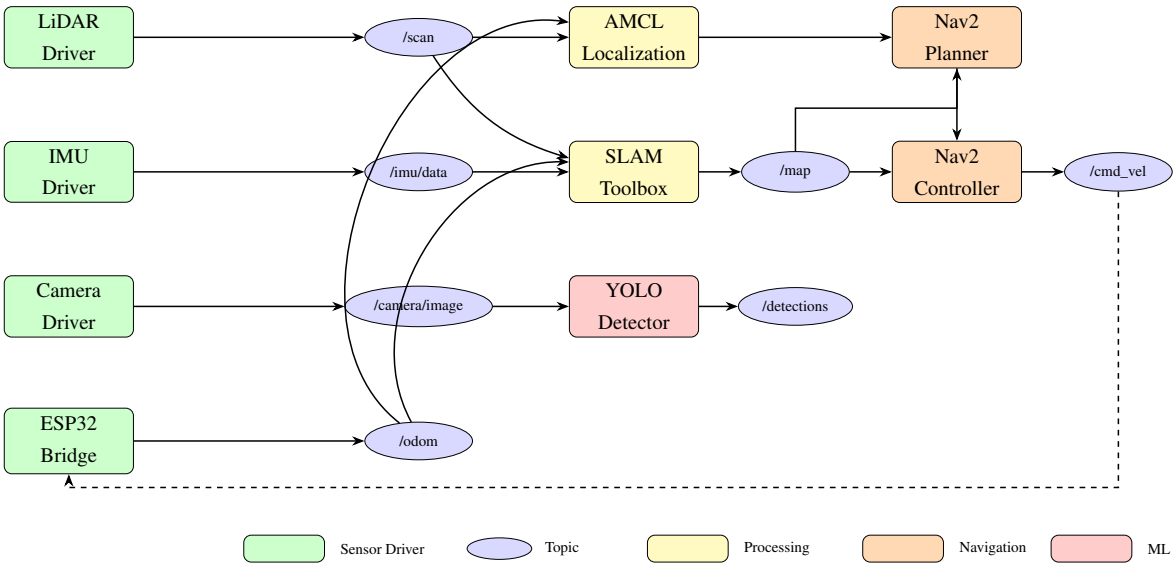


Figure 83: ROS 2 Node Graph for STAR Robot

15.4.7 Navigation Stack (Nav2)

Nav2 is the ROS 2 navigation framework providing path planning, trajectory following, and behavior coordination (Nav2 Documentation; Macenski et al.).

Table 79: Nav2 Component Configuration

Component	Plugin	Function
BT Navigator	NavigateToPose	Behavior tree coordination
Planner Server	NavFn	Global A* path planning
Controller Server	DWA	Local trajectory + obstacle avoidance
Costmap 2D	Layered	Static + LiDAR + inflation + ultrasonic
Recovery Server	Spin, Backup	Unstick behaviors

DWA Local Planner The Dynamic Window Approach (DWA) is the local planner for real-time obstacle avoidance (Fox et al.). At 20 Hz, DWA samples velocity commands within kinematic constraints, simulates trajectories, and selects the highest-scoring collision-free option.

Table 80: DWA Configuration Parameters

Parameter	Value	Rationale
v_{max}	0.5 m/s	Safe indoor speed; 25 cm stopping distance
ω_{max}	1.0 rad/s	180° in 3 s
$\dot{v}_{max}$	0.5 m/s^2	Prevents wheel slip
Robot footprint	0.38 m × 0.38 m	1.25 ft square
Simulation time	2.0 s	Look-ahead horizon

Velocity Selection: 0.5 m/s max ensures adequate sensor response (5 cm travel between 10 Hz LiDAR scans) and stopping distance ($d = v^2/2a = 0.25$ m at 0.5 m/s^2 decel).

DWA Algorithm Flow The DWA controller samples velocities, simulates trajectories, and selects the optimal path at 20 Hz:

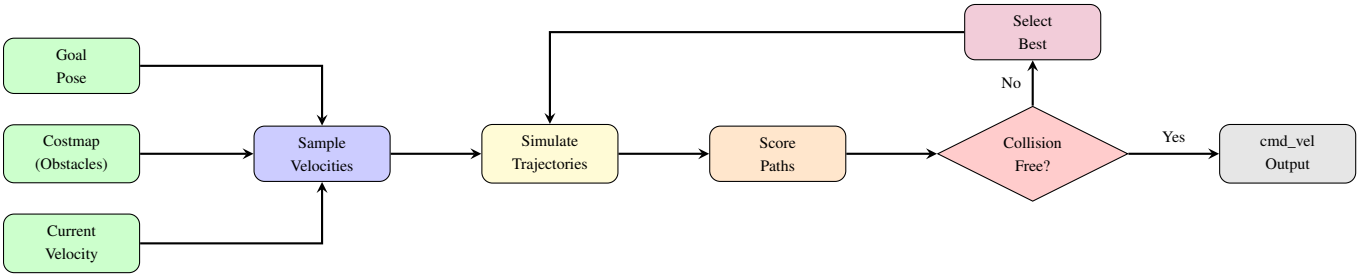


Figure 84: DWA Local Planner Algorithm Flow

Ultrasonic Integration Seven HC-SR04 sensors provide short-range detection for DWA, particularly for obstacles below the LiDAR plane (table legs, pets).

Table 81: HC-SR04 Range Utilization

Range	Accuracy	Action
2–30 cm	±3 mm	Emergency stop
30–150 cm	±1 cm	DWA costmap integration
>150 cm	±3+ cm	Ignored (unreliable)

15.4.8 Control Systems

The STAR robot implements a hierarchical control architecture with cascaded control loops operating at different frequencies.

Control Hierarchy The control system uses established robotics control theory to implement velocity control and navigation. The following equations, derived from standard references (Åström and Murray; Siciliano et al.), will be used to model the control system during implementation.

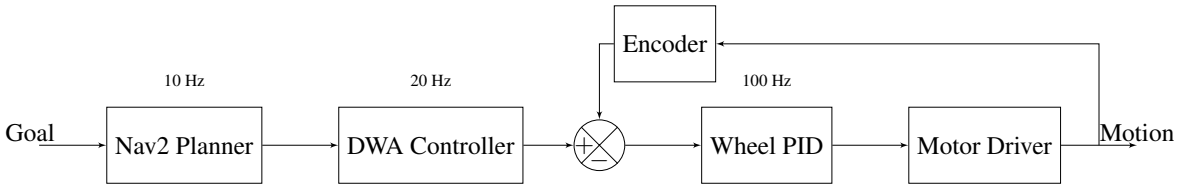


Figure 85: Hierarchical Control Architecture

Velocity Control (PID) Each wheel will be controlled by an independent PID velocity controller running on the ESP32 at 100 Hz (Åström and Murray):

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \tag{2}$$

where $u(t)$ is the PWM duty cycle, $e(t) = v_{cmd} - v_{actual}$ is the velocity error, and K_p, K_i, K_d are tunable gains.

Differential Drive Kinematics The 4-wheel differential drive configuration follows the standard 2-wheel kinematic model (Siciliano et al.):

Forward Kinematics:

$$v = \frac{v_R + v_L}{2}, \quad \omega = \frac{v_R - v_L}{L} \tag{3}$$

Inverse Kinematics:

$$v_R = v + \frac{\omega L}{2}, \quad v_L = v - \frac{\omega L}{2} \tag{4}$$

where v is linear velocity, ω is angular velocity, v_R/v_L are wheel velocities, and L is the effective wheel base.

Odometry Calculation Wheel odometry integrates encoder counts to estimate robot pose (x, y, θ):

$$\Delta s = \frac{\Delta s_R + \Delta s_L}{2}, \quad \Delta\theta = \frac{\Delta s_R - \Delta s_L}{L} \tag{5}$$

$$x_{k+1} = x_k + \Delta s \cdot \cos(\theta_k + \Delta\theta/2), \quad y_{k+1} = y_k + \Delta s \cdot \sin(\theta_k + \Delta\theta/2) \tag{6}$$

15.4.9 Machine Learning: Person Detection

The STAR robot incorporates real-time person detection to support first responder applications. Detected persons are marked on the generated map, providing situational awareness for emergency response teams.

Use Case Requirements

- **Primary Function**: Detect and localize people within camera field of view
- **Output**: Position markers on the occupancy grid map indicating person locations
- **Safety Consideration**: Robot should slow or stop when persons are detected in the navigation path
- **Data Logging**: Detection timestamps and estimated positions stored for post-mission analysis

YOLO Model Selection YOLO (You Only Look Once) is the selected object detection framework due to its real-time performance characteristics (Ultralytics). We evaluated multiple YOLO variants for deployment on the Raspberry Pi 5.

Table 82: YOLO Model Comparison for Raspberry Pi 5

Model	Parameters	mAP@50	Pi 5 FPS	Memory
YOLOv8n	3.2M	37.3%	8–12	6.3 MB
YOLOv8s	11.2M	44.9%	3–5	22.5 MB
YOLOv8m	25.9M	50.2%	1–2	52.0 MB
YOLOv8l	43.7M	52.9%	<1	87.7 MB
YOLOv8n (INT8)	3.2M	35.5%	15–20	3.2 MB

FPS estimates for 640×480 input on Pi 5 CPU. INT8 quantization uses TensorFlow Lite. mAP@50 = mean Average Precision at 50% IoU threshold on COCO dataset.

Table 83: YOLO Model Decision Matrix

Criterion	Weight	v8n	v8s	v8m	v8n INT8
Real-time Performance ($\geq$5 FPS)	30%	4	2	1	**5**
Detection Accuracy	25%	3	4	**5**	3
Memory Footprint	20%	4	3	2	**5**
CPU Headroom (for SLAM)	15%	3	2	1	**5**
Implementation Complexity	10%	**5**	5	5	3
Weighted Score	100%	3.60	2.95	2.40	**4.30**

Decision Matrix

Selection: YOLOv8n INT8 YOLOv8n with INT8 quantization selected for Pi 5: 15–20 FPS, 3.2 MB model, minimal CPU impact (30–50%), 1.8% mAP reduction acceptable for person detection. Uses TensorFlow Lite runtime (Google, TensorFlow Lite: Post-training Quantization).

Table 84: Local vs. Remote Inference Trade-offs

Factor	Pi 5 (Local)	Server (Remote)
Latency	50–100 ms	100–300 ms (network dependent)
FPS	15–20 (INT8)	60+ (FP16)
Model Size	YOLOv8n only	YOLOv8m/l possible
Accuracy	35.5% mAP	50+ % mAP
Network Dependency	None	Required
CPU Impact	30–50%	Minimal

Local vs. Remote Inference Runtime switching between local/remote based on network availability and CPU load.

15.4.10 Remote Server Architecture

Optional server for ML offloading, data persistence, and UI hosting.

Table 85: Server Component Stack

Component	Technology	Protocol	Function
Gateway	Kotlin/Ktor	WebSocket	ROS 2 bridge, session mgmt, REST API, JWT auth
ML Server	PyTorch/CUDA	gRPC	YOLOv8m/l inference on RTX 5070
Database	PostgreSQL+PostGIS	SQL	Maps, detections, trajectories, metadata
Web UI	Vue 3 + TypeScript	HTTP/WS	Real-time monitoring and control

Table 86: Robot-Server Data Streams

Stream	Rate	Content
Telemetry	10 Hz	Pose, velocity, sensor status
Map Updates	On change	Incremental occupancy grid deltas
Camera Frames	5 FPS	JPEG-compressed (when remote ML enabled)
Commands	On demand	Velocity, goals, mode switches
Heartbeat	1 Hz	Keep-alive with auto-reconnect

Communication Protocol 4G cellular backup available when WiFi unavailable (+100–500 ms latency).

15.4.11 User Interface

Web-based UI for real-time monitoring, control, and visualization. Built with Vue 3, TypeScript, Pinia, Tailwind CSS, and WebSocket streaming (Vue.js, Vue.js 3 Documentation; Microsoft).

Web UI Architecture The STAR Web UI follows a reactive single-page application architecture. Figure 86 illustrates the data flow from robot sensors through the UI rendering pipeline.

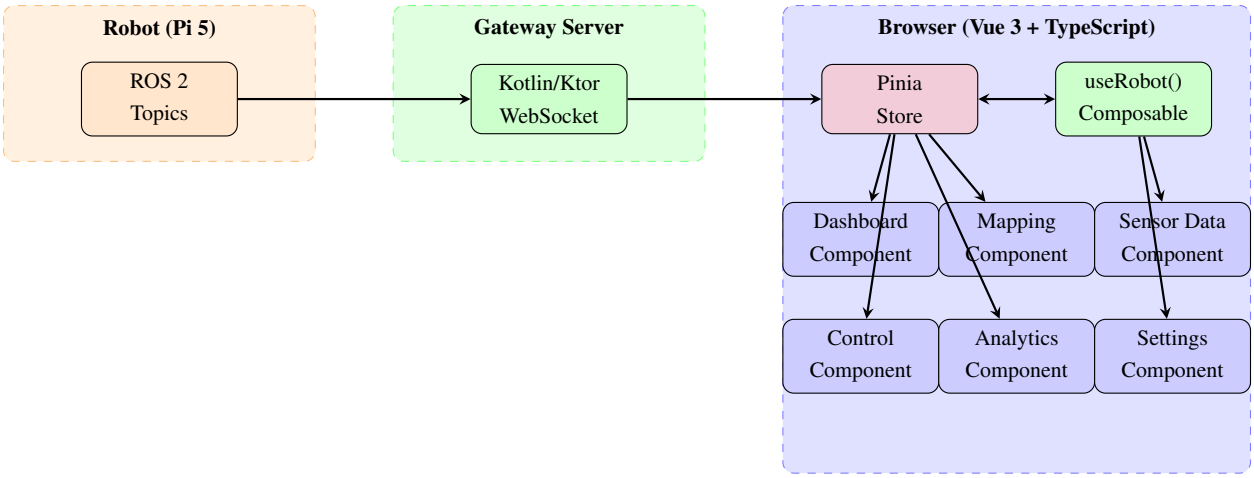


Figure 86: Web UI Data Flow Architecture

Dashboard and Sensor Views The Dashboard page (Figure 87) provides a real-time mapping visualization with concentric range rings (2–10 m), robot position indicator, and live LiDAR scan overlay. Status cards display battery level (with estimated runtime), WiFi signal strength, connection status, and current operating mode. The IMU Orientation panel shows pitch, roll, yaw angles and velocity with a 3D model preview indicating robot attitude.

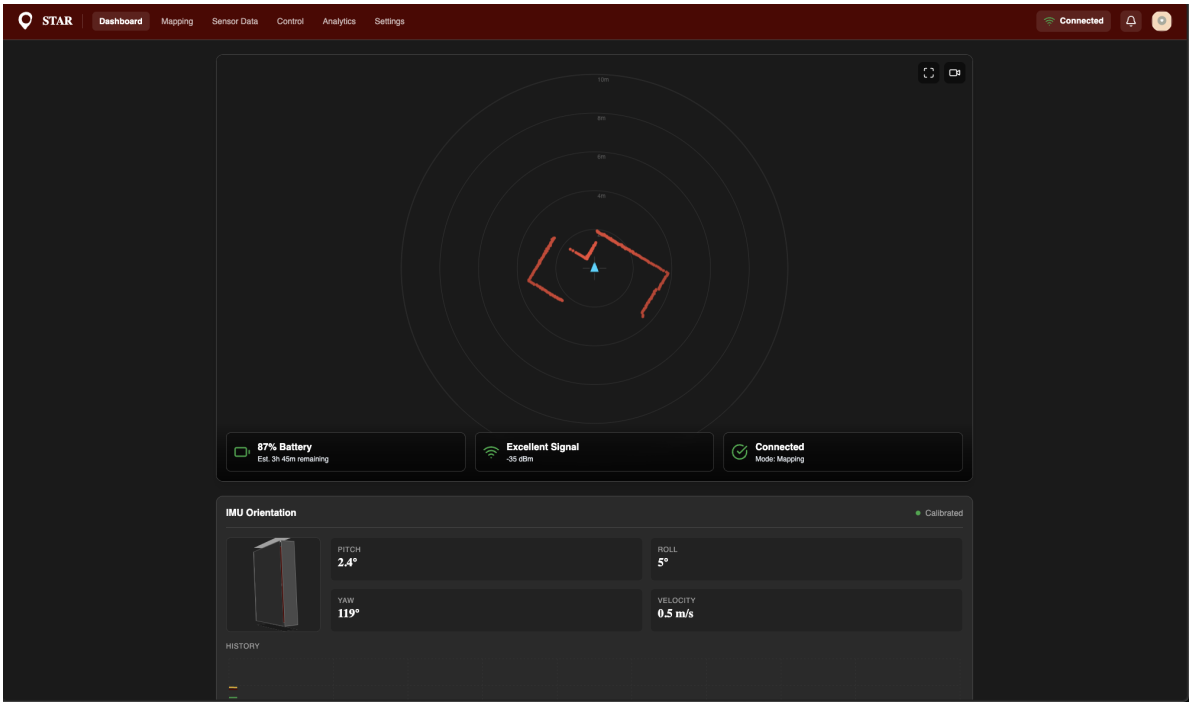


Figure 87: STAR Web UI Dashboard: Real-time mapping view with IMU orientation display

The Sensor Data page (Figure 88) displays all telemetry streams: GPS coordinates (latitude,

longitude, altitude), internal temperature with thermal warning indicators, CPU/RAM utilization gauges, and motor load percentage. The Raw Data Log provides timestamped system events for debugging and mission review.

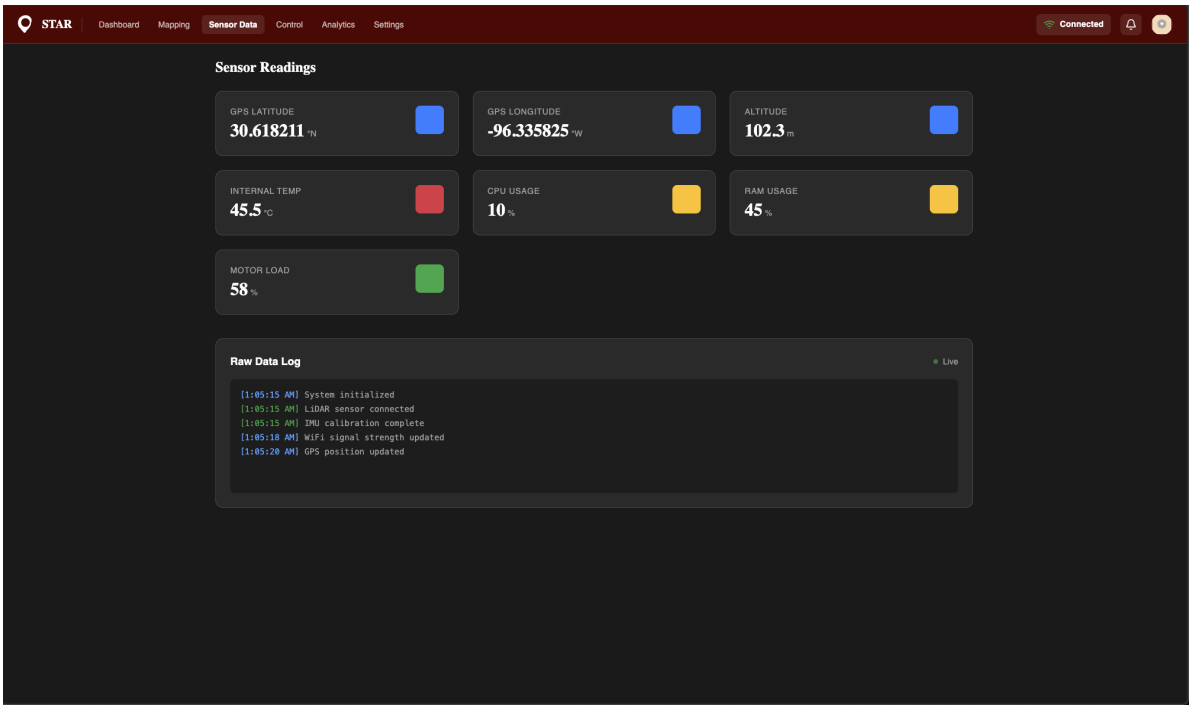


Figure 88: STAR Web UI Sensor Data: Telemetry dashboard with GPS, thermal, and system metrics

Table 87: Web UI Pages

Page	Features
Dashboard	Video feed, system status, battery, connection indicator
Mapping	Live occupancy grid, robot pose, LiDAR overlay, person markers, path history
Control	Virtual joystick, WASD keys, speed presets (0.1/0.3/0.5 m/s), e-stop
Sensors	LiDAR scan, IMU orientation, ultrasonic readings
Analytics	Mission stats, coverage %, detection counts
Settings	Robot config, network, map export (PGM, PNG, PDF, GeoJSON)

Responsive design: desktop (full dashboard), tablet (field-optimized), mobile (control-focused).

15.4.12 Software Implementation Summary

Table 88: Software Stack Summary

Subsystem	Implementation
OS	Custom Buildroot Linux (ROS 2 Jazzy)
SLAM	SLAM Toolbox with loop closure
LiDAR	RPLiDAR C1 (prototype) / SICK TiM561 (industrial)
Navigation	Nav2 + DWA + layered costmaps
Control	Hierarchical PID at 100 Hz
ML	YOLOv8n INT8 local; GPU offload optional
Server	Kotlin gateway + Vue UI

Demonstration Goals: Autonomous mapping, DWA obstacle avoidance with sensor fusion, real-time person detection, web-based teleoperation.

15.5 Mechanical Preliminary Implementation

Section Author(s): MN

15.5.1 Chassis Design

The mechanical design centers on a compact mobile platform capable of operating in confined or partially obstructed environments where first responders require quick mapping before entry. The chassis features a rigid, low-profile frame that houses the LiDAR, electronics, power system, and drivetrain in a protected configuration.

Design Requirements:

- LiDAR mounted with unobstructed 270° field of view
- Raspberry Pi and electronics positioned centrally for balance
- Low center of gravity for stability during navigation
- Modular layout for maintenance and testing

Materials:

- **Base structure:** U-channel and sheet aluminum for rigidity
- **Upper structure:** 3D-printed PETG for lightweight, cost-effective fabrication, and machinability

15.5.2 Drivetrain

Multiple configurations were evaluated, including tracks, eight wheels, and six wheels with various sizes and placements. The four-wheel differential drive design was selected for its simplicity and adequate performance. Torque and RPM calculations confirmed the design meets requirements with a safety factor of three.

Drive System Features:

- Four-wheel independent drive for differential steering
- 3D-printed bevel gears transfer motor torque to wheel axes (cost savings)
- Wheel assemblies provide traction on tile, concrete, and other indoor surfaces
- Capable of traversing small obstacles

15.5.3 Component Integration

The internal layout is designed for easy reconfiguration:

- **Motor control subsystem:** ESP32 isolated to reduce electrical noise and manage thermal behavior

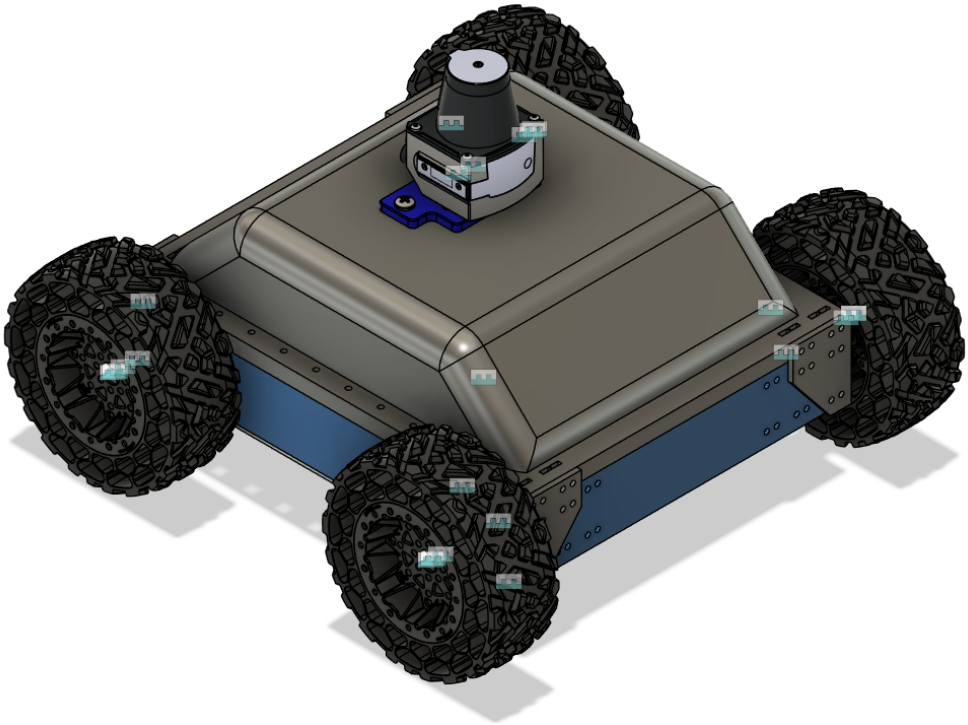


Figure 89: CAD Main Version 1

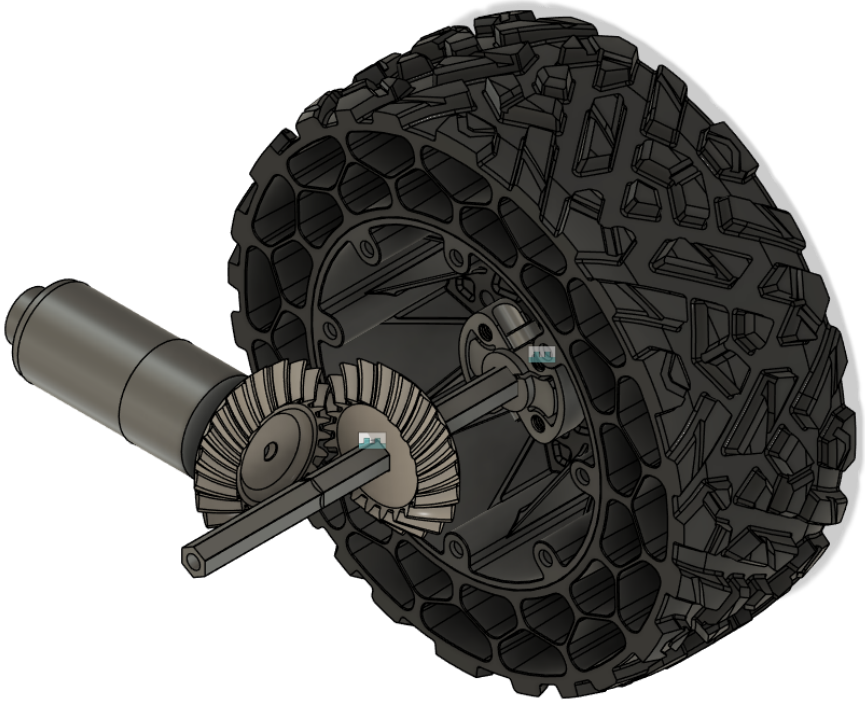


Figure 90: Drive System CAD with Custom Bevel Gears

- **Battery housing:** Positioned at bottom center for low center of gravity; tray design enables easy battery replacement
- **Thermal management:** Ventilation pathways around Raspberry Pi for heat dissipation during sustained workloads

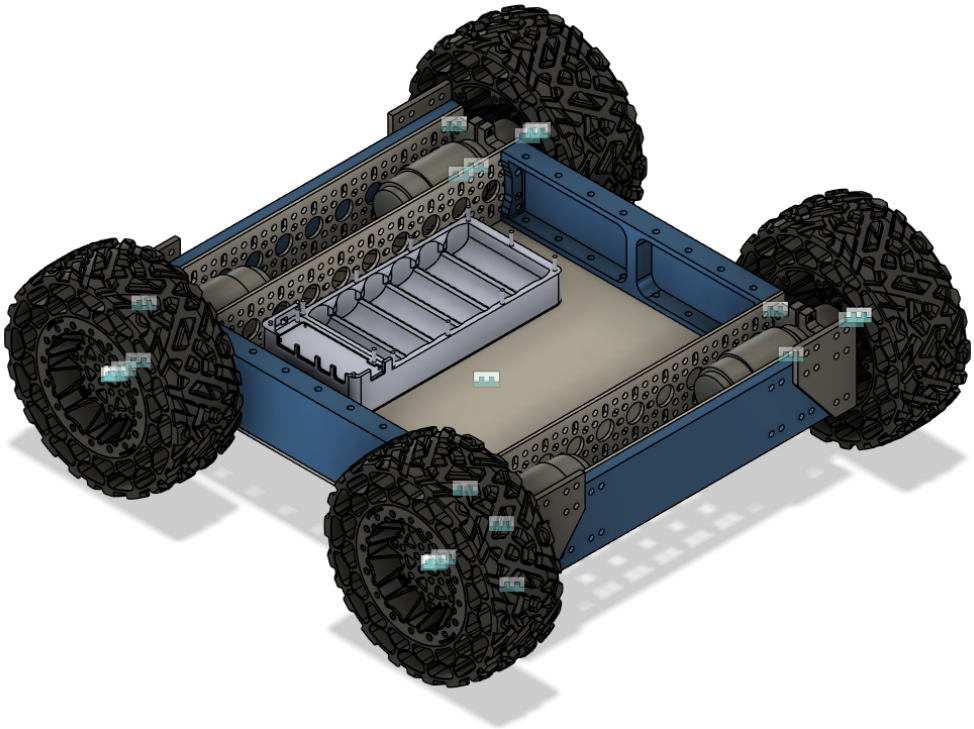


Figure 91: Initial CAD Skeleton Structure

The mechanical design supports the system's functional objectives through a stable, maintainable platform optimized for reliable mapping in operational environments.

16.0 Test Plan and Validation

Section Author(s): BS, CM, JB, MN, JH, SC

16.1 Test Objectives and Scope

The primary objective of this test plan is to systematically verify that the STAR Robot meets all functional and non-functional requirements defined in Section 4.0. Testing serves two distinct purposes: verification (confirming the system is built correctly according to technical specifications) and validation (confirming the correct system is built to satisfy project requirements).

The scope of testing encompasses:

- **Unit Verification:** Individual firmware drivers, mechanical components, and circuit modules tested in isolation.
- **Subsystem Integration:** Interaction between the ESP32-S3 firmware, Raspberry Pi 5 compute platform, electrical power systems, and mechanical chassis.
- **System Validation:** Full-stack ROS 2 navigation, SLAM Toolbox performance, and autonomous behavior under realistic indoor conditions.

The following items are explicitly out of scope: third-party ROS 2 libraries (assumed stable and treated as black-box components), manufacturing defects of off-the-shelf PCBs (assumed tested by vendor), and outdoor operation testing.

16.2 Test Methodology: The V-Model Approach

To ensure a structured validation process, this project utilizes the Systems Engineering V-Model, a framework that links each phase of the design lifecycle to a corresponding testing phase. The V-Model derives its name from its characteristic shape when visualized: the left arm represents progressive decomposition during design (from high-level requirements down to detailed component specifications), while the right arm represents progressive integration during testing (from unit tests up to acceptance testing).

The fundamental principle behind the V-Model is *traceability*: every design decision should be verifiable through a corresponding test. This approach prevents the common pitfall of discovering defects late in development, where they are expensive to fix. Instead, the V-Model ensures that:

1. Component Design is verified by Unit Testing: for example, verifying that the DRV8243-Q1 motor driver responds correctly to PWM signals from the ESP32-S3.

2. Subsystem Design is verified by Integration Testing: for example, confirming that the ESP32-S3 correctly commands the motor drivers and receives encoder feedback via the SPI interface to the Raspberry Pi 5.
3. System Requirements are validated by System Testing: for example, demonstrating that the robot can autonomously navigate a 50m path while avoiding obstacles (FR-001, FR-002).
4. User Requirements are validated by Acceptance Testing: demonstrating the complete system to Dr. Porter and the review panel under real-world conditions.

Figure 92 illustrates this relationship. The horizontal dashed lines represent traceability links between design phases and their corresponding tests.

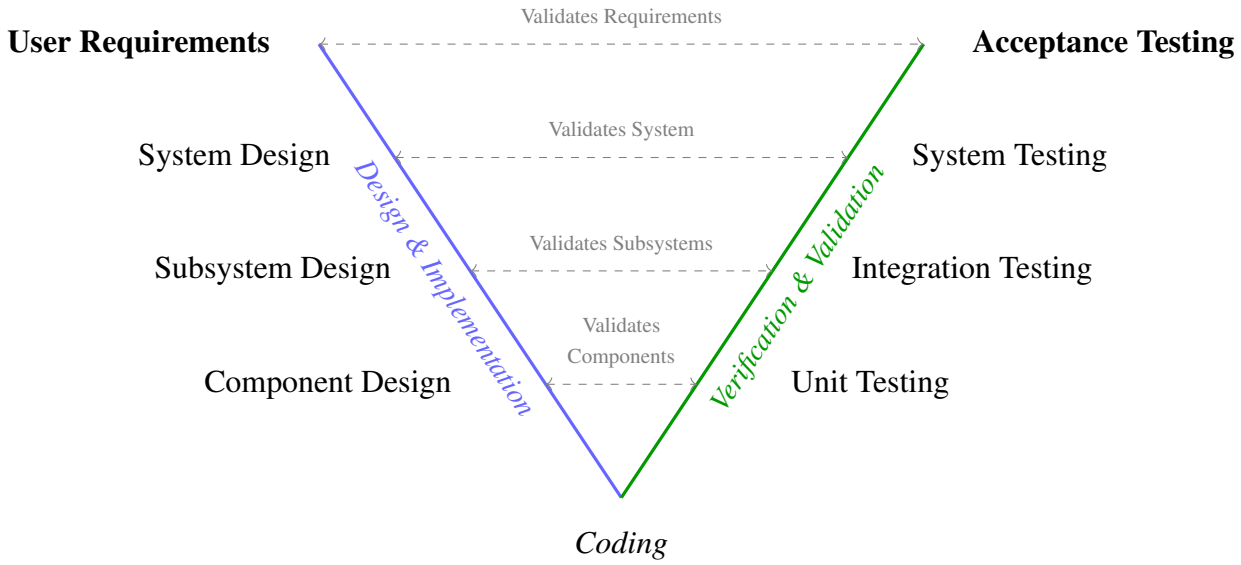


Figure 92: The V-Model approach applied to the STAR Robot validation strategy. The left arm represents design decomposition from user requirements to component specifications. The right arm represents verification and validation, progressing from unit tests to acceptance testing. Horizontal dashed lines indicate traceability between design phases and their corresponding tests.

16.3 Test Strategy: The Testing Pyramid

The test strategy follows the Testing Pyramid principle, which recommends a high volume of fast, automated unit tests at the base, fewer integration tests in the middle, and a small number of comprehensive system and acceptance tests at the top. This structure optimizes for rapid feedback: most defects are caught early by unit tests (which are fast and cheap to run), while integration and system tests catch issues that only emerge when components interact. Integration testing employs two complementary approaches: Hardware-in-Loop (HIL) testing runs software against physical hardware, while Software-in-Loop (SIL) testing uses simulation environments like Gazebo.

Figure 93 illustrates the pyramid structure. The width of each level represents the relative quantity of tests, while the height represents increasing integration complexity.

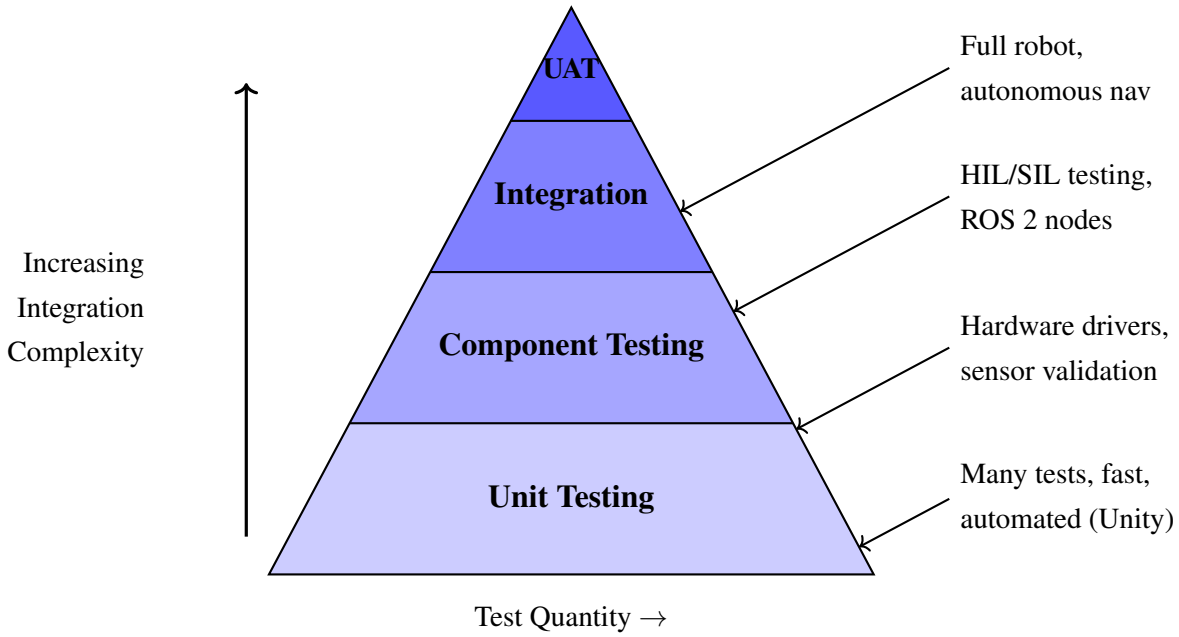


Figure 93: The Testing Pyramid showing the distribution of test types. Unit tests form the broad base with many fast, automated tests. Each successive level has fewer tests but validates increasingly complex integration. UAT = User Acceptance Testing.

Table 89 summarizes the five test levels employed in this project, from foundational unit tests to final acceptance testing.

Table 89: Test Strategy Levels and Their Scope

Level	Test Type	Scope and Purpose
5	Acceptance Testing	Validation against project requirements in real-world conditions (Final Demonstration, May 8, 2026)
4	System Testing	Complete integrated robot system operating autonomously in the test arena
3	Integration Testing	Interactions between subsystems using Hardware-in-Loop (HIL) and Software-in-Loop (SIL) methods
2	Component Testing	Individual hardware components and firmware drivers on the development bench
1	Unit Testing	Individual functions and modules tested in isolation using Unity Test Framework

16.4 Requirements Traceability

Every test case is explicitly linked to one or more requirements from Tables 2 and 3 to ensure complete coverage. This traceability serves two purposes: it guarantees that all requirements are tested, and it allows impact analysis when requirements change.

The coverage objectives are:

- 100% of Critical and High priority requirements covered by at least one passed test case.
- $\geq$80% code coverage for ESP32-S3 firmware modules (measured via Unity test framework).
- 100% of hardware interfaces exercised during integration testing.
- All functional requirements (FR-001 through FR-009) validated by system-level tests.

16.5 Test Items

This section identifies all components subject to testing, organized by subsystem.

16.5.1 Embedded Firmware Components

Table 90 lists the STAR firmware modules under test. Each module implements a specific abstraction layer (core services, bus communication, or sensor drivers) within the layered architecture described in Section 15.3.

Table 90: Embedded Firmware Test Items

ID	Component	Description
ESP-CORE-001	star_core	Abstract interfaces and system initialization
ESP-ERR-001	star_error_handler	Error handling with exponential backoff
ESP-PIN-001	star_pin_validator	GPIO conflict detection and validation
ESP-BUS-001	star_bus	Unified I2C/SPI/UART bus abstraction
ESP-DRV-003	star_sensor_hcsr04	Ultrasonic distance sensor driver (7 sensors)
ESP-DRV-004	star_sensor_bno055_bmp280	10-DOF IMU + barometric pressure sensor
ESP-DRV-005	star_bms_bq78350	Battery Management System SMBus driver

16.5.2 Electrical Subsystems

Table 91 identifies the electrical components requiring verification. Priority reflects the impact of failure on system safety and functionality.

Table 91: Electrical Subsystem Test Items

ID	Component	Specification	Priority
ELE-PWR-001	Power Distribution Board	11.1V input, 3.3/5/6V regulated outputs	Critical
ELE-CHG-001	BQ25798 Buck-Boost Charger	USB-C PD input, 3S LiPo charging	Critical
ELE-BMS-001	BQ78350 Fuel Gauge	SMBus, state-of-charge monitoring	Critical
ELE-AFE-001	BQ76920 AFE	Cell balancing, protection thresholds	Critical
ELE-MTR-001	DRV8243-Q1 Motor Drivers ($\times$4)	12V nominal, 8A peak per channel	Critical
ELE-IMU-002	BNO055 + BMP280 Module	I2C, 9-axis fusion + pressure	Med
ELE-USS-001	HC-SR04 Ultrasonic Sensors ($\times$7)	GPIO trigger/echo, 2–400cm range	Med
ELE-CAM-001	IMX219-83 Stereo Camera	CSI-2 interface, V4L2 driver	High

16.5.3 Mechanical Assemblies

Table 92 lists the mechanical assemblies requiring structural and functional verification.

Table 92: Mechanical Assembly Test Items

ID	Component	Material/Type	Priority
MEC-CHS-001	Robot Chassis	Aluminum frame, PETG panels	Critical
MEC-WHL-001	Wheel Assemblies ($\times$4)	Rubber tires, custom bevel gears	Critical
MEC-ENC-001	Wheel Encoders ($\times$4)	Optical quadrature, 360 CPR	High
MEC-MNT-001	LiDAR Mount (SICK TiM561)	Machined aluminum	High
MEC-MNT-002	Camera Mount (IMX219-83)	3D printed PETG, adjustable angle	Med
MEC-MNT-003	Electronics Enclosure	3D printed PETG	Med

16.6 Phase 1: Hardware and Electrical Verification

Before any software deployment, the physical platform undergoes rigorous continuity testing and stress testing. This phase identifies manufacturing defects, assembly errors, and design issues that would otherwise cause confusing failures during software integration. All signal measurements utilize the RIGOL DHO804 digital oscilloscope, leveraging its 12-bit vertical resolution (4096 quantization levels) to detect low-amplitude noise and transients that 8-bit instruments would miss.

Table 93 summarizes the Phase 1 hardware verification tests. Complete test procedures and pass criteria are provided in Appendix 17.12.1.

Table 93: Phase 1: Hardware Verification Test Summary

Category	Test Range	Count	Key Verification Items
PCB Assembly	TC-PCB-001–006	6	Power rails, signal traces, solder quality
Power System	TC-PWR-001–007	7	Rail accuracy ($\pm$3–10%), protection circuits
Communication	TC-COM-001–006	6	I2C/SPI/UART integrity, bus enumeration
Motor Control	TC-MTR-001–007	7	Driver function, PWM, encoder feedback
Mechanical	TC-MEC-001–008	8	Structural load, alignment, thermal
Total		**34**	

16.7 Phase 2: Embedded Firmware Integration

With hardware verified, Phase 2 tests the Hardware Abstraction Layer (HAL) and low-level drivers on the ESP32-S3. These tests execute on real hardware (Hardware-in-Loop testing) to verify that firmware correctly interfaces with the physical sensors and actuators. Table 94 summarizes the firmware integration tests; complete specifications are in Appendix 17.12.2.

Table 94: Phase 2: Firmware Integration Test Summary

Category	Test Range	Count	Key Verification Items
IMU Validation	TC-FW-001–002	2	Gravity vector, gyroscope bias
Odometry	TC-FW-003–004	2	Distance accuracy ($<2\%$), PID tuning
Timing/Safety	TC-FW-005–008	4	Control loop (100Hz), watchdog, error handling
Sensors	TC-FW-009	1	Temperature-compensated ultrasonic
Total		**9**	

16.8 Phase 3: System Integration and ROS 2 Validation

Phase 3 validates the complete robot stack, including the ROS 2 Humble middleware, SLAM Toolbox algorithms, Nav2 navigation planning, and the web-based user interface. These tests execute in the field test environment (described in Section 16.9) using the fully assembled robot. Table 95 summarizes the system integration tests; complete specifications are in Appendix 17.12.3.

Table 95: Phase 3: System Integration Test Summary

Category	Test Range	Count	Key Verification Items
System Boot	TC-SYS-001–003	3	Boot time, node health, latency
SLAM/Navigation	TC-SYS-004–008	5	Map quality, LiDAR, obstacle avoidance
User Interface	TC-SYS-009–012	4	Network, web UI, teleoperation
Safety/Endurance	TC-SYS-013–015	3	E-stop, battery life, CPU load
Total		**15**	

16.9 Test Environment and Tools

Reproducible testing requires strictly controlled and documented environments. This section specifies the hardware, software, and physical spaces used for testing.

16.9.1 Physical Test Environment

Table 96: Physical Test Environment Specifications

Environment	Configuration
Development Bench	Dedicated workstation with bench power supply, oscilloscope, and logic analyzer. Used for all Phase 1 and Phase 2 testing. Located in university fabrication lab.
Simulation (SIL)	Gazebo Sim running on Ubuntu 22.04 with ROS 2 Humble. Used for algorithm validation before hardware deployment. Custom Buildroot image tested in QEMU.
Field Arena (HIL)	4m $\times$ 4m enclosed arena in university lab with variable lighting conditions (200–800 lux). Floor surfaces: tile (primary), concrete (secondary). Obstacles: static wooden blocks (various sizes) and wheeled carts for dynamic testing.
Extended Testing	TEEX Disaster City complex available for realistic post-disaster scenario testing if schedule permits.

16.9.2 Test Equipment

Table 97: Laboratory Test Equipment

Equipment	Model	Purpose and Specification
Digital Oscilloscope	RIGOL DHO804	Signal integrity, timing, and power analysis. 4 channels, 70MHz bandwidth, 12-bit vertical resolution (4096 levels).
Logic Analyzer	HiLetgo USB	Bus protocol analysis (I2C, SPI, UART, SMBus). 8 channels, 24MHz sampling.
Digital Multimeter	OWON XDM1041	Precision voltage/current/resistance measurement. True RMS, 22000 count.
Bench Power Supply	ATRIX MPS-6010H-1C ($\times 2$)	Controlled power for HIL testing. 60V, 10A per unit.
FTDI Adapter	FT232RL	Serial communication debug. 3.3V logic levels.

16.9.3 Software Test Environment

Table 98: Software and Tools Configuration

Tool	Version	Purpose
ESP-IDF	5.x	ESP32-S3 SDK and toolchain
PlatformIO	Latest	Build system and test runner for embedded
Unity Test Framework	2.5.x	Unit testing framework for C firmware
Python	3.10+	Test automation scripts and data analysis
ROS 2	Humble Hawksbill	Robot middleware on custom Buildroot image
Gazebo	Sim (Ignition)	Physics simulation for SIL testing
Git/GitHub	Latest	Version control and issue tracking
RViz2	Humble	Real-time visualization of sensor data and maps

16.10 Defect Management

When a test case fails, the defect enters a defined lifecycle to ensure systematic resolution and prevent regression.

16.10.1 Severity Classifications

Defect severity determines prioritization and blocking behavior:

- **Critical (Blocker):** Safety hazard, system cannot boot, or data corruption. Testing of dependent components is blocked until resolved. Examples: motor runaway condition, power supply overvoltage, firmware crash on startup.
- **Major:** Key functionality fails (e.g., SLAM produces unusable maps), but workarounds exist or other subsystems can still be tested.
- **Minor:** Cosmetic issues, edge-case performance degradation, or documentation errors. Does not block other testing.

16.10.2 Defect Lifecycle

All defects follow this resolution process:

1. **Log:** Defect recorded in GitHub Issues with severity label, reproduction steps, and expected vs. actual behavior.
2. **Triage:** Team lead assigns priority and owner within 24 hours (Critical) or 72 hours (Major/Minor).
3. **Isolate:** Developer reproduces the defect, preferably in the SIL environment for faster iteration.
4. **Fix:** Code is patched and committed to a feature branch with a reference to the issue number.
5. **Regression Test:** The original failing test *and* all related tests are re-executed to ensure the fix does not introduce new faults.
6. **Close:** Issue is marked closed only after the regression test passes and code is merged to main branch.

16.11 Entry and Exit Criteria

Clear criteria govern transitions between test phases and final acceptance.

16.11.1 Entry Criteria

Each phase has prerequisites that must be satisfied before testing begins:

- **Unit Testing:** Source code committed, compiles without errors or warnings, code review completed and approved.
- **Integration Testing:** All relevant unit tests pass (100%), hardware available and powered, interface specifications documented.
- **System Testing:** All integration tests pass, robot fully assembled, power system verified safe by BS (Electrical Lead).
- **Acceptance Testing:** All system tests pass, user documentation complete, demonstration environment prepared.

16.11.2 Exit Criteria

Table 99 specifies the quantitative thresholds required to exit each phase.

Table 99: Exit Criteria by Test Phase

Phase	Execution	Pass Rate	Critical Defects	Coverage
Unit Testing	100%	$\geq 95\%$	0 open	$\geq 80\%$ code
Integration Testing	100%	$\geq 90\%$	0 open	100% interfaces
System Testing	100%	$\geq 85\%$	0 open	100% requirements
Acceptance Testing	100%	100%	0 open	Sponsor sign-off

16.12 Risk Assessment

Table 100 identifies risks that could impact the testing schedule or outcomes, along with mitigation strategies. These risks complement the project-level risks documented in Section 12.0.

Table 100: Test Risk Register

ID	Risk Description	Prob.	Impact	Mitigation Strategy
TR-001	Hardware damage during testing	Med	High	Maintain spare components; enforce current-limited power-up procedures
TR-002	Firmware deadline overrun	Med	High	Begin integration testing early; parallelize with hardware verification
TR-003	Real-time performance insufficient	Med	High	Profile early; optimize critical paths; reduce logging in production
TR-004	GPIO overvoltage damage	Low	Critical	Implement strict pre-power voltage verification; use protective diodes
TR-005	Power system fault	Low	Critical	Fused connections; BMS monitoring; thermal shutoff via NTC thermistors
TR-006	SLAM accuracy below requirements	Med	High	Calibrate LiDAR carefully; tune SLAM Toolbox parameters; improve odometry
TR-007	Test environment unavailable	Low	Med	Schedule lab time in advance; maintain Gazebo simulation as backup

16.13 Quality Assurance Practices

Quality is assured through multiple complementary mechanisms applied throughout the development and test cycle:

- **Code Reviews:** All firmware and ROS 2 node changes require peer review and approval

before merge to main branch.

- **Automated Regression:** CI/CD pipeline executes unit tests on every commit; integration tests run nightly on available hardware.
- **Design Reviews:** Formal review at each phase gate (PDR, Design Review, Final Demonstration) with documented decisions and action items.
- **Traceability:** Bidirectional links between requirements (FR/NFR), design documents, code, and test cases.
- **Documentation Review:** Technical documentation peer-reviewed for accuracy and completeness before milestone submissions.

16.14 Tools and Development Environment

Table 101: Development Tools

Category	Tool	Purpose
Version Control	Git/GitHub	Source code management, issue tracking
IDE	CLion/VS Code	Development environment
Testing	PlatformIO + Unity	Automated unit testing for ESP32-S3
Documentation	LaTeX, Doxygen	Technical documentation, API references
Project Management	Projectum	Custom task tracking aligned with WBS
Visualization	RViz2	Real-time sensor data and map visualization

16.15 Deployment and Acceptance Plan

Final deployment follows a structured sequence aligned with project milestones (Table 25):

1. **Final Assembly (M5):** Complete system integration with production firmware and calibrated sensors by March 31, 2026.
2. **Software Integration (M6):** Functional ROS 2 software with navigation and mapping capabilities by April 15, 2026.
3. **System Testing (M7):** Execute all system test cases in the university lab environment. Document results and resolve all Critical defects by May 1, 2026.
4. **Documentation Delivery:** Provide user manual, assembly instructions, and software installation guide.
5. **Final Demonstration (M8):** Present fully functional prototype to Dr. Porter and review panel on May 8, 2026. Execute acceptance test cases demonstrating autonomous mapping in real-world conditions.

6. **Formal Handover:** Obtain signed acceptance; release all source code and hardware designs under MIT License on GitHub.

17.0 Appendix

Section Author(s): All

17.1 Acronyms and Definitions

The following acronyms and technical terms are used throughout this document:

1. **STAR** (Simultaneous Tracking and Robotics): The autonomous indoor mapping robot developed by Locked Inc. as the subject of this senior design project.
2. **SLAM** (Simultaneous Localization and Mapping): An algorithmic technique that enables a robot to map its environment while simultaneously determining its own location within that environment.
3. **ETID** (Department of Engineering Technology and Industrial Distribution): The academic department at Texas A&M University responsible for the senior design program.
4. **RPLiDAR** (Robotic Perception LiDAR): A compact and affordable 2D LiDAR sensor used as a backup mapping sensor in the STAR Robot system.
5. **PCB** (Printed Circuit Board): A board that mechanically supports and electrically connects electronic components using conductive pathways etched into copper layers.
6. **LiDAR** (Light Detection and Ranging): A remote sensing technology that uses laser light to measure distances and create detailed 3D maps of environments.
7. **ROS2** (Robot Operating System 2): A flexible framework for writing robot software, providing libraries and tools for building robot applications.
8. **CAD** (Computer-Aided Design): Software tools used to create digital designs and technical drawings of mechanical and electrical components.
9. **MIT License**: A permissive open-source software license that allows free use, modification, and distribution with minimal restrictions, originating from the Massachusetts Institute of Technology.
10. **GitHub**: A web-based platform for version control and collaborative software development where project source code is publicly accessible and managed.
11. **AMR** (Autonomous Mobile Robot): A mobile robot capable of understanding and navigating its environment without human intervention.
12. **MIT** (Massachusetts Institute of Technology): A prestigious research university; also refers to the MIT License, a permissive open-source software license.
13. **RViz2** (ROS Visualization 2): A 3D visualization tool for ROS 2 that displays sensor data, robot models, navigation paths, and generated maps in real-time during robot operation.

14. **Nav2** (Navigation 2): The ROS 2 navigation stack providing autonomous navigation capabilities including path planning, obstacle avoidance, and behavior trees.

Note: Additional acronyms specific to sections are defined at their first occurrence in the document.

17.2 Work Breakdown Structure

The complete Work Breakdown Structure diagram is shown below.

17.2.1 Research Phase WBS

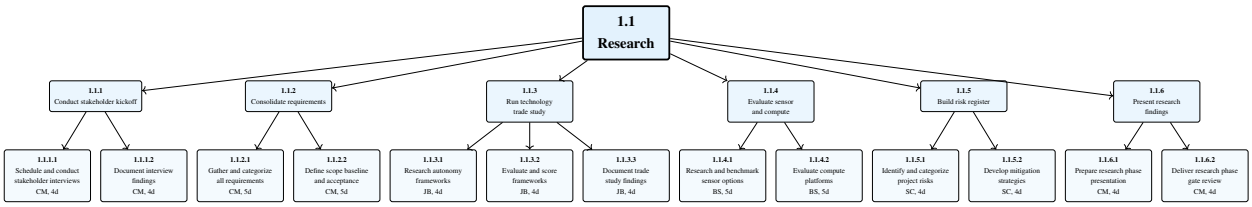


Figure 94: Research Phase Work Breakdown Structure (Full Size)

17.2.2 Design Phase WBS

The detailed Design Phase WBS diagrams are presented in Section 6.0 (Work Breakdown Structure). The Design phase contains 32 elements with 24 work packages covering system architecture, mechanical design, PCB design, software architecture, integration planning, and design review activities.

17.2.3 Build Phase WBS

The detailed Build Phase WBS diagrams are presented in Section 6.0 (Work Breakdown Structure). The Build phase contains hardware assembly, sensor integration, firmware development, and system integration activities.

17.2.4 Test Phase WBS

The detailed Test Phase WBS diagrams are presented in Section 6.0 (Work Breakdown Structure). The Test phase covers unit testing, integration testing, system validation, and performance verification activities.

17.2.5 Close Phase WBS

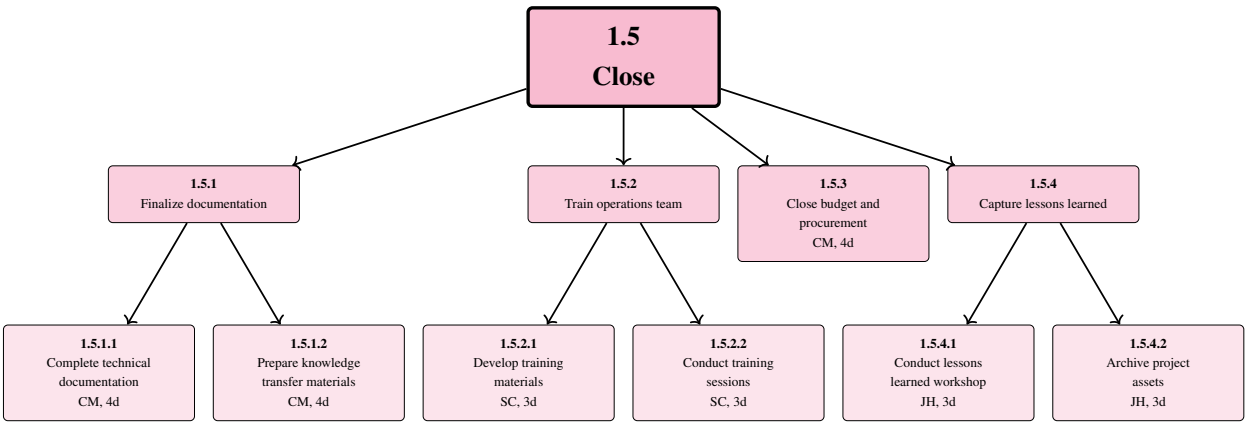


Figure 95: Close Phase Work Breakdown Structure (Full Size)

17.3 Network Logic Diagram

The complete Network Logic Diagram is shown below.

Figure 96: Complete Network Logic Diagram (Full Size)

Figure 97: Network Logic Diagram - Critical Path Highlighted

C.1 CPM Calculation Table

The following table shows the Critical Path Method (CPM) calculations for all project activities. Tasks on the critical path (Float = 0) are highlighted.

Table 102: Complete CPM Calculation Results

WBS ID	Task Name	Dur	ES	EF	LS	LF	Float
Research Phase							

Table 102 (Continued)

WBS ID	Task Name	Dur	ES	EF	LS	LF	Float
1.1.1.1	**Schedule and conduct s...**	**4**	**0**	**4**	**0**	**4**	**0 (CP)**
1.1.1.2	**Document interview fin...**	**4**	**4**	**8**	**4**	**8**	**0 (CP)**
1.1.2.1	**Gather and categorize ...**	**5**	**8**	**13**	**8**	**13**	**0 (CP)**
1.1.2.2	**Define scope baseline ...**	**5**	**13**	**18**	**13**	**18**	**0 (CP)**
1.1.3.1	**Research autonomy fram...**	**4**	**18**	**22**	**18**	**22**	**0 (CP)**
1.1.3.2	**Evaluate and score fra...**	**4**	**22**	**26**	**22**	**26**	**0 (CP)**
1.1.3.3	**Document trade study f...**	**4**	**26**	**30**	**26**	**30**	**0 (CP)**
1.1.4.1	Research and benchmark...	5	18	23	20	25	2
1.1.4.2	Evaluate compute platf...	5	23	28	25	30	2
1.1.5.1	Identify and categoriz...	4	18	22	22	26	4
1.1.5.2	Develop mitigation str...	4	22	26	26	30	4
1.1.6.1	**Prepare research phase...**	**4**	**30**	**34**	**30**	**34**	**0 (CP)**
1.1.6.2	**Deliver research phase...**	**4**	**34**	**38**	**34**	**38**	**0 (CP)**
Design Phase							
1.2.1.1	**Define high-level syst...**	**4**	**38**	**42**	**38**	**42**	**0 (CP)**
1.2.1.2	**Design subsystem inter...**	**4**	**42**	**46**	**42**	**46**	**0 (CP)**
1.2.1.3	**Document interface con...**	**4**	**46**	**50**	**46**	**50**	**0 (CP)**
1.2.2.1	**Create initial chassis...**	**5**	**50**	**55**	**50**	**55**	**0 (CP)**
1.2.2.2	**Design drivetrain and ...**	**5**	**55**	**60**	**55**	**60**	**0 (CP)**
1.2.2.3	**Design sensor mounting...**	**5**	**60**	**65**	**60**	**65**	**0 (CP)**
1.2.2.4	**Complete CAD assembly ...**	**5**	**65**	**70**	**65**	**70**	**0 (CP)**
1.2.3.1	Define power budget an...	4	50	54	54	58	4
1.2.3.2	Design PCB schematic a...	5	54	59	58	63	4
1.2.3.3	Complete PCB layout an...	4	59	63	63	67	4
1.2.3.4	Generate manufacturing...	3	63	66	67	70	4
1.2.4.1	Define ROS 2 node arch...	4	50	54	54	58	4
1.2.4.2	Design message types a...	4	54	58	58	62	4
1.2.4.3	Design navigation and ...	4	58	62	62	66	4
1.2.4.4	Document software arch...	4	62	66	66	70	4
1.2.5.1	Define integration seq...	4	50	54	54	58	4
1.2.5.2	Design hardware integr...	4	54	58	58	62	4
1.2.5.3	Design software integr...	4	58	62	62	66	4

Table 102 (Continued)

WBS ID	Task Name	Dur	ES	EF	LS	LF	Float
1.2.5.4	Document test strategy...	4	62	66	66	70	4
1.2.6.1	**Prepare design review ...**	**5**	**70**	**75**	**70**	**75**	**0 (CP)**
1.2.6.2	**Conduct design review ...**	**5**	**75**	**80**	**75**	**80**	**0 (CP)**
1.2.7	Configure Raspberry Pi...	5	66	71	111	116	45
1.2.8.1	Configure LiDAR driver...	4	71	75	116	120	45
1.2.8.2	Test SLAM algorithm in...	4	75	79	120	124	45
Build Phase							
1.3.1.1	**Procure chassis materi...**	**5**	**80**	**85**	**80**	**85**	**0 (CP)**
1.3.1.2	**Fabricate primary chas...**	**5**	**85**	**90**	**85**	**90**	**0 (CP)**
1.3.1.3	**Assemble drivetrain an...**	**5**	**90**	**95**	**90**	**95**	**0 (CP)**
1.3.1.4	**Install mounting brack...**	**5**	**95**	**100**	**95**	**100**	**0 (CP)**
1.3.2.1	Receive and inspect PC...	4	80	84	82	86	2
1.3.2.2	Build main power distr...	5	84	89	86	91	2
1.3.2.3	Build signal and data ...	5	89	94	91	96	2
1.3.2.4	Test harness continuit...	4	94	98	96	100	2
1.3.3.1	**Install compute platfo...**	**5**	**100**	**105**	**100**	**105**	**0 (CP)**
1.3.3.2	**Mount and connect LIDA...**	**5**	**105**	**110**	**105**	**110**	**0 (CP)**
1.3.3.3	**Install additional sen...**	**5**	**110**	**115**	**110**	**115**	**0 (CP)**
1.3.3.4	**Route and secure all c...**	**5**	**115**	**120**	**115**	**120**	**0 (CP)**
1.3.3.5	**Verify all electrical ...**	**4**	**120**	**124**	**120**	**124**	**0 (CP)**
1.3.4.1	**Deploy ROS 2 software ...**	**5**	**124**	**129**	**124**	**129**	**0 (CP)**
1.3.4.2	**Validate SLAM navigati...**	**5**	**129**	**134**	**129**	**134**	**0 (CP)**
1.3.5.1	**Execute mechanical int...**	**4**	**134**	**138**	**134**	**138**	**0 (CP)**
1.3.5.2	**Execute electrical int...**	**4**	**138**	**142**	**138**	**142**	**0 (CP)**
1.3.5.3	**Execute software integ...**	**4**	**142**	**146**	**142**	**146**	**0 (CP)**
1.3.6.1	**Prepare build phase re...**	**5**	**146**	**151**	**146**	**151**	**0 (CP)**
1.3.6.2	**Conduct build phase ga...**	**5**	**151**	**156**	**151**	**156**	**0 (CP)**
Test Phase							
1.4.1.1	**Review and finalize te...**	**4**	**156**	**160**	**156**	**160**	**0 (CP)**
1.4.1.2	**Prepare test environme...**	**4**	**160**	**164**	**160**	**164**	**0 (CP)**
1.4.1.3	**Set up data collection...**	**4**	**164**	**168**	**164**	**168**	**0 (CP)**
1.4.2.1	**Execute navigation fun...**	**4**	**168**	**172**	**168**	**172**	**0 (CP)**

Table 102 (Continued)

WBS ID	Task Name	Dur	ES	EF	LS	LF	Float
1.4.2.2	**Execute sensor integra...**	**4**	**172**	**176**	**172**	**176**	**0 (CP)**
1.4.2.3	**Execute power system t...**	**4**	**176**	**180**	**176**	**180**	**0 (CP)**
1.4.2.4	**Document test results ...**	**4**	**180**	**184**	**180**	**184**	**0 (CP)**
1.4.3.1	Execute indoor navigat...	4	168	172	198	202	30
1.4.3.2	Execute multi-room map...	4	172	176	202	206	30
1.4.3.3	Execute obstacle avoid...	4	176	180	206	210	30
1.4.3.4	Analyze field trial da...	4	180	184	210	214	30
1.4.4.1	**Triage and prioritize ...**	**5**	**184**	**189**	**184**	**189**	**0 (CP)**
1.4.4.2	**Implement fixes and ve...**	**5**	**189**	**194**	**189**	**194**	**0 (CP)**
1.4.5.1	**Prepare acceptance dem...**	**4**	**194**	**198**	**194**	**198**	**0 (CP)**
1.4.5.2	**Deliver demo and final...**	**4**	**198**	**202**	**198**	**202**	**0 (CP)**
Close Phase							
1.5.1.1	**Complete technical doc...**	**4**	**202**	**206**	**202**	**206**	**0 (CP)**
1.5.1.2	**Prepare knowledge tran...**	**4**	**206**	**210**	**206**	**210**	**0 (CP)**
1.5.2.1	Develop training mater...	3	202	205	208	211	6
1.5.2.2	Conduct training sessi...	3	205	208	211	214	6
1.5.3	**Close budget, procurem...**	**4**	**210**	**214**	**210**	**214**	**0 (CP)**
1.5.4.1	Conduct lessons learne...	3	202	205	208	211	6
1.5.4.2	Archive project assets...	3	205	208	211	214	6

Legend:

- **ES** = Early Start: Earliest day the task can begin
- **EF** = Early Finish: Earliest day the task can complete (ES + Duration)
- **LS** = Late Start: Latest day the task can begin without delaying the project
- **LF** = Late Finish: Latest day the task can complete without delaying the project
- **Float** = Total Float/Slack: LS - ES (days of flexibility)
- **CP** = Critical Path: Tasks with zero float (bold rows)

17.4 Gantt Chart

The complete project Gantt chart is shown below.

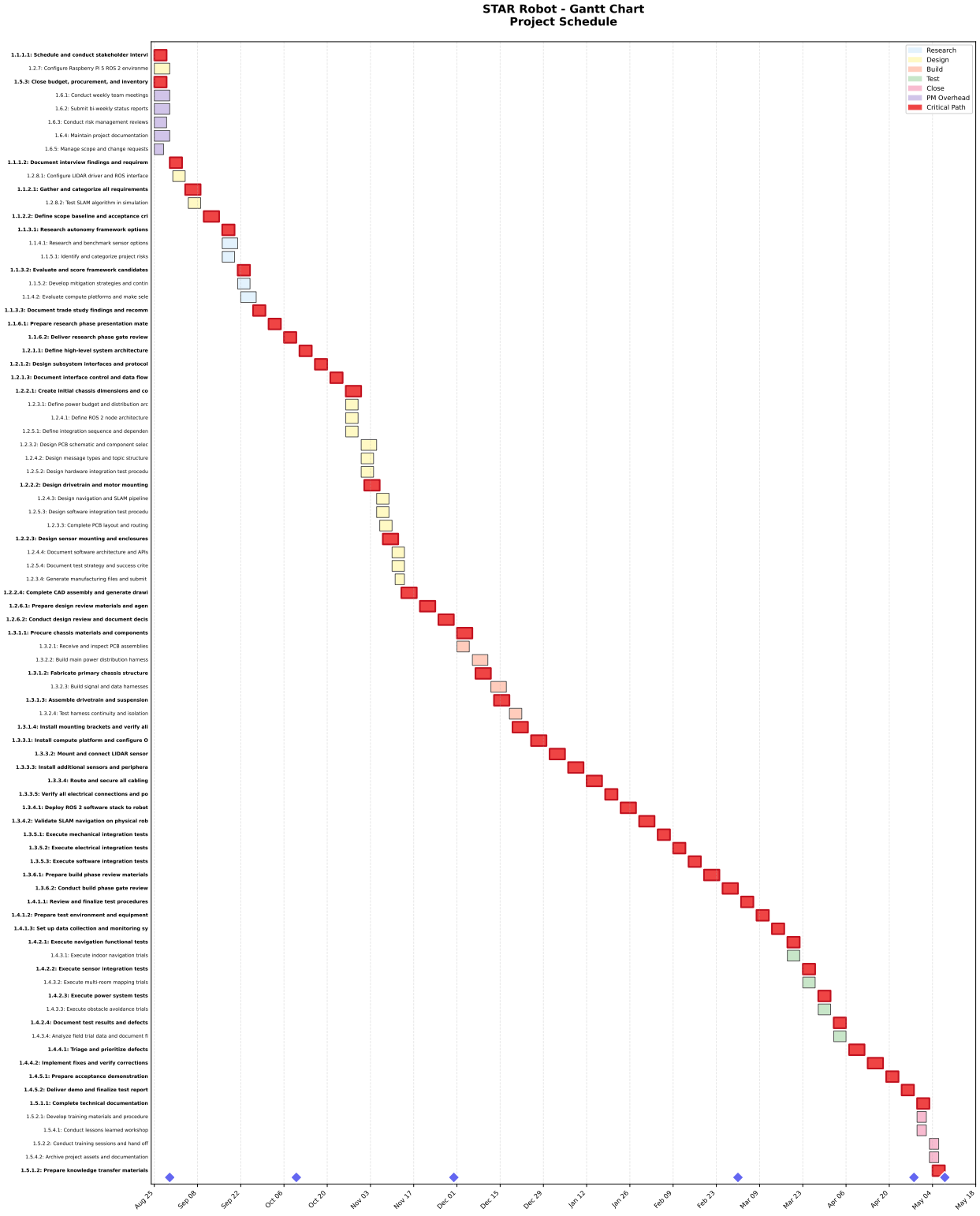


Figure 98: Complete Gantt Chart (Full Size)

17.5 Resource Allocation Matrix

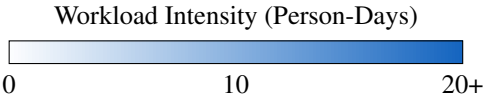
This appendix provides the complete Resource Allocation Matrix with detailed monthly workload distribution. Values show primary assignment effort (bold) with secondary support contributions (+N).

17.5.1 Monthly Resource Distribution Heat Map

Table 103 presents the full monthly allocation by team member with both primary and secondary effort. Primary values represent direct task ownership; secondary values indicate support effort at approximately 20% of primary.

Table 103: Detailed Monthly Resource Allocation (Primary + Secondary Person-Days)

Member	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Total
CM	**2**	**10**$_{+2}$	**14**$_{+3}$	**12**$_{+4}$	**6**$_{+2}$	**8**$_{+3}$	**10**$_{+3}$	**10**$_{+4}$	**12**$_{+4}$	**12**$_{+3}$	**96**
JH	**0**	**2**$_{+1}$	**3**$_{+2}$	**6**$_{+3}$	**4**$_{+2}$	**8**$_{+4}$	**12**$_{+5}$	**16**$_{+6}$	**17**$_{+6}$	**6**$_{+3}$	**74**
JB	**0**	**6**$_{+2}$	**6**$_{+2}$	**12**$_{+4}$	**8**$_{+3}$	**9**$_{+3}$	**8**$_{+3}$	**5**$_{+2}$	**5**$_{+2}$	**2**$_{+1}$	**61**
SC	**0**	**4**$_{+1}$	**4**$_{+1}$	**6**$_{+2}$	**4**$_{+1}$	**4**$_{+1}$	**4**$_{+2}$	**8**$_{+3}$	**8**$_{+3}$	**4**$_{+2}$	**46**
BS	**0**	**5**$_{+2}$	**5**$_{+2}$	**8**$_{+3}$	**6**$_{+2}$	**6**$_{+2}$	**6**$_{+2}$	**4**$_{+2}$	**4**$_{+1}$	**0**	**44**
MN	**0**	**0**	**0**	**10**$_{+3}$	**5**$_{+2}$	**5**$_{+2}$	**10**$_{+3}$	**6**$_{+2}$	**4**$_{+1}$	**0**	**40**
Primary	**2**	**27**	**32**	**54**	**33**	**40**	**50**	**49**	**50**	**24**	**361**



17.5.2 Phase-by-Phase Allocation

The following tables show resource allocation broken down by project phase.

Table 104: Research Phase Resource Allocation

Member	CM	JH	JB	SC	BS	MN	Phase Total
Primary Days	26	5	12	8	5	–	**56**

Research Phase (August – October 2025)

Table 105: Design Phase Resource Allocation

Member	CM	JH	JB	SC	BS	MN	Phase Total
Primary Days	22	16	29	12	16	8	**103**

Design Phase (November 2025 – February 2026)

Table 106: Build Phase Resource Allocation

Member	CM	JH	JB	SC	BS	MN	Phase Total
Primary Days	10	24	15	10	19	16	**94**

Build Phase (January – April 2026)

Table 107: Test Phase Resource Allocation

Member	CM	JH	JB	SC	BS	MN	Phase Total
Primary Days	8	23	5	10	4	12	**62**

Test Phase (March – May 2026)

Table 108: Close Phase Resource Allocation

Member	CM	JH	JB	SC	BS	MN	Phase Total
Primary Days	12	6	–	2	–	4	**24**

Close Phase (May 2026)

Table 109: PM Overhead Resource Allocation

Member	CM	JB	BS	JH	SC	MN	Phase Total
Primary Days	18	–	–	–	4	–	**22**

PM Overhead (Throughout Project)

17.5.3 Total Effort Summary

Table 110: Total Effort Summary by Phase and Team Member

Phase	CM	JH	JB	SC	BS	MN	Total
Research	26	5	12	8	5	0	56
Design	22	16	29	12	16	8	103
Build	10	24	15	10	19	16	94
Test	8	23	5	10	4	12	62
Close	12	6	0	2	0	4	24
PM Overhead	18	0	0	4	0	0	22
Total	**96**	**74**	**61**	**46**	**44**	**40**	**361**

17.6 Supporting Documents

Additional supporting documents:

- Team Fact Sheet
- Component Datasheets
- Design Calculations
- Meeting Minutes

17.6.1 Team Contact Information

Table 111: Team Member GitHub and LinkedIn Profiles

Name	GitHub	LinkedIn
Cesar Magana	IMMZEK	cesarmagana2002
Jared Bartz	jared-bartz-tamu	jared-bartz
Brighton Sikarskie	bsikar	bsikar
Michael Norton	mcnorton1	michael-norton
Jeremie Hews	Jhews6	jeremie-hews
Shawn Ciaciura	Scicada46	shawn-ciaciura

17.6.2 Team Fact Sheet

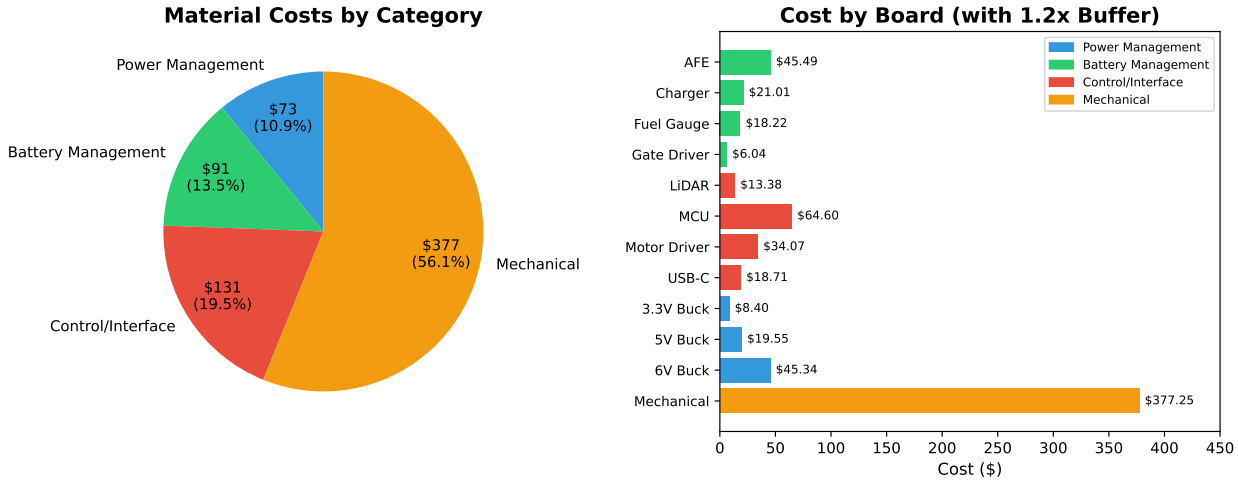
Table 112: Locked Inc. Team Contact Information

Name	Role	Phone	Email
Cesar Magana	Project Manager	915-270-0299	cesmag02@tamu.edu
Brighton Sikarskie	Hardware Engineer	682-251-4445	brighton.sikarskie@tamu.edu
Jeremie Hews	Hardware Engineer	469-720-6652	jhews6@tamu.edu
Michael Norton	Mechanical Engineer	512-839-2098	mnorton1029@tamu.edu
Shawn Ciaciura	Test Engineer	509-999-3045	Sciaciura@tamu.edu
Jared Bartz	Software Engineer	469-406-9596	jared_bartz@tamu.edu

Team Email: lockedinc67@gmail.com
Sponsor/Advisor: Dr. Logan Porter

17.7 Budget and Cost Analysis

The complete budget breakdown and cost analysis is shown below. This includes material costs, labor estimates, and cash flow projections.



Total Materials: $672.06 (Electronics: $294.81 + Mechanical: $377.25)

Figure 99: Complete Budget Breakdown and Cost Analysis

17.8 Risk Assessment

The complete risk assessment for the STAR Robot project is presented below. This appendix provides detailed risk analysis including probability assessments, impact evaluations, and mitigation strategies for all identified project risks.

17.8.1 Technical Risks

Table 113: Technical Risk Register

Risk	Probability	Impact	Mitigation Strategy
SLAM Performance Degradation	3	4	Test in varied environments; implement fallback mapping modes
LiDAR Reliability Issues	2	4	Select industrial-grade sensor; implement data filtering
Processing Resource Limits	3	3	Optimize algorithms; consider compute offloading
3D-Printed Gear Wear	4	5	Design with safety factors; maintain spare parts inventory
Integration Complexity	Medium	High	Modular architecture; comprehensive integration testing

17.8.2 Schedule Risks

Table 114: Schedule Risk Register

Risk	Probability	Impact	Mitigation Strategy
Component Lead Times	4	3	Order critical components early; identify backup suppliers
Testing Delays	3	3	Build buffer time into schedule; parallel testing where possible
Scope Creep	2	4	Strict change control; prioritize core requirements

17.8.3 Resource Risks

Table 115: Resource Risk Register

Risk	Probability	Impact	Mitigation Strategy
Budget Overrun	2	3	Track expenses closely; maintain contingency reserve
Team Availability	3	3	Cross-train team members; document all work
Equipment Failure	2	4	Maintain backup equipment; regular maintenance schedule

17.9 Bill of Materials

Complete Bills of Materials for all PCB designs. Each table includes reference designators, values, quantities, manufacturer part numbers, and supplier pricing from Digi-Key and Mouser. **Cost calculations use the lower price between suppliers. A 1.2× spare parts buffer (20% additional) is applied in budget totals (Section 13.0).**

17.9.1 Motor Driver Board BOM

Table 116: Motor Driver Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1–C4	56μF	4	865080343007	732-8417-1-ND	710-865080343007	0.26	0.27
C5–C8	220nF	4	CL10B224KB8NNNC	1276-CL10B224KB8NNNCCT-ND	187-CL10B224KB8NNNC	0.11	0.11
C9–C12	100nF	4	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
D1–D4	Red LED	4	150060RS75000	732-4978-1-ND	710-150060RS75000	0.15	0.16
J1,J4–J6	WR-WTB 8P	4	61200821621	732-5395-ND	710-61200821621	0.97	1.08
J2	WR-WTB 6P	1	61200621621	732-5394-ND	710-61200621621	0.43	0.47
J3,J10–J12	JST XH 6P	4	B6B-XH-A	455-B6B-XH-A-ND	–	0.20	–
J7	Phoenix 2P	1	1017491	277-17493-ND	651-1017491	1.06	0.72
R1–R4	10kΩ	4	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R5–R8	68Ω	4	RT0603FRE0768RL	13-RT0603FRE0768RLCT-ND	603-RT0603FRE0768RL	0.10	0.10

Continued on next page

Table 116 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
R9–R12	3.16kΩ	4	ERA-3AEB3161V	P3.16KDBCT-ND	667-ERA-3AEB3161V	0.10	0.10
U1–U4	DRV8243S	4	DRV8243SQDGQRQ1	296-DRV8243SQDGQRQ1CT-ND	595-DRV8243SQDGQRQ1	4.72	6.37
					Board Total (Best Price):		**$28.39**

17.9.2 USB-C PD Controller Board BOM

Table 117: USB-C Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1,C5,C8,C14,C17,C24,C28,C29	100nF	8	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C2	1μF	1	UMK107AB7105KA-T	587-3247-1-ND	–	0.08	0.10
C3–C13,C22,C25	10μF	11	GRM21BR61H106KE3L	490-18663-1-ND	81-GRM21BR61H106KE3L	0.28	0.28
C15,C16	330pF	2	CL10B331KB8WPNC	1276-CL10B331KB8WPNCCT-ND	187-CL10B331KB8WPNC	0.10	0.10
C18–C21	10nF	4	CL10B103KB8NNNL	1276-CL10B103KB8NNNLCT-ND	187-CL10B103KB8NNNL	0.10	0.10
C23	22μF	1	T527T226M025ATE100	–	80-T527T226M25ATE100	–	1.13
C26,C27,C30,C31	4.7μF	4	GRT21BR61H475KE3K	490-GRT21BR61H475KE13KCT-ND	81-GRT21BR61H475KE3K	0.17	0.17
D1,D2	TVS	2	82306050029	732-3040-1-ND	710-82306050029	0.73	0.73
FB1	22Ω	1	BLM18SN220TN1D	490-13961-1-ND	81-BLM18SN220TN1D	0.19	0.19
J1,J2	Phoenix 2P	2	1017491	277-17493-ND	651-1017491	1.06	0.72
J4	WR-WTB 6P	1	61200621621	732-5394-ND	710-61200621621	0.43	0.47
J5	Header 2P	1	M20-9990246	952-2262-ND	855-M20-9990246	0.10	0.10
J7	USB-C	1	USB4216-03-A	2073-USB4216-03-ACT-ND	640-USB4216-03-A	0.62	0.72
R1–R4	10kΩ	4	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R5,R6	3.3kΩ	2	–	311-2366-1-ND	603-RT0603DRD073K3L	0.13	0.13
R7	100kΩ	1	AC0603JR-07100KL	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
U1	EEPROM	1	CAT24C512WI-GT3	CAT24C512WI-GT3OSCT-ND	863-CAT24C512WI-GT3	0.59	0.59
U2	TPS25751D	1	TPS25751DREFR	296-TPS25751DREFRCT-ND	595-TPS25751DREFR	2.90	2.91

Continued on next page

Table 117 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
U3	TVS	1	TVS2200DRVR	296-52187-1-ND	595-TVS2200DRVR	0.53	0.83
					Board Total (Best Price):		**$15.59**

17.9.3 AFE Board BOM

Table 118: AFE Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1–C4	220nF	4	CL10B224KB8NNNC	1276-CL10B224KB8NNNCCT-ND	187-CL10B224KB8NNNC	0.11	0.11
C5,C6,C8–C10,C14,C15	100nF	7	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C7,C12	1μF	2	UMK107AB7105KA-T	587-3247-1-ND	UMK107AB7105KA-T	0.08	0.10
C11,C13	10μF	2	GRM21BR61H106KE3L	490-18663-1-ND	81-GRM21BR61H106KE3L	0.28	0.28
C16	470pF	1	VJ0603Y471KXAPW1BC	720-VJ0603Y471KXAPW1BCCT-ND	77-VJ0603Y471KXAPBC	0.10	0.10
C17,C18	4.7nF	2	CGA3E2X7R1H472K080AA	445-5661-1-ND	810-CGA3E2X7R1H472K	0.10	0.10
D1–D3	Yellow LED	3	SML-D12Y1WT86	SML-D12Y1WT86CT-ND	755-SML-D12Y1WT86	0.19	0.19
D4–D6	5.6V Zener	3	DDZ9690-7	DDZ9690DICT-ND	621-DDZ9690-7	0.10	0.10
D7	1N4148W	1	1N4148W	4878-1N4148WCT-ND	637-1N4148W	0.10	0.10
D8,D10	TVS	2	PTVS14VP1UP,115	1727-4427-1-ND	771-PTVS14VP1UP115	0.49	0.49
D9	3.3V Zener	1	MMSZ5226C-E3-08	MMSZ5226C-E3-08GICT-ND	78-MMSZ5226C-E3-08	0.18	0.18
D11	Schottky	1	V5N3202HM3/I	112-V5N3202HM3/ICT-ND	625-V5N3202HM3/I	0.99	1.00
D12	Red LED	1	150060RS75000	732-4978-1-ND	710-150060RS75000	0.15	0.16
F1,F2	Fuse	2	3568	36-3568-ND	534-3568	1.20	1.09
FB1–FB4	220Ω	4	BBPY00201209221Y00	553-BBPY00201209221Y00CT-ND	603-BBPY201209221Y00	0.10	0.10
J1	Phoenix 4P	1	1017495	277-17497-ND	651-1017495	2.86	2.17
J3	MicroSD	1	694106301002	732-5930-ND	710-694106301002	0.92	1.02
J4,J5	WR-WTB 8P	2	61200821621	732-5395-ND	710-61200821621	0.97	1.08
J6,J7	Phoenix 2P	2	1017491	277-17493-ND	651-1017491	1.06	0.72
Q1–Q3	DMN2004K-7	3	DMN2004K-7	DMN2004KDICT-ND	621-DMN2004K-7	0.41	0.41
Q4,Q5,Q7,Q8	CSD19531Q5AT	4	CSD19531Q5AT	296-37749-1-ND	595-CSD19531Q5AT	3.15	3.21
Q6	ZXMP10A13FTA	1	ZXMP10A13FTA	ZXMP10A13FCT-ND	522-ZXMP10A13FTA	0.76	0.72
Q9	FDD4141	1	FDD4141	4518-FDD4141CT-ND	512-FDD4141	1.00	1.45

Continued on next page

Table 118 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
R1–R3	100Ω	3	CRCW0603100RJNEA	541-100GCT-ND	71-CRCW0603J-100-E3	0.10	0.10
R4–R6	33Ω	3	CRCW060333R0FKEAC	541-3988-1-ND	71-CRCW060333R0FKEAC	0.10	0.10
R7,R9,R12,R14,R22,R24	10kΩ	6	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R8,R10,R11,R13,R17,R26	1kΩ	6	AC0603JR-131KL	13-AC0603JR-131KLCT-ND	603-AC0603JR-131KL	0.10	0.10
R15,R18,R20	100Ω	3	CRCW0603100RJNEA	541-100GCT-ND	71-CRCW0603J-100-E3	0.10	0.10
R16,R21,R23	10MΩ	3	RK73H1JTTD1005F	2019-RK73H1JTTD1005FCT-ND	660-RK73H1JTTD1005F	0.10	0.10
R19	2mΩ	1	WSK1216L2000FEA	541-WSK1216L2000FEACT-ND	71-WSK1216L2000FEA	1.41	1.41
R25	510kΩ	1	CRCW0603510KJNEA	541-510KGCT-ND	71-CRCW0603J-510K-E3	0.10	0.10
R27	24Ω	1	RT0603BRD0724RL	13-RT0603BRD0724RLCT-ND	603-RT0603BRD0724RL	0.10	0.10
R28	3.01kΩ	1	RC0603DR-073K01L	13-RC0603DR-073K01LCT-ND	603-RC0603DR-073K01L	0.10	0.10
S1	Button	1	SKRPACE010	4809-SKRPACE010CT-ND	688-SKRPACE010	0.21	0.21
U1	BQ76920	1	BQ7692000PWR	296-39958-1-ND	595-BQ7692000PWR	3.56	3.56
					Board Total (Best Price):		**$37.91**

17.9.4 MCU Board BOM

Table 119: MCU Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1,C3,C4	100nF	3	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C2	1μF	1	UMK107AB7105KA-T	587-3247-1-ND	UMK107AB7105KA-T	0.08	0.10
C5	10μF	1	GRM21BR61H106KE3L	490-18663-1-ND	81-GRM21BR61H106KE3L	0.28	0.28
D1,D2	TVS	2	82306050029	732-3040-1-ND	710-82306050029	0.73	0.73
J1–J7	HC-SR04	7	4007	1528-2832-ND	485-4007	3.95	3.95
J8	Pi GPIO	1	61304021121	732-5310-ND	710-61304021121	1.81	2.01
J9	9-pin	1	61300911121	732-5322-ND	710-61300911121	0.39	0.43
J10,J12	SWD	2	PH1-04-UA	2057-PH1-04-UA-ND	737-PH1-04-UA	0.10	0.10
J11	WR-WTB 6P	1	61200621621	732-5394-ND	710-61200621621	0.43	0.47
J13–J16,J19	WR-WTB 8P	5	61200821621	732-5395-ND	710-61200821621	0.97	1.08

Continued on next page

Table 119 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
J17	USB-C	1	USB4216-03-A	2073-USB4216-03-ACT-ND	640-USB4216-03-A	0.62	0.72
J18,J21	WR-WTB 6P	2	61200621621	732-5394-ND	710-61200621621	0.43	0.47
J20	JST XH 5P	1	S5B-XH-A	455-S5B-XH-A-ND	–	0.22	–
R1–R5,R8	10kΩ	6	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R6,R7	5.1kΩ	2	RT0603BRD075K1L	YAG1696CT-ND	603-RT0603BRD075K1L	0.10	0.10
S1	Reset	1	SKRPACE010	4809-SKRPACE010CT-ND	688-SKRPACE010	0.21	0.21
S2	Boot	1	SKRPACE010	4809-SKRPACE010CT-ND	688-SKRPACE010	0.21	0.21
U1	DS18B20	1	DS18B20+	DS18B20+PAR-ND	700-DS18B20+	7.00	6.59
U2	ESP32-S3	1	ESP32-S3-WROOM-1-N16	5407-ESP32-S3-WROOM-1-N16CT-ND	356-ESP32S3WROOM1N16	5.92	5.99
U3	74HC139	1	74HC139D,653	1727-5943-1-ND	771-74HC139D-T	0.36	0.36
U4	74HC153	1	SN74HC153DR	296-14835-1-ND	595-SN74HC153DR	0.59	1.28
					Board Total (Best Price):		**$53.83**

17.9.5 Buck-Boost Charger Board BOM

Table 120: Buck-Boost Charger Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1	4.7μF	1	GRT21BR61H475KE13K	490-GRT21BR61H475KE13KCT-ND	81-GRT21BR61H475KE3K	0.17	0.17
C2,C3	47nF	2	CC0603KPX7R9BB473	13-CC0603KPX7R9BB473CT-ND	603-CC603KPX7R9BB473	0.10	0.10
C4,C14,C19,C23	100nF	4	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C5–C13,C16–C18,C20–C22	10μF	13	GRM21BR61H106KE3L	490-18663-1-ND	81-GRM21BR61H106KE3L	0.28	0.28
C15	1nF	1	CC0603KPX7R9BB102	13-CC0603KPX7R9BB102CT-ND	603-CC603KPX7R9BB102	0.10	0.10
C24–C26	22μF	3	T527T226M025ATE100	–	80-T527T226M25ATE100	–	1.13
D1	Red LED	1	150060RS75000	732-4978-1-ND	710-150060RS75000	0.15	0.16
J2–J4	Phoenix 2P	3	1017491	277-17493-ND	651-1017491	1.06	0.72
L1	1μH	1	VLS6045EX-1R0N	445-173043-1-ND	810-VLS6045EX-1R0N	0.34	0.30
R1	5.3kΩ	1	TNPW06035K30BEEA	541-4550-1-ND	71-TNPW06035K30BEEA	0.23	0.23

Continued on next page

Table 120 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
R2	30.1kΩ	1	AT0603FRE0730K1L	13-AT0603FRE0730K1LCT-ND	603-AT0603FRE0730K1L	0.10	0.10
R3	137kΩ	1	RC0603FR-07137KL	311-137KHRCT-ND	603-RC0603FR-07137KL	0.10	0.10
R4	100kΩ	1	AC0603JR-07100KL	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
R5	10kΩ	1	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R6	68Ω	1	RT0603FRE0768RL	13-RT0603FRE0768RLCT-ND	603-RT0603FRE0768RL	0.10	0.10
R7	10.5kΩ	1	RT0603FRE0710K5L	13-RT0603FRE0710K5LCT-ND	603-RT0603FRE0710K5L	0.10	0.10
R8	100Ω	1	CRCW0603100RJNEA	541-100GCT-ND	71-CRCW0603J-100-E3	0.10	0.10
TH1	NTC	1	NTCGS163JX103DT7	445-NTCGS163JX103DT7CT-ND	810-NTCGS163JX103DT7	0.30	0.26
U1	BQ25798	1	BQ25798RQMR	296-BQ25798RQMRCT-ND	595-BQ25798RQMR	5.05	5.05
					Board Total (Best Price):		**$17.51**

17.9.6 Fuel Gauge Board BOM

Table 121: Fuel Gauge Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1,C2	3.3nF	2	GCM188R71H332KA37D	490-8023-1-ND	81-GCM188R71H332KA7D	0.12	0.10
C3	4.7μF	1	GRT21BR61H475KE13K	490-GRT21BR61H475KE13KCT-ND	81-GRT21BR61H475KE3K	0.17	0.17
C4,C7	1μF	2	UMK107AB7105KA-T	587-3247-1-ND	UMK107AB7105KA-T	0.08	0.10
C5,C6,C8	100nF	3	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
D1–D5	YelGreen LED	5	SML-D12M1WT86	SML-D12M1WT86CT-ND	755-SML-D12M1WT86	0.19	0.19
D6,D7	5.6V Zener	2	DDZ9690-7	DDZ9690DICT-ND	621-DDZ9690-7	0.10	0.10
D8,D9	TVS	2	82306050029	732-3040-1-ND	710-82306050029	0.73	0.73
J1	WR-WTB 8P	1	61200821621	732-5395-ND	710-61200821621	0.97	1.08
J5,J6	WR-WTB 6P	2	61200621621	732-5394-ND	710-61200621621	0.43	0.47
Q1	ZXMP10A13FTA	1	ZXMP10A13FTA	ZXMP10A13FCT-ND	522-ZXMP10A13FTA	0.76	0.72
Q2	BSS123	1	BSS123	BSS123NCT-ND	512-BSS123	0.35	0.36

Continued on next page

Table 121 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
Q3	MMBT3904	1	MMBT3904	4878-MMBT3904CT-ND	637-MMBT3904	0.11	0.11
R1,R2,R4	300kΩ 0.1%	3	ERA-3AEB304V	P300KDBCT-ND	667-ERA-3AEB304V	0.10	0.10
R3,R6,R9,R10	100kΩ	4	AC0603JR-07100KL	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
R5	24.9kΩ 0.1%	1	RT0603BRD0724K9L	YAG1599CT-ND	603-RT0603BRD0724K9L	0.12	0.10
R7	10kΩ 0.1%	1	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R8,R18	510kΩ	2	CRCW0603510KJNEA	541-510KGCT-ND	71-CRCW0603J-510K-E3	0.10	0.10
R11–R16	100Ω	6	CRCW0603100RJNEA	541-100GCT-ND	71-CRCW0603J-100-E3	0.10	0.10
R13,R14	1MΩ	2	AC0603FR-071ML	311-1MLDCT-ND	603-AC0603FR-071ML	0.10	0.10
R17	220kΩ	1	CRCW0603220KFKEA	541-220KHCT-ND	71-CRCW0603-220K-E3	0.10	0.10
R19,R20	10kΩ	2	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
S1	Button	1	SKRPACE010	4809-SKRPACE010CT-ND	688-SKRPACE010	0.21	0.21
U1	BQ78350	1	BQ78350DBT-R1A	296-BQ78350DBT-R1A-ND	595-BQ78350DBT-R1A	5.83	5.83
U2	PCA9306	1	PCA9306D,118	568-4215-1-ND	771-PCA9306D-T	0.72	0.69
					Board Total (Best Price):		**$15.18**

17.9.7 Gate Driver Board BOM

Table 122: Gate Driver Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1,C2	10nF	2	CL10B103KB8NNNL	1276-CL10B103KB8NNNLCT-ND	187-CL10B103KB8NNNL	0.10	0.10
C3	1μF	1	UMK107AB7105KA-T	587-3247-1-ND	–	0.08	0.10
J1	WR-WTB 8P	1	61200821621	732-5395-ND	710-61200821621	0.97	1.08
J2	WR-WTB 6P	1	61200621621	732-5394-ND	710-61200621621	0.43	0.47
R1–R3	100Ω	3	–	541-100GCT-ND	71-CRCW0603J-100-E3	0.10	0.10
R4–R6	620Ω	3	–	13-RC0603FR-10620RLCT-ND	603-RC0603FR-10620RL	0.10	0.10
U1	BQ76200PW	1	BQ76200PW	296-43316-5-ND	595-BQ76200PWR	3.23	2.75
					Board Total (Best Price):		**$5.03**

17.9.8 3.3V Buck Regulator Board BOM

Table 123: 3.3V Buck Regulator Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1,C2,C4	22μF	3	T527T226M025ATE100	N/A	80-T527T226M25ATE100	–	1.13
C3,C5,C7	100nF	3	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C6	10μF	1	GRM21BR61H106KE3L	490-18663-1-ND	81-GRM21BR61H106KE3L	0.28	0.28
C8	4.7μF	1	GRT21BR61H475KE3K	490-GRT21BR61H475KE13KCT-ND	81-GRT21BR61H475KE3K	0.17	0.17
J1	Phoenix 2P	1	1017491	277-17493-ND	651-1017491	1.06	0.72
J2	WR-WTB 6P	1	61200621621	732-5394-ND	710-61200621621	0.43	0.47
J3	WR-WTB 8P	1	61200821621	732-5395-ND	710-61200821621	0.97	1.08
L1	1μH	1	VLS6045EX-1R0N	445-173043-1-ND	810-VLS6045EX-1R0N	0.34	0.30
R1	100kΩ	1	–	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
R2	453kΩ	1	–	13-RT0603FRE07453KLCT-ND	603-RT0603FRE07453KL	0.10	0.10
U1	TLV62569ADRLR	1	TLV62569ADRLR	296-51692-1-ND	595-TLV62569ADRLR	0.24	0.24
					Board Total (Best Price):		**$7.00**

17.9.9 5V Buck Regulator Board BOM

Table 124: 5V Buck Regulator Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1–C6,C12–C14	22μF	9	T527T226M025ATE100	N/A	80-T527T226M25ATE100	–	1.13
C7–C9	100nF	3	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C10	1μF	1	UMK107AB7105KA-T	587-3247-1-ND	–	0.08	0.10
C11	33pF	1	–	311-4034-1-ND	603-CC603KRNPO9BN330	0.10	0.10
J1,J3–J5	Phoenix 2P	4	1017491	277-17493-ND	651-1017491	1.06	0.72
L1	2.2μH	1	PA4343.222NLT	553-3451-1-ND	673-PA4343.222NLT	1.50	1.50
R1	100kΩ	1	–	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
R2	220kΩ	1	–	541-220KHCT-ND	71-CRCW0603-220K-E3	0.10	0.10
R3	30kΩ	1	–	13-RT0603FRE0730KLCT-ND	603-RT0603FRE0730KL	0.10	0.10

Continued on next page

Table 124 (continued)

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
U1	TPS566238PRQFR	1	TPS566238PRQFR	296-TPS566238PRQFRCT-ND	595-TPS566238PRQFR	1.79	0.96
					Board Total (Best Price):		**$16.29**

17.9.10 6V Buck Regulator Board BOM

Table 125: 6V Buck Regulator Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C117–C124,C133–C138	10μF	14	T521B106M025ATE100	399-16632-2-ND	80-T521B106M25ATE100	1.74	1.74
C125,C132	1μF	2	UMK107AB7105KA-T	587-3247-1-ND	–	0.08	0.10
C126	3.3nF	1	GCM188R71H332KA7D	490-8023-1-ND	81-GCM188R71H332KA7D	0.12	0.10
C127	2.2μF	1	GRT188R61H225KE3D	490-12332-1-ND	81-GRT188R61H225KE3D	0.16	0.16
C128	220pF	1	–	13-CC0603KPX7R9BB221CT-ND	603-CC603KPX7R9BB221	0.10	0.10
C129	6.8nF	1	–	BC1251CT-ND	77-VJ0603Y682KXACBC	0.10	0.10
C130	100nF	1	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
C131	1nF	1	–	13-CC0603KPX7R9BB102CT-ND	603-CC603KPX7R9BB102	0.10	0.10
J1,J3	Phoenix 2P	2	1017491	277-17493-ND	651-1017491	1.06	0.72
L4	0.44μH	1	SPB1308-R44M	118-SPB1308-R44MCT-ND	652-SPB1308-R44M	1.58	1.89
Q13,Q14	CSD19531Q5AT	2	CSD19531Q5AT	296-37749-1-ND	595-CSD19531Q5AT	3.15	3.21
R82	100kΩ	1	–	311-100KLECT-ND	603-AC0603JR-07100KL	0.10	0.10
R83	1.37kΩ	1	–	YAG4804CT-ND	603-RT0603DRE071K37L	0.10	0.10
R84	7.68kΩ	1	–	13-RT0603FRE077K68LCT-ND	603-RT0603FRE077K68L	0.10	0.10
R85	309Ω	1	ERA-3AEB3090V	P309DBCT-ND	667-ERA-3AEB3090V	0.10	0.10
R86	10kΩ	1	RT0603BRE0710KL	YAG2313CT-ND	603-RT0603BRE0710KL	0.14	0.10
R87	1kΩ	1	–	13-AC0603JR-131KLCT-ND	603-AC0603JR-131KL	0.10	0.10
U14	TPS40305DRCT	1	TPS40305DRCT	296-25487-1-ND	595-TPS40305DRCT	2.68	2.68
					Board Total (Best Price):		**$37.78**

17.9.11 LiDAR Breakout Board BOM

Table 126: LiDAR Breakout Board Complete BOM

Ref	Value	Qty	MPN	DigiKey PN	Mouser PN	DK $	MO $
C1	4.7μF	1	GRT21BR61H475KE3K	490-GRT21BR61H475KE13KCT-ND	81-GRT21BR61H475KE3K	0.17	0.17
C2	1μF	1	UMK107AB7105KA-T	587-3247-1-ND	–	0.08	0.10
C3	100nF	1	CC603JRX7R9BB104	311-1779-1-ND	603-CC603JRX7R9BB104	0.10	0.10
J2	M12 5P	1	1028585	277-1028585-ND	651-1028585	10.08	13.90
Q1	ZXMP10A13FTA	1	ZXMP10A13FTA	ZXMP10A13FCT-ND	522-ZXMP10A13FTA	0.76	0.72
					Board Total (Best Price):		**$11.15**

17.9.12 Electronics Cost Summary

Table 127: PCB Electronics Cost Summary

Board	Parts	Raw Cost	With 1.2× Buffer
Power Management			
3.3V Buck Regulator	15	$7.00	$8.40
5V Buck Regulator	23	$16.29	$19.55
6V Buck Regulator	34	$37.78	$45.34
Battery Management			
Analog Front-End (AFE)	81	$37.91	$45.49
Buck-Boost Charger	43	$17.51	$21.01
Fuel Gauge	46	$15.18	$18.22
Gate Driver	12	$5.03	$6.04
Control/Interface			
LiDAR Breakout	5	$11.15	$13.38
MCU Board	42	$53.83	$64.60
Motor Driver	42	$28.39	$34.07
USB-C PD Controller	51	$15.59	$18.71
Electronics Total	**394**	**$245.66**	**$294.81**

17.9.13 Mechanical Components

Table 128: Mechanical Components Bill of Materials

Item	Qty	Unit Cost	Total Cost
1611 Series Flanged Ball Bearing (8mm ID, 14mm OD, 5mm Thick), 2-Pack	4	$4.00	$16.00
8mm REX Shaft with E-Clip (88mm, Stainless Steel)	4	$5.29	$21.16
1309 Series Sonic Hub (8mm REX Bore)	4	$8.00	$32.00
2910 Series Aluminum Clamping Collar (8mm REX ID)	4	$5.00	$20.00
2807 Series Stainless Steel Shim (12-Pack)	1	$2.69	$2.69
Prusament PETG Filament (1 kg)	1	$30.00	$30.00
1120 Series U-Channel (288 mm, 11-Hole)	2	$15.00	$30.00
6061 Aluminum Sheet (12in × 12in, 0.080in thick)	1	$21.00	$21.00
Wasteland Wheel (144mm × 52mm)	4	$25.00	$100.00
Metal DC Geared Motor w/ Encoder (6V, 210 RPM, 10 kg·cm)	4	$20.00	$80.00
2802 Series Steel Button-Head Screw (M4×12mm), 25-Pack	2	$3.60	$7.20
M4 × 0.7mm Nylock Nut	2	$3.00	$6.00
1501 Series M4 Standoff (10mm), 4-Pack	4	$2.80	$11.20
Total Cost			**$377.25**

Note: Pricing shown is per-unit at time of design; verify current pricing before ordering.

17.10 ESP32 Firmware Source Code

This appendix contains the complete source code for the STAR ESP32 firmware framework. The firmware implements a production-ready embedded system with dependency injection, thread-safe communication, and comprehensive error handling. The full repository with all libraries is available at the project GitHub.

17.10.1 Dependency Inversion Principle (DIP) in Embedded C

The STAR firmware implements the **Dependency Inversion Principle (DIP)**, the "D" in SOLID, in pure C without object-oriented language features. DIP states:

1. **High-level modules should not depend on low-level modules.** Both should depend on abstractions.
2. **Abstractions should not depend on details.** Details should depend on abstractions.

In traditional embedded C, sensor drivers often directly call I^2C or SPI functions, creating tight coupling that makes testing and hardware changes difficult. DIP inverts this relationship:

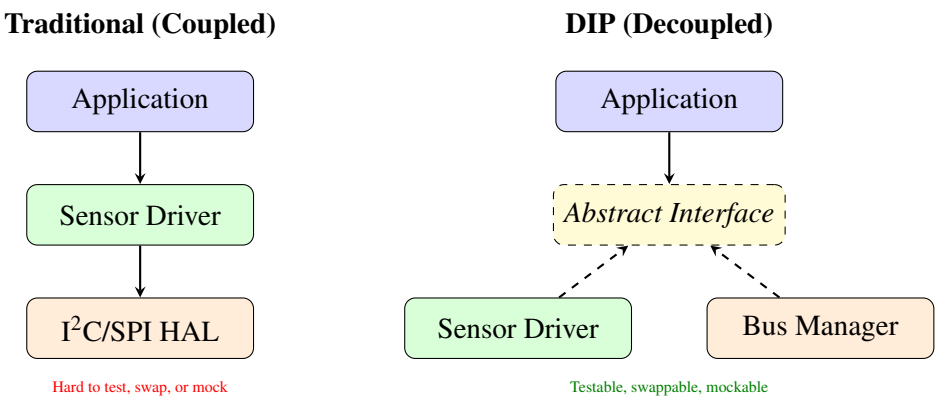


Figure 100: Traditional vs. DIP Architecture Comparison

Implementation in C. DIP is implemented using `structs` with function pointers. Each interface defines a contract that implementations must fulfill:

Code Snippet 6: Abstract Interface Definition

```c
/* star_error_interface.h - Abstract contract for error handling */
typedef struct star_error_interface {
    /* Function pointers define the contract */
    esp_err_t (*record_error)(void* ctx, esp_err_t error, const char* message,
                              const char* file, int line, const char* func);
    bool (*can_retry)(void* ctx);
    esp_err_t (*reset_state)(void* ctx);

    /* Opaque pointer to concrete implementation - could be ANY error handler */
    void* ctx;
} star_error_interface_t;
```

Concrete implementations provide the actual behavior:

Components receive interfaces via constructor injection:

Benefits in Practice.

- **Unit Testing**: Pass mock interfaces that record calls instead of touching hardware

Code Snippet 7: Concrete Implementation Binding

```c
/* star_error_handler.c - Concrete implementation with retry logic */
void error_handler_get_interface(star_error_interface_t* iface,
                                 error_handler_t* handler) {
    /* Bind function pointers to concrete implementation */
    iface->record_error = internal_record_error;
    iface->can_retry    = internal_can_retry;
    iface->reset_state  = internal_reset_state;
    iface->ctx          = handler;  /* Pass concrete instance */
}
```

Code Snippet 8: Dependency Injection via Constructor

```c
/* Sensor driver receives abstract interfaces, not concrete implementations */
esp_err_t star_sensor_hcsr04_init(hcsr04_handle_t* handle,
                                  star_bus_manager_t* manager,  /* Bus abstraction */
                                  const char* bus_name,
                                  star_error_interface_t* error_iface,  /* DIP */
                                  const hcsr04_config_t* config) {
    /* Store interface reference - driver doesn't know the concrete type */
    handle->error_iface = error_iface;

    /* Use interface without knowing implementation details */
    if (error_iface != NULL) {
        STAR_IFACE_RECORD_ERROR(error_iface, ESP_OK, "HC-SR04 initialized");
    }
    return ESP_OK;
}
```

- **Hardware Swapping**: Replace BNO055 with MPU6050 by providing a different implementation of the same interface
- **Optional Features**: Pass `NULL` for optional interfaces (e.g., error handler) for minimal builds
- **No Circular Dependencies**: Interfaces live in `star_core`; implementations depend on interfaces, not vice versa

17.10.2 System Call Graphs

The following call graphs were generated using GNU cflow, Egypt, and Graphviz to visualize the firmware's execution flow and validate the layered architecture. The complete call graph shows all function relationships in the codebase.

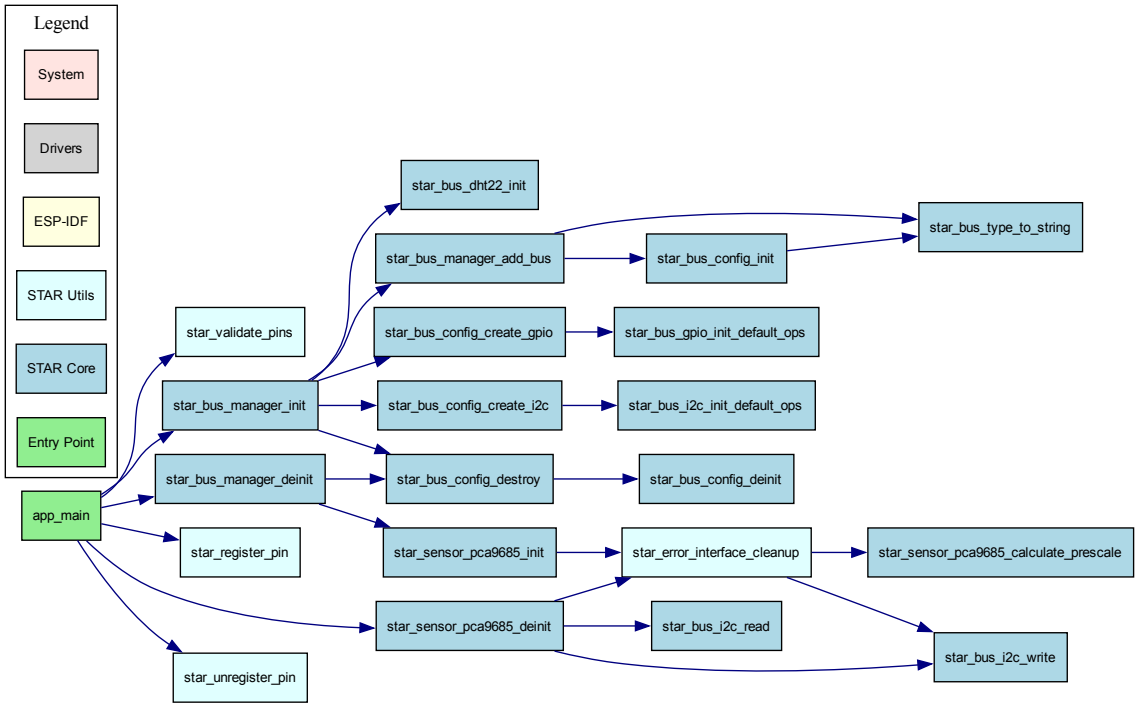


Figure 101: Application Layer Call Graph: Entry Point and Task Initialization

17.10.3 Main Application (`main.c`)

The main application orchestrates system initialization following the dependency injection pattern. It creates abstract interfaces, initializes the bus manager, configures hardware buses, and starts FreeRTOS tasks.

Code Snippet 9: STAR Firmware Main Application (main.c)

```c
#include <esp_log.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>

#include "include/system_config.h"
#include "modules/include/shared_data.h"
#include "star_bus_config.h"
#include "star_bus_dht22_proprietary.h"
#include "star_bus_gpio.h"
#include "star_bus_manager.h"
#include "star_error_handler.h"
#include "star_error_interface.h"
#include "star_pin_interface.h"
#include "star_pin_validator.h"
#include "star_sensor_pca9685.h"
#include "tasks/include/dht22_task.h"
#include "tasks/include/led_task.h"
#include "tasks/include/sensor_task.h"
#include "tasks/include/watchdog_task.h"

/* Error Handler Configuration for Fast LED Response */
static const uint8_t  s_main_fast_error_retries    = 2;
```

```c
static const uint32_t s_main_fast_error_base_delay = 10;
static const uint32_t s_main_fast_error_max_delay  = 50;
static const uint32_t s_main_loop_sleep_ms = 10000;

static const char* s_TAG = "MAIN";

static esp_err_t initialize_hardware_buses(star_bus_manager_t* const bus_manager)
{
  esp_err_t ret;

  /* Initialize DHT22 proprietary bus */
  ESP_LOGI(s_TAG, "Setting up DHT22 on GPIO %d", STAR_SYSTEM_CONFIG.gpio.dht22_data);
  star_dht22_config_t dht22_config = STAR_DHT22_CONFIG_DEFAULT();
  dht22_config.gpio_pin            = STAR_SYSTEM_CONFIG.gpio.dht22_data;

  ret = star_bus_dht22_init(bus_manager, STAR_SYSTEM_CONFIG.bus_names.dht22, &dht22_config);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to init DHT22: %s", esp_err_to_name(ret));
    return ret;
  }

  /* Initialize GPIO bus for HC-SR04 sensors */
  const gpio_num_t hcsr04_pins[] = {
    STAR_SYSTEM_CONFIG.gpio.left_trig,  STAR_SYSTEM_CONFIG.gpio.left_echo,
    STAR_SYSTEM_CONFIG.gpio.right_trig, STAR_SYSTEM_CONFIG.gpio.right_echo};
  star_bus_config_t* hcsr04_gpio_bus = star_bus_config_create_gpio(
    STAR_SYSTEM_CONFIG.bus_names.hcsr04, hcsr04_pins, 4);

  if (hcsr04_gpio_bus == NULL) {
    ESP_LOGE(s_TAG, "Failed to create GPIO bus for HC-SR04");
    return ESP_ERR_NO_MEM;
  }

  ret = star_bus_manager_add_bus(bus_manager, hcsr04_gpio_bus);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to add GPIO bus: %s", esp_err_to_name(ret));
    star_bus_config_destroy(hcsr04_gpio_bus);
    return ret;
  }

  /* Initialize I2C bus for PCA9685 PWM controller */
  star_bus_config_t* pwm_bus = star_bus_config_create_i2c(
    STAR_SYSTEM_CONFIG.bus_names.pca9685,
    STAR_SYSTEM_CONFIG.pca9685.i2c_port,
    STAR_SYSTEM_CONFIG.pca9685.i2c_addr,
    STAR_SYSTEM_CONFIG.gpio.pca9685_sda,
    STAR_SYSTEM_CONFIG.gpio.pca9685_scl,
    STAR_SYSTEM_CONFIG.pca9685.i2c_frequency);

  if (pwm_bus == NULL) {
    ESP_LOGE(s_TAG, "Failed to create I2C bus for PCA9685");
    return ESP_ERR_NO_MEM;
  }

  ret = star_bus_manager_add_bus(bus_manager, pwm_bus);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to add I2C bus: %s", esp_err_to_name(ret));
    star_bus_config_destroy(pwm_bus);
    return ret;
  }

  ESP_LOGI(s_TAG, "All hardware buses initialized successfully");
  return ESP_OK;
}

static esp_err_t initialize_pca9685(star_bus_manager_t* const bus_manager,
                                    star_error_interface_t* const error_iface,
                                    pca9685_handle_t* const pca_handle)
{
  const pca9685_config_t pca9685_config = {
    .i2c_addr      = STAR_SYSTEM_CONFIG.pca9685.i2c_addr,
    .pwm_freq      = STAR_SYSTEM_CONFIG.pca9685.pwm_frequency_hz,
    .output_mode   = k_pca9685_output_totem_pole,
    .ext_clock     = false,
```

```
    .invert_output = false};

  esp_err_t ret = star_sensor_pca9685_init(
    pca_handle, bus_manager, STAR_SYSTEM_CONFIG.bus_names.pca9685,
    error_iface, STAR_SYSTEM_CONFIG.gpio.pca9685_oe, false, &pca9685_config);

  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to init PCA9685: %s", esp_err_to_name(ret));
    return ret;
  }

  ESP_LOGI(s_TAG, "PCA9685 initialized successfully");
  return ESP_OK;
}

static esp_err_t start_system_tasks(star_bus_manager_t* const bus_manager,
                                    star_error_interface_t* const error_iface,
                                    const pca9685_handle_t* pca_handle)
{
  esp_err_t ret;

  ret = dht22_task_start(bus_manager, STAR_SYSTEM_CONFIG.bus_names.dht22);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to start DHT22 task: %s", esp_err_to_name(ret));
    return ret;
  }

  ret = sensor_task_start(bus_manager, error_iface, STAR_SYSTEM_CONFIG.bus_names.hcsr04);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to start sensor task: %s", esp_err_to_name(ret));
    dht22_task_stop();
    return ret;
  }

  ret = led_task_start(pca_handle);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to start LED task: %s", esp_err_to_name(ret));
    sensor_task_stop();
    dht22_task_stop();
    return ret;
  }

  ret = watchdog_task_start();
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to start watchdog task: %s", esp_err_to_name(ret));
    led_task_stop();
    sensor_task_stop();
    dht22_task_stop();
    return ret;
  }

  ESP_LOGI(s_TAG, "All system tasks started successfully");
  return ESP_OK;
}

void app_main(void)
{
  ESP_LOGI(s_TAG, "Starting STAR Firmware - Task-Based Architecture");
  ESP_LOGI(s_TAG, "System: %d HC-SR04 sensors, DHT22 temperature, RGB LED feedback",
           STAR_SYSTEM_CONFIG.num_hcsr04_sensors);

  /* Initialize shared data (FreeRTOS queues, mutexes) */
  esp_err_t ret = shared_data_init();
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to initialize shared data: %s", esp_err_to_name(ret));
    return;
  }

  shared_data_set_system_state(k_system_state_initializing);

  /* Get pin validator interface (DIP) */
  star_pin_interface_t pin_iface;
  pin_validator_get_interface(&pin_iface);

```

```
  /* Create fast-retry error handler (DIP) */
  error_handler_t* fast_error_handler = error_handler_create_custom(
    s_main_fast_error_retries, s_main_fast_error_base_delay,
    s_main_fast_error_max_delay, NULL, NULL);
  if (fast_error_handler == NULL) {
    ESP_LOGE(s_TAG, "Failed to create fast error handler");
    shared_data_deinit();
    return;
  }

  star_error_interface_t* error_iface = malloc(sizeof(star_error_interface_t));
  if (error_iface == NULL) {
    ESP_LOGE(s_TAG, "Failed to allocate error interface");
    error_handler_destroy_default(fast_error_handler);
    shared_data_deinit();
    return;
  }

  error_handler_get_interface(error_iface, fast_error_handler);

  /* Initialize bus manager with injected interfaces */
  star_bus_manager_t bus_manager;
  ret = star_bus_manager_init(&bus_manager, "main_bus_mgr", error_iface, &pin_iface);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Failed to init bus manager: %s", esp_err_to_name(ret));
    free(error_iface);
    error_handler_destroy_default(fast_error_handler);
    shared_data_deinit();
    return;
  }

  /* Initialize all hardware buses */
  ret = initialize_hardware_buses(&bus_manager);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Hardware bus initialization failed");
    star_bus_manager_deinit(&bus_manager);
    free(error_iface);
    error_handler_destroy_default(fast_error_handler);
    shared_data_deinit();
    return;
  }

  /* Initialize PCA9685 PWM controller */
  pca9685_handle_t pca9685_handle;
  ret = initialize_pca9685(&bus_manager, error_iface, &pca9685_handle);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "PCA9685 initialization failed");
    star_bus_manager_deinit(&bus_manager);
    free(error_iface);
    error_handler_destroy_default(fast_error_handler);
    shared_data_deinit();
    return;
  }

  /* Start all FreeRTOS tasks */
  ret = start_system_tasks(&bus_manager, error_iface, &pca9685_handle);
  if (ret != ESP_OK) {
    ESP_LOGE(s_TAG, "Task initialization failed");
    star_sensor_pca9685_deinit(&pca9685_handle);
    star_bus_manager_deinit(&bus_manager);
    free(error_iface);
    error_handler_destroy_default(fast_error_handler);
    shared_data_deinit();
    return;
  }

  shared_data_mark_system_ready();
  ESP_LOGI(s_TAG, "System initialization complete - all tasks are now running");

  /* Keep main task alive to prevent FreeRTOS handle corruption */
  while (1) {
    vTaskDelay(pdMS_TO_TICKS(s_main_loop_sleep_ms));
  }
}
```

17.10.4 System Configuration (`system_config.h`)

The configuration header defines type-safe structures for all hardware parameters, GPIO mappings, and task configurations.

Code Snippet 10: System Configuration Header (system_config.h)

```
#ifndef SYSTEM_CONFIG_H
#define SYSTEM_CONFIG_H

#include <driver/gpio.h>
#include <driver/i2c.h>
#include <stdbool.h>
#include <stdint.h>

/* Task Priorities */
typedef enum {
  k_star_task_priority_sensors  = 1,  /* Lowest - data collection */
  k_star_task_priority_dht22    = 2,  /* Low - optional sensor */
  k_star_task_priority_leds     = 3,  /* High - user responsiveness */
  k_star_task_priority_watchdog = 4   /* Highest - system safety */
} star_task_priority_t;

/* LED Channel Mappings */
typedef enum {
  k_star_led_channel_left_red    = 0,
  k_star_led_channel_left_green  = 1,
  k_star_led_channel_left_blue   = 2,
  k_star_led_channel_right_red   = 3,
  k_star_led_channel_right_green = 4,
  k_star_led_channel_right_blue  = 5,
  k_star_led_channel_max         = 6
} star_led_channel_t;

/* System Health Status */
typedef enum {
  k_star_health_status_ok,
  k_star_health_status_degraded,
  k_star_health_status_critical
} star_health_status_t;

/* GPIO Pin Configuration */
typedef struct {
  const gpio_num_t left_trig, left_echo, right_trig, right_echo;  /* HC-SR04 */
  const gpio_num_t dht22_data;                                     /* DHT22 */
  const gpio_num_t pca9685_sda, pca9685_scl, pca9685_oe;          /* PCA9685 */
} star_gpio_config_t;

/* PCA9685 Configuration */
typedef struct {
  const uint8_t i2c_addr;
  const uint32_t i2c_frequency;
  const i2c_port_t i2c_port;
  const uint32_t pwm_frequency_hz;
} star_pca9685_config_t;

/* Task Configuration */
typedef struct {
  const char* name;
  const uint32_t stack_size;
  const star_task_priority_t priority;
  const uint32_t interval_ms;
} star_task_config_t;

/* Master Configuration Structure */
typedef struct star_system_config {
  const star_gpio_config_t gpio;
  const star_pca9685_config_t pca9685;
  /* ... distance, pwm, tasks, health, freertos, bus_names, log_tags ... */
  const uint32_t num_hcsr04_sensors;
} star_system_config_t;

```

```c
/* Global Configuration Instance */
extern const star_system_config_t STAR_SYSTEM_CONFIG;

#define STAR_SYSTEM_HCSR04_SENSOR_COUNT (2)

/* Inline Validation Functions */
static inline bool star_validate_sensor_index(uint32_t idx) {
  return idx < STAR_SYSTEM_CONFIG.num_hcsr04_sensors;
}

static inline bool star_validate_temperature(float temperature) {
  return (temperature >= -40.0f && temperature <= 80.0f);
}

#endif /* SYSTEM_CONFIG_H */
```

17.10.5 Error Interface (`star_error_interface.h`)

The error interface defines the abstract contract for error handling, enabling dependency injection:

Code Snippet 11: Error Handler Interface (DIP Pattern)

```c
#ifndef STAR_ERROR_INTERFACE_H
#define STAR_ERROR_INTERFACE_H

#include <stdbool.h>
#include "esp_err.h"

/**
 * @brief Error handler interface - abstract operations for error handling
 *
 * Example Usage:
 *   // 1. Create concrete error handler
 *   error_handler_t error_handler;
 *   error_handler_init(&error_handler, 3, 100, 5000, NULL, NULL);
 *
 *   // 2. Get interface from implementation
 *   star_error_interface_t error_iface;
 *   error_handler_get_interface(&error_iface, &error_handler);
 *
 *   // 3. Inject into components
 *   star_bus_manager_init(&bus_manager, "main", &error_iface, &pin_iface);
 */
typedef struct star_error_interface {
  esp_err_t (*record_error)(void* ctx, esp_err_t error, const char* message,
                            const char* file, int line, const char* func);
  bool (*can_retry)(void* ctx);
  esp_err_t (*reset_state)(void* ctx);
  void* ctx;  /* Opaque pointer to concrete implementation */
} star_error_interface_t;

#define STAR_IFACE_RECORD_ERROR(iface, err, msg) \
  ((iface) && (iface)->record_error \
     ? (iface)->record_error((iface)->ctx, (err), (msg), __FILE__, __LINE__, __func__) \
     : (err))

#endif /* STAR_ERROR_INTERFACE_H */
```

17.10.6 Watchdog Task (`watchdog_task.c`)

The watchdog task monitors system health and triggers recovery actions:

Code Snippet 12: Watchdog Task Implementation (Excerpt)

```c
typedef enum {
```

```
  k_watchdog_health_good = 0,
  k_watchdog_health_degraded,
  k_watchdog_health_critical,
  k_watchdog_health_fault
} watchdog_health_level_t;

static watchdog_health_level_t internal_assess_sensor_health(void)
{
  shared_context_t* shared_ctx = shared_data_get_context();
  uint32_t hcsr04_failures = 0, hcsr04_reads = 0;

  if (xSemaphoreTake(shared_ctx->health_mutex,
                     pdMS_TO_TICKS(STAR_SYSTEM_CONFIG.freertos.mutex_timeout_ms)) == pdTRUE) {
    for (uint8_t i = 0; i < STAR_SYSTEM_CONFIG.num_hcsr04_sensors; i++) {
      hcsr04_failures += shared_ctx->health_data.sensor_read_failures[i];
    }
    hcsr04_reads = shared_ctx->health_data.total_sensor_reads;
    xSemaphoreGive(shared_ctx->health_mutex);
  }

  if (hcsr04_reads > 0) {
    const float failure_rate = (float)hcsr04_failures / hcsr04_reads;
    if (failure_rate > STAR_SYSTEM_CONFIG.health.failure_rate_critical) {
      return k_watchdog_health_critical;
    } else if (failure_rate > STAR_SYSTEM_CONFIG.health.failure_rate_degraded) {
      return k_watchdog_health_degraded;
    }
  }
  return k_watchdog_health_good;
}

static esp_err_t internal_take_recovery_action(watchdog_health_level_t level)
{
  switch (level) {
    case k_watchdog_health_degraded:
      shared_data_set_system_state(k_system_state_error_recovery);
      break;
    case k_watchdog_health_critical:
      shared_data_set_system_state(k_system_state_error_recovery);
      if (led_task_is_running()) led_task_emergency_off();
      break;
    case k_watchdog_health_fault:
      shared_data_set_system_state(k_system_state_fault);
      vTaskDelay(pdMS_TO_TICKS(STAR_SYSTEM_CONFIG.health.restart_delay_ms));
      esp_restart();  /* System restart */
      break;
    case k_watchdog_health_good:
      if (shared_data_get_system_state() == k_system_state_error_recovery) {
        shared_data_set_system_state(k_system_state_running);
      }
      break;
  }
  return ESP_OK;
}

static void internal_watchdog_task_loop(void* parameter)
{
  watchdog_task_context_t* ctx = (watchdog_task_context_t*)parameter;
  TickType_t last_wake_time = xTaskGetTickCount();

  while (ctx->task_running) {
    watchdog_health_level_t health = internal_assess_overall_health();
    ctx->health_checks_performed++;

    if (health != k_watchdog_health_good) {
      internal_take_recovery_action(health);
    }

    vTaskDelayUntil(&last_wake_time,
                    pdMS_TO_TICKS(STAR_SYSTEM_CONFIG.tasks.watchdog.interval_ms));
  }
  vTaskDelete(NULL);
}
```

17.10.7 Sensor Task (`sensor_task.c`)

The sensor task reads HC-SR04 ultrasonic sensors and publishes data via queues:

Code Snippet 13: Sensor Task Implementation (Excerpt)

```
static esp_err_t internal_read_single_sensor(sensor_task_context_t* ctx, uint8_t idx)
{
  float distance_cm;
  esp_err_t result = star_sensor_hcsr04_read_distance(
    &ctx->sensor_handles[idx], &distance_cm);

  sensor_data_t sensor_data = shared_data_create_sensor_data(idx, distance_cm, result);
  shared_data_set_sensor_reading(idx, &sensor_data);
  shared_data_update_health(idx, (result == ESP_OK));

  /* Publish to queue for LED task */
  shared_context_t* shared_ctx = shared_data_get_context();
  if (shared_ctx != NULL && shared_ctx->sensor_data_queue != NULL) {
    xQueueSend(shared_ctx->sensor_data_queue, &sensor_data, 0);
  }

  return result;
}

static void internal_sensor_task_loop(void* parameter)
{
  sensor_task_context_t* ctx = (sensor_task_context_t*)parameter;

  while (ctx->task_running) {
    TickType_t start_time = xTaskGetTickCount();

    for (uint8_t i = 0; i < STAR_SYSTEM_CONFIG.num_hcsr04_sensors; i++) {
      if (!ctx->task_running) break;
      internal_read_single_sensor(ctx, i);
      if (i < STAR_SYSTEM_CONFIG.num_hcsr04_sensors - 1) {
        vTaskDelay(pdMS_TO_TICKS(5));  /* Inter-sensor delay */
      }
    }

    TickType_t elapsed = xTaskGetTickCount() - start_time;
    TickType_t target = pdMS_TO_TICKS(STAR_SYSTEM_CONFIG.tasks.sensors.interval_ms);
    if (elapsed < target) vTaskDelay(target - elapsed);
  }
  vTaskDelete(NULL);
}
```

17.10.8 LED Task (`led_task.c`)

The LED task consumes sensor data and updates RGB LEDs based on distance:

Code Snippet 14: LED Task Implementation (Excerpt)

```
static esp_err_t internal_update_led_from_sensor_data(led_task_context_t* ctx,
                                                      const sensor_data_t* data)
{
  if (data->is_valid && data->read_result == ESP_OK) {
    return led_controller_update_distance(&ctx->led_controller,
                                          (led_index_t)data->sensor_index,
                                          data->distance_cm);
  } else {
    return led_controller_turn_off(&ctx->led_controller,
                                   (led_index_t)data->sensor_index);
  }
}

static void internal_process_sensor_data_queue(led_task_context_t* ctx)
{
```

```
  shared_context_t* shared_ctx = shared_data_get_context();
  sensor_data_t sensor_data;
  uint32_t processed = 0;
  const uint32_t max_per_cycle = 8;

  while (processed < max_per_cycle &&
         xQueueReceive(shared_ctx->sensor_data_queue, &sensor_data, 0) == pdTRUE) {
    internal_update_led_from_sensor_data(ctx, &sensor_data);
    processed++;
  }
}

static void internal_led_task_loop(void* parameter)
{
  led_task_context_t* ctx = (led_task_context_t*)parameter;
  TickType_t last_wake_time = xTaskGetTickCount();

  while (ctx->task_running) {
    if (ctx->leds_initialized) {
      internal_process_sensor_data_queue(ctx);
    }
    vTaskDelayUntil(&last_wake_time,
                    pdMS_TO_TICKS(STAR_SYSTEM_CONFIG.tasks.leds.interval_ms));
  }
  vTaskDelete(NULL);
}
```

17.10.9 HC-SR04 Driver API (`star_sensor_hcsr04.h`)

The ultrasonic sensor driver provides ISR-based timing with temperature compensation:

Code Snippet 15: HC-SR04 Sensor Driver API

```
typedef struct {
  gpio_num_t trigger_pin;
  gpio_num_t echo_pin;
  float temperature_c;  /* For speed of sound correction */
} hcsr04_config_t;

typedef struct hcsr04_handle {
  star_bus_manager_t* manager;
  const char* bus_name;
  gpio_num_t trigger_pin, echo_pin;
  float temperature_c;
  star_error_interface_t* error_iface;
  SemaphoreHandle_t mutex;
  bool initialized, owns_error_handler;
  volatile uint32_t echo_start_time, echo_end_time;
  volatile bool measurement_complete, timeout_occurred;
} hcsr04_handle_t;

/* Initialize with dependency injection */
esp_err_t star_sensor_hcsr04_init(hcsr04_handle_t* handle,
                                  star_bus_manager_t* manager,
                                  const char* bus_name,
                                  star_error_interface_t* error_iface,
                                  const hcsr04_config_t* config);

/* Blocking distance read */
esp_err_t star_sensor_hcsr04_read_distance(hcsr04_handle_t* handle,
                                           float* distance_cm);

/* Non-blocking measurement */
esp_err_t star_sensor_hcsr04_trigger(hcsr04_handle_t* handle);
esp_err_t star_sensor_hcsr04_is_complete(const hcsr04_handle_t* handle, bool* complete);
esp_err_t star_sensor_hcsr04_get_result(const hcsr04_handle_t* handle, float* distance_cm);

/* Temperature compensation (v_sound = 331.3 + 0.606 * T) */
esp_err_t star_sensor_hcsr04_set_temperature(hcsr04_handle_t* handle, float temperature_c);
```

17.10.10 Shared Data Module (`shared_data.h`)

The shared data module provides thread-safe inter-task communication:

Code Snippet 16: Shared Data Module API

```c
typedef struct {
  uint8_t sensor_index;
  float distance_cm;
  esp_err_t read_result;
  uint32_t timestamp_ms;
  bool is_valid;
} sensor_data_t;

typedef enum {
  k_system_state_initializing,
  k_system_state_running,
  k_system_state_error_recovery,
  k_system_state_fault
} system_state_t;

typedef struct {
  QueueHandle_t sensor_data_queue;
  QueueHandle_t temperature_data_queue;
  SemaphoreHandle_t health_mutex;
  SemaphoreHandle_t state_mutex;
  TaskHandle_t dht22_task_handle, sensors_task_handle;
  TaskHandle_t leds_task_handle, watchdog_task_handle;
  system_health_t health_data;
  system_state_t system_state;
  float current_temperature_c;
  bool temperature_available;
  sensor_data_t latest_sensors[STAR_SYSTEM_HCSR04_SENSOR_COUNT];
  bool system_ready;
} shared_context_t;

/* Thread-safe API */
esp_err_t shared_data_init(void);
void shared_data_deinit(void);
shared_context_t* shared_data_get_context(void);

void shared_data_update_health(uint8_t sensor_index, bool success);
system_state_t shared_data_get_system_state(void);
void shared_data_set_system_state(system_state_t state);
bool shared_data_get_temperature(float* temperature_c);
void shared_data_set_temperature(float temperature_c, bool valid);
bool shared_data_get_sensor_reading(uint8_t sensor_index, sensor_data_t* data);
void shared_data_set_sensor_reading(uint8_t sensor_index, const sensor_data_t* data);

sensor_data_t shared_data_create_sensor_data(uint8_t idx, float dist, esp_err_t result);
uint32_t shared_data_get_uptime_ms(void);
bool shared_data_is_system_ready(void);
void shared_data_mark_system_ready(void);
```

Note: The complete firmware repository with all 11 custom libraries (29,000+ lines of code), Doxygen documentation, and build configurations is available at the project GitHub repository. The firmware compiles for both ESP32-WROOM (4 MB flash) and ESP32-S3 (16 MB flash + 8 MB PSRAM) targets using PlatformIO.

17.11 Appendix K: Complete Software Process Map

This appendix contains the complete integrated software process map showing all six layers and their interconnections. For detailed explanations of each layer, refer to Section 75 (Software Implementation).

17.11.1 Full System Process Map

The complete software architecture with all data flows, decision points, and inter-layer communication paths is documented in Section 17 (Software Implementation). Each of the six layers has a detailed native TikZ process flow diagram:

- **Layer 1: User Interface**: Figure 76
- **Layer 2: Navigation & Control**: Figure 77
- **Layer 3: SLAM & Mapping**: Figure 78
- **Layer 4: Computer Vision Safety**: Figure 79
- **Layer 5: Hardware & Sensors**: Figure 80
- **Layer 6: ESP32-Firmware Demo**: Figure 81

17.11.2 Layer Summary

Table 129: Software Process Map Layer Summary

Layer	Name	Primary Responsibilities
1	User Interface	Map display, camera feeds, teleoperation, E-Stop handling
2	Navigation & Control	A* path planning, DWA local planning, motion execution
3	SLAM & Mapping	LiDAR processing, Hector-SLAM, loop closure, map updates
4	Computer Vision	Stereo depth, YOLO detection, person-/obstacle safety
5	Hardware & Sensors	Motor control (SPI), IMU (I2C), power monitoring
6	ESP32 Demo	STAR Framework, FreeRTOS tasks, demo sensors/actuators

17.11.3 Legend

Shapes

- **Ellipse**: Start/End nodes
- **Rectangle**: Process steps
- **Diamond**: Decision points
- **Parallelogram**: Data input/output

Colors

- **Yellow**: Start/End/Decision nodes
- **Blue**: Initialization steps
- **Green**: Processing steps
- **Orange**: Optimization/Safety operations
- **Red**: Emergency handling
- **Purple**: Output/Communication
- **Gray**: Data flow

Line Types

- **Solid**: Normal control flow
- **Dashed**: Data flow or feedback
- **Bold Red**: Emergency paths

17.11.4 Abbreviations and Interfaces

Table 130: Process Map Abbreviations

Abbreviation	Meaning
SLAM	Simultaneous Localization and Mapping (Hector-SLAM)
A*	A-star path planning algorithm
DWA	Dynamic Window Approach (local planner)
LiDAR	SICK TiM561 over Ethernet (TCP/IP)
IMU	Inertial Measurement Unit over I2C
SPI	Serial Peripheral Interface (motor commands)
SMBus	System Management Bus (battery monitoring)
ADC	Analog-to-Digital Converter (temperature)
GPIO	General Purpose Input/Output
ICP	Iterative Closest Point (scan matching)
RANSAC	Random Sample Consensus
FPS	Frames Per Second
PWM	Pulse Width Modulation

Table 131: ESP32-Firmware Demo Components

Component	Description
DHT22	Temperature/Humidity sensor (proprietary protocol)
HC-SR04	Ultrasonic distance sensors (GPIO)
PCA9685	16-channel PWM controller (I2C)
STAR Framework	Bus Manager + Error Handler + Pin Validator
DIP	Dependency Inversion Principle architecture
FreeRTOS Tasks	DHT22, Sensor, LED, Watchdog tasks

17.12 Detailed Test Cases

This appendix provides the complete test case specifications for all verification and validation activities. For an overview of the test strategy and methodology, refer to Section 16.0.

17.12.1 Phase 1: Hardware and Electrical Verification

Table 132: PCB Continuity Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-PCB-001	Power rail continuity	Multimeter continuity; probe VCC at multiple points	Beep, $R < 1\Omega$	Critical
TC-PCB-002	Power rail isolation	Probe between VCC/GND; between 3.3V, 5V, 6V rails	No continuity	Critical
TC-PCB-003	Signal trace continuity	Probe each trace from connector to IC pin	All traces continuous	Critical
TC-PCB-004	Solder bridge detection	Visual inspection; probe adjacent IC pins	No shorts detected	Critical
TC-PCB-005	Component orientation	Verify IC pin-1 markers; check diode/cap polarity	All correct	Critical
TC-PCB-006	Solder joint quality	Visual for dull joints; probe for mechanical integrity	Shiny, concave joints	High

PCB Continuity Test Cases

Table 133: Power System Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-PWR-001	3.3V regulation	Measure TLV62569 output, 0–500mA load	$3.3V \pm 3\%$	Critical
TC-PWR-002	5V regulation	Measure TPS566238 output, 0–2A load	$5V \pm 5\%$	Critical
TC-PWR-003	6V motor rail	Measure TPS40305, no-load and full motor load	$6V \pm 10\%$	Critical
TC-PWR-004	Rail ripple (6V)	DHO804 AC coupled, 20MHz BW, full load	$< 100mV_{pp}$	Critical
TC-PWR-005	Undervoltage lockout	Reduce battery voltage; monitor BQ76920 DCHG	Disconnect at $9.6V \pm 0.1V$	Critical
TC-PWR-006	Overcurrent protection	Controlled short via 0.1Ω shunt	Activates $< 10ms$	Critical
TC-PWR-007	Power sequencing	Trigger on 3.3V rise; capture all rails	3.3V → 5V → 6V	High

Power System Test Cases

Table 134: Communication Bus Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-COM-001	I2C enumeration	Execute `i2cdetect`; compare addresses	0x28 (BNO055), 0x77 (BMP280)	Critical
TC-COM-002	I2C signal integrity	DHO804 on SDA/SCL; measure rise time	Rise $< 300ns$; no ringing	High
TC-COM-003	SPI clock accuracy	Measure SCLK on Pi 5 to ESP32-S3 link	±5% of 1MHz	High
TC-COM-004	UART reliability	Transmit 10,000 byte pattern; verify	BER $< 0.01\%$	Critical
TC-COM-005	GPIO logic levels	Measure HIGH/LOW under 1mA load	HIGH $> 2.4V$; LOW $< 0.4V$	Critical
TC-COM-006	SMBus to BQ78350	Read fuel gauge registers via SMBus	Valid SOC/voltage readings	Critical

Communication Bus Test Cases

Table 135: Motor Control Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-MTR-001	Driver enable	Set EN HIGH; apply 50% PWM; measure current	Rotates; 0.5–2A	Critical
TC-MTR-002	PWM speed control	Duty cycles: 25%, 50%, 75%; measure RPM	Proportional $\pm 10\%$	High
TC-MTR-003	Direction control	Command FORWARD then REVERSE via PH pin	Reverses correctly	High
TC-MTR-004	Current sensing	Apply known load; read IPROPI voltage	$\pm 10\%$ of shunt	High
TC-MTR-005	Overcurrent protection	Stall motor; monitor nFAULT	Latches within 10ms	Critical
TC-MTR-006	Encoder feedback	Rotate wheel one revolution; count pulses	Count = 360 ± 1	High
TC-MTR-007	Center-aligned PWM	Verify MCPWM waveforms on scope	Symmetric, reduced EMI	High

Motor Control Test Cases

Table 136: Mechanical System Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-MEC-001	Chassis load	Apply 5kg distributed; measure deflection	$< 1mm$ at center	Critical
TC-MEC-002	Wheel alignment	Drive 3m on smooth floor; measure deviation	$< 10cm$ over 3m	High
TC-MEC-003	Encoder alignment	Rotate wheel slowly; monitor encoder on scope	Clean transitions	High
TC-MEC-004	Cable strain relief	Flex cables 100 times; test continuity	All continuous	Med
TC-MEC-005	Vibration resistance	Max speed 10 min; inspect fasteners	None loosened	Med
TC-MEC-006	Sensor mount rigidity	Apply 2kg lateral to LiDAR mount	Deflection < 2	High
TC-MEC-007	Thermal performance	Full load 15 min; measure driver temp	$< 60C$	High
TC-MEC-008	Doorway clearance	Measure robot dimensions	W $< 76cm$, H $< 100cm$	Critical

Mechanical System Test Cases

17.12.2 Phase 2: Embedded Firmware Integration

Table 137: Firmware and Driver Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-FW-001	IMU gravity vector	Level surface; read BNO055 accel for 10s	$1g \pm 0.05g$; Z dominant	Critical
TC-FW-002	IMU gyro bias	Hold stationary; record gyro 60s	Drift $<$ 1/min	Critical
TC-FW-003	Odometry precision	Drive exactly 1m; compare encoder	Error $< 2\%$ (FR-007)	Critical
TC-FW-004	Motor PID tuning	Step from 0 to target RPM; record	Overshoot $< 10\%$; settle $< 200ms$	High
TC-FW-005	Control loop timing	Toggle GPIO at loop start/end	Period $10ms \pm 0.5ms$	High
TC-FW-006	Watchdog recovery	Inject infinite loop; verify reset	Reset within 2s	Critical
TC-FW-007	Error handler backoff	Trigger I2C failures; monitor retry	Exponential backoff	High
TC-FW-008	Bus disconnection	Disconnect I2C sensor; monitor system	Graceful degradation	Critical
TC-FW-009	HC-SR04 temp comp	Read DS18B20+; verify distance calc	$\pm 0.5\%$ correction	Med

17.12.3 Phase 3: System Integration and ROS 2 Validation

Table 138: System Integration Test Cases (Complete)

TC-ID	Test Case	Procedure	Pass Criteria	Pri
TC-SYS-001	Cold boot	Apply power; time until ROS 2 ready	Boot $< 60s$	Med
TC-SYS-002	ROS 2 node health	Start stack; verify via `ros2 node list`	All running; no respawns	Critical
TC-SYS-003	End-to-end latency	Publish `cmd_vel`; time to wheel motion	Latency $< 100ms$	Critical
TC-SYS-004	SLAM map quality	Teleoperate 20m loop; evaluate closure	Error $< 5cm$ (FR-003/005)	Critical
TC-SYS-005	LiDAR scan rate	Monitor TiM561 via `ros2 topic hz`	$\geq$15Hz (FR-004)	Critical
TC-SYS-006	Obstacle detection	Place 10cm obstacle; command toward it	Detect at $> 50cm$ (FR-002)	Critical
TC-SYS-007	Dynamic avoidance	Walk across planned path	Replan within 1s	High
TC-SYS-008	Autonomous nav	Set goal 50m away; start Nav2	Complete (FR-001)	Critical
TC-SYS-009	Network robustness	Disconnect WiFi 5s; observe	Reconnect $< 10s$; no crash	High
TC-SYS-010	Web UI response	Load dashboard; interact	Load $< 3s$; responsive	High
TC-SYS-011	Map export	Complete mapping; export via map_saver	Valid .pgm/.yaml (FR-006)	Med
TC-SYS-012	Teleoperation	Control via web UI	Latency $< 100ms$ (FR-008)	High
TC-SYS-013	Emergency stop	Press E-Stop during motion	Stop $< 500ms$; $< 15cm$	Critical
TC-SYS-014	Battery endurance	Continuous nav until shutdown	$\geq$60 min (NFR-001)	Critical
TC-SYS-015	CPU utilization	Monitor `htop` during SLAM	$< 80\%$ (NFR-004)	High

18.0 Works Cited

18.1 Project Management & Methodology

Asana. “Critical Path Method (CPM): A Project Management Essential”, 2024, Comprehensive guide to identifying the critical path and calculating float in project schedules, asana.com/resources/critical-path-method. Accessed 8 Dec. 2025.

Atlassian. “RACI Chart: Definitions, Uses and Examples for Project Managers”, 2024, Responsibility Assignment Matrix guide for defining roles: Responsible, Accountable, Consulted, Informed, www.atlassian.com/work-management/project-management/raci-chart. Accessed 8 Dec. 2025.

BarD Software. “GanttProject: Free Project Management Software”, 2024, Open-source desktop project management application under GPL3 license with Gantt charts, PERT charts, and critical path analysis, ganttproject.biz. Accessed 8 Dec. 2025.

Cohn, Mike. *Succeeding with Agile: Software Development Using Scrum*. Introduces the Testing Pyramid concept: 70% unit tests, 20% integration tests, 10% end-to-end tests, Addison-Wesley Professional, 2009.

Defense Acquisition University. “Control Account (CA)”, 2024, www.dau.edu/acquipedia-article/control-account-ca. Accessed 6 Dec. 2025.

INCOSE. *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*. 5th ed., Defines V-Model for systems engineering with verification (did we build the system right?) and validation (did we build the right system?), John Wiley & Sons, 2023.

Kelley, James E., and Morgan R. Walker. “Critical-Path Planning and Scheduling”. *Proceedings of the Eastern Joint Computer Conference*, 1959, Original Critical Path Method (CPM) paper introducing forward/backward pass and float calculations, pp. 160–73. https://doi.org/10.1145/1460299.1460318.

Project Management Academy. “Free Float vs Total Float: Understanding the Difference”, 2024, Explains total float (TF = LS - ES) and free float (FF = ES_successor - EF_current) calculations for CPM, projectmanagementacademy.net/resources/blog/free-float-vs-total-float. Accessed 8 Dec. 2025.

Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th ed., Project Management Institute, 2021.

———. “RACI Chart: Roles and Responsibilities”, 2024, www.pmi.org/learning/library/best-practices-managing-people-quality-management-7012. Accessed 6 Dec. 2025.

Smartsheet. “The Essential Guide to Work Breakdown Structure (WBS)”, 2024, Guide to creating hierarchical WBS with 100% rule and control account structure, www.smartsheet.com/content/construction-work-breakdown-structure. Accessed 8 Dec. 2025.

Wikipedia. “SMART Criteria”, 2024, Specific, Measurable, Achievable, Relevant, and Time-bound goal-setting framework, en.wikipedia.org/wiki/SMART_criteria. Accessed 8 Dec. 2025.

———. “V-Model (Software Development)”, 2024, Systems engineering lifecycle model with verification and validation phases, en.wikipedia.org/wiki/V-model. Accessed 8 Dec. 2025.

WorkBreakdownStructure.com. “Work Breakdown Structure Guide”, 2024, www.workbreakdownstructure.com/. Accessed 6 Dec. 2025.

18.2 Industry Statistics & Market Research

Grand View Research. “Autonomous Mobile Robot Market Size, Share & Trends Analysis Report By Component, By Robot Type, By End Use, By Region, And Segment Forecasts, 2025 - 2030”, 2024, Market valued at USD 4.07 billion in 2024, projected to reach USD 9.56 billion by 2030, www.grandviewresearch.com/industry-analysis/autonomous-mobile-robots-market. Accessed 7 Dec. 2025.

Standard Bots. “Robot Machine Price: How Much Do They Cost in 2025”, 2025, Comprehensive pricing guide for industrial and mobile robots, standardbots.com/blog/robot-machine-price-how-much-do-they-cost. Accessed 7 Dec. 2025.

U.S. Bureau of Labor Statistics. “Census of Fatal Occupational Injuries Summary, 2021”, 2022, 5,283 fatal work injuries recorded in the United States in 2023, www.bls.gov/news.release/cfoi.nr0.htm. Accessed 7 Dec. 2025.

U.S. Department of Labor. “Statement by Assistant Secretary for Occupational Safety and Health on 2021 Census of Fatal Occupational Injuries”, 2022, www.dol.gov/newsroom/releases/osha/osha20221216-1. Accessed 7 Dec. 2025.

UtilitiesOne. “The Impact of Mobile Mapping Systems on Geospatial Data Collection”, 2024, Analysis of efficiency gains from mobile mapping vs traditional surveying, utilitiesone.com/the-impact-of-mobile-mapping-systems-on-geospatial-data-collection. Accessed 7 Dec. 2025.

18.3 Microcontrollers & Computing Platforms

Amazon Web Services. “FreeRTOS API Reference”, 2025, Complete API reference for task creation, queues, semaphores, and timers, www.freertos.org/Documentation/02-Kernel/04-API-references. Accessed 8 Dec. 2025.

———. “FreeRTOS Documentation”, 2025, Real-time operating system kernel for embedded devices with task management, queues, and semaphores, www.freertos.org/Documentation/00-Overview. Accessed 8 Dec. 2025.

Espressif Systems. “ESP-IDF Programming Guide”, 2025, Official development framework for ESP32 with FreeRTOS integration, docs.espressif.com/projects/esp-idf/en/stable/esp32. Accessed 8 Dec. 2025.

———. ESP32 Technical Reference Manual. 2024, www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Accessed 6 Dec. 2025.

———. ESP32-S3-WROOM-1 and ESP32-S3-WROOM-1U Datasheet. Version 1.6. N16 variant: Dual-core Xtensa LX7 at 240 MHz, 512 KB SRAM, 16 MB Flash, two MCPWM units with 3 timers each, GPIO35-37 available (no PSRAM), flexible GPIO matrix for PWM routing, 2024, www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf. Accessed 8 Dec. 2025.

———. “FreeRTOS (ESP-IDF)”, 2025, ESP-IDF FreeRTOS implementation with SMP support for dual-core ESP32, docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos.html. Accessed 8 Dec. 2025.

———. “Motor Control Pulse Width Modulator (MCPWM) - ESP-IDF Programming Guide”, 2025, docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/mcpwm.html. Accessed 6 Dec. 2025.

KiCad Project. “KiCad 9.0 Release Notes”, 2025, Open-source EDA suite for schematic capture and PCB layout, www.kicad.org/blog/2025/02/Version-9.0.0-Released. Accessed 8 Dec. 2025.

PlatformIO Labs. “PlatformIO Documentation”, 2025, Cross-platform embedded development IDE supporting ESP32, Arduino, and unit testing, docs.platformio.org/en/latest. Accessed 8 Dec. 2025.

———. “PlatformIO Unit Testing”, 2025, Unit testing framework for embedded systems with native and target device execution, docs.platformio.org/en/latest/advanced/unit-testing/index.html. Accessed 8 Dec. 2025.

Raspberry Pi Ltd. “Raspberry Pi 5 Documentation”, 2024, www.raspberrypi.com/documentation/computers/raspberry-pi-5.html. Accessed 6 Dec. 2025.

———. Raspberry Pi 5 Product Brief. 2023, datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf. Accessed 6 Dec. 2025.

Raspberry Pi Spy. “Simple Guide to the RPi GPIO Header and Pins”, 2024, GPIO header pinout reference for Raspberry Pi boards including Pi 5, www.raspberrypi-spy.co.uk/2012/06/simple-guide-to-the-rpi-gpio-header-and-pins. Accessed 8 Dec. 2025.

18.4 Power Management & Battery Systems

Texas Instruments. BQ25798 I2C Controlled, 1- to 4-Cell, 5-A Buck-Boost Battery Charger with Dual-Input Selector and USB PD 3.0 OTG Output. Document SLUSDV2B, Rev. January 2023. Supports 1-4S Li-ion, 5A charge current, true MPPT for solar, 96.5% peak efficiency, 2023, www.ti.com/lit/ds/symlink/bq25798.pdf. Accessed 8 Dec. 2025.

———. BQ25798 User's Guide. Document SLUUCB5C. Detailed register map, I2C programming examples, charging algorithm documentation, 2024, www.ti.com/lit/ug/sluucb5c/sluucb5c.pdf. Accessed 8 Dec. 2025.

———. BQ76200 High-Side N-Channel FET Driver for Battery Pack Protection. Document SLUSC16B. Charge pump generates 9-14V above PACK+, controls CHG/DSG/PCHG FETs, works with BQ769x0 series AFEs for complete battery protection, 2022, www.ti.com/lit/ds/symlink/bq76200.pdf. Accessed 8 Dec. 2025.

———. BQ78350 CEDV Li-Ion Gas Gauge and Battery Management Controller. 2024, www.ti.com/lit/ds/slusb48/slusb48.pdf. Accessed 6 Dec. 2025.

———. TLV62569A 2-A, High-Efficiency Synchronous Step-Down Converter in SOT-5X3 Package. Document SLVSE95B, Rev. March 2020. 2.5V-5.5V input, 2A output, 1.5MHz switching, 95% peak efficiency, Forced PWM mode, 2020, www.ti.com/lit/ds/symlink/tlv62569a.pdf. Accessed 8 Dec. 2025.

———. TPS40303/TPS40304/TPS40305 2.95V to 6V Input, 25A Synchronous Buck Controller with D-CAP2 Mode. Document SLUS964D. Voltage-mode with input feed-forward, up to 1.2MHz switching, 25A capability with appropriate FETs, adaptive on-time control, 2023, www.ti.com/lit/ds/symlink/tps40303.pdf. Accessed 8 Dec. 2025.

———. TPS566231/TPS566238 4.5V to 17V Input, 6A Synchronous Buck Converter with D-CAP3 Mode. Document SLVSFD3B. D-CAP3 control with forced CCM mode, 600kHz switching, 6A continuous output, excellent transient response, 2023, www.ti.com/lit/ds/symlink/tps566231.pdf. Accessed 8 Dec. 2025.

18.5 USB Type-C & Power Delivery

———. "Common TPS25751 Use Cases and Setting Using Embedded Controller", 2024, Application Note SLVAFV8. DRP implementation with EC control, 4CC command sequences, GPIO configuration, www.ti.com/lit/pdf/slvafv8. Accessed 8 Dec. 2025.

———. Integrated USB Type-C, USB Power Delivery and Charging Reference Design for 2-4 Cell Batteries. Reference Design TIDUEY1C, Rev. C. Complete DRP implementation with TPS25751D and BQ25798, supports 100W sink/45W source, 2025, www.ti.com/lit/pdf/tiduey1. Accessed 8 Dec. 2025.

———. “TPS25751 and TPS26750 EEPROM Update Over I2C”, 2024, Application Note SLVAFL1. Dual-region EEPROM architecture and safe update procedures, www.ti.com/lit/pdf/slvafl1. Accessed 8 Dec. 2025.

———. TPS25751 Evaluation Module User’s Guide. Document SLVUCP9A. Hardware setup, programming procedures, and DRP evaluation, 2024, www.ti.com/lit/ug/slvucp9a/slvucp9a.pdf. Accessed 8 Dec. 2025.

———. TPS25751 Host Interface Technical Reference Manual. Document SLVUCR8A, Rev. A. 79-page register map and 4CC command interface reference, 2024, www.ti.com/lit/ug/slvucr8a/slvucr8a.pdf. Accessed 8 Dec. 2025.

———. TPS25751D USB Type-C and USB Power Delivery Controller with Integrated Power Switches. Document SLVSH93A, Rev. March 2024. USB-IF Certified for PD 3.1 (TID#10306), supports up to 100W (20V/5A), 2024, www.ti.com/lit/ds/symlink/tps25751.pdf. Accessed 8 Dec. 2025.

———. “USBCPD Application Customization Tool”, 2024, Web-based GUI for TPS25751D configuration, generates firmware patch bundles and VIF files, dev.ti.com/gallery/info/USBPD/USBCPD_Application_Customization_Tool. Accessed 8 Dec. 2025.

18.6 Motor Control & Drivers

DFRobot. “Metal DC Geared Motor w/Encoder - 6V 210RPM 10Kg.cm”, 2024, www.dfrobot.com/product-1617.html. Accessed 6 Dec. 2025.

NXP Semiconductors. PCA9685 16-channel, 12-bit PWM Fm+ I2C-bus LED Controller. Rev 4 (April 2015). 12-bit PWM resolution (4096 steps), 24-1526 Hz programmable frequency, I2C Fast-mode Plus up to 1 MHz, 62 devices per bus via A0-A5 pins, 25 MHz internal oscillator, 2015, cdn-shop.adafruit.com/datasheets/PCA9685.pdf. Accessed 8 Dec. 2025.

Texas Instruments. DRV8243-Q1 Automotive H-Bridge Driver with Integrated Current Sense and Diagnostics. 2024, www.ti.com/lit/ds/symlink/drv8243-q1.pdf. Accessed 6 Dec. 2025.

———. “So, Which PWM Technique is Best?”, 2016, Technical Blog Series on PWM Techniques for Motor Control, e2e.ti.com/blogs_/b/industrial_strength/posts/so-which-pwm-technique-is-best-part-4. Accessed 6 Dec. 2025.

18.7 LiDAR Sensors

SICK AG. “sick_scan_xd ROS 2 Driver”, 2024, Generic ROS 2 driver for SICK 2D LiDAR sensors including TiM5xx series, github.com/SICKAG/sick_scan_xd. Accessed 8 Dec. 2025.

———. TiM561-2050101 2D LiDAR Sensor Datasheet. 10m range, 270-degree FOV, 15 Hz scan rate, IP65 rated, 2024, www.sick.com/media/pdf/6/46/446/dataSheet_TiM561-2050101_1071419_en.pdf. Accessed 8 Dec. 2025.

———. TiM561/TiM571 2D LiDAR Sensors Operating Instructions. 2024, www.sick.com/us/en/catalog/products/lidar-and-radar-sensors/lidar-sensors/tim/tim561-2050101/p/p369446. Accessed 6 Dec. 2025.

SLAMTEC. "RPLiDAR C1 - DTOF LiDAR 360 Degree Laser Range Scanner", 2024, www.dfrobot.com/product-2803.html. Accessed 6 Dec. 2025.

———. RPLiDAR C1 Development Kit User Manual. 2024, www.slamtec.com/en/Lidar/C1. Accessed 6 Dec. 2025.

———. "RPLiDAR Public SDK for C++", 2024, Official SDK supporting RPLiDAR A-series and C-series sensors with cross-platform support, github.com/Slamtec/rplidar_sdk. Accessed 8 Dec. 2025.

———. "RPLiDAR ROS 2 Package", 2024, ROS 2 driver for RPLiDAR sensors publishing LaserScan messages, github.com/Slamtec/rplidar_ros/tree/ros2. Accessed 8 Dec. 2025.

18.8 Environmental & Proximity Sensors

Analog Devices. DS18B20 Programmable Resolution 1-Wire Digital Thermometer. Rev 6 (July 2019). -55 to 125C range, 0.5C accuracy (-10 to 85C), 9-12 bit resolution (93.75-750ms conversion), 1-Wire protocol, enables ultrasonic temperature compensation: v = 331.3 + 0.606*T m/s, 2019, www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf. Accessed 8 Dec. 2025.

Bosch Sensortec. BMP280 Digital Pressure Sensor. Document BST-BMP280-DS001-26, Rev 1.26 (October 2021). 300-1100 hPa range, 1.0 hPa absolute accuracy, 0.12 hPa relative accuracy (1m altitude), I2C up to 3.4 MHz, configurable oversampling 1x-16x, 2021, www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf. Accessed 8 Dec. 2025.

———. BNO055 Intelligent 9-axis Absolute Orientation Sensor. Document BST-BNO055-DS000-18, Rev 1.8 (October 2021). ARM Cortex-M0+ on-chip sensor fusion, 100 Hz quaternion/Euler output, I2C address 0x28/0x29 at 400 kHz, auto-calibration for gyro drift and magnetic disturbances, 2021, www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bno055-ds000.pdf. Accessed 8 Dec. 2025.

DFRobot. "Gravity: 10 DOF IMU AHRS BNO055 + BMP280", 2024, www.dfrobot.com/product-1793.html. Accessed 6 Dec. 2025.

Elecfreaks. HC-SR04 Ultrasonic Ranging Module. 2-400cm range, 15-degree effective beam angle, 40kHz ultrasonic frequency, requires temperature compensation for speed of sound variation, 2024, cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf. Accessed 6 Dec. 2025.

Mouser Electronics. "HC-SR04 Ultrasonic Sensor Technical Specifications", 2024, Complete specifications: 5V operation, 15mA working current, 2cm-400cm range, 3mm resolution at best conditions, www.mouser.com/datasheet/2/813/HCSR04-1022824.pdf. Accessed 8 Dec. 2025.

18.9 Communication Protocols

Analog Devices. "Guide to Comparing I2C Bus to the SMBus", 2024, www.analog.com/en/resources/design-notes/guide-to-comparing-ic-bus-to-the-smbus.html. Accessed 6 Dec. 2025.

———. "I2C Communication Protocol: Understanding I2C Primer, PMBus, and SMBus", 2024, Comprehensive I2C tutorial covering addressing, timing, and protocol variants, www.analog.com/en/resources/analog-dialogue/articles/i2c-communication-protocol-understanding-i2c-primer-pmbus-and-smbus.html. Accessed 8 Dec. 2025.

———. "Introduction to SPI Interface", 2024, SPI fundamentals including full-duplex operation and clock polarity/phase modes, www.analog.com/en/resources/analog-dialogue/articles/introduction-to-spi-interface.html. Accessed 8 Dec. 2025.

i2c-bus.org. "I2C Bus Specification", 2024, Reference for I2C timing, addressing modes, and electrical specifications, www.i2c-bus.org/specification. Accessed 8 Dec. 2025.

NXP Semiconductors. "UM10204 I2C-bus Specification and User Manual", 2021, Rev. 7.0. Standard-mode (100 kbit/s), Fast-mode (400 kbit/s), Fast-mode Plus (1 Mbit/s), High-speed mode (3.4 Mbit/s), www.nxp.com/docs/en/user-guide/UM10204.pdf. Accessed 8 Dec. 2025.

System Management Interface Forum. "System Management Bus (SMBus) Specification Version 3.0", 2014, smbus.org/specs/SMBus_3_0_20141220.pdf. Accessed 6 Dec. 2025.

Texas Instruments. "SMBus Compatibility With an I2C Device", 2009, Application Report SLOA132, www.ti.com/lit/an/sloa132/sloa132.pdf. Accessed 6 Dec. 2025.

Wikipedia. "Serial Peripheral Interface", 2024, SPI protocol overview with CPOL/CPHA clock modes and multi-device addressing, en.wikipedia.org/wiki/Serial_Peripheral_Interface. Accessed 8 Dec. 2025.

———. "USB-C", 2024, USB Type-C connector specifications and USB Power Delivery overview, en.wikipedia.org/wiki/USB-C. Accessed 8 Dec. 2025.

18.10 Cellular Communication

DFRobot. “Gravity: CAT1 A7670G Global 4G IoT Communication Module”, 2024, www.dfrobot.com/product-2802.html. Accessed 6 Dec. 2025.

———. “GSM Antenna with Magnetic Base (3m)”, 2024, www.dfrobot.com/product-1365.html. Accessed 6 Dec. 2025.

18.11 Cameras & Imaging

Linux Kernel Documentation. “Video for Linux API (V4L2)”, 2024, Linux kernel API for video capture devices, camera drivers, and streaming I/O, docs.kernel.org/userspace-api/media/v4l/v4l2.html. Accessed 8 Dec. 2025.

MIPI Alliance. MIPI Camera Serial Interface 2 (CSI-2) Specification. Industry standard for camera-to-processor interfaces supporting D-PHY (up to 4.5 Gbps/lane) and C-PHY (up to 6 Gsps/trio), used by Raspberry Pi Camera Module and IMX sensors, 2022, www.mipi.org/specifications/csi-2. Accessed 8 Dec. 2025.

Waveshare. “IMX219-83 Stereo Camera 8MP Binocular Camera Module”, 2024, www.waveshare.com/imx219-83-stereo-camera.htm. Accessed 6 Dec. 2025.

18.12 SLAM Algorithms

Google. “Cartographer ROS Integration Documentation”, 2024, Algorithm walkthrough for 2D and 3D SLAM with loop closure, google-cartographer-ros.readthedocs.io/en/latest/algo_walkthrough.html. Accessed 8 Dec. 2025.

Hess, Wolfgang, et al. “Real-Time Loop Closure in 2D LIDAR SLAM”. *IEEE International Conference on Robotics and Automation (ICRA)*. Google Cartographer SLAM algorithm, 2016, pp. 1271–78, https://doi.org/10.1109/ICRA.2016.7487258, Google Cartographer SLAM algorithm.

Kohlbrecher, Stefan, et al. “A Flexible and Scalable SLAM System with Full 3D Motion Estimation”. *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. Hector SLAM algorithm using scan matching without odometry, 2011, pp. 155–60, https://doi.org/10.1109/SSRR.2011.6106777, Hector SLAM algorithm using scan matching without odometry.

Macenski, Steve, and Ivona Jambrecic. “SLAM Toolbox: SLAM for the Dynamic World”. *Journal of Open Source Software*. Default SLAM solution for ROS2 Nav2, 2021, p. 2783, https://doi.org/10.21105/joss.02783, Default SLAM solution for ROS2 Nav2.

Wikipedia. “Shakey the Robot”, 2024, First mobile robot with AI reasoning (SRI International, 1966-1972), pioneered A* pathfinding, en.wikipedia.org/wiki/Shakey_the_robot. Accessed 8 Dec. 2025.

18.13 ROS 2 & Navigation

Automatic Addison. “Calculating Wheel Odometry for a Differential Drive Robot”, 2024, Tutorial on encoder-based odometry calculation with ICC kinematics, automaticaddison.com/calculating-wheel-odometry-for-a-differential-drive-robot. Accessed 8 Dec. 2025.

Fox, Dieter, et al. “The Dynamic Window Approach to Collision Avoidance”. *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997, Original DWA algorithm for mobile robot navigation, pp. 23–33. https://doi.org/10.1109/100.580977.

Hart, Peter E., et al. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, 1968, Original A* pathfinding algorithm paper, foundational for robot navigation and motion planning, pp. 100–107. https://doi.org/10.1109/TSSC.1968.300136.

Macenski, Steve, et al. “The Marathon 2: A Navigation System”. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Nav2 navigation framework for ROS2, 2020, pp. 2718–25, https://doi.org/10.1109/IROS45743.2020.9341207, Nav2 navigation framework for ROS2.

Open Robotics. “Configuring DWB Controller”, 2024, Dynamic Window Approach controller configuration for local path planning, docs.nav2.org/configuration/packages/configuring-dwb-controller.html. Accessed 8 Dec. 2025.

———. “Configuring NavFn Planner”, 2024, NavFn global planner using Dijkstra or A* algorithms on costmap, docs.nav2.org/configuration/packages/configuring-navfn.html. Accessed 8 Dec. 2025.

———. “diagnostic_common_diagnostics — ROS 2 Package”, 2024, ROS 2 package providing CPU monitor, memory monitor, and sensors monitor nodes, index.ros.org/p/diagnostic_common_diagnostics/. Accessed 7 Dec. 2025.

———. “geometry_msgs Package”, 2024, ROS 2 message definitions for Twist (cmd_vel), Pose, and Transform, docs.ros.org/en/jazzy/p/geometry_msgs. Accessed 8 Dec. 2025.

———. “nav_msgs Package”, 2024, ROS 2 message definitions for OccupancyGrid, Path, and Odometry, docs.ros.org/en/jazzy/p/nav_msgs. Accessed 8 Dec. 2025.

———. “Nav2 Costmap 2D Configuration”, 2024, Configuration guide for 2D cost maps including inflation layers and obstacle detection, docs.nav2.org/configuration/packages/configuring-costmaps.html. Accessed 8 Dec. 2025.

———. “Nav2 Documentation”, 2024, docs.nav2.org/. Accessed 6 Dec. 2025.

———. “ROS 2 Documentation: Jazzy Jalisco”, 2024, docs.ros.org/en/jazzy/. Accessed 6 Dec. 2025.

———. “ROS 2 Jazzy Jalisco Released”, 2024, Long-term support release of ROS 2 with 5-year EOL (May 2029), www.openrobotics.org/blog/2024/5/ros-jazzy-jalisco-released. Accessed 8 Dec. 2025.

———. “teleop_twist_joy Package”, 2024, Generic joystick teleop for twist robots, publishing geometry_msgs/Twist, docs.ros.org/en/jazzy/p/teleop_twist_joy. Accessed 8 Dec. 2025.

18.14 Machine Learning & Computer Vision

Google. “TensorFlow Lite: Post-training Quantization”, 2024, www.tensorflow.org/lite/performance/post_training_quantization. Accessed 6 Dec. 2025.

Jocher, Glenn, et al. “Ultralytics YOLOv8”, 2023, Version 8.0.0, github.com/ultralytics/ultralytics. Accessed 6 Dec. 2025.

OpenCV Team. “OpenCV Deep Neural Networks (dnn module)”, 2024, docs.opencv.org/4.x/d2/d58/tutorial_table_of_content_dnn.html. Accessed 6 Dec. 2025.

Ultralytics. “YOLOv8 Documentation”, 2024, docs.ultralytics.com/. Accessed 6 Dec. 2025.

18.15 Control Systems & Robotics Theory

Åström, Karl Johan, and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. 2nd ed., Open-access textbook on control systems including PID, Princeton UP, 2021, fbswiki.org/.

Columbia University. “ICC Kinematics for Differential Drive Robots”, 2019, Lecture notes on Instantaneous Center of Curvature (ICC) for two-wheeled robots, www.cs.columbia.edu/~allen/F19/NOTES/icckinematics.pdf. Accessed 8 Dec. 2025.

Imperix. “Model Predictive Control (MPC) Implementation”, 2024, Practical guide to implementing MPC for real-time control applications, imperix.com/doc/implementation/model-predictive-control. Accessed 8 Dec. 2025.

Siciliano, Bruno, et al. *Robotics: Modelling, Planning and Control*. Comprehensive robotics textbook covering kinematics and control, Springer, 2009, https://doi.org/10.1007/978-1-84628-642-1.

Thrun, Sebastian, et al. *Probabilistic Robotics*. Foundational text on probabilistic approaches to robotics including SLAM, MIT P, 2005.

Wikipedia. “Dynamic Window Approach”, 2024, Local path planning algorithm considering robot dynamics and velocity constraints, en.wikipedia.org/wiki/Dynamic_window_approach. Accessed 8 Dec. 2025.

———. “Proportional-Integral-Derivative Controller”, 2024, Overview of PID control theory, tuning methods, and anti-windup techniques, en.wikipedia.org/wiki/Proportional-integral-derivative_controller. Accessed 8 Dec. 2025.

———. “Ziegler-Nichols Method”, 2024, Classic PID tuning method using ultimate gain and oscillation period, en.wikipedia.org/wiki/Ziegler-Nichols_method. Accessed 8 Dec. 2025.

18.16 Web Technologies

JetBrains. “Ktor: Asynchronous Framework for Kotlin”, 2024, ktor.io/. Accessed 6 Dec. 2025.

Microsoft. “TypeScript Documentation”, 2024, www.typescriptlang.org/docs/. Accessed 6 Dec. 2025.

Vue.js. “Pinia: The Intuitive Store for Vue.js”, 2024, Official state management library for Vue.js 3, successor to Vuex with TypeScript support and Composition API integration, pinia.vuejs.org/. Accessed 8 Dec. 2025.

———. “Vue.js 3 Documentation”, 2024, vuejs.org/guide/introduction.html. Accessed 6 Dec. 2025.

18.17 Embedded Linux & Build Systems

Buildroot Developers. “Buildroot: Making Embedded Linux Easy”, 2025, buildroot.org/. Accessed 6 Dec. 2025.

———. The Buildroot User Manual. 2025, buildroot.org/downloads/manual/manual.html. Accessed 6 Dec. 2025.

18.18 Hardware Acceleration

Sugiura, Keisuke, and Hiroki Matsutani. “A Unified Accelerator Design for LiDAR SLAM Algorithms for Low-end FPGAs”. *IEEE International Symposium on Circuits and Systems (ISCAS)*. Keio University, Department of ICS, 2022, pp. 1–5, https://doi.org/10.1109/ISCAS48785.2022.9937658, Keio University, Department of ICS.

18.19 Physics & Acoustics

Engineering ToolBox. “Speed of Sound in Air”, 2024, Temperature and humidity effects on sound velocity for ultrasonic sensor compensation, www.engineeringtoolbox.com/air-speed-sound-d_603.html. Accessed 8 Dec. 2025.

Kinsler, Lawrence E., et al. *Fundamentals of Acoustics*. 4th ed., John Wiley & Sons, 2000.

The Physics Classroom. “The Speed of Sound”, 2024, Educational resource on acoustic wave propagation in different media, www.physicsclassroom.com/class/sound/lesson-2/the-speed-of-sound. Accessed 8 Dec. 2025.

Wikipedia. “Braking Distance”, 2024, Stopping distance calculation: d = v^2 / (2*mu*g), used for robot safety margins, en.wikipedia.org/wiki/Braking_distance. Accessed 8 Dec. 2025.

———. “Speed of Sound”, 2024, v = 331.3 + 0.606*T (m/s) for dry air, where T is temperature in Celsius, en.wikipedia.org/wiki/Speed_of_sound. Accessed 8 Dec. 2025.

18.20 Materials & Manufacturing

ASTM International. ASTM B209/B209M - Standard Specification for Aluminum and Aluminum-Alloy Sheet and Plate. 6061-T6: tensile strength 290-310 MPa (42-45 ksi), yield strength 240-276 MPa (35-40 ksi), 10-12% elongation, used for robot chassis structural components, 2014, www.astm.org/b0209m-14.html. Accessed 8 Dec. 2025.

Prusa Research. “Prusament PETG Technical Data Sheet”, 2023, PETG: tensile strength 40-50 MPa, glass transition 75-85C, heat deflection 70C, superior toughness vs PLA for functional robot parts, prusament.com/materials/prusament-petg/. Accessed 8 Dec. 2025.

18.21 Documentation Tools

van Heesch, Dimitri. “Doxygen: Generate Documentation from Source Code”, 2024, Standard tool for API documentation generation from C/C++/Python source code, supports call graphs via Graphviz integration, www.doxygen.nl/. Accessed 8 Dec. 2025.

18.22 Licensing & Platforms

GitHub, Inc. “GitHub: Let’s Build from Here”, 2024, github.com. Accessed 5 Dec. 2025.

Open Source Initiative. “The MIT License”, 2024, opensource.org/license/mit. Accessed 5 Dec. 2025.